

distribution that maximizes the likelihood of the training data (or maximizes the probability that the model assigns to the training data). *No* in the sense that we only outlined an *explicit* way to approximate our desired joint probability distribution. This is one school of thought. In general, a model that defines an explicit and tractable probability density function allows us to operate directly on the log-likelihood of the training data, compute its gradient, and apply available optimization algorithms to search for the maximum. There are other models that provide an explicit but intractable probability density function, in which case we must use approximations to maximize the likelihood. How do we solve an optimization problem *approximately*? We can either use a deterministic approximation, relying on *variational methods* (variational autoencoder models), or use a stochastic approximation, relying on Markov chain Monte Carlo methods. Finally, there are *implicit* ways to approximate our desired joint probability distribution. Implicit models learn to sample from the unknown distribution without ever explicitly defining a formula for it. Generative adversarial networks fall into this category.

Nowadays, the three most popular approaches to generative modeling are:

Generative adversarial networks

These are implicit density models.

Variational models that provide an explicit but intractable probability density function

We approximate the solution of the optimization problem within the framework of probabilistic graphical models, where we maximize a lower bound on the log-likelihood of the data, since immediately maximizing the log-likelihood of the data is intractable.

Fully visible belief networks

These provide explicit and tractable probability density functions, such as **Pixel Convolutional Neural Networks (PixelCNN) 2016** and **WaveNet (2016)**. These models learn the joint probability distribution by decomposing it into a product of one-dimensional probability distributions for each individual dimension, conditioned on those that preceded it, and learning each of these distributions one at a time. This decomposition is thanks to the product rule or chain rule for probabilities. For example, PixelCNN trains a network that learns the conditional probability distribution of every individual pixel in an image given previous pixels (to the left and to the top of it), and WaveNet trains a network that learns the conditional probability distribution of every individual audio signal in a sound wave conditioned on those that preceded it. The drawbacks here are that these models generate the samples only one entry at a time, and they disallow parallelization. This slows down the generation process considerably. For example, it takes WaveNet two minutes of computation time to generate one second of audio, so we cannot use it for live back-and-forth conversations.

There are other generative models that fall into the above categories but are less popular, due to expensive computational requirements or difficulties in selecting the density function and/or its transformations. These include models that require a change of variables, such as nonlinear independent component estimation (explicit and tractable density model), Boltzmann machine models (explicit and intractable density model, with a stochastic Markov chain approximation to the solution of the maximization problem), and generative stochastic network models (implicit density model, again depending on a Markov chain to arrive at its approximate maximum likelihood). We survey these models briefly toward the end of this chapter. In practice and away from mathematical theory and analysis, Markov chain approaches are out of favor due to their computational cost and reluctance to converge rapidly.

Shifting Our Brain from Deterministic Thinking to Probabilistic Thinking

In this chapter, we are slowly shifting our brains from deterministic thinking to probabilistic thinking. So far in this book, we have only used deterministic functions to make our predictions. The training functions were linear combinations of data features, sometimes composed with nonlinear activators, the loss functions were deterministic discriminators between the true values and the predicted ones, and the optimization methods were based on deterministic gradient descent methods. Stochasticity, or randomness, was only introduced when we needed to make the computations of the deterministic components of our model less expensive, such as stochastic gradient descent or stochastic singular value decomposition; when we split our data sets into training, validation, and test subsets; when we selected our minibatches; when we traversed some hyperparameter spaces; or when we passed the scores of data samples into the softmax function, which is a deterministic function, and interpreted the resulting values as probabilities. In all of these settings, stochasticity and the associated probability distributions related only to specific components of the model, serving only as a means to an end: enabling the practical implementation and computation of the deterministic model. They never constituted a model's core makeup.

Generative models are different than the models that we have seen in previous chapters in the sense that they are probabilistic at their core. Nevertheless, we still have the training, loss, and optimization structure, except that now the model learns a probability distribution (explicitly or implicitly) as opposed to learning a deterministic function. Our loss function then measures the error between the true and the predicted probability distributions (at least for the explicit density models), so we must understand how to define and compute some sort of error function between probabilities instead of deterministic values. We must also learn how to optimize and take derivatives in this probabilistic setting.

In mathematics, it is a much easier problem to evaluate a given function (forward problem) than to find its inverse (inverse problem), let alone when we only have access to a few observations of the function values, such as our data samples. In our probabilistic setting, the forward problem looks like this: given a certain probability distribution, sample some data. The inverse problem is the one we care about: given this finite number of realizations (data samples) of a probability distribution that we do not know, find the probability distribution that most likely generated them. One difficulty that comes to mind is the issue of uniqueness: there could be more than one distribution that fits our data. Moreover, the inverse problem is usually much harder because in essence we have to act backward and undo the process that the forward function followed to arrive at the given observations. The issue is that most processes cannot be undone, and this is somehow bigger than us, embedded in the laws of nature: the universe tends to increase entropy. On top of the hardship inherent in solving inverse problems, the probability distributions that we usually try to estimate for AI applications are high dimensional, with many variables, and we are not even sure that our probabilistic model has accounted for all the variables (but that is problematic for deterministic models as well). These difficulties should not deter us. Representing and manipulating high-dimensional probability distributions is important for many math, science, finance, engineering, and other applications. We must dive into generative models.

Throughout the rest of this chapter, we will differentiate the case when our estimated probability distribution is given with an explicit formula, and when we do not have a formula but instead we numerically generate new data samples from an implicit distribution. Note that in the previous chapters, with all of our deterministic models, we always had explicit formulas for our training functions, including the ones given by decision trees, fully connected neural networks, and convolutional neural networks. Back then, once we estimated these deterministic functions from the data, we could answer questions like: what is the predicted value of the target variable? In probabilistic models, we answer a different question: what is the probability that the target variable assumes a certain value, or lies in a certain interval? The difference is that we do not know how our model combined the variables to produce our result, as in the deterministic case. What we try to estimate in probabilistic models is the probability that the model's variables occur together with the target variable (their joint probability), ideally for all ranges of all variables. This will give us the probability distribution of the target variable, without having to explicitly formulate how the model's variables interact to produce this result. This purely depends on observing the data.

Maximum Likelihood Estimation

Many generative models either directly or indirectly rely on the *maximum likelihood principle*. For probabilistic models, the goal is to learn a probability distribution that approximates the true probability distribution of the observed data. One way to do this is to specify an explicit probability distribution $p_{\text{model}}(\vec{x}; \vec{\theta})$ with some unknown parameters $\vec{\theta}$, then solve for the parameters $\vec{\theta}$ that make the training data set as likely to be observed as possible. That is, we need to find the $\vec{\theta}$ that *maximizes the likelihood* of the training data, assigning a high probability for these samples. If there are m training data points, we assume that they are sampled independently, so that the probability of observing them together is just the product of the probabilities of all the individual samples. So we have:

$$\vec{\theta}_{\text{optimal}} = \arg \max_{\vec{\theta}} p_{\text{model}}(\vec{x}^1; \vec{\theta}) p_{\text{model}}(\vec{x}^2; \vec{\theta}) \cdots p_{\text{model}}(\vec{x}^m; \vec{\theta})$$

Recall that each probability is a number between zero and one. If we multiply all of these probabilities together, we would obtain numbers extremely small in magnitude, which introduces numerical instabilities and runs the risk of underflow (when the machine stores a very small number as zero, essentially removing all significant digits). The *log* function always solves this problem, transforming all numbers whose magnitude is extremely large or extremely small back to the reasonable magnitude realm. The good news is that the log transformation for our probabilities does not affect the values of the optimal $\vec{\theta}$, since the log function is an increasing function. That is, if $f(\vec{\theta}_{\text{optimal}}) \geq f(\vec{\theta})$ for all $\vec{\theta}$, then $\log(f(\vec{\theta}_{\text{optimal}})) \geq \log(f(\vec{\theta}))$ for all $\vec{\theta}$ as well. Composing with increasing functions does not change the inequality sign. The point is that the maximum likelihood solution becomes equivalent to the maximum log-likelihood solution. Now recall that the log function transforms products to sums, so we have:

$$\vec{\theta}_{\text{optimal}} = \arg \max_{\vec{\theta}} \log(p_{\text{model}}(\vec{x}^1; \vec{\theta})) + \log(p_{\text{model}}(\vec{x}^2; \vec{\theta})) + \cdots + \log(p_{\text{model}}(\vec{x}^m; \vec{\theta}))$$

Note that this expression wants to increase each of $p_{\text{model}}(\vec{x}, \vec{\theta})$ for each data sample. That is, it prefers that the values of $\vec{\theta}$ *push up* the graph of $p_{\text{model}}(\vec{x}, \vec{\theta})$ above each

data point $\vec{x}^i$. However, we cannot push up indefinitely. There must be a downward compensation since the hyper-area of the region under the graph has to add up to 1, knowing that $p_{model}(\vec{x}, \vec{\theta})$ is a probability distribution.

We can reformulate our expression in terms of expectation and conditional probabilities:

$$\vec{\theta}_{optimal} = \arg \max_{\vec{\theta}} \mathbb{E}_{x \sim p_{data}} \log \left(p_{model}(\vec{x} | \vec{\theta}) \right)$$

The deterministic models that we discussed in the previous chapters find the models' parameters (or weights) by minimizing a loss function that measures the error between the models' predictions and the true values provided by the data labels, or in other words, between y_{model} and y_{data} . In this chapter, we care about finding the parameters that maximize the log-likelihood of the data. It would be nice if there was a formulation of log-likelihood maximization that is analogous to minimizing a quantity that measures an error between the probability distributions p_{model} and p_{data} , so that the analogy between this chapter and the previous chapters is obvious. Luckily, there is. The maximum likelihood estimation is the same as minimizing the *Kullback-Leibler (KL) divergence* between the probability distribution that generated the data and the model's probability distribution:

$$\vec{\theta}_{optimal} = \arg \min_{\vec{\theta}} Divergence_{KL} \left(p_{data}(\vec{x}) || p_{model}(\vec{x}; \vec{\theta}) \right)$$

If p_{data} happens to be a member of the family of distributions $p_{model}(\vec{x}; \vec{\theta})$, and if we were able to perform the minimization precisely, then we would recover the exact distribution that generated the data, namely p_{data} . However, in practice, we do not have access to the data-generating the distribution; in fact, it is the distribution that we are trying to approximate. We only have access to m samples from p_{data} . These samples define the empirical distribution $\hat{p}_{data}$ that places mass only on exactly these m samples. Now maximizing the log-likelihood of the training set is exactly equivalent to minimizing the KL divergence between $\hat{p}_{data}$ and $p_{model}(\vec{x}; \vec{\theta})$:

$$\vec{\theta}_{optimal} = \arg \min_{\vec{\theta}} Divergence_{KL} \left(\hat{p}_{data}(\vec{x}) || p_{model}(\vec{x}; \vec{\theta}) \right)$$

At this point we might be confused between three optimization problems that are in fact mathematically equivalent, they just happen to come from different subdisciplines and subcultures of mathematics, statistics, natural sciences, and computer science:

- Maximizing the log-likelihood of the training data
- Minimizing the KL divergence between the empirical distribution of the training data and the model's distribution
- Minimizing the cross-entropy loss function between the training data labels and the model outputs, when we are classifying into multiple classes using composition with the softmax function.

Do not be confused. The parameters that minimize the KL divergence are the same as the parameters that minimize the cross-entropy and the negative log-likelihood.

Explicit and Implicit Density Models

The goal of maximum log-likelihood estimation (or minimum KL divergence) is to find a probability distribution $p_{\text{model}}(\vec{x}; \vec{\theta})$ that best explains the observed data.

Generative models use this learned $p_{\text{model}}(\vec{x}; \vec{\theta})$ to generate new data. There are two approaches here, one explicit and the other implicit:

Explicit density models

Define the formula for the probability distribution *explicitly* in terms of $\vec{x}$ and $\vec{\theta}$, then find the values of $\vec{\theta}$ that maximize the log-likelihood of the training data samples by following the gradient vector (the partial derivatives with respect to the components of $\vec{\theta}$) uphill. One glaring difficulty here is coming up with a formula for the probability density that is able to capture the complexity in the data, while at the same time staying amiable to computing the log-likelihood with its gradient.

Implicit density models

Sample directly from $p_{\text{model}}(\vec{x}; \vec{\theta})$ without ever writing a formula for this distribution. Generative stochastic networks do this based on a Markov chain framework, which is slow to converge and thus unpopular for practical applications. Using this approach, the model stochastically transforms an existing sample to obtain another sample from the same distribution. Generative adversarial networks interact indirectly with the model's probability distribution without explicitly defining it. They set up a zero-sum game between two networks, where

one network generates a sample and the other network acts like a classifier determining whether the generated sample is from the correct distribution or not.

Explicit Density-Tractable: Fully Visible Belief Networks

These models admit an explicit probability density function with tractable log-likelihood optimization. They rely on the **chain rule of probability** to decompose the joint probability distribution $p_{model}(\vec{x})$ into a product of one-dimensional probability distributions:

$$p_{model}(\vec{x}) = \prod_{i=1}^n p_{model}(x_i | x_1, x_2, \dots, x_{i-1})$$

The main drawback here is that samples must be generated one component at a time (one pixel of an image, or one character of a word, or one entry of a discrete audio wave), therefore, the cost of generating one sample is $O(n)$.

Example: Generating Images via PixelCNN and Machine Audio via WaveNet

PixelCNN trains a convolutional neural network that models the conditional distribution of every individual pixel, given previous pixels (to the left and to the top of the target pixel). **Figure 8-1** illustrates this.

WaveNet trains a convolutional neural network that models the conditional distribution of each entry of an audiowave, given the previous entries. We will only elaborate on WaveNet. It is the one-dimensional analog of PixelCNN and captures the essential ideas.

The goal of WaveNet is to generate wideband raw audio waveforms. So we must learn the joint probability distribution of an audio waveform $\vec{x} = (x_1, x_2, \dots, x_T)$ from a certain genre.

We use the product rule to decompose the joint distribution into a product of single variable distributions where we condition each entry of the audio waveform on those that preceded it:

$$p_{model}(\vec{x}) = \prod_{t=1}^T p_{model}(x_t | x_1, x_2, \dots, x_{t-1})$$

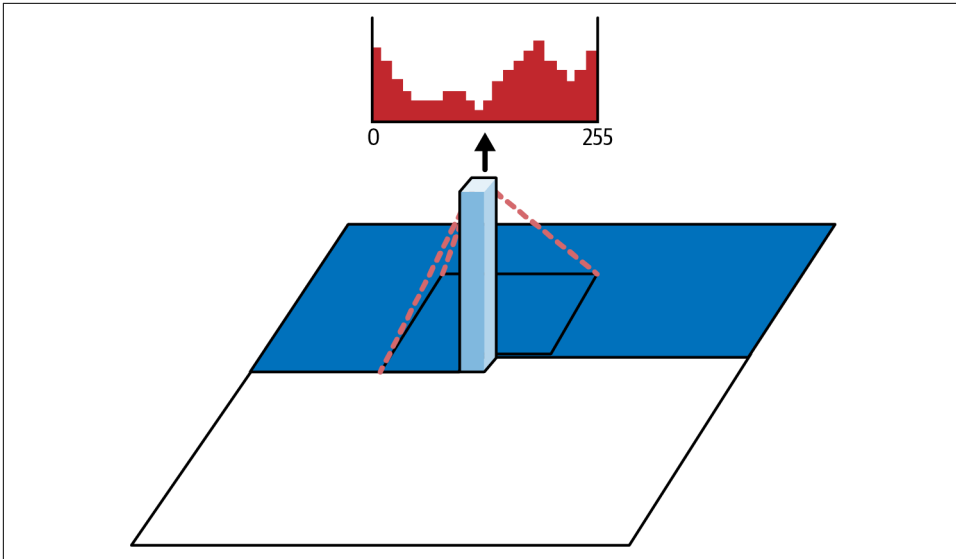


Figure 8-1. PixelCNN learning the conditional distribution of the n th pixel conditioned on the previous $n-1$ pixels (*image source*)

One difficulty is that audio waveforms have very high temporal resolution, with at least 16,000 entries per 1 second of audio (so one data sample that is a minute long is a vector with $T = 960,000$ entries). Each entry represents one time step of discretized raw audio, and is usually stored as a 16-bit integer. That is, each entry can assume any value between 0 and 65,535. If we keep this range, the network has to learn the probability for each entry, so the softmax function at the output level has to output 65,536 probability scores for every single entry. The total number of entries we have to do this for, along with the computational complexity of the network itself, become very expensive. To make this more tractable, we must quantize, which in electronics means approximate a continuously varying signal by one whose amplitude is restricted to a prescribed set of values. WaveNet transforms the raw data to restrict the entries' values to 256 options each, ranging from 0 to 255, similar to the pixel range for digital images. Now, during training the network must learn the probability distribution of each entry over these 256 values, given the preceding entries, and during audio generation it samples from these learned distributions one entry at a time.

The last complication is that if the audio signal represents anything meaningful, then the vector representing it has long-range dependencies over multiple time scales. To capture these long-range dependencies, WaveNet uses dilated convolutions. These are one-dimensional kernels or filters that skip some entries to cover a wider range without increasing the number of parameters (see [Figure 8-2](#) for an illustration).

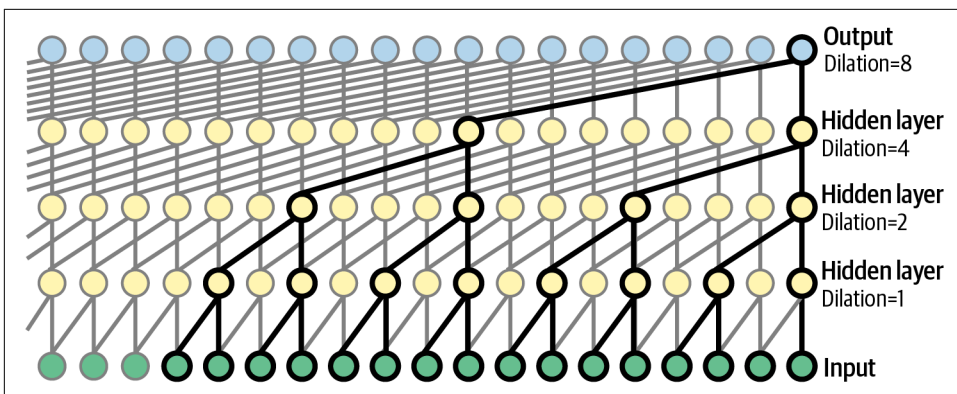


Figure 8-2. Dilated convolution with kernel size equals two. At each layer the kernel has only two parameters but it skips entries for larger coverage (*image source that has a nice animation*).

Note also that the network cannot peek into the future, so the filters at each layer cannot use entries from the training sample that are ahead of the target entry. In one dimension we just stop filtering earlier at each convolutional layer, so it is a simple time shift. In two dimensions we use *masked* filters, which have zeros to the right and to the bottom of the central entry.

WaveNet learns a total of T probability distributions, one for each entry of the audio waveform conditioned on those entries that preceded it: $p_{model}(x_1), p_{model}(x_2|x_1), p_{model}(x_3|x_1, x_2), \dots$ and $p_{model}(x_T|x_1, x_2, \dots, x_{T-1})$. During training, these distributions can be computed in parallel.

Now suppose we need to learn the probability distribution of the 100th entry, given the previous 99 entries. We input batches of audio samples from the training data, and the convolutional network uses only the first 99 entries of each sample, computing linear combinations (the filters linearly combine), passing through non-linear activation functions from one layer to the next to the next using some skip connections and residual layers to battle vanishing gradients, and finally passing the result through a softmax function and outputting a vector of length 256 containing probability scores for the value of the 100th entry. This is the probability distribution for the 100th entry output by the model. After comparing this output distribution with the empirical distribution of the data for the 100th entry from the training batch, the parameters of the network get adjusted to decrease the error (lower the cross-entropy or increase the likelihood). As more batches of data and more epochs pass through the network, the probability distribution for the 100th entry, given the previous 99, will approach the empirical distribution from the training data. What we save within the network after training are the values of the parameters. Now we can use the trained network to generate machine audio, one entry at a time:

1. Sample a value x_1 from the probability distribution $p_{model}(x_1)$.
2. Augment (x_1) with zeros to establish the required length for the network's input and pass the vector through the network. We will get as an output $p_{model}(x_2|x_1)$, from which we can sample x_2 .
3. Augment (x_1, x_2) with zeros and pass the vector through the network. We will get as an output $p_{model}(x_3|x_1, x_2)$, from which we can sample x_3 .
4. Keep going.

We can condition WaveNet on a certain speaker identity, so we can generate different voices using one model.

The fact that we can train WaveNet in parallel but use it to generate audio only sequentially is a major shortcoming. This has been rectified with **Parallel WaveNet**, which is deployed online by Google Assistant, including serving multiple English and Japanese voices.

To summarize and place this discussion in the same mathematical context as this chapter, PixelCNN and WaveNet are models that aim to learn the joint probability distribution of image data or audio data from certain genres. They do so by decomposing the joint distribution into a product of one-dimensional probability distributions for each entry of their data, conditioned on all the preceding entries. To find these one-dimensional conditional distributions, they use a convolutional network to learn the way the observed entries interact together to produce a distribution of the next entry. This way, the input to the network is deterministic, and its output is a probability mass function. The network itself is also a deterministic function. We can view the network together with its output as a probability distribution with parameters that we tweak. As the training evolves, the output gets adjusted until it reaches an acceptable agreement with the empirical distribution of the training data. Therefore, we are not applying a deterministic function to a probability distribution and tweaking the function's parameters until we agree with the distribution of the training data. We are instead starting with an explicit formula for a probability distribution with many parameters (the network's parameters), then tweaking the parameters until this explicit probability distribution reasonably agrees with training data. We do this for each conditional probability distribution corresponding to each entry.

Explicit Density-Tractable: Change of Variables Nonlinear Independent Component Analysis

The main idea here is that we have the random variable representing the observed training data $\vec{x}$ and we want to learn the source random variable $\vec{s}$ that generated

it. We assume that there is a deterministic transformation $g(\vec{s}) = \vec{x}$ that is invertible and differentiable that transforms the unknown $\vec{s}$ to the observed $\vec{x}$. That is, $\vec{s} = g^{-1}(\vec{x})$. Now we need to find an appropriate g to find the probability distribution of $\vec{s}$. Moreover, we assume that $\vec{s}$ has independent entries, or components, so that its probability distribution is nothing but the product of the distributions of its components.

The formula that relates the probability distribution of a random variable with the probability distribution of a deterministic transformation of it is:

$$\begin{aligned} p_s(\vec{s}) &= p_x(\vec{x}) \times \text{determinant}(\text{Jacobian}) \\ &= p_x(g(\vec{s})) \left| \det \left(\frac{\partial g(\vec{s})}{\partial \vec{s}} \right) \right| \end{aligned}$$

Multiplying by the determinant of the Jacobian of the transformation accounts for the change in volume in space due to the transformation.

Nonlinear independent component estimation models the joint probability distribution as the nonlinear transformation of the data $\vec{s} = g^{-1}(\vec{x})$. The transformation g is learned such that g^{-1} maps the data to a latent space where it conforms to a factorized distribution; that is, the mapping results in independent latent variables. The transformation g^{-1} is parameterized to allow for easy computation of the determinant of the Jacobian and the inverse Jacobian. g^{-1} is based on a deep neural network and its parameters are learned by optimizing the log-likelihood, which is tractable.

Note that the requirement that the transformation g must be invertible means that the latent variables $\vec{s}$ must have the same dimension as the data features (length of $\vec{x}$). This imposes restrictions on the choice of the function g and is a disadvantage of nonlinear independent component analysis models.

In comparison, generative adversarial networks impose very few requirements on g , and, in particular, allow $\vec{s}$ to have more dimensions than $\vec{x}$.

Explicit Density-Intractable: Variational Autoencoders Approximation via Variational Methods

Deterministic autoencoders are composed of an encoder that maps the data from x space to latent z space of lower dimension, and a decoder that in turn maps the data from z space to $\hat{x}$ space, with the objective of not losing much information, or

reducing the reconstruction error, which means keeping x and $\hat{x}$ close, for example, in the Euclidean distance sense. In this sense, we can view *principal component analysis*, which is based on the singular value decomposition $X = U\Sigma V^t$, as a linear encoder, where the decoder is simply the transpose of the encoding matrix. Encoding and decoding functions can be nonlinear and/or neural networks.

For deterministic autoencoders, we cannot use the decoder as a data generator. At least, if we do, then we have to pick some z from latent z space and apply the decoder function to it. We are unlikely to get any $\hat{x}$ that is close to how the desired data x looks, unless we picked a z that corresponds to a coded x due to overfitting. We need a regularization that provides us with some control over z space, giving us the benefit of avoiding overfitting and using autoencoders as a data generator. We accomplish this by shifting from deterministic autoencoding to probabilistic autoencoding.

Variational autoencoders are probabilistic autoencoders: the encoder outputs probability distributions over the latent space z instead of single points. Moreover, during training, the loss function includes an extra regularization term that controls the distribution over the latent space. Therefore, the loss function for variational autoencoders contains a reconstruction term (such as mean squared distance) and a regularization term to control the probability distribution output by the encoder. The regularization term can be a KL divergence from a Gaussian distribution, since the underlying assumption is that simple probabilistic models best describe the training data. In other words, complex relationships can be probabilistically simple. We have to be careful here, since this introduces a bias: the simple assumption on the data distribution in the latent variable can be a drawback if it is too weak. That is, when the assumption on the prior distribution or the assumption on the approximate posterior distribution is too weak, even with a perfect optimization algorithm and infinite training data, the gap between the estimate $\mathcal{L}$ and the true log-likelihood can lead to p_{model} learning a completely different distribution than the true p_{data} .

Mathematically, we maximize a lower bound $\mathcal{L}$ on the log-likelihood of the data. In science, variational methods define lower bounds on an energy functional that we want to maximize, or upper bounds on an energy functional that we want to minimize. These bounds are usually easier to obtain and have tractable optimization algorithms, even when the log-likelihood does not. At the same time, they provide good estimates for the optimal values that we are searching for:

$$\mathcal{L}(\vec{x}, \vec{\theta}) \leq \log p_{model}(\vec{x}, \vec{\theta})$$

Variational methods often achieve very good likelihood, but subjective evaluation of samples regard their generated samples as having lower quality. They are also considered more difficult to optimize than fully visible belief networks. Moreover, people find their mathematics more difficult than that of fully visible belief networks and of generative adversarial networks (discussed soon).

Explicit Density-Intractable: Boltzman Machine Approximation via Markov Chain

Boltzmann machines (originating in the 1980s) are a family of generative models that rely on Markov chains to train generative models. This is a sampling technique that happens to be more expensive than the simple sampling of a mini-batch from a data set to estimate a loss function. We will discuss Markov chains in the context of reinforcement learning in [Chapter 11](#). In the context of data generation, they have many disadvantages that caused them to fall out of favor: high computational cost, impractical and less efficient to extend to higher dimensions, slow to converge, and no clear way to know whether the model has converged or not, even when the theory says it must converge. Markov chain methods have not scaled to problems like ImageNet generation.

A Markov chain has a transition operator q that encodes the probability of transitioning from one state of the system to another. This transition operator q needs to be explicitly defined. We can generate data samples by repeatedly drawing a sample $x' \sim q(x'|x)$, updating x' sequentially according to the transition operator q . This sequential nature of generation is another disadvantage compared to single step generation. Markov chain methods can sometimes guarantee that x' will eventually converge to a sample from $p_{model}(x)$, even though the convergence might be slow.

Some models, such as deep Boltzman machines, employ both Markov chain and variational approximations.

Implicit Density-Markov Chain: Generative Stochastic Network

Generative stochastic networks ([Bengio et al. 2014](#)) do not explicitly define a density function, and instead use a Markov chain transition operator that interacts indirectly with $p_{model}(x)$ by sampling from the training data. This Markov chain operator must be run several times to obtain a sample from $p_{model}(x)$. These methods still suffer from the shortcomings of Markov chain methods mentioned in the previous section.

Implicit Density-Direct: Generative Adversarial Networks

Currently the most popular generative models are:

- Fully visible deep belief networks, such as PixelCNN, WaveNet, and their variations.
- Variational autoencoders, consisting of a probabilistic encoder-decoder architecture.
- Generative adversarial networks, which have received a lot of attention from the scientific community due to the simplicity of their concept and the good quality of their generated samples. We discuss them now.

Generative adversarial networks were introduced in 2014 by Ian Goodfellow et al. The mathematics involved is a beautiful mixture between probability and game theory. Generative adversarial networks avoid some disadvantages associated with other generative models:

- Generating samples all at once, in parallel, as opposed to feeding a new pixel back into the network to predict the one, such as in PixelCNN.
- The generator function has few restrictions. This is an advantage relative to Boltzmann machines, for which few probability distributions admit tractable Markov chain sampling, and relative to nonlinear independent component analysis, for which the generator must be invertible and the latent variables z must have the same dimension as the samples x .
- Generative adversarial networks do not need Markov chains. This is an advantage relative to Boltzmann machines and to generative stochastic networks.
- While variational autoencoders might never converge to the true data generating distribution if they assume prior or posterior distributions that are too weak, generative adversarial networks converge to the true p_{data} given that we have infinite training data and a large enough model. Moreover, generative adversarial networks do not need variational bounds, and the specific model families used within the generative adversarial network framework are already known to be universal approximators. Thus, generative adversarial networks are already known to be asymptotically consistent. On the other hand, some variational autoencoders are conjectured to be asymptotically consistent, but this still needs to be proven.

The disadvantage of generative adversarial networks is that training them requires spotting the Nash equilibrium of a game, which is more difficult than just optimizing an objective function. Moreover, the solution tends to be numerically unstable. This was improved in 2015 by Alec Radford et al. in their paper

“Unsupervised Representation Learning with Deep Convolutional Generative Adversarial Networks”. This approach led to more stable models.

During training, generative adversarial networks formulate a game between two separate networks: a generator network and a discriminator network that tries to classify generator samples as either coming from the true distribution $p_{data}(x)$ or from the model $p_{model}(x)$. The loss functions of the two networks are related, so that the discriminator communicates the discrepancy between the two distributions, and the generator adjusts its parameters accordingly until it exactly reproduces the true data distribution (in theory) so that the discriminator’s classifications are no better than random guesses.

The generator network wants to maximize the probability that the discriminator assigns the wrong label in its classification, whether the sample is from the training data or from the model, while the discriminator network wants to minimize that probability. This is a two-player zero-sum game, where one player’s gain is another’s loss. We end up solving a minimax problem instead of a purely maximizing or minimizing problem. A unique solution exists.

How Do Generative Adversarial Networks Work?

Keeping the goal of learning the generator’s probability distribution $p_g(\vec{x}; \vec{\theta})$ over the data, here’s how the learning progresses for generative adversarial networks:

1. Start with a random sample $\vec{z}$ from a prior probability distribution $p_z(\vec{z})$, which could be just uniform random noise for each component of $\vec{z}$.
2. Start also with a random sample $\vec{x}$ from the training data, so it is a sample from the probability distribution $p_{data}(\vec{x})$ that the generator is trying to learn.
3. Apply to $\vec{z}$ the deterministic function $G(\vec{z}, \vec{\theta}_g)$ representing the generative neural network. The parameters $\vec{\theta}_g$ are the ones we need to tweak via backpropagation until the output $G(\vec{z}, \vec{\theta}_g)$ looks similar to samples from the training data set.
4. Pass the output $G(\vec{z}, \vec{\theta}_g)$ into another deterministic function D representing the discriminative neural network. Now we have the new output $D(G(\vec{z}, \vec{\theta}_g), \vec{\theta}_d)$ that is just a number closer to one or zero, signifying whether this sample came from the generator or from the training data. Thus, for this input from the

generator, $D\left(G\left(\vec{z}, \vec{\theta}_g\right), \vec{\theta}_d\right)$ must return a number close to one. The parameters $\vec{\theta}_d$ are the ones we need to tweak via backpropagation until D returns the wrong classification around half of the time.

5. Pass also the sample $\vec{x}$ from the training data to D , so we evaluate $D\left(\vec{x}, \vec{\theta}_d\right)$. For this input, $D\left(\vec{x}, \vec{\theta}_d\right)$ must return a number close to zero.
6. What is the loss function for these two networks, that has in its formula both sets of parameters $\vec{\theta}_g$ and $\vec{\theta}_d$, along with the sampled vectors $\vec{x}$ and $\vec{z}$? The discriminator function D wants to get it right for both types of inputs, $\vec{x}$ and $G\left(\vec{z}, \vec{\theta}_g\right)$. So its parameters $\vec{\theta}_d$ must be selected so that a number close to 1 is assigned a large score when the input is $G\left(\vec{z}, \vec{\theta}_g\right)$, and a number close to 0 is assigned a large value when the input is $\vec{x}$. In both cases, we can use the negative of the log function since that is a function that is large near 0 and small near 1. Therefore, D needs the parameters $\vec{\theta}_d$ that maximize:

$$\mathbb{E}_{\vec{x} \sim p_{data}(\vec{x})} \left[\log D\left(\vec{x}, \vec{\theta}_d\right) \right] + \mathbb{E}_{\vec{z} \sim p_z(\vec{z})} \left[\log \left(1 - D\left(G\left(\vec{z}, \vec{\theta}_g\right), \vec{\theta}_d\right) \right) \right]$$

At the same time, G needs the parameters $\vec{\theta}_g$ that minimize $\log \left(1 - D\left(G\left(\vec{z}, \vec{\theta}_g\right), \vec{\theta}_d\right) \right)$. Combined, D and G engage in a two-player minimax game with value function $V(D, G)$:

$$\min_{\vec{\theta}_g} \max_{\vec{\theta}_d} \mathbb{E}_{\vec{x} \sim p_{data}(\vec{x})} \left[\log D\left(\vec{x}, \vec{\theta}_d\right) \right] + \mathbb{E}_{\vec{z} \sim p_z(\vec{z})} \left[\log \left(1 - D\left(G\left(\vec{z}, \vec{\theta}_g\right), \vec{\theta}_d\right) \right) \right]$$

This is a very simple mathematical structure, where setting up a discriminator network allows us to get closer to the true data distribution without ever explicitly defining it or assuming anything about it.

Finally, we note that generative adversarial networks are highly promising for many applications. One example is the dramatic enhancement they have for semi-supervised learning, where the “[NIPS 2016 Tutorial: Generative Adversarial Networks](#)” (Goodfellow 2016) reports:

We introduce an approach for semi-supervised learning with generative adversarial networks that involves the discriminator producing an additional output indicating the label of the input. This approach allows us to obtain state of the art results on MNIST, SVHN, and CIFAR-10 in settings with very few labeled examples. On MNIST, for example, we achieve 99.14% accuracy with only 10 labeled examples per class with a fully connected neural network—a result that’s very close to the best known results with fully supervised approaches using all 60,000 labeled examples. This is very promising because labeled examples can be quite expensive to obtain in practice.

Another far-reaching application of generative adversarial networks (and machine learning in general) is simulating data for high energy physics. We discuss this next.

Example: Machine Learning and Generative Networks for High Energy Physics

The following discussion is inspired by and borrows from the [Machine Learning for Jet Physics Workshop 2020](#) and the two articles “[Deep Learning and Its Application to LHC Physics](#)” (Guest et al. 2018) and “[Graph Generative Adversarial Networks for Sparse Data Generation in High Energy Physics](#)” (Kansal et al. 2021).

Before the deep learning revolution began in 2012, the field of high energy physics traditionally relied in its analyses and computations on physical considerations and human intuition, boosted decision trees, handcrafted data feature engineering and dimensionality reduction, and traditional statistical analysis. These techniques, while insightful, are naturally far from optimal and hard to automate or extend to higher dimensions. Several studies have demonstrated that traditional shallow networks based on physics-inspired engineered *high-level* features are outperformed by deep networks based on the higher-dimensional *lower-level* features that receive less pre-processing. Many areas of the [Large Hadron Collider](#) data analysis have suffered from long-standing suboptimal feature engineering, and deserve reexamination. Thus, the high energy physics field is a breeding ground ripe for machine learning applications. A lot of progress is taking place on this front. The field is employing several machine learning techniques, including artificial neural networks, kernel density estimation, support vector machines, genetic algorithms, boosted decision trees, random forests, and generative networks.

The experimental program of the Large Hadron Collider probes the most fundamental questions in modern physics: the nature of mass, the dimensionality of space, the unification of the fundamental forces, the particle nature of dark matter, and the fine-tuning of the [Standard Model](#). One driving goal is to understand the most fundamental structure of matter. Part of that entails searching for and studying exotic particles, such as the top quark and Higgs boson, produced in collisions at accelerators such as the Large Hadron Collider. Specific benchmarks and challenges include mass reconstruction, jet substructure, and jet-flavor classification. For example, one can

identify jets from heavy (c , b , t) or light (u , d , s) quarks, gluons, and W , Z , and H bosons.

Running high energy particle experiments and collecting the resulting data is extremely expensive. The data collected is enormous in terms of the number of collisions and the complexity of each collision. In addition, the bulk of accelerator events does not produce interesting particles (signal particles versus background particles). Signal particles are rare, so high data rates are necessary. For example, the Large Hadron Collider detectors have $O(10^8)$ sensors used to record the large number of particles produced after each collision. It is thus of paramount importance to extract maximal information from experimental data (think regression and classification models), to accurately select and identify events for effective measurements, and to produce reliable methods for simulating new data similar to data produced by experiments (think generative models). High energy physics data is characterized by its high dimensionality, along with the complex topologies of many signal events.

This discussion ties into our chapter through the nature of collisions and the interaction of their products with Large Hadron Collider detectors. They are quantum mechanical, and therefore the observations resulting from a particular interaction are fundamentally probabilistic. The resulting data analysis must then be framed in statistical and probabilistic terms.

In our chapter, our aim is to learn the probability distribution $p(\vec{\theta} | \vec{x})$ of the model's parameters given the observed data. If the data is fairly low dimensional, such as less than five dimensions, the problem of estimating the unknown statistical model from the simulated samples would not be difficult, using histograms or kernel-based density estimates. However, we cannot easily extend these simple methods to higher dimensions, due to the curse of dimensionality. In a single dimension, we would need N samples to estimate the source probability density function, but in d dimensions, we would need $O(N^d)$. The consequence is that if the dimension of the data is greater than 10 or so, it is impractical or even impossible to use naive methods to estimate the probability distribution, requiring a prohibitive amount of computational resources.

High energy physicists have traditionally dealt with the curse of dimensionality by reducing the dimension of the data through a series of steps that operate both on individual collision events and collections of events. These established approaches were based on specific, hand-engineered features in the data to a number small enough to allow the estimation of the unknown probability distribution $p(x | \theta)$ using samples generated by simulation tools. Obviously, due to complexity of the data and the rarity of potential new physics, along with its subtle signatures, this traditional approach is probably suboptimal. Machine learning eliminates the need for hand-engineering features and manual dimensionality reduction that can miss crucial

information in the lower-level higher-dimensional data. Moreover, the structure of lower-level data obtained directly from the sensors fits very well with well-established neural network models, such as convolutional neural networks and graph neural networks; for example, the projective tower structure of calorimeters present in nearly all modern high energy physics detectors is similar to the pixels of an image.

Note, however, that while the image-based approach has been successful, the actual detector geometry is not perfectly regular, thus some data preprocessing is required to represent jet images. In addition, jet images are typically very sparse. Both irregular geometry and sparsity can be addressed using *graph-based convolutional networks* instead of the usual convolutional networks for our particle data modeling. Graph convolutional networks extend the application of convolutional neural networks to irregularly sampled data. They are able to handle sparse, permutation invariant data with complex geometries. We will discuss graph networks in [Chapter 9](#). They always come with nodes, edges, and a matrix encoding the relationships in the graph, called an *adjacency matrix*. In the context of high energy physics, the particles of a jet represent the nodes of the graph, and the edges encode how close the particles are in a learned adjacency matrix. In high energy physics, graph-based networks have been successfully applied to classification, reconstruction, and generation tasks.

The subject of our chapter is generative models, or generating data similar to a given data set. Generating or simulating data faithful to the experimental data collected in high energy physics is of great importance. In “[Graph Generative Adversarial Networks for Sparse Data Generation in High Energy Physics](#)”, the authors develop graph-based generative models, using a generative adversarial network framework, for simulating sparse data sets like those produced at the [CERN](#) Large Hadron Collider.

The authors illustrate their approach by training on and generating sparse representations of MNIST handwritten digit images and jets of particles in proton-proton collisions like those at the Large Hadron Collider. The model successfully generates sparse MNIST digits and particle jet data. The authors use two metrics to quantify agreement between real and generated data: a graph-based [Fréchet inception distance](#) and the particle and jet feature-level [1-Wasserstein distance](#).

Other Generative Models

We have discussed state-of-the-art generative models (as of 2022), but this chapter will be incomplete if we do not go over Naive Bayes, Gaussian mixture, and Boltzmann machine models. There are many others. That being said, Yann LeCun (VP and chief AI scientist at Meta) offers [his perspective](#) on some of these models:

Researchers in speech recognition, computer vision, and natural language processing in the 2000s were obsessed with accurate representations of uncertainty. This led to a flurry of work on probabilistic generative models such as Hidden Markov Models

in speech, Markov random fields and constellation models in vision, and probabilistic topic models in NLP, e.g., with latent Dirichlet analysis. There were debates at computer vision workshops about generative models vs discriminative models. There were heroic-yet-futile attempts to build object recognition systems with non-parametric Bayesian methods. Much of this was riding on previous work on Bayesian networks, factor graphs and other graphical models. That's how one learned about exponential family, belief propagation, loopy belief propagation, variational inference, etc. Chinese restaurant process, Indian buffet process, etc. But almost none of this work was concerned with the problem of learning representations. Features were assumed to be given. The structure of the graphical model, with its latent variables, was assumed to be given. All one had to do was to compute some sort of log-likelihood by linearly combining features, and then use one of the above mentioned sophisticated inference methods to produce marginal distributions over the unknown variables, one of which being the answer, e.g., a category. In fact, exponential family pretty much means shallow: the log-likelihood can be expressed as a linearly parameterized function of features (or simple combinations thereof). Learning the parameters of the model was seen as just another variational inference problem. It's interesting to observe that almost none of this is relevant to today's top speech, vision, and NLP systems. As it turned out, solving the problem of learning hierarchical representations and complex functional dependencies was a much more important issue than being able to perform accurate probabilistic inference with shallow models. This is not to say that accurate probabilistic inference is not useful.

In the same vein, he continues:

Generative Adversarial Networks are nice for producing pretty pictures (though they are being edged out by diffusion models, or “multistep denoising auto-encoders” as I like to call them), but for recognition and representation learning, GANs have been a big disappointment.

Nevertheless, there is a lot of math to be learned from all these models. In my experience, we understand and retain math at a much deeper level when we see it developed and utilized for specific purposes, as opposed to only train the neurons of the brain. Many mathematicians claim to experience pleasure while proving theories that have yet to find applications. I was never one of those.

Naive Bayes Classification Model

The Naive Bayes model is a very simple classification model that we can also use as a generative model, since it ends up computing a joint probability distribution $p(\vec{x}, y_k)$ for the data to determine its classification. The training data has features $\vec{x}$ and labels y_k . Therefore, we can use the Naive Bayes model to generate new data points, together with labels, by sampling from this joint probability distribution.

The goal of a Naive Bayes model is to compute the probability of the class y_k given the data features $\vec{x}$, which is the conditional probability $p(y_k | \vec{x})$. For data with

many features (high-dimensional $\vec{x}$), this is expensive to compute, so we use Bayes' Rule and exploit the reverse conditional probability, which in turn leads to the joint probability distribution. That is:

$$p(y_k | \vec{x}) = \frac{p(y_k)p(\vec{x} | y_k)}{p(\vec{x})} = \frac{p(\vec{x}, y_k)}{p(\vec{x})}$$

The Naive Bayes model makes the very strong and naive assumption, which in practice works better than one might expect, that the data features are mutually independent when conditioned on the class label y_k . This assumption helps simplify the joint probability distribution in the numerator tremendously, especially when we expand it as a product of single variable conditional probabilities. The feature independence assumptions conditional on the class label y_k means:

$$p(x_i | x_{i+1}, x_{i+2}, \dots, x_n, y_k) = p(x_i | y_k)$$

Thus, the joint probability distribution factors into:

$$\begin{aligned} p(\vec{x}, y_k) &= p(x_1 | x_2, \dots, x_n, y_k) p(x_2 | x_3, \dots, x_n, y_k) \cdots p(x_n | y_k) p(y_k) \\ &= p(x_1 | y_k) p(x_2 | y_k) \cdots p(x_n | y_k) p(y_k) \end{aligned}$$

We can now estimate these single feature probabilities conditioned on each category of the data easily from the training data. We can similarly estimate the probability of each class $p(y_k)$ from the training data, or we can assume the classes are equally likely, so that $p(y_k) = \frac{1}{\text{number of classes}}$.

Note that in general, generative models find the joint probability distribution $p(\vec{x}, \vec{y}_k)$ between labels y_k and data $\vec{x}$. Classification models, on the other hand, calculate the conditional probabilities $p(y_k | \vec{x})$. They focus on calculating the decision boundaries between different classes in the data by returning the class y_k with the highest probability. So for the Naive Bayes classifier, it returns the label y_* with the highest value for $p(y_k | \vec{x})$, which is the same as the highest value for $p(x_1 | y_k) p(x_2 | y_k) \cdots p(x_n | y_k) p(y_k)$.

Gaussian Mixture Model

In a Gaussian mixture model, we assume that all the data points are generated from a mixture of a finite number of Gaussian distributions with unknown parameters (means and covariance matrices). We can think of mixture models as being similar to

k-means clustering, but here we include information about the centers of the clusters (means of our Gaussians) along with the shape of the spread of the data in each cluster (determined by the covariance of the Gaussians). To determine the number of clusters in the data, Gaussian mixture models sometimes implement **the Bayesian information criterion**. We can also restrict our model to control the covariance of the different Gaussians in the mixture: full, tied, diagonal, tied diagonal, and spherical (see **Figure 8-3** for an illustration).

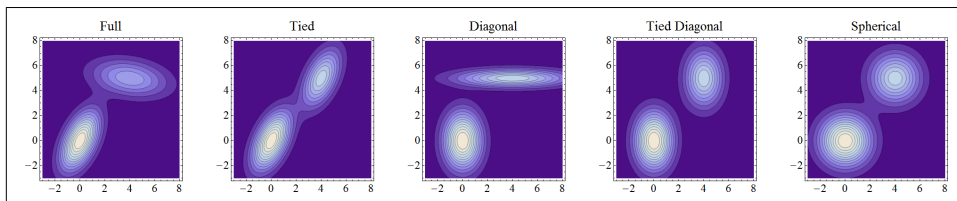


Figure 8-3. Gaussian mixture covariance types (image source)

We finally need to maximize the likelihood of the data to estimate unknown parameters of the mixture (means and entries of the covariance matrices).

Maximum likelihood becomes intractable when there are latent or hidden variables in the data (variables that are not directly measured or observed). The way around this is to use an **expectation maximization (EM) algorithm** to estimate the maximum likelihood. The expectation maximization algorithm works as follows:

1. Estimate the values for the latent variables by creating a function for the expectation of the log-likelihood using the current estimate for the unknown parameters.
2. Optimize: compute new parameters that maximize the expected log-likelihood evaluated in step 1.
3. Repeat steps 1 and 2 until convergence.

We can see how Gaussian mixture models can be used as clustering, generative, or classification models. For clustering, this is the main part of the model buildup. For generation, sample new data points from the mixture after computing the unknown parameters via expectation maximization. For classification, given a new data point, the model assigns it to the Gaussian to which it most probably belongs.

The Evolution of Generative Models

In this section, we tell the story that contributed to ending the winter of neural networks, and ultimately led to modern probabilistic deep learning models, such as variational autoencoders, fully visible deep belief networks, and generative adversarial networks. We encounter the progression from Hopfield nets to Boltzmann machines to restricted Boltzmann machines. I have a special affinity for these models: in

addition to their historical value, and learning the joint probability distribution of the data features by assembling a network of basic computational units, they employ the mathematical machinery of the extremely neat and well-developed field of *statistical mechanics*, my initial area of research.

In statistical mechanics, we define probability distributions in terms of energy functions. The probability of us finding a system in a certain state $\vec{x}$ depends on its energy $E(\vec{x})$ at that state. More precisely, high energy states are less probable, which manifests itself in the negative sign in the exponential in the following formula:

$$p(\vec{x}) = \frac{\exp(-E(\vec{x}))}{Z}$$

The exponential function guarantees that p is positive, and the *partition function* Z in the denominator ensures that the sum (or integral if x is continuous) of $p(\vec{x})$ overall states $\vec{x}$ is 1, making p a valid probability distribution. Machine learning models that define joint probability distributions this way are called *energy-based models*, for obvious reasons. They differ in how they assign the energy at each state, meaning in the specific formula they use for $E(\vec{x})$, which in turn affects the formula for the partition function Z . The formula for $E(\vec{x})$ contains the parameters of the model $\vec{\theta}$, which we need to compute from the data using maximum likelihood estimation. In fact, it is better if we have the dependence of p , E , and Z on $\vec{\theta}$ explicit in the joint probability distribution formula:

$$p(\vec{x}, \vec{\theta}) = \frac{\exp(-E(\vec{x}, \vec{\theta}))}{Z(\vec{\theta})}$$

In most cases, it is not possible to compute a closed formula for the partition function Z , rendering the maximum likelihood estimation intractable. More precisely, when we maximize the log-likelihood, we need to compute its gradient, which includes computing its gradient with respect to the parameter $\vec{\theta}$, which in turn forces us to compute the gradient of the partition function Z with respect to $\vec{\theta}$. The following quantity appears frequently in these computations:

$$\nabla_{\vec{\theta}} \log Z(\vec{\theta}) = \mathbb{E}_{\vec{x} \sim p(\vec{x})} \left(\nabla_{\vec{\theta}} \log \left(\text{numerator}(\vec{x}, \vec{\theta}) \right) \right)$$

where in our case the numerator in the formula of the energy-based joint probability distribution is $\exp\left(-E\left(\vec{x}, \vec{\theta}\right)\right)$, but this can also differ among models.

The cases where the partition function is intractable urges us to resort to approximation methods such as stochastic maximum likelihood and contrastive divergence. Other methods sidestep approximating the partition function and compute conditional probabilities without knowledge of the partition function. They take advantage of the ratio definition of conditional probabilities, along with the ratio in the definition of an energy-based joint probability distribution, effectively canceling out the partition function. These methods include score matching, ratio matching, and denoising score matching.

Other methods, such as noise contrastive estimation, annealed importance sampling, bridge sampling, or a combination of these relying on the strengths of each, approximate the partition function directly, not the log of its gradient.

We will not discuss any of these methods here. Instead we refer interested readers to *Deep Learning* by Ian Goodfellow et al. (2016).

Back to Hopfield nets and Boltzmann machines. These are the stepping stones to deep neural networks that are trained through backpropagation that the recent deterministic and probabilistic deep learning models rely on. These methods form the original *connectionist* (of neurons) approach to learning arbitrary probability distributions, initially only over binary vectors of zeros and ones, and later over vectors with arbitrary real number values.

Hopfield Nets

Hopfield nets take advantage of the elegant mathematics of statistical mechanics by identifying the states of neurons of an artificial neural network with the states of elements in a physical system. Even though Hopfield nets eventually proved to be computationally expensive and of limited practical use, they are the founding fathers of the modern era of neural networks, and are worth exploring if only to gauge the historical evolution of the AI field. Hopfield nets have no hidden units, and all their (visible) units are connected to each other. Each unit can be found in an *on* or *off* state (one or zero), and collectively they encode information about the whole network (or the system).

Boltzmann Machine

A Boltzmann machine is a Hopfield net, but with the addition of hidden units. We are already familiar with the structure of input units and hidden units in neural networks, so no need to explain them, but this is where they started. Similar to the Hopfield net, both input and hidden units are binary, where the states are either 0 or

1 (modern versions implement units that take real number values, not only binary values).

All Boltzmann machines have an intractable partition function, so we approximate the maximum likelihood gradient using the techniques surveyed at the introduction of this section.

Boltzmann machines rely only on the computationally intensive *Gibbs sampling* for their training. Gibbs is a name that appears repetitively in the statistical mechanics field. Gibbs sampling provides unbiased estimates of the weights of the network, but these estimates have high variance. In general, there is a trade-off between bias and variance, and this trade-off highlights the advantages and disadvantages of methods relying on each.

Restricted Boltzmann Machine (Explicit Density and Intractable)

Boltzmann machines have a very slow learning rate due to the many inter-connections within visible layers and within hidden layers (think a very messy back-propagation). This makes their training very slow and prohibits their application to practical problems. Restricted Boltzmann machines, which restrict connections only to those between different layers, solve this problem. That is, there are no connections within each layer of a restricted Boltzmann machine, allowing all of the units in each layer to be updated simultaneously. Therefore, for two connected layers, we can collect co-occurrence statistics by alternately updating all of the units in each layer. In practice, there are larger savings because of minimal sampling procedures, such as contrastive divergence.

Conditional independence

The lack of connections within each layer means that the states of all units in the hidden layer do not depend on each other, but they do depend on the states of units in the previous layer. In other words, given the states of the previous layer's units, the state of each hidden unit is independent of the states of the other units in the hidden layer. This conditional independence allows us to factorize the joint probability of the state of a hidden layer $p(\vec{h} \mid \vec{h}_{previous})$ as the product of the conditional probabilities of the states of individual hidden units. For example, if we have three units in a hidden layer, $p(\vec{h} \mid \vec{h}_{previous}) = p(h_1 \mid \vec{h}_{previous})p(h_2 \mid \vec{h}_{previous})p(h_3 \mid \vec{h}_{previous})$. The other way is also true, the states of the units of a previous layer are conditionally independent of each other given the states of the current layer. This conditional independence means that we can sample unit states instead of iteratively updating them for long periods of time.

Universal approximation

The restricted connections in restricted Boltzmann machines allow for their stacking, that is, having a series of multiple hidden layers that are able to extract more complex features. We can see now how the architecture of the modern multilayer artificial neural network slowly emerged. Recall that in [Chapter 4](#), we discussed the universal approximation of neural networks for a wide range of deterministic functions. In this chapter, we would like our networks to represent (or learn) joint probability distributions instead of deterministic function. In 2008, Le Roux and Bengio proved that Boltzmann machines can approximate any discrete probability distribution to an arbitrary accuracy. This result also applies to restricted Boltzmann machines. Moreover, under certain mild conditions, each additional hidden layer increases the value of the log-likelihood function, thus allowing the model distribution to be closer to the true joint probability distribution of the training set.

In 2015, Eldan and Shamir verified empirically that increasing the number of layers of the neural networks is exponentially more valuable than increasing the width of network layers by the number of units in each layer (depth versus width). We also know from practice (without proofs) that it is possible to train a network with hundreds of hidden layers, where deeper layers represent higher-order features. Historically, the problem of vanishing gradients had to be overcome in order to train deep networks.

The Original Autoencoder

The autoencoder architecture aims to compress the information of the input into its lower-dimensional hidden layers. The hidden layers should retain the same amount of information as the input layers, even when they have fewer units than the input layers. We have discussed modern variational autoencoders, which provide an efficient method for training autoencoder networks. During training, each vector should be mapped to itself (unsupervised), and the network tries to learn the best *encoding*. The input and the output layers must then have the same number of units. A Boltzmann machine set up with a certain number of input units, fewer number of hidden units, and an output layer with the same number of units as the input layer describes an original network autoencoder architecture. From a historical perspective this is significant: the autoencoder is one of the first examples of a network successfully learning a code, implicit in the states of the hidden units, to represent its inputs. This makes it possible to force a network to compress its input into a hidden layer with minimal loss of information. This is now an integral part of neural networks that we take for granted. The autoencoder architecture, with and without Boltzmann machines (with and without energy-based joint probability distribution), is still very influential in the deep learning world.

Earlier in this chapter, we discussed variational autoencoders. From a historical point of view, these synthesize the ideas of Boltzmann machine autoencoders, deep autoencoder networks, *denoising autoencoders*, and the information bottleneck (Tishby et al. 2000), which have their roots in the idea of analysis by synthesis (Selfridge 1958). Variational autoencoders use fast variational methods for their learning. In the context of bias-variance trade-off, variational methods provide biased estimates for the network's weights that have low variance.

Probabilistic Language Modeling

A natural connection between this chapter and [Chapter 7](#), which focused almost exclusively on natural language processing and the various ways to extract meaning from natural language data, is to survey the fundamentals behind probabilistic language models, then highlight the models from [Chapter 7](#) that adhere to these fundamentals.

This chapter started with maximum likelihood estimation. One of the reasons this appears everywhere when we need to estimate probability distributions is that the probability distribution attained via maximum likelihood estimation is supported by mathematical theory, under a couple conditions: maximum likelihood estimation does converge to the true distribution $p_{data}(\vec{x})$ that generated the data, in the limit as the number of data samples goes to infinity (that is, assuming we have a ton of data), and provided that the model probability distribution $p_{model}(\vec{x}, \vec{\theta})$ already includes the true probability distribution. That is, in the limit as the number of samples goes to infinity, the model parameters $\vec{\theta}^*$ that will maximize the likelihood of the data satisfy $p_{model}(\vec{x}, \vec{\theta}^*) = p_{data}(\vec{x})$.

In language models, the training data is samples of text from some corpus and/or genre, and we would like to learn its probability distribution so that we can generate similar text. It is important to keep in mind that the true data distribution is most likely *not* included in the family of distributions provided by $p_{model}(\vec{x}, \vec{\theta})$, so the theoretical result in the previous paragraph might never hold in practice; however, this doesn't deter us and we usually settle for models that are useful enough for our purposes. Our goal is to build a model that assigns probabilities to pieces of language. If we randomly assemble some pieces of language, we most likely end up with gibberish. What we actually want is to find the distribution of those sentences that mean something. A good language model is one that assigns high probabilities to sentences that are meaningful, even when these sentences are not among the training data. People usually compute the *perplexity* of a language model on the training data set to evaluate its performance.

Language models are based on the assumption that the probability distribution of the next word depends on the $n-1$ words that preceded it, for some fixed n , so we care about calculating $p_{model}(x_n | x_1, x_2, \dots, x_{n-1})$. If we are using a word2vec model that embeds the meaning of each word in vector, then each of these x 's is represented by a vector. Words that mean things or are frequently used in similar contexts tend to have similar vector values. We can use the transformer model from [Chapter 7](#) to predict the next word vector based on the preceding word vectors.

Frequency-based language models construct conditional probability tables by counting the number of times words appear together in the training corpus. For example, we can estimate the conditional probability $p(\text{morning}|\text{good})$ of the word *morning* appearing after the word *good*, by counting the number of times *good morning* appears in the corpus divided by the number of times *good* appears in the corpus. That is:

$$p(\text{morning}|\text{good}) = \frac{p(\text{good}, \text{morning})}{p(\text{good})}$$

This breaks down for very large corpuses or for unstructured text data such as tweets, Facebook comments, or SMS messages where the usual rules of grammar/spelling, etc. are not totally adhered to.

We can formalize the notion of a probabilistic language model this way:

1. Specify the vocabulary V of your language. This could be a set of characters, spaces, punctuations, symbols, unique words, and/or n-grams. Mathematically, it is a finite discrete set that includes a stopping symbol signifying the end of a thought or sentence, like a period in English (even though a period does not always mean the end of a sentence in English, such as when it is used for abbreviations).
2. Define a sentence (which could be meaningful or not) as a finite sequence of symbols $\vec{x} = (x_1, x_2, \dots, x_m)$ from the vocabulary V ending in the stop symbol. Each x_i can assume any value from the vocabulary V . We can specify m as a maximum length for our sentences.
3. Define our language space $l^m = \{(x_1, x_2, \dots, x_m), x_i \in V\}$ as the set of all sentences of length less than or equal to m . The overwhelming majority of these sentences will mean nothing, and we need to define a language model that only captures the sentences that mean something: high probabilities for meaningful sentences and low probabilities for nonmeaningful ones.
4. Let $\mathcal{L}$ be the collection of all subsets of l^m . This accounts for collections of all meaningful and meaningless sentences of maximal length m .

5. In rigorous probability theory, we usually start with probability triples: a space, a *sigma algebra* containing some subsets of that space, and a probability measure assigned to each member of the chosen sigma algebra (do not worry about these details in this chapter). A language model, in this context, is the probability triple: the language space $\mathcal{L}$, the sigma algebra made up of all the subsets of the language space $\mathcal{L}$, and a probability measure P that we need to assign to each member of $\mathcal{L}$. Since our language space is discrete and finite, it is easier to assign a probability p to each member of $\mathcal{L}$ instead, that is, a probability to each sentence $\vec{x} = (x_1, x_2, \dots, x_m)$ (since this will in turn induce a probability measure P on the collection of all subsets $\mathcal{L}$, so we will never worry about this for language models). It is this p that we need to learn from the training data. The usual approach is to select a full family of probability distributions $p(\vec{x}; \vec{\theta})$ parameterized by $\vec{\theta}$.
6. Finally, we need to estimate the parameter $\vec{\theta}$ by maximizing the likelihood of the training data set that contains many sentence samples from $\mathcal{L}$. Since the probabilities of meaningful sentences are very small numbers, we use the logarithm of these probabilities instead to avoid the risk of underflow.

For consistency, it is a nice exercise to check log-linear models and log-bilinear models (GloVe) and latent Dirichlet allocation from [Chapter 7](#) in the context of this section.

Summary and Looking Ahead

This was another foundational chapter in our journey to pinpoint the mathematics that is required for the state-of-the-art AI models. We shifted from learning deterministic functions in earlier chapters to learning joint probability distributions of data features. The goal is to use those to generate new data similar to the training data. We learned, still without formalizing, a lot of properties and rules for probability distributions. We surveyed the most relevant models, along with some historical evolution that led us here. We made the distinction between models that provide explicit formulas for their joint distributions and models that interact indirectly with the underlying distribution, without explicitly writing down formulas. For models with explicit formulas, computing the log-likelihood and its gradients can be tractable or intractable, each of which requires its own methods. The goal is always the same: capture the underlying true joint probability distribution of the data by finding a model that maximizes its log-likelihood.

None of this would have been necessary if our data was low dimensional, with one or two features. Histograms and kernel density estimators do a good job of estimating

probability distributions for low-dimensional data. One of the best accomplishments in machine learning is the ability to model high-dimensional joint probability distributions from a big volume of data.

All of the approaches that we presented in this chapter have their pros and cons. For example, variational autoencoders allow us to perform both learning and efficient Bayesian inference in probabilistic graphical models with hidden (latent) variables. However, they generate lower-quality samples. Generative adversarial networks generate better samples, but they are more difficult to optimize due to their unstable training dynamics. They search for an unstable saddle point instead of a stable maximum or minimum. Deep belief networks such as PixelCNN and WaveNet have a stable training process, optimizing the softmax loss function. However, they are inefficient during sampling and don't organically embed data into lower dimensions, as autoencoders do.

Two-player zero-sum games from game theory appeared naturally in this chapter due to the setup of generative adversarial networks.

Looking ahead into the next chapter on *graphical modeling*, we note that the connections in the graph of a neural network dictate the way we can write conditional probabilities, easily pointing out the various dependencies and conditional independences. We saw this while discussing restricted Boltzmann machines in this chapter. In the next chapter, we focus exclusively on graphical modeling, which we have managed to avoid for a good three-quarters of the book.

Graph Models

Now this is something we all want to learn.

—H.

Graphs, diagrams, and networks are all around us: cities and roadmaps, airports and connecting flights, electrical networks, the power grid, the World Wide Web, molecular networks, biological networks such as our nervous system, social networks, terrorist organization networks, schematic representations of mathematical models, artificial neural networks, and many, many others. They are easily recognizable, with distinct nodes representing some entities that we care for, which are then connected by directed or undirected edges indicating the presence of some relationship between the connected nodes.

Data that has a natural graph structure is better understood by a mechanism that exploits and preserves that structure, building functions that operate directly on graphs (however they are mathematically represented), as opposed to feeding graph data into machine learning models that artificially reshape it before analyzing it. This inevitably leads to loss of valuable information. This is the same reason convolutional neural networks are successful with image data, recurrent neural networks are successful with sequential data, and so on.

Graph-based models are very attractive for data scientists and engineers. Graph structures offer a flexibility that is not afforded in spaces with a fixed underlying coordinate system, such as in Euclidean spaces or in relational databases, where the data along with its features is forced to adhere to a rigid and predetermined form. Moreover, graphs are the natural setting that allows us to investigate the relationships between the points in a data set. So far, our machine learning models consumed data represented as isolated data points. Graph models, on the other hand, consume isolated data points, *along with the connections between them*, allowing for deeper understanding and more expressive models.

The human brain naturally internalizes graphical structures: it is able to model entities and their connections. It is also flexible enough to generate new networks, or expand and enhance existing ones, for example, when city planning, project planning, or when continuously updating transit networks. Moreover, humans can transition from natural language text to graph models and vice versa seamlessly. When we read something new, we find it natural to formulate a graphical representation to better comprehend it or illustrate it to other people. Conversely, when we see graph schematics, we are able to describe them via natural language. There are currently models that generate natural language text based on knowledge graphs and vice versa. This is called reasoning over knowledge graphs.

At this point we are pretty comfortable with the building blocks of neural networks, along with the types of data and tasks they are usually suited for:

- Multilayer perceptron or fully connected neural network ([Chapter 4](#))
- Convolutional layers ([Chapter 5](#))
- Recurrent layers ([Chapter 7](#))
- Encoder-decoder components ([Chapter 7](#))
- Adversarial components and two-player zero-sum games ([Chapter 8](#))
- Variational components ([Chapter 8](#))

The main tasks are mostly classification, regression, clustering, coding and decoding, or new data generation, where the model learns the joint probability distribution of the data features.

We are also familiar with the fact that we can mix and match some of the components of neural networks to construct new models that are geared toward specific tasks. The good news is that graph neural networks use the same exact ingredients, so we do not need to go over any new machine learning concepts in this chapter. Once we understand how to mathematically represent graph data along with its features in a way that can be fed into a neural network, either for analysis or for new network (graph) data generation, we are good to go. We will therefore avoid going down a maze of surveys for all the graph neural networks out there. Instead, we will focus on the simple mathematical formulation, popular applications, common tasks for graph models, available data sets, and model evaluation methods. Our goal is to develop a strong intuition for the workings of the subject. The main challenge is, yet again, lowering the dimensionality of the problem in a way that makes it amenable to computation and analysis, while preserving the most amount of information. In other words, for a network with millions of users, we cannot expect our models to take as input vectors or matrices with millions of dimensions. We need efficient representation methods for graph data.

If you want to dive deeper and fast track into graph neural networks, the survey paper “A Comprehensive Survey on Graph Neural Networks” (Wu et al. 2019) is an excellent place to start (of course, only after carefully reading this chapter).

Graphs: Nodes, Edges, and Features for Each

Graphs are naturally well suited to model any problem where the goal is to understand a discrete collection of objects (with emphasis on discrete and not continuous) through the relationships among them. Graph theory is a relatively young discipline in discrete mathematics and computer science with virtually unlimited applications. This field is in need of more brains to tackle its many unsolved problems.

A graph (see Figure 9-1) is made up of:

Nodes or vertices

Bundled together in a set as $Nodes = \{node_1, node_2, \dots, node_n\}$. This can be as little as a handful of nodes (or even one node), or as massive as billions of nodes.

Edges

Connecting any two nodes (this can include an edge from a node to itself, or multiple edges connecting the same two nodes) in a directed (pointing from one node to the other) or undirected way (the edge has no direction from either node to the other). The set of edges is $Edges = \{edge_{ij} = (node_i, node_j)\}$, such that there is an edge pointing from $node_i$ to $node_j$.

Node features

We can assign to each $node_i$ a list of, say, d features (such as the age, gender, and income level of a social media user) bundled together in a vector $\overrightarrow{features_{node_i}}$. We can then bundle all the feature vectors of all the n nodes of the graph in a matrix $Features_{Nodes}$ of size $d \times n$.

Edge features

Similarly, we can assign to each $edge_{ij}$ a list of, say, c features (such as the length of a road, its speed limit, and whether it is a toll road or not) bundled together in a vector $\overrightarrow{features_{edge_{ij}}}$. We can then bundle all the feature vectors of all the m edges of the graph in a matrix $Features_{Edges}$ of size $c \times m$.

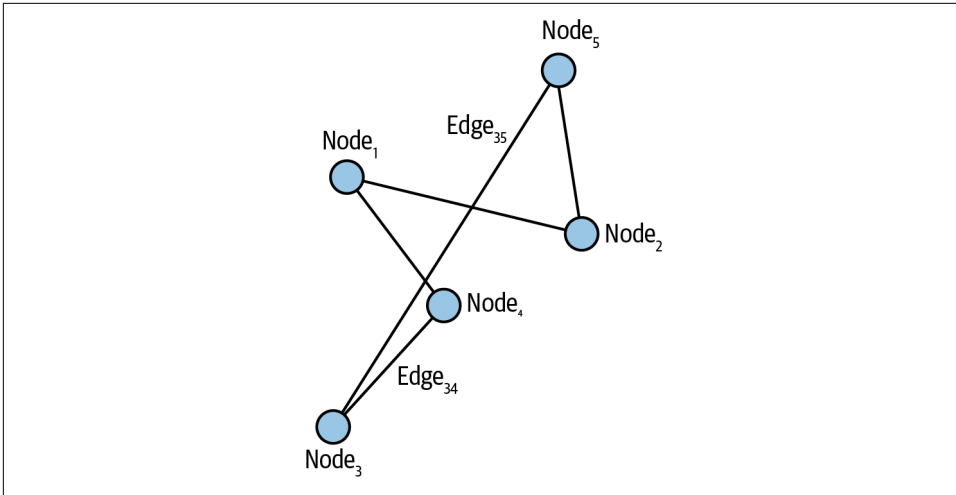


Figure 9-1. A graph is made up of nodes and directed or undirected edges connecting the nodes

Graph models are powerful because they are flexible and not necessarily forced to adhere to a rigid grid-like structure. We can think of their nodes as *floating through space* with no coordinates whatsoever. They are only held together by the edges that connect them. However, we need a way to represent their intrinsic structure. There are software packages that visualize graphs given their sets of nodes and edges, but we cannot do analysis and computations on these pretty (and informative) pictures. There are two popular graph representations that we can use as inputs to machine learning models: a graph's *adjacency matrix* and its *incidence matrix*.

There are other representations that are useful for graph theoretic algorithms, such as *edge listing*, *two linear arrays*, and *successor listing*. All of these representations convey the same information but differ in their storage requirements and the efficiency of graph retrieval, search, and manipulation. Most graph neural networks take as input the adjacency matrix along with the feature matrices for the nodes and the edges. Many times, they must do a dimension reduction (called graph representation or graph embedding) before feeding the graph data into a model. Other times, the dimension reduction step is part of the model itself.

Adjacency matrix

One algebraic way to store the structure of a graph on a machine and study its properties is through an adjacency matrix, which is an $n \times n$ whose entries $adjacency_{ij} = 1$ if there is an edge from $node_i$ to $node_j$, and $adjacency_{ij} = 0$ if there is no edge from $node_i$ to $node_j$. Note that this definition is able to accommodate a self edge, which is an edge from a vertex to itself, but not multiple edges between two distinct nodes, unless we decide to include the numbers 2, 3, etc.

as entries in the adjacency matrix. This, however, can mess up some results that graph theorists have established using the adjacency matrix.

Incidence matrix

This is another algebraic way to store the structure of the graph and retain its full information. Here, we list both the nodes and the edges, then formulate a matrix whose rows correspond to the vertices and whose columns correspond to the edges. An entry $incidence_{ij}$ of the matrix is 1 if $edge_j$ connects $node_i$ to some other node, and zero otherwise. Note that this definition is able to accommodate multiple edges between two distinct nodes, but not a self edge from a node to itself. Since many graphs have more edges than vertices, this matrix tends to be very wide and larger in size than the adjacency matrix.

The *Laplacian matrix* is another matrix that is associated with an undirected graph. It is an $n \times n$ symmetric matrix where each node has a corresponding row and column. The diagonal entries of the Laplacian matrix are equal to the degree of each node, and the off-diagonal entries are zero if there is no edge between nodes corresponding to that entry, and -1 if there is an edge between them. This is the discrete analog of the continuous Laplace operator from calculus and partial differential equations where the discretization happens at the nodes of the graph.

The Laplacian matrix takes into account the second derivatives of a continuous (and twice differentiable) function, which measure the concavity of a function, or how much its value at a point differs from its value at the surrounding points. Similar to the continuous Laplacian operator, the Laplacian matrix provides a measure of the extent a graph differs at one node from its values at nearby nodes. The Laplacian matrix of a graph appears when we investigate random walks on graphs and when we study electrical networks and resistances. We will see these later in this chapter.

We can easily infer simple node and edge statistics from the adjacency and incidence matrices, such as the degrees of nodes (the degree of a node is the number of edges connected to this node). The degree distribution $P(k)$ reflects the variability in the degrees of all the nodes. $P(k)$ is the empirical probability that a node has exactly k edges. This is of interest for many networks, such as web connectivity and biological networks.

For example, if the distribution of nodes of degree k in a graph follows a power law of the form $P(k) = k^{-\alpha}$, then such graphs have few nodes of high connectivity, or hubs, which are central to the network topology, holding it together, along with many nodes with low connectivity, which connect to the hubs.

We can also add time dependency, and think of dynamic graphs whose properties change as time evolves. Currently, there are models that add time dependency to the node and/or edge feature vectors (so each entry of these vectors becomes time dependent). For example, for a GPS system that predicts travel routes, the edge

features connecting one point on the map to another change with time depending on the traffic situation.

Now that we have a mathematical framework for graph objects, along with their node and edge features, we can feed these representative vectors and matrices (and labels for supervised models) into machine learning models and do business as usual. Most of the time, half of the story is having a good representation for the objects at hand. The other half of the story is the expressive power of machine learning models in general, where we can get good results without encoding (or even having to learn) the rules that lead to these results. For the purposes of this chapter, this means that we can jump straight into graph neural networks *before* learning proper graph theory.



Directed Graphs

For directed graphs, on the one hand, we are interested in the same properties as undirected graphs, such as their spanning trees, fundamental circuits, cut sets, planarity, thickness, and others. On the other hand, directed graphs have their own unique properties that are different than undirected graphs, such as strong connectedness, arborescence (a directed form of rooted tree), decyclization, and others.

Example: PageRank Algorithm

PageRank is a retired algorithm (expired in 2019) that Google used to rank web pages in its search engine results. It provides a measure for the importance of a web page based on how many other pages link to it. In graph language, the nodes are the web pages, and the directed edges are the links pointing from one page to another. According to PageRank, node is important when it has many other web pages pointing to it, that is, when its incoming degree is large (see **Figure 9-2**).

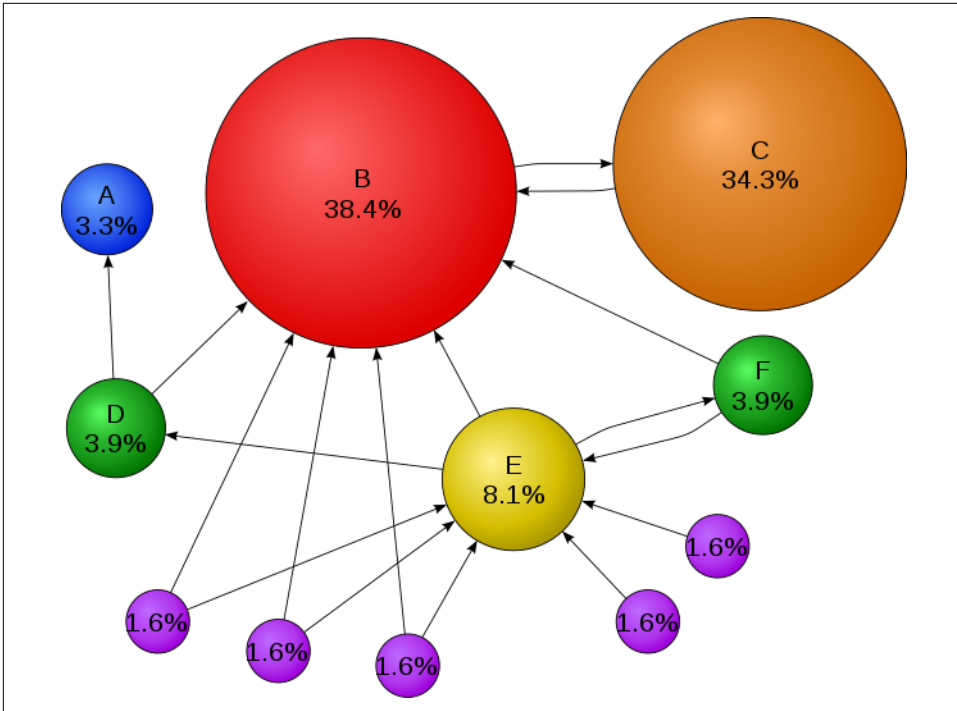


Figure 9-2. PageRank gives a higher score for pages with more pages pointing (or linking) to them (*image source*)

As a concrete example involving graphs, adjacency matrix, linear algebra, and the web, let's walk through the PageRank algorithm for an absurdly simplified World Wide Web consisting of only four indexed web pages, such as in Figure 9-3, as opposed to billions.

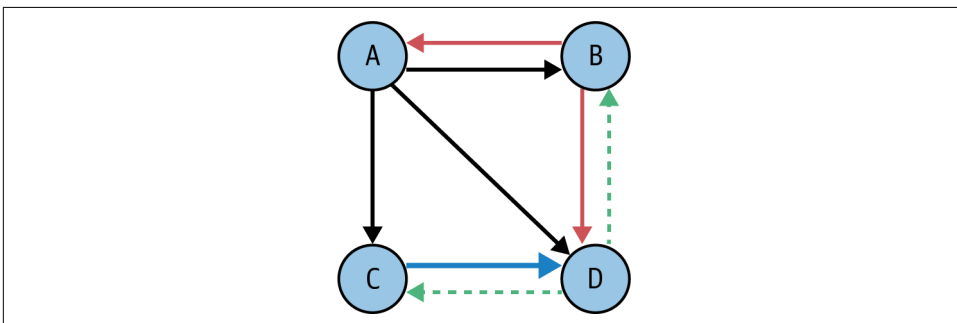


Figure 9-3. A fictitious World Wide Web consisting of only four indexed web pages (adapted from *Coursera: Mathematics for Machine Learning*).

In the graph of [Figure 9-3](#), only B links to A; A and D link to B; A and D link to C; A, B, and C link to D; A links to B, C, and D; B links to A and D; C links to D; and D links to B and C.

Let's think of a web surfer who starts at some page then randomly clicks on a link from that page, then a link from this new page, and so on. This surfer simulates a *random walk on the graph of the web*.

In general, on the graph representing the World Wide Web, such a random surfer traverses the graph from a certain node to one of its neighbors (or back to itself if there are links pointing back to the page). We will encounter the World Wide Web one more time in this chapter and explore the kind of questions that we like to understand about the nature of its graph. We need a matrix for the random walk, which for this application we call the *linking matrix*, but in reality it is the adjacency matrix weighted by the degree of each vertex. We use this random walk matrix, or linking matrix, to understand the long-term behavior of the random walk on the graph. Random walks on graphs will appear throughout this chapter.

Back to the four-page World Wide Web of [Figure 9-3](#). If the web surfer is at page A, there is a one-third chance the surfer will move to page B, one-third chance to move to C, and one-third chance to move to D. Thus, the *outward linking* vector of page A is:

$$\overrightarrow{\text{linking}_A} = \begin{pmatrix} 0 \\ 1/3 \\ 1/3 \\ 1/3 \end{pmatrix}$$

If the web surfer is at page B, there is a one-half chance they will move to page A and a one-half chance they will move to page D. Thus, the outward linking vector of page B is:

$$\overrightarrow{\text{linking}_B} = \begin{pmatrix} 1/2 \\ 0 \\ 0 \\ 1/2 \end{pmatrix}$$

Similarly, the outward linking vectors of pages C and D are:

$$\overrightarrow{\text{linking}_C} = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 1 \end{pmatrix} \text{ and } \overrightarrow{\text{linking}_D} = \begin{pmatrix} 0 \\ 1/2 \\ 1/2 \\ 0 \end{pmatrix}$$

We bundle the linking vectors of all the web pages together to create a linking matrix:

$$\text{Linking} = \begin{pmatrix} 0 & 1/2 & 0 & 0 \\ 1/3 & 0 & 0 & 1/2 \\ 1/3 & 0 & 0 & 1/2 \\ 1/3 & 1/2 & 1 & 0 \end{pmatrix}$$

Note that the columns of the linking matrix are the outward linking probabilities, and the rows of A are the *inward linking* probabilities. How can a surfer end up at page A? They can only be at B, and from there there's only a 0.5 probability they will end up at A.

Now we can *rank* page A by adding up the ranks of all the pages pointing to A, each weighted by the probability a surfer will end up at page A from that page; that is, a page with many highly ranked pages pointing to it will also rank high. The ranks of all four pages are therefore:

$$\begin{aligned} \text{rank}_A &= 0\text{rank}_A + 1/2\text{rank}_B + 0\text{rank}_C + 0\text{rank}_D \\ \text{rank}_B &= 1/3\text{rank}_A + 0\text{rank}_B + 0\text{rank}_C + 1/2\text{rank}_D \\ \text{rank}_C &= 1/3\text{rank}_A + 0\text{rank}_B + 0\text{rank}_C + 1/2\text{rank}_D \\ \text{rank}_D &= 1/3\text{rank}_A + 1/2\text{rank}_B + 1\text{rank}_C + 0\text{rank}_D \end{aligned}$$

To find the numerical value for the rank of each web page, we have to solve that system of linear equations, which is the territory of linear algebra. In matrix vector notation, we write the system as:

$$\overrightarrow{\text{ranks}} = \text{Linking} \overrightarrow{\text{ranks}}$$

Therefore, the vector containing all the ranks of all the web pages is an eigenvector of the linking matrix of the graph of the web pages (where the nodes are the web pages, and the directed edges are the links between them) with eigenvalue 1. Recall that in reality, the graph of the web is enormous, which means that the linking matrix is enormous, and devising efficient ways to find its eigenvectors becomes of immediate interest.

Computing eigenvectors and eigenvalues of a given matrix is one of the most important contributions of numerical linear algebra, with immediate applications in many fields. A lot of the numerical methods for finding eigenvectors and eigenvalues involve repeatedly multiplying a matrix with a vector. When dealing with huge matrices, this is expensive, and we have to use every trick in the book to make the operations cheaper. We take advantage of the sparsity of the matrix (many entries are zeros, so it is a waste to multiply with these entries and *then* discover that they are just zeros); we introduce randomization or stochasticity, and venture into the fields of high-dimensional probability and large random matrices (we will get a flavor of these in [Chapter 11](#) on probability). For now, we reemphasize the iterative method we introduced in [Chapter 6](#) on singular value decompositions: we start with a random vector $\overrightarrow{ranks_0}$, then produce a sequence of vectors iteratively by multiplying by the linking matrix:

$$\overrightarrow{ranks_{i+1}} = \text{Linking } \overrightarrow{ranks_i}$$

For our four-page World Wide Web, this converges to the vector:

$$\overrightarrow{ranks} = \begin{pmatrix} 0.12 \\ 0.24 \\ 0.24 \\ 0.4 \end{pmatrix}$$

which means that page D is ranked highest, and in a search engine query with similar content it will be the first page returned. We can then redraw the diagram in [Figure 9-3](#) with the size of each circle corresponding to the importance of the page.

When the PageRank algorithm was in use, the real implementation included a damping factor d , a number between 0 and 1, usually around 0.85, which takes into account only an 85% chance that the web surfer clicks on a link from the page they are currently at, and a 15% chance that they start at a completely new page that has no links from the page they are currently at. This modifies the iterative process to find the rankings of the pages of the web in a straightforward way:

$$\overrightarrow{ranks_{i+1}} = d \left(\text{Linking } \overrightarrow{ranks_i} \right) + \frac{1-d}{\text{total number of pages}} \overrightarrow{ones}$$

Finally, you might be wondering whether Google keeps searching the web for new web pages and indexing them, and does it keep checking all indexed web pages for new links? The answer is yes, and the following excerpts are from [Google's "In-Depth Guide to How Google Search Works"](#):

Google Search is a fully-automated search engine that uses software known as web crawlers that explore the web regularly to find pages to add to our index. In fact, the vast majority of pages listed in our results aren't manually submitted for inclusion, but are found and added automatically when our web crawlers explore the web. [...] There isn't a central registry of all web pages, so Google must constantly look for new and updated pages and add them to its list of known pages. This process is called "URL discovery". Some pages are known because Google has already visited them. Other pages are discovered when Google follows a link from a known page to a new page: for example, a hub page, such as a category page, links to a new blog post. Still other pages are discovered when you submit a list of pages (a sitemap) for Google to crawl. [...] When a user enters a query, our machines search the index for matching pages and return the results we believe are the highest quality and most relevant to the user. Relevancy is determined by hundreds of factors, which could include information such as the user's location, language, and device (desktop or phone). For example, searching for "bicycle repair shops" would show different results to a user in Paris than it would to a user in Hong Kong.

The more data we collect, the more complex searching it becomes. Google rolled out RankBrain in 2015. It uses machine learning to vectorize the text on the web pages, similar to what we did in [Chapter 7](#). This process adds context and meaning to the indexed pages, so that the search returns more accurate results. The bad thing that this process adds is the much higher dimensions associated with meaning vectors. To circumvent the difficulty of checking every vector at every dimension before returning the web pages closest to the query, Google uses an approximate nearest neighbor algorithm, which helps return excellent results in milliseconds—the experience we have now.

Inverting Matrices Using Graphs

Many problems in the applied sciences involve writing a discrete linear system $A\vec{x} = \vec{b}$ and solving it, which is equivalent to inverting the matrix A and finding the solution $\vec{x} = A^{-1}\vec{b}$. But for large matrices, this is a computationally expensive operation, along with having high storage requirements and poor accuracy. We are always looking for efficient ways to invert matrices, sometimes leveraging the special characteristics of the particular matrices at hand.

The following is a graph theoretic method that computes the inverse of a matrix of a decent size (for example, a hundred rows and a hundred columns):

1. Replace each nonzero entry in matrix A with a 1. We obtain a binary matrix.
2. Permute the rows and the corresponding columns of the resulting binary matrix to make all diagonal entries 1's.

3. We think of the matrix obtained as the adjacency matrix of a directed graph (where we delete the self-loops corresponding to 1's along the diagonal from the graph).
4. The resulting directed graph is partitioned into its fragments.
5. If a fragment is too large, we tear it into smaller fragments by removing an appropriate edge.
6. We invert the smaller matrices.
7. Apparently this leads to the inverse of the original matrix.

We will not explain why and how, but this method is so cute, so it made its way into this chapter.

Cayley Graphs of Groups: Pure Algebra and Parallel Computing

Graphs of groups, also called Cayley graphs or Cayley diagrams, can be helpful in designing and analyzing network architectures for parallel computers, routing problems, and routing algorithms for interconnected networks. The paper “[Processor Interconnection Networks from Cayley Graphs](#)” (Schibell et al. 2011) is an interesting and easy read on earlier designs applying Cayley graphs for parallel computing networks, and explains how to construct Cayley graphs that meet specific design parameters. Cayley graphs have also been applied for [classification of data](#).

We can represent every group with n elements as a connected directed graph of n nodes, where each node corresponds to an element from the group, and each edge represents a multiplication by a generator from the group. The edges are labeled (or colored) depending on which generator from the group we are multiplying by (see [Figure 9-4](#)). This directed graph uniquely defines the group: each product of elements in the group corresponds to following a sequence of directed edges on the graph. For example, the graph of a cyclic group of n elements is a directed circuit of n nodes in which every edge represents multiplication by one generator of the group.

From a pure math perspective, Cayley graphs are useful for visualizing and studying abstract groups, encoding their full abstract structure and all of their elements in a visual diagram. The symmetry of Cayley graphs makes them useful for constructing more involved abstract objects. These are central tools for combinatorial and geometric group theory. For more on Cayley graphs, check out this [Wolfram Mathworld page](#).

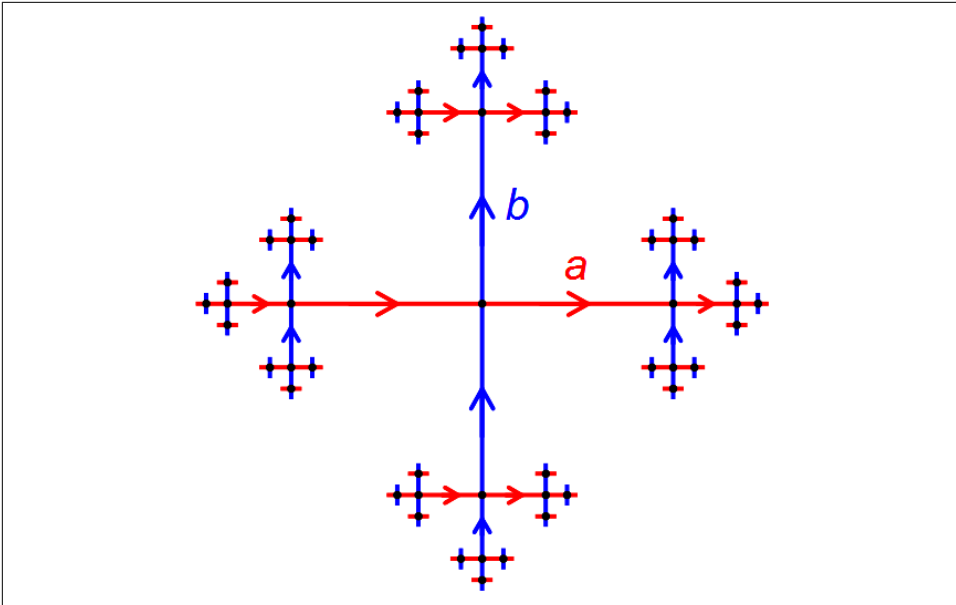


Figure 9-4. Cayley graph for the free group on two generators a and b ; each node represents an element of the free group, and each edge represents multiplication by a or b (image source)

Message Passing Within a Graph

The *message passing framework* is a useful approach for modeling the propagation of information within graphs, as well as neatly aggregating the information conveyed in the nodes, edges, and the structure of the graph itself into vectors of a certain desired dimension. Within this framework, we update every node with information from the feature vectors of its neighboring nodes and the edges connected to them. A graph neural network performs multiple rounds of message passing; each round propagates a single node's information further. Finally, we combine the latent features of each individual node to obtain its unified vector representation and represent the whole graph.

More concretely, for a specific node, we choose a function that takes as input the node's feature vector, the feature vector of one of its neighboring nodes (those connected to it by an edge), and the feature vector of the edge that connects it to this neighboring node, and outputs a new vector that contains within it information from the node, the neighbor, and their connecting edge. We apply the same function to all of the node's neighbors, then add the resulting vectors together, producing a *message vector*. Finally, we update the feature vector of our node by combining its original feature vector with the message vector within an update function that we also choose. When we do this for each node in the graph, each node's new feature vector will

contain information from itself, all its neighbors, and all its connecting edges. Now, when we repeat this process one more time, the node's most recent feature vector will contain information from itself, all its neighbors *and its neighbor's neighbors*, and all the corresponding connecting edges. Thus, the more message passing rounds we do, the more each node's feature vector contains information from farther nodes within the graph, moving one edge separation at a time. The information diffuses successively across the entire network.

The Limitless Applications of Graphs

Applications for graph neural networks, and graph models in general, are ubiquitous and so important that I am a bit regretful I did not start the book with graphs. In any graph model, we start by answering the following questions:

- What are the nodes?
- What is the relationship that links two nodes, that establishes directed or undirected edge(s) between them?
- Should the model include feature vectors for the nodes and/or the edges?
- Is our model dynamic, where the nodes, edges, and their features evolve with time, or is it static in time?
- What are we interested in? Classifying (for example cancerous or noncancerous; fake news spreader or real news spreader)? Generating new graphs (for example for drug discovery)? Clustering? Embedding the graph into a lower-dimensional and structured space?
- What kind of data is available or needed, and is the data organized and/or labeled? Does it need preprocessing?

We survey few applications in this section, but there are many more that genuinely lend themselves to a graph modeling structure. It is good to read the abstracts of the linked publications since they help capture common themes and ways of thinking about these models. The following list gives a good idea of the common tasks for graph neural networks, which include:

- Node classification
- Graph classification
- Clustering and community detection
- New graph generation
- Influence maximization
- Link prediction



Image Data as Graphs

We might encounter graph neural networks tested on the **MNIST data set for handwritten digits**, which is one of the benchmark sets for computer vision. If you wonder how image data (stored as three-dimensional tensors of pixel intensities across each channel) manages to fit into a graph structure, here's how it works. Each pixel is a node, and its features are the respective intensities of its three channels (if it is a color image, otherwise it only has one feature). The edges connect each pixel to the three, five, or eight pixels surrounding it, depending on whether the pixel is located at a corner, edge, or in the middle of the image.

Brain Networks

One of the main pursuits in neuroscience is understanding the network organization of the brain. Graph models provide a natural framework and many tools for analyzing the complex networks of the brain, both in terms of their anatomy and their functionality.

To create artificial intelligence on par with human intelligence, we must understand the human brain on many levels. One aspect is the brain's network connectivity, how connectivity affects the brain's functionality, and how to replicate that, building up from small computational units to modular components to a fully independent and functional system.

Human brain anatomical networks demonstrate short path length (conservation of wiring costs) along with high-degree cortical *hubs*, that is, high clustering. This is on both the cellular scale and on the whole brain scale. In other words, the brain network seems to have organized itself in a way that maximizes the efficiency of information transfer and minimizes connection cost. The network also demonstrates modular and hierarchical topological structures and functionalities. The topological structures of the brain networks and their functionalities are interdependent over both short and long time scales. The dynamic properties of the networks are affected by their structural connectivity, and over a longer time scale, the dynamics affect the topological structure of the network.

The most important questions are: what is the relationship between the network properties of the brain and its cognitive behavior? What is the relationship between the network properties and brain and mental disorders? For example, we can view neuropsychiatric disorders as disconnectivity syndromes, where graph theory can help quantify weaknesses, vulnerability to lesions, and abnormalities in the network structures. In fact, graph theory has been applied to study the structural and functional network properties in schizophrenia, Alzheimer's, and other disorders.

Spread of Disease

As we have all learned from the COVID-19 pandemic, it is of crucial importance to be able to forecast disease incidents accurately and reliably for mitigation purposes, quarantine measures, policy, and many other decision factors. A graph model can consider either individuals or entire geographic blocks as nodes, and contact occurrences between these individuals or blocks as edges. Recent models for predicting COVID-19 spread, for example, the article “Combining Graph Neural Networks and Spatio-temporal Disease Models to Improve the Prediction of Weekly COVID-19 Cases in Germany” (Fritz et al. 2022), incorporate human mobility data from Facebook, Apple, and Google to model interactions between the nodes in their models.

There is plenty of data that can be put to good use here. Facebook’s “Data for Good” resource has a wealth of data on population densities, social mobility and travel patterns, social connectedness, and others. Google’s COVID-19 Community Mobility Reports draw insights from Google Maps and other products into a data set that charts movement trends over time by geography across different categories of places, such as retail and recreation, groceries and pharmacies, parks, transit stations, workplaces, and residential areas. Similarly, Apple’s and Amazon’s mobility data serve a similar purpose with the goal of aiding efforts to limit the spread of COVID-19.

Spread of Information

We can use graphs to model the spread of information, disease, rumors, gossip, computer viruses, innovative ideas, or others. Such a model is usually a directed graph, where each node corresponds to an individual, and the edges are tagged with information about the interaction between individuals. The edge tags, or weights, are usually probabilities. The weight w_{ij} of the edge connecting $node_i$ to $node_j$ is the probability of a certain effect (disease, rumor, computer virus) propagating from $node_i$ to $node_j$.

Detecting and Tracking Fake News Propagation

Graph neural networks perform better in the task of detecting fake news (see Figure 9-5) than content-based natural language processing approaches. The abstract of the paper “Fake News Detection on Social Media using Geometric Deep Learning” (Monti et al. 2019) is informative:

Social media are nowadays one of the main news sources for millions of people around the globe due to their low cost, easy access, and rapid dissemination. This however comes at the cost of dubious trustworthiness and significant risk of exposure to ‘fake news’, intentionally written to mislead the readers. Automatically detecting fake news poses challenges that defy existing content-based analysis approaches. One of the main reasons is that often the interpretation of the news requires the knowledge of political or social context or ‘common sense’, which current natural language processing

algorithms are still missing. Recent studies have empirically shown that fake and real news spread differently on social media, forming propagation patterns that could be harnessed for the automatic fake news detection. Propagation based approaches have multiple advantages compared to their content based counterparts, among which is language independence and better resilience to adversarial attacks. In this paper, we show a novel automatic fake news detection model based on geometric deep learning. The underlying core algorithms are a generalization of classical convolutional neural networks to graphs, allowing the fusion of heterogeneous data such as content, user profile and activity, social graph, and news propagation. Our model was trained and tested on news stories, verified by professional fact checking organizations, that were spread on Twitter. Our experiments indicate that social network structure and propagation are important features allowing highly accurate (92.7% ROC AUC) fake news detection. Second, we observe that fake news can be reliably detected at an early stage, after just a few hours of propagation. Third, we test the aging of our model on training and testing data separated in time. Our results point to the promise of propagation based approaches for fake news detection as an alternative or complementary strategy to content based approaches.

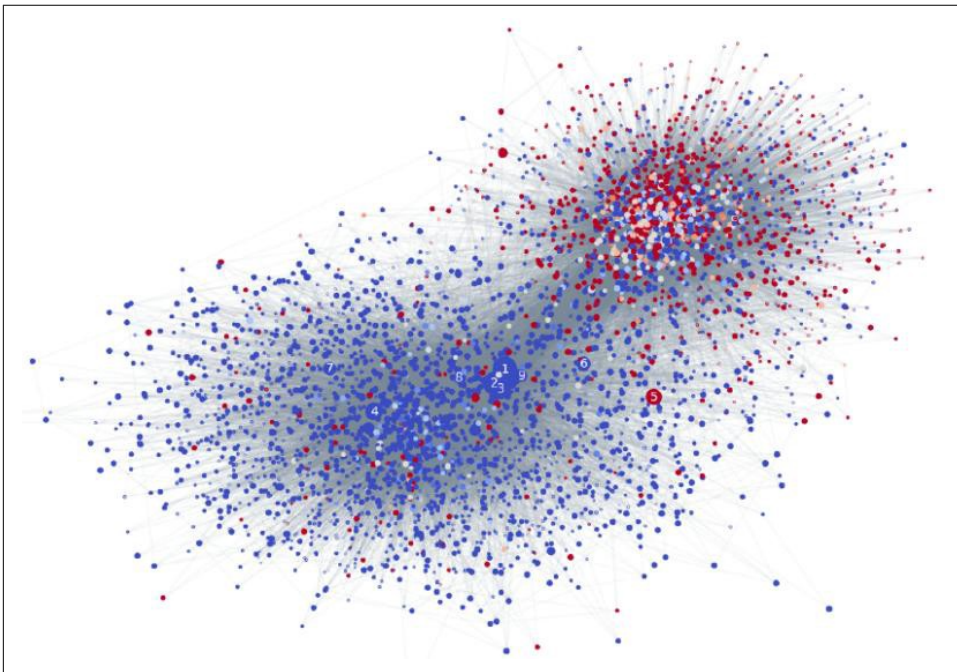


Figure 9-5. Nodes that spread fake news are labeled in red color; people who think alike cluster together in social networks (see [image source](#) for a color version of this image)

Web-Scale Recommendation Systems

Since 2018, Pinterest has been using the **PinSage graph convolutional network**. This curates users' home feed and makes suggestions for new and relevant pins. The authors utilize *random walks on graphs* in their model, which we will discuss later in this chapter. Here is the full abstract:

Recent advancements in deep neural networks for graph-structured data have led to state-of-the-art performance on recommender system benchmarks. However, making these methods practical and scalable to web-scale recommendation tasks with billions of items and hundreds of millions of users remains a challenge. Here we describe a large-scale deep recommendation engine that we developed and deployed at Pinterest. We develop a data efficient Graph Convolutional Network (GCN) algorithm PinSage, which combines efficient random walks and graph convolutions to generate embeddings of nodes (i.e., items) that incorporate both graph structure as well as node feature information. Compared to prior GCN approaches, we develop a novel method based on highly efficient random walks to structure the convolutions and design a novel training strategy that relies on harder-and-harder training examples to improve robustness and convergence of the model. We deploy PinSage at Pinterest and train it on 7.5 billion examples on a graph with 3 billion nodes representing pins and boards, and 18 billion edges. According to offline metrics, user studies and A/B tests, PinSage generates higher-quality recommendations than comparable deep learning and graph-based alternatives. To our knowledge, this is the largest application of deep graph embeddings to date and paves the way for a new generation of web-scale recommender systems based on graph convolutional architectures.

Fighting Cancer

In the article “**HyperFoods: Machine Intelligent Mapping of Cancer-Beating Molecules in Foods**” (Veselkov et al. 2019), the authors use protein, gene, and drug interaction data to identify the molecules that help prevent and beat cancer. They also map the foods that are the richest in cancer-beating molecules (see **Figure 9-6**). Again, the authors utilize random walks on graphs. Here's the abstract of the paper:

Recent data indicate that up to 30–40% of cancers can be prevented by dietary and life-style measures alone. Herein, we introduce a unique network-based machine learning platform to identify putative food-based cancer-beating molecules. These have been identified through their molecular biological network commonality with clinically approved anti-cancer therapies. A machine-learning algorithm of random walks on graphs (operating within the supercomputing DreamLab platform) was used to simulate drug actions on human interactome networks to obtain genome-wide activity profiles of 1962 approved drugs (199 of which were classified as “anti-cancer” with their primary indications). A supervised approach was employed to predict cancer-beating molecules using these ‘learned’ interactome activity profiles. The validated model performance predicted anti-cancer therapeutics with classification accuracy of 84–90%. A comprehensive database of 7962 bioactive molecules within foods was fed into the model, which predicted 110 cancer-beating molecules (defined by anti-cancer drug

likeness threshold of >70%) with expected capacity comparable to clinically approved anti-cancer drugs from a variety of chemical classes including flavonoids, terpenoids, and polyphenols. This in turn was used to construct a ‘food map’ with anti-cancer potential of each ingredient defined by the number of cancer-beating molecules found therein. Our analysis underpins the design of next-generation cancer preventative and therapeutic nutrition strategies.

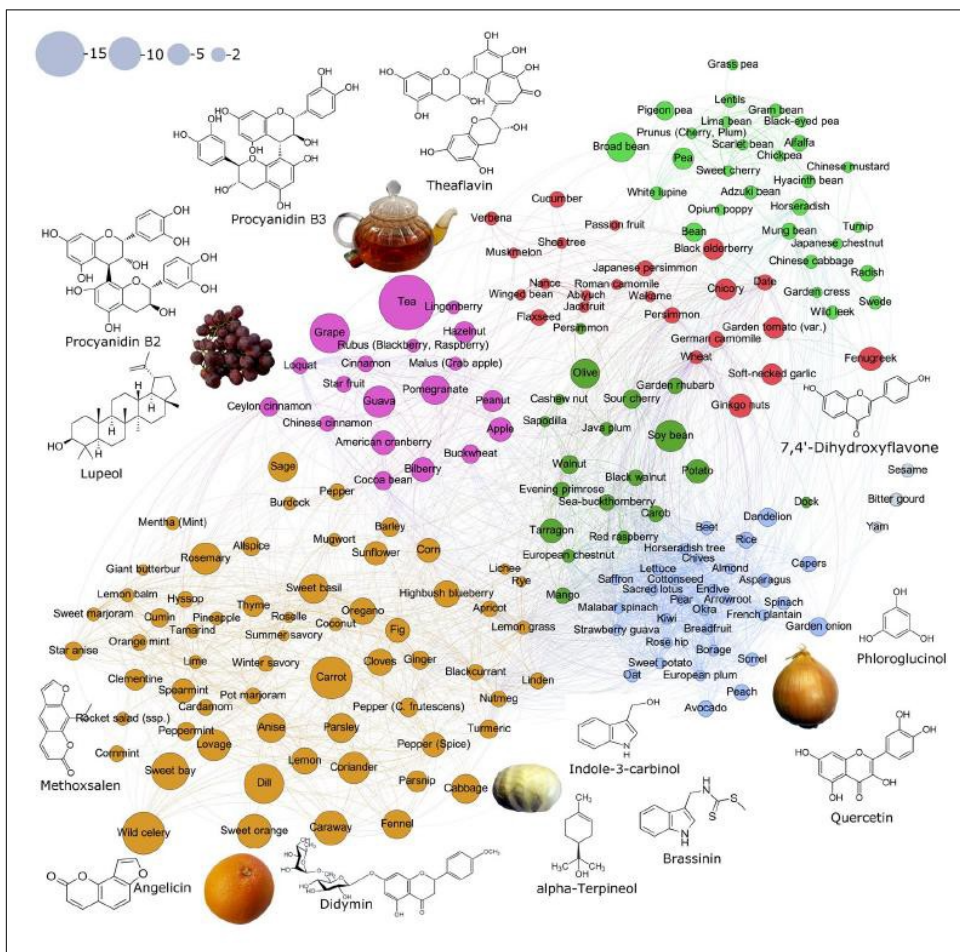


Figure 9-6. Machine intelligent mapping of cancer-beating molecules in foods; the bigger the node, the more diverse the set of cancer-beating molecules (*image source*)

Biochemical Graphs

We can represent molecules and chemical compounds as graphs where the nodes are the atoms, and the edges are the chemical bonds between them. Data sets from this chemoinformatics domain are useful for assessing a classification model's

performance. For example, the **NCII** data set, containing around 4,100 chemical compounds, is useful for anti-cancer screens where the chemicals are labeled as positive or negative to hinder cell lung cancer. Similar labeled graph data sets for proteins and other compounds are available on the same website, along with the papers that employ them and the performance of different models on these data sets.

Molecular Graph Generation for Drug and Protein Structure Discovery

In the last chapter we learned how generative networks such as variational autoencoders and adversarial networks learn joint probability distributions from the data in order to generate similar-looking data for various purposes. Generative networks for graphs build on similar ideas; however, they are a bit more involved than networks generating images per se. Generative graph networks generate new graphs either in a sequential manner, outputting nodes and edges step-by-step, or in a global manner, outputting a whole graph's adjacency matrix at once. See, for example, this [survey paper on generative graph networks \(2020\)](#) for details on the topic.

Citation Networks

In citation networks, the nodes could be the authors, and the edges are their coauthorships; or the nodes are papers, and the (directed) edges are the citations between them. Each paper has directed edges pointing to the papers it cites. Features for each paper include its abstract, authors, year, venue, title, field of study, and others. Tasks include node clustering, node classification, and link prediction. Popular data sets for paper citation networks include CoRA, CiteSeerX, and PubMed. The **CoRA data set** contains around three thousand machine learning publications grouped into seven categories. Each paper in the citation networks is represented by a one-hot vector indicating the presence or absence of a word from a prespecified dictionary, or by a term frequency-inverse document frequency (TF-IDF) vector. These data sets are updated continuously as more papers join the networks.

Social Media Networks and Social Influence Prediction

Social media networks, such as Facebook, Twitter, Instagram, and Reddit, are a distinctive feature of our time (after 2010). The **Reddit data set** is an example of the available data sets. This is a graph where the nodes are the posts and the edges are between two posts that have comments from the same user. The posts are also labeled with the community to which they belong.

Social media networks and their social influence have a substantial impact on our societies, ranging from advertising to winning presidential elections to toppling political regimes. One important task for a graph model representing social networks is to predict the social influence of the nodes in the network. Here, the nodes are the users, and their interactions are the edges. Features include users' gender, age, sex, location,

activity level, and others. One way to quantify social influence, the target variable, is through predicting the actions of a user given the actions of their near neighbors in the network. For example, if a user's friends buy a product, what is the probability that they will buy the same product after a given period of time? Random walks on graphs help predict the social influence of certain nodes in a network.

Sociological Structures

Social diagrams are directed graphs that represent relationships among individuals in a society or among groups of individuals. The nodes are the members of the society or the groups, and the directed edges are the relationships between these members, such as admiration, association, influence, and others. We are interested in the connectedness, separability, size of fragments, and so forth in these social diagrams. One example is from anthropological studies where a number of tribes are classified according to their kinship structures.

Bayesian Networks

Later in this chapter we will discuss Bayesian networks. These are probabilistic graph models whose goal is one we are very familiar with in the AI field: to learn joint probability distribution of the features of a data set. Bayesian networks consider this joint probability distribution as a product of single variable distributions *conditional only on a node's parents* in a graph representing the relationships between the features of the data. That is, the nodes are the feature variables and the edges are between the features that *we believe* to be connected. Applications include spam filtering, voice recognition, and coding and decoding, to name a few.

Traffic Forecasting

Traffic prediction is the task of predicting traffic volumes using historical roadmaps, road speed, and traffic volume data. There are **benchmark traffic data sets** that we can use to track progress and compare models. For example, the **METR-LA data set** is a spatial-temporal graph, containing four months of traffic data collected by 207 sensors on the highways of Los Angeles County. The traffic network is a graph, where the nodes are the sensors, and the edges are the road segments between these sensors. At a certain time t , the features are traffic parameters, such as velocity and volume. The task of a graph neural network is to predict the features of the graph after a certain time has elapsed.

Other traffic forecasting models employ **Bayesian networks**, such as traffic flows among adjacent road links. The model uses information from adjacent road links to analyze the trends of focus links.

Logistics and Operations Research

We can model and solve many problems in operations research, such as transportation problems and activity networks, using graphs. The graphs involved are usually weighted directed graphs. Operations research problems are combinatorial in nature, and are always trivial if the network is small. However, for large real-world networks, the challenge is finding efficient algorithms that can sift through the enormous search space and quickly rule out big parts of it. A large part of the research literature deals with estimating the computational complexity of such algorithms. This is called combinatorial optimization. Typical problems include the traveling salesman problem, supply chain optimization, shared rides routing and fares, job matching, and others. Some of the graph methods and algorithms used for such problems are minimal spanning trees, shortest paths, max-flow min-cuts, and matching in graphs. We will visit operations research examples later in this book.

Language Models

Graph models are relevant for a variety of natural language tasks. These tasks seem different at the surface but many of them boil down to clustering, for which graph models are very well suited.

For any application we must first choose what the nodes, edges, and features for each represent. For natural language, these choices reveal hidden structures and regularities in the language and in the language corpuses.

Instead of representing a natural language sentence as a sequence of tokens for recurrent models or as a vector of tokens for transformers, in graph models we embed sentences in a graph, then employ graph deep learning (or graph neural networks).

One example from computational linguistics is constructing diagrams for parsing language, as shown in [Figure 9-7](#).

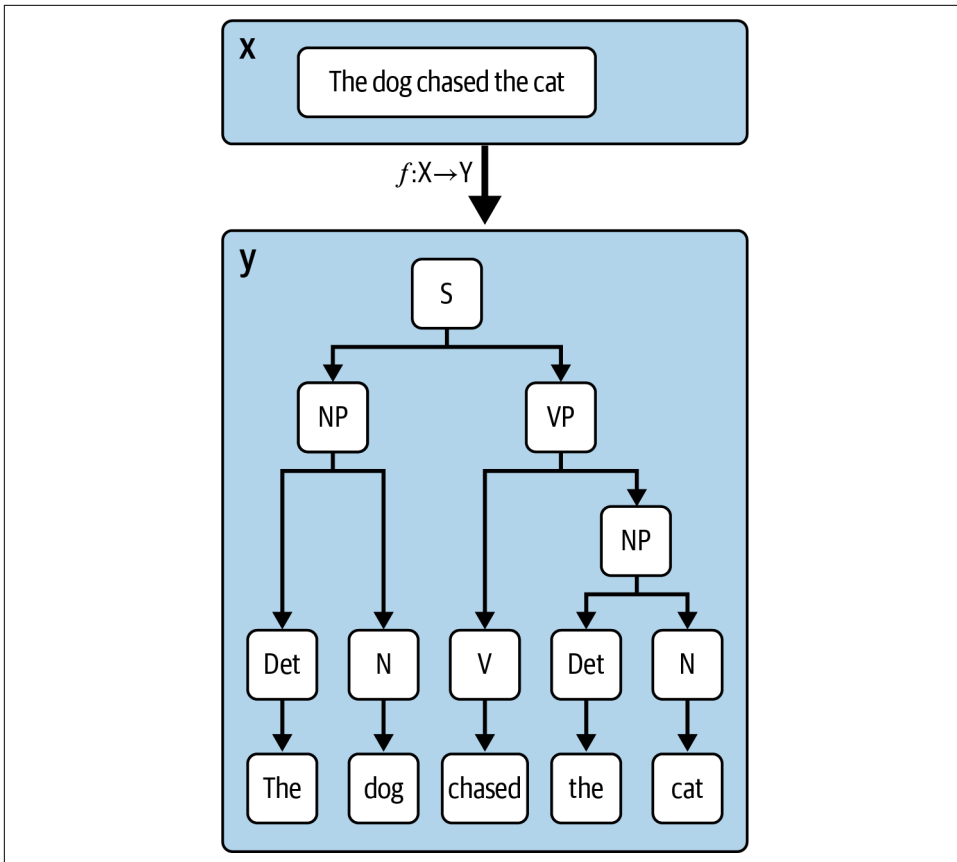


Figure 9-7. A parsed sentence

The nodes are words, n-grams, or phrases, and the edges are the relationships between them, which depend on the language grammar or syntax (article, noun, verb, etc.). A language is defined as the set of all strings correctly generated from the language vocabulary according to its grammar rules. In that sense, computer languages are easy to parse (they are built that way), while natural languages are much harder to specify completely due to their complex nature.



Parsing

Parsing means converting a stream of input into a structured or formal representation so it can be automatically processed. The input to a parser might be sentences, words, or even characters. The output is a tree diagram containing information about the function of each part of the input. Our brain is a great parser for language inputs. Computers parse programming languages.

Another example is news clustering or article recommendations. Here, we use graph embeddings of text data to determine text similarity. The nodes can be words and the edges can be semantic relationships between the words, or just their co-occurrences. Or the nodes can be words and documents, and the edges can again be semantic or co-occurrence relationships. Features for nodes and edges can include authors, topics, time periods, and others. Clusters emerge naturally in such graphs.

Another type of parsing that does not depend on the syntax or grammar of a language is abstract meaning representation (AMR). It relies instead on semantic representation, in the sense that sentences that are similar in meaning should be assigned the same abstract meaning representation, even if they are not identically worded. Abstract meaning representation graphs are rooted, labeled, directed, acyclic graphs, representing full sentences. These are useful for machine translation and natural language understanding. There are packages and libraries for abstract meaning representation parsing, visualization, and surface generation, as well as publicly available data sets.

For other natural language applications, the following survey paper is a nice reference that is easy to read to learn more about the subject: [“A Survey of Graphs in Natural Language Processing”](#) (Nastase et al. 2015).

Graph Structure of the Web

Since the inception of the World Wide Web in 1989, it has grown enormously and has become an indispensable tool for billions of people around the world. It allows access to billions of web pages, documents, and other resources using an internet web browser. With billions of pages linking to each other, it is of great interest to investigate the graph structure of the web. Mathematically, this vast and expansive graph is fascinating in its own right. But understanding this graph is important for more reasons than a beautiful mental exercise, providing insights into algorithms for crawling, indexing, and ranking the web (as in the PageRank algorithm that we saw earlier in this chapter), searching communities, and discovering the social and other phenomena that characterize its growth or decay.

The World Wide Web graph has:

Nodes

Web pages, on the scale of billions

Edges

These are directed from one page linking to another page, on the scale of hundreds of billions

We are interested in:

- What is the average degree of the nodes?
- Degree distributions of the nodes (for both the in degree and out degree, which can be very different). Are they power laws? Some other laws?
- Connectivity of the graph: what is the percentage of connected pairs?
- Average distances between the nodes.
- Is the observed structure of the web dependent or independent of the particular crawl used?
- Particular structures of weakly and strongly connected components.
- Is there a giant strongly connected component? What is the proportion of nodes that can reach or be reached from this giant component?

Automatically Analyzing Computer Programs

We can use graphs for verification of computer programs, program reasoning, reliability theory, fault diagnosis in computers, and studying the structure of computer memory. The paper “[Graph Neural Networks on Program Analysis](#)” (Allamanis 2021) is one example:

Program analysis aims to determine if a program’s behavior complies with some specification. Commonly, program analyses need to be defined and tuned by humans. This is a costly process. Recently, machine learning methods have shown promise for probabilistically realizing a wide range of program analyses. Given the structured nature of programs, and the commonality of graph representations in program analysis, graph neural networks (GNN) offer an elegant way to represent, learn, and reason about programs and are commonly used in machine learning-based program analyses. This article discusses the use of graph neural networks for program analysis, highlighting two practical use cases: Variable misuse detection and type inference.

Data Structures in Computer Science

A data structure in computer science is a structure that stores, manages, and organizes data. There are different data structures, and they are usually chosen in a way that makes it efficient to access the data (read, write, append, infer or store relationships, etc.).

Some data structures use graphs to organize data, computational devices in a cluster, and represent the flow of data and computation or the communication network. There are also graph databases geared toward storing and querying graph data. Other databases transform graph data to more structured formats (such as relational formats).

Here are some examples for graph data structures:

PageRank algorithm

We have already encountered the PageRank algorithm, along with the link structure of a website represented as a directed graph, where the nodes are the web pages and the edges represent the links from one page to another. A database keeping all the web pages along with their link structures can be either graph structured, where the graph is stored as is using the linking matrix or adjacency matrix with no transformation necessary, or it can be transformed to fit the structure of other nongraphical databases.

Binary search trees for organizing files in a database

Binary search trees are ordered data structures that are efficient for both random and sequential access of records, and for file modification. The inherent order of a binary search tree speeds up search time: we cut the amount of data to sort through by half at each level of the tree. It also speeds up insertion time: unlike an array, when we add a node to the binary tree data structure, we create a new piece in memory and link to it. This is faster than creating a new large array, then inserting the data from the smaller array to the new, larger one.

Graph-based information retrieval systems

In some information retrieval systems we assign a certain number of index terms to each document. We can think of these as the document's indicators, descriptors, or keywords. These index terms will be represented as nodes of a graph. We connect two index terms with an undirected edge if these two happen to be closely related, such as the indices *graph* and *network*. The resulting *similarity graph* is very large, and is possibly disconnected. The maximally connected subgraphs of this graph are its *components*, and they naturally classify the documents in this system. For information retrieval, our query specifies some index terms, that is, certain nodes of the graph, and the system returns the maximal complete subgraph that includes the corresponding nodes. This gives the complete list of index terms, which in turn specify the documents we are searching for.

Load Balancing in Distributed Networks

The computational world has grown from Moore's law to parallel computing to cloud computing. In cloud computing, our data, our files, and the machines executing our files and doing computations on our data are not near us. They are not even near each other. As applications become more complex and as network traffic increases, we need the software or hardware analog of a network traffic cop that distributes network traffic across multiple servers so that no single server bears a heavy load, enhancing performance in terms of application response times, end user experience, and so on.

As traffic increases, more appliances, or nodes, need to be added to handle the volume. Network traffic distributing needs to be done while preserving data security and privacy, and should be able to predict traffic bottlenecks before they happen. This is exactly what load balancers do. It is not hard to imagine the distributed network as a graph with the nodes as the connected servers and appliances. Load balancing is then a traffic flow problem on a given graph, and there are a variety of algorithms for allocating the load. All algorithms operate on the network's graph. Some are static, allocating loads without updating the network's current state in terms of loads or malfunctioning units, while others are dynamic, but require constant communication within the network about the nodes' statuses. The following are some algorithms:

Least connection algorithm

This method directs traffic to the server with the fewest active connections.

Least response time algorithm

This method directs traffic to the server with the fewest active connections and the lowest average response time.

Round-robin algorithm

This algorithm allocates load in a rotation on the servers. It directs traffic to the first available server, then moves that server to the bottom of the queue.

IP hash

This method allocates servers based on the IP address of the client.

Artificial Neural Networks

Finally, artificial neural networks are graphs where the nodes are the computational units and the edges are inputs and outputs of these units. [Figure 9-8](#) summarizes popular artificial neural network models as graphs.

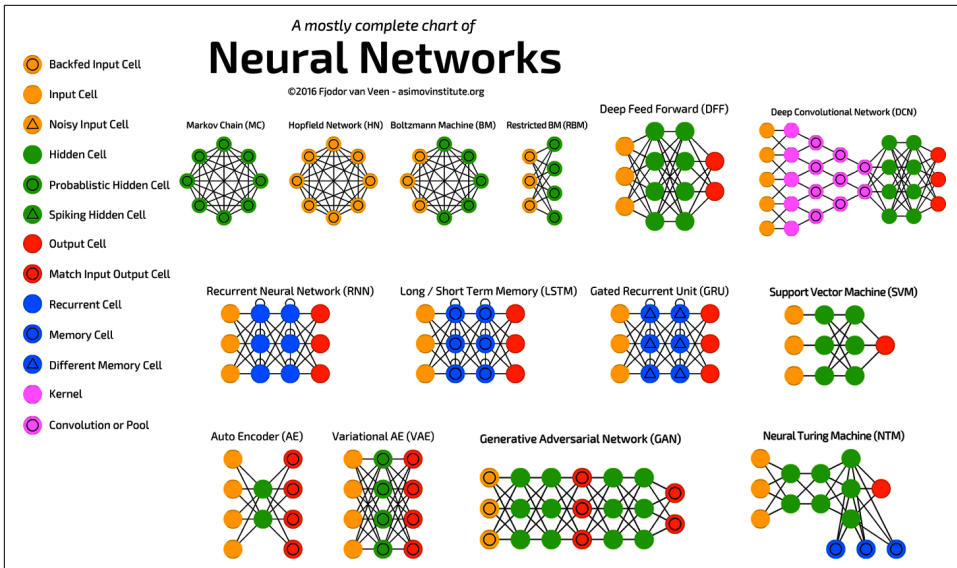


Figure 9-8. Neural networks as graphs (*image source*)

Random Walks on Graphs

A random walk on a graph (Figure 9-9) means exactly what it says: a sequence of steps that starts at some node, and at each time step, chooses a neighboring node (using the adjacency matrix) with probability proportional to the weights of the edges, and moves there.

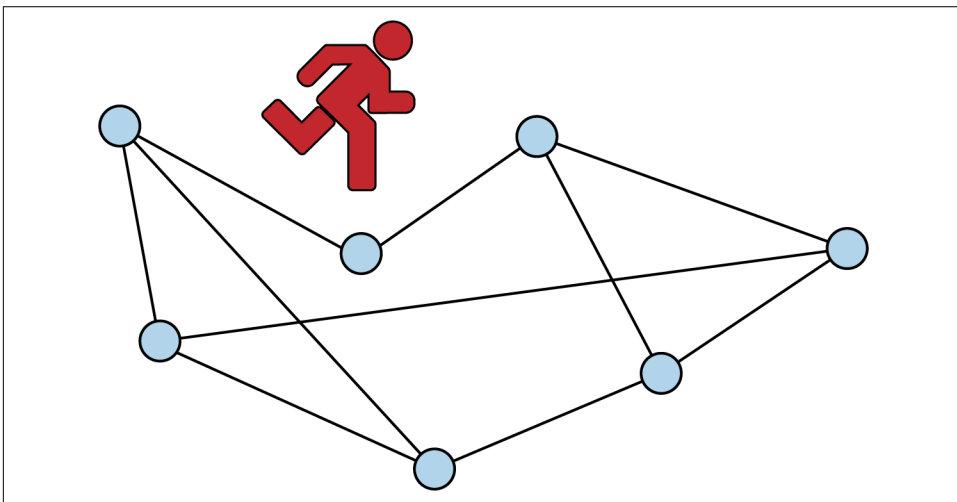


Figure 9-9. A random walker on an undirected graph

If the edges are unweighted, then the neighboring nodes are all equally likely to be chosen for the walk's move. At any time step, the walk can stay at the same node in the case that there is a self edge, or when it is a lazy random walk with a positive probability the walk stays at a node instead of moving to one of its neighbors. We are interested in the following:

- What is the list of nodes visited by a random walk, in the order they are visited? Here, the starting point and the structure of the graph matter in how much a walk covers or whether the walk can ever reach certain regions of the graph. In graph neural networks, we are interested in learning a representation for a given node based on its neighbors' features. In large graphs where nodes have more neighbors than is computationally feasible, we employ random walks. However, we have to be careful since different parts of the graph have different random walk expansion speeds, and if we do not take that into account by adjusting the number of steps of a random walk according to the subgraph structure, we might end up with low-quality representations for the nodes, and undesirable outcomes as these go down the work pipeline.
- What is the expected behavior of a random walk, that is, the probability distribution over the visited nodes after a certain number of steps? We can study basic properties of a random walk, such as its long-term behavior, by using *the spectrum of the graph*, which is the set of eigenvalues of its adjacency matrix. In general, the spectrum of an operator helps us understand what happens when we repeatedly apply the operator. Randomly walking on a graph is equivalent to repeatedly applying a normalized version of the adjacency matrix to the node of the graph where we started. Every time we apply this *random walk matrix*, we walk one step further on the graph.
- How does a random walk behave on different types of graphs, such as paths, trees, two fully connected graphs joined by one edge, infinite graphs, and others?
- For a given graph, does the walk ever return to its starting point? If so, how long do we have to walk until we return?
- How long do we have to walk until we reach a specific node?
- How long do we have to walk until we visit all the nodes?
- How does a random walk *expand*? That is, what is the *influence distribution* of certain nodes that belong in certain regions of the graph? What is the size of their influence?
- Can we design algorithms based on random walks that are able to reach *obscure* parts of large graphs?



Random Walks and Brownian Motion

In the limit where the size of steps of a random walk goes to zero, we obtain a Brownian motion. Brownian motion usually models the random fluctuations of particles suspended in a medium such as a fluid, or price fluctuations of derivatives in financial markets. We frequently encounter the term Brownian motion with the term *Weiner process*, which is a continuous stochastic process with a clear mathematical definition of how the motion (of a particle or of a price fluctuation in finance) starts (at zero), how the next step is sampled (from the normal distribution and with independent increments), and assumptions about its continuity as a function of time (it is almost surely continuous). Another term it is associated with is *martingale*. We will see these in [Chapter 11](#) on probability.

We did encounter a random walk once in this chapter when discussing the PageRank algorithm, where a random web page surfer randomly chooses to move from the page they are at to a neighboring page on the web. We noticed that the long-term behavior of the walk is discovered when we repeatedly apply the linking matrix of the graph, which is the same as the adjacency matrix normalized by the degrees of each node. In the next section we see more uses of random walks for graph neural networks.

We can use random walks (on directed or undirected graphs, weighted or unweighted graphs) for community detection and influence maximization in small networks, where we would only need the graph's adjacency matrix (as opposed to node embedding into feature vectors, then clustering).

Node Representation Learning

Before implementing any graph tasks on a machine, we must be able to represent the nodes of the graph as vectors that contain information about their position in the graph and their features relative to their locality within the graph. A node's representation vector is usually aggregated from the node's own features and the features of the nodes surrounding it.

There are different ways to aggregate features, transform them, or even choose which of the neighboring nodes contribute to the feature representation of a given node. Let's go over a few methods:

- Traditional node representation methods rely on subgraph summary statistics.
- In other methods, nodes that occur together on short random walks will have similar vector representations.
- Other methods take into account that random walks tend to spread differently on different graph substructures, so a node's representation method adapts to the

local substructure that the node belongs in, deciding on an appropriate radius on influence for each node depending on the topology of the subgraph it belongs to.

- Yet other methods produce a multiscale representation by multiplying the feature vector of a node with powers of a random walk matrix.
- Other methods allow for nonlinear aggregation of the feature vectors of a node and its neighbors.

It is also important to determine how large of a neighborhood a node draws information from (influence distribution), that is, to find the range of nodes whose features affect a given node's representation. This is analogous to sensitivity analysis in statistics, but here we need to determine the sensitivity of a node's representation to changes in the features of the nodes surrounding it.

After creating a node representation vector, we feed it into another machine learning model during training, such as a support vector machine model for classification, just like feeding other features of the data into the model. For example, we can learn the feature vector of every user in a social network, then pass these vectors into a classification model along with other features to predict if the user is a fake news spreader or not. However, we do not have to rely on a machine learning model downstream to classify nodes. We can do that directly from the graph structural data where we can predict a node's class depending on its association with other local nodes. The graph can be only partially labeled, and the task is to predict the rest of the labels. Moreover, the node representation step can either be a preprocessing step, or one part of an end-to-end model, such as a graph neural network.

Tasks for Graph Neural Networks

After going through the linear algebra formulation of graphs, applications of graph models, random walks on graphs, and vector node representations that encode node features along with their zones of influence within the graph, we should have a good idea about the kind of tasks that graph neural networks can perform. Let's go through some of these.

Node Classification

The following are examples of node classification tasks:

- In an articles citation network, such as in CiteSeerX or CoRA, where the nodes are academic papers (given as bag-of-words vectors) and the directed edges are the citations between the papers, classify each paper into a specific discipline.
- In the Reddit data set, where the nodes are comment posts (given as word vectors) and undirected edges are between comments posted by the same user, classify each post according to the community it belongs to.

- The protein-protein interaction network data set contains 24 graphs where the nodes are labeled with the gene ontology sets (do not worry about the medical technical names, focus on the math instead. This is the nice thing about mathematical modeling, it works the same way for all kinds of applications from all kinds of fields, which validates it as a potential underlying language of the universe). Usually 20 graphs from the protein-protein interaction network data set are used for training, 2 graphs are used for validation, and the rest for testing, each corresponding to a human tissue. The features associated with the nodes are positional gene sets, motif gene sets, and immunological signatures. Classify each node according to its gene ontology set.
- There are wildlife trade monitoring networks such as [Traffic.org](https://www.traffic.org/), analyzing dynamic wildlife trade trends and using and updating data sets such as the [CITES Wildlife Trade Database](https://www.cites.org/) or the [USDA Ag Data Commons](https://www.usda.gov/datacommons/) (this data set includes more than a million wildlife or wildlife product shipments, representing more than 60 biological classes and more than 3.2 billion live organisms). One classification task on the trade network is to classify each node, representing a trader (a buyer or a seller) as being engaged in illegal trade activity or not. The edges in the network represent a trade transaction between the buyer and seller. Features for the nodes include personal information for the traders, bank account numbers, locations, etc.; and features for the edges would include transaction identification numbers, dates, price tags, the species bought or sold, and so on. If we already have a subset of the traders labeled as illegal traders, our model's task would then be to predict the labels of other nodes in the network based on their connections with other nodes (and their features) in the network.

Node classification examples lend themselves naturally to semi-supervised learning, where only a few nodes in the data set come with their labels, and the task is to label the rest of the nodes. Clean labeled data is what we all should be advocating for our systems to be more accurate, reliable, and transparent.

Graph Classification

Sometimes we want to label a whole graph as opposed to labeling the individual nodes of a graph. For example, in the PROTEINS data set we have a collection of chemical compounds each represented as a graph and labeled as either an enzyme or not an enzyme. For a graph learning model we would input the nodes, edges, their features, the graph structure, and the label for each graph in the data set, thus creating a whole graph representation or embedding, as opposed to a single node representation.

Clustering and Community Detection

Clustering in graphs is an important task that discovers communities or groups in networks, such as terrorist organizations. One way is to create node and graph representations, then feed them into traditional clustering methods such as k-means clustering. Other ways produce node and graph representations that take into account the goal of clustering within their design. These can include encoder-decoder designs and attention mechanisms similar to methods we encountered in previous chapters. Other methods are spectral, which means that they rely on the eigenvalues of the Laplacian matrix of the graph. Note that for nongraph data, principal component analysis is one method that we used for clustering that is also spectral, relying on the singular values of the data table. Computing eigenvalues of anything is always an expensive operation, so the goal becomes finding ways around having to do it. For graphs, we can employ graph theoretic methods such as *max-flow min-cut* (we will see this later in this chapter). Different methods have their own sets of strengths and shortcomings; for example, some employ time-proven graph theoretic results but fail to include the node or edge features because the theory was not developed with any features in mind, let alone a whole bunch of them. The message is to always be honest about what our models account and do not account for.

Graph Generation

Graph generation is very important for drug discovery, materials design, and other applications. Traditionally, graph generation approaches used handcrafted families of random graph models using a simple stochastic generation process. These models are well understood mathematically due to their simple properties. However, for the same reason, they are limited in their ability to capture real-world graphs with more complex dependencies, or even the correct statistical properties, such as heavy tailed distribution for the node degrees that many real-world networks exhibit. More recent approaches, such as generative graph neural networks, integrate graph and node representations with generative models that we went over in the previous chapter. These have a greater capacity to learn structural information from the data and generate complex graphs, such as molecules and compounds.

Influence Maximization

Influence maximization is a subfield of network diffusion, where the goal is to maximize the diffusion of something, such as information or a vaccine, through a network, while only giving the thing to few initial nodes, or seeds. The objective is to locate the few nodes that have an overall maximal influence. Applications include information propagation such as job openings, news, advertisements, and vaccinations. Traditional methods for locating the seeds choose the nodes based on highest degree, closeness, betweenness, and other graph structure properties. Others

employ the field of discrete optimization, obtaining good results and proving the existence of approximate optimizers. More recent approaches employ graph neural networks and adversarial networks when there are other objectives competing with the objective of maximizing a node's influence, for example, reaching a specific portion of the population, such as a certain minority group, that is not necessarily strongly connected with the natural hubs of the graph.

Link Prediction

Given two nodes of a graph, what is the probability that there is an edge linking them? Note that proximity in the sense of sharing common neighbors is not necessarily an indicator for a link (or an interaction). In a social network, people tend to run in the same circles, so two people sharing many common friends are likely to interact and are likely connected as well. But in some biological systems, such as in studying protein-protein interactions, the opposite is true: proteins sharing more common neighbors are less likely to interact. Therefore, computing similarity scores based on basic properties such as graph distance, degrees, common neighbors, and so on does not always produce correct results. We need neural networks to learn node and graph embeddings along with classifying whether two nodes are linked or not. One example of such networks is in the paper “[Link Prediction with Graph Neural Networks and Knowledge Extraction](#)” (Zhang et al. 2020).

Dynamic Graph Models

Many of the applications we have discussed in this chapter would benefit from including time dependency in our graph models, since they are dynamic in nature. Examples include traffic forecasting, load balancing for distributed networks, simulating all kinds of interacting particle systems, and illegal wildlife trade monitoring. In dynamic graph models, node and edge features are allowed to evolve with time, and some nodes or edges can be added or removed. This modeling captures information such as the latest trade trends in a market, fluctuations, new criminal activity in certain networks, and new routes or connections in transportation systems.

Thinking about how to model dynamic graphs and extract information from them is not new; see, for example, the article “[Dynamic Graph Models](#)” (Harray et al. 1997). However, the introduction of deep learning makes knowledge extraction from such systems more straightforward. Current approaches for dynamic graphs integrate graph convolutions to capture spatial dependencies with recurrent neural networks or convolutional neural networks to model temporal dependencies.

The paper “[Learning to Simulate Complex Physics with Graph Networks](#)” (Sanchez-Gonzalez et al. 2020) is a great example with wonderful high-resolution results that employs a dynamic graph neural network to simulate any system of interacting particles on a much larger scale, in terms of both the number of involved particles

and the time the system is allowed to (numerically) evolve, than what was done before. The particles, such as sand or water particles, are the nodes of the graph, with attributes such as position, velocity, pressure, external forces, etc., and the edges connect the particles that are allowed to interact with each other. The input to the neural network is a graph, and the output is a graph with the same nodes and edges but with updated attributes of particle positions and properties. The network learns the dynamics, or the update rule at each time step, via message passing. The update rule depends on the system's state at the current time step, and on a parameterized function whose parameters are optimized for some training objective that depends on the specific application, which is the main step in any neural network. The prediction targets for supervised learning are the average acceleration for each particle.

Bayesian Networks

Bayesian networks are graphs that are perfectly equipped to deal with uncertainty, encoding probabilities in a mathematically sound way. Figures 9-10 and 9-11 are examples of two Bayesian networks. In Chapter 8, we presented multiple models that attempt to learn the joint probability distribution, ranging from explicit and tractable all the way to implicit and intractable. A Bayesian network is a very simple explicit model for the joint probability distribution, where we explicitly prespecify how each variable interacts with the others.

In a Bayesian network:

- The nodes are variables that we believe our model should include.
- The edges are directed, pointing from the *parent* node to the *child* node, or from a *higher neuron* to a *lower neuron*, in the sense that we know the probability of the child variable is conditional on observing the parent variable.
- No cycles are allowed in the graph of the network.
- Heavy reliance on Bayes' Rule: if there is an arrow from A to B, then $P(B|A)$ is the forward probability, and $P(A|B)$ is the inverse probability. Think of this as $P(\text{evidence}|\text{hypothesis})$ or $P(\text{symptoms}|\text{disease})$. We can calculate the inverse probability from Bayes' Rule:

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

- If there is no arrow pointing to a variable (if it has no parents), then all we need is the *prior* probability of that variable, which we compute from the data or from expert knowledge, such as 13% of women in the US develop breast cancer.
- If we happen to obtain more data on one of the variables in the model, or more *evidence*, we update the node corresponding to that variable (the conditional

probability), then propagate that information following the connections in the network, updating the conditional probabilities at each node, in two different ways, depending on whether the information is propagating from parent to child, or from child to parent. The update in each direction is very simple: comply with Bayes' Rule.

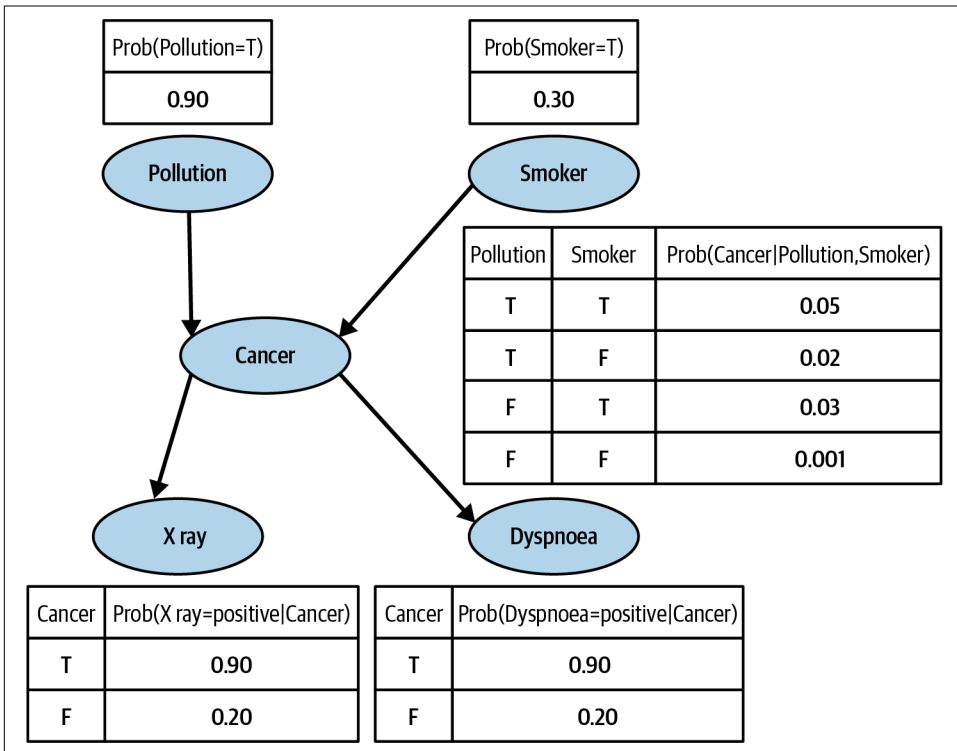


Figure 9-10. A Bayesian network

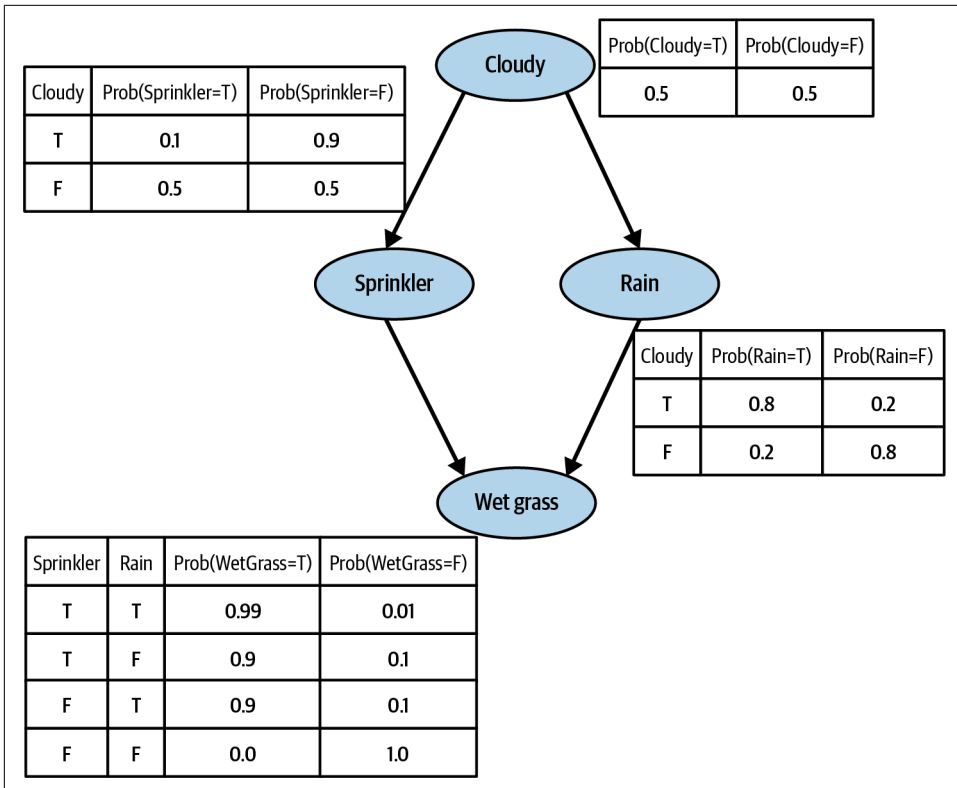


Figure 9-11. Another Bayesian network

A Bayesian Network Represents a Compactified Conditional Probability Table

What a Bayesian network represents is a compactified conditional probability table. Usually when we model a real-world scenario, each discrete variable can assume certain discrete values or categories, and each continuous variable can assume any value in a given continuous range. In theory, we can construct a giant conditional probability table that gives the probability of each variable, assuming a certain state of given fixed values of the other variables. In reality, even for a reasonably small number of variables, this is infeasible and expensive both for storage and computations. Moreover, we do not have access to all the information required to construct the table. Bayesian networks get around this hurdle by allowing variables to interact with only a few neighboring variables, so we only have to compute the probability of a variable given the states of those variables directly connected to it in the graph of the network, albeit both forward and backward. If new evidence about any variable in the network arrives, then the graph structure, together with Bayes' Rule, guides us to update the probabilities of all the variables in the network in a systematic,

explainable, and transparent way. *Sparsifying* the network this way is a feature of Bayesian networks that has allowed for their success.

In summary, a Bayesian network's graph specifies the joint probability distribution of the model's variables (or the data's features) as a product of local conditional probability distributions, one for each node:

$$P(x_1, x_2, \dots, x_n) = \prod_{i=1}^n P(x_i | \text{parents of } x_i)$$

Making Predictions Using a Bayesian Network

Once a Bayesian network is set and the conditional probabilities initiated (and continuously updated using more data), we can get quick results simply by searching these conditional probability distribution tables along Bayes' Rule or the product rule, depending on the query. For example, what is the probability that this email is spam given the words it contains, the sender location, the time of the day, the links it includes, the history of interaction between the sender and the recipient, and other values of the spam detection variables? What is the probability that a patient has breast cancer given the mammogram test result, family history, symptoms, blood tests, etc.? The best part here is that we do not need to consume energy executing large programs or employ large clusters of computers to get our results. That is, our phone and tablet batteries will last longer because they don't have to spend much computational power coding and decoding messages; instead, they apply Bayesian networks for using the *turbo decoding forward error correction algorithm*.

Bayesian Networks Are Belief Networks, Not Causal Networks

In Bayesian networks, although a pointed arrow from parent variable to child variable is preferably causal, in general it *is not* causal. All it means is that we can model the probability distribution of a child variable given the states of its parent(s), and we can use Bayes' Rule to find the inverse probability: the probability distribution of a parent given the child. This is usually the more difficult direction because it is less intuitive and harder to observe. One way to think about this is that it is easier to calculate the probability distribution of a child's traits given that we know the parents' traits, $P(\text{child}|\text{mother},\text{father})$, even before having the child, than inferring the parents' traits given that we know the child's traits $P(\text{father}|\text{child})$ and $P(\text{mother}|\text{child})$. The Bayesian network in this example (Figure 9-12) has three nodes, mother, father, and child, with an edge pointing from the mother to the child, and another edge pointing from the father to the child.

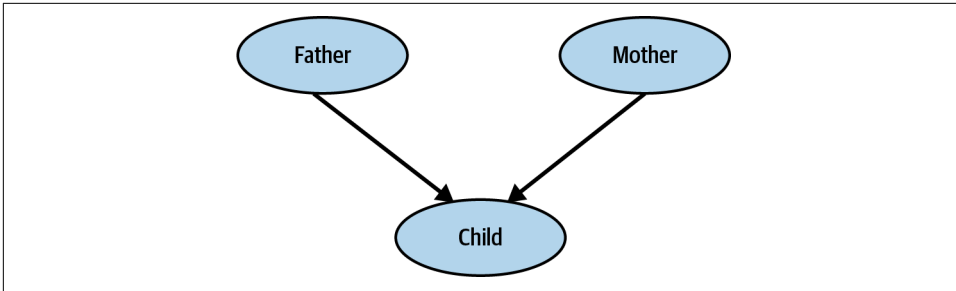


Figure 9-12. The child variable is a collider in this Bayesian network

There is no edge between the mother and the father because there is no reason for their traits to be related. Knowing the mother's traits gives us no information about the father's traits; however, knowing the mother's traits and the child's traits allows us to know slightly more about the father's traits, or the distribution $P(\text{father} | \text{mother}, \text{child})$. This means that the mother's and father's traits, which were originally independent, are conditionally dependent given knowing the child's traits. Thus, a Bayesian network models the dependencies between variables in a graph structure, providing a map for how the variables *are believed* to relate to each other: their conditional dependencies and independencies. This is why Bayesian networks are also called belief networks.

Keep This in Mind About Bayesian Networks

Let's keep the following in mind about Bayesian networks:

- Bayesian networks have no causal direction and are limited in answering causal questions, or “why” questions, such as: what caused the onset of a certain disease? That said, we will soon learn that we can use a Bayesian network for causal reasoning, and to predict the consequences of intervention. Whether used for causal reasoning or not, how we update a Bayesian network, or how we propagate the *belief*, always works in the same way.
- If some variables have missing data, Bayesian networks can handle it because they are designed to propagate information efficiently from variables with abundant information about them to variables with less information.

Chains, Forks, and Colliders

The building blocks of Bayesian networks (with three or more nodes) are three types of *junctions*: chain, fork, and collider, illustrated in [Figure 9-13](#).

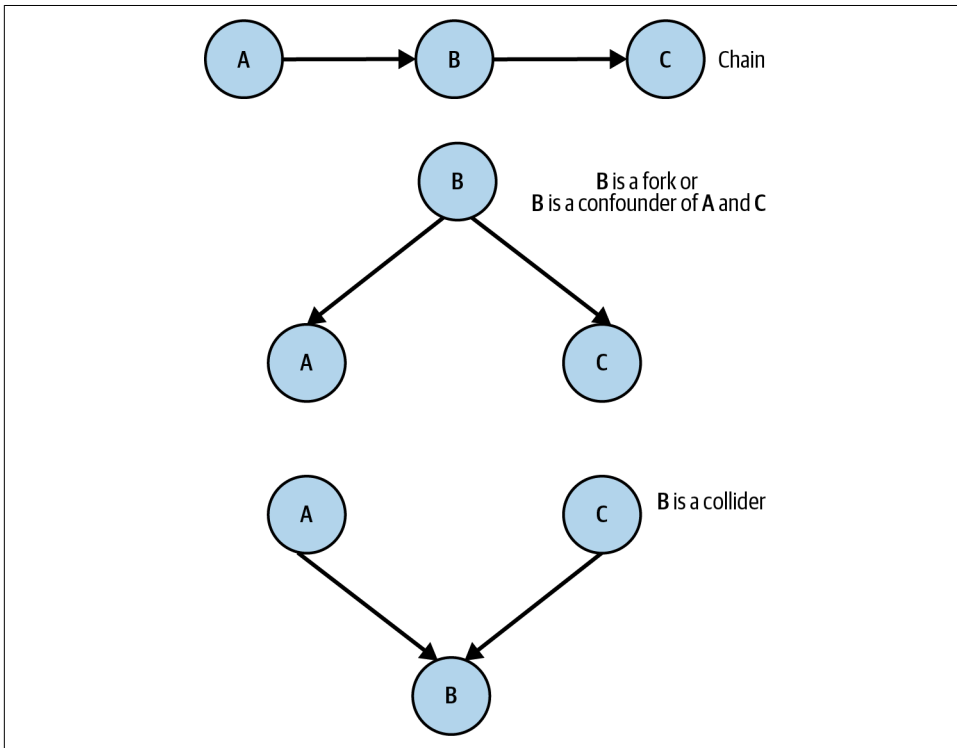


Figure 9-13. The three types of junctions in a Bayesian network

Chain: $A \rightarrow B \rightarrow C$

In this chain, B is a mediator. If we know the value of B, then learning about A does not increase or decrease our belief in C. Thus, A and C are conditionally independent, given that we know the value of the mediator B. Conditional independence allows us, and a machine using a Bayesian network, to focus only on the relevant information.

Fork: $B \rightarrow A$ and $B \rightarrow C$

That is, B is a common parent or a confounder of A and C. The data will show that A and C are statistically correlated even though there is no causal relationship between them. We can expose this fake correlation by conditioning on the confounder B.

Collider: $A \rightarrow B$ and $C \rightarrow B$

Colliders are different than chains or forks when we condition on the variable in the middle. We saw this in the example of parents pointing to a child. If A and C are originally independent, conditioning on B makes them dependent! This unexpected and noncausal transfer of information is one characteristic of Bayesian networks conditioning: conditioning on a collider happens to open a dependence path between its parents.

Another thing we need to be careful about: when the outcome and mediator are confounded. Conditioning on the mediator in this case is different than holding it constant.

Given a Data Set, How Do We Set Up a Bayesian Network for the Involved Variables?

The graph structure of a Bayesian network can be decided on manually by us, or learned from the data by algorithms. Algorithms for Bayesian networks are very mature and there are commercial ones. Once the network's structure is set in place, if a new piece of information about a certain variable in the network arrives, it is easy to update the conditional probabilities at each node by following the diagram and propagating the information through the network, updating the belief about each variable in the network. The inventor of Bayesian networks, **Judea Pearl**, likens this updating process to living organic tissue, and to a biological network of neurons, where if you excite one neuron, the whole network reacts, propagating the information from one neuron to its neighbors.

Finally, we can think of neural networks as Bayesian networks.



Models Learning Patterns from the Data

It is important to note that Bayesian networks, and any other models learning the joint probability distributions of the features of the given data, such as the models we encountered in the previous chapter on generative models, and the deterministic models in earlier chapters, only detect patterns from the data and learn associations, as opposed to learning *what caused* these patterns to begin with. For an AI agent to be truly intelligent and reason like humans, it must ask the questions *how*, *why*, and *what if* about what it sees and what it does, and it must seek answers, like humans do at a very early age. In fact, for humans, there is the age of *why* early in their development: it is the age when children drive their parents crazy asking *why* about everything and anything. An AI agent should have a *causal model*. This concept is so important for attaining general AI. We will visit it in the next section and one more time in **Chapter 11** on probability.

Graph Diagrams for Probabilistic Causal Modeling

Since taking our first steps in statistics, all we have heard was: *correlation is not causation*. Then we go on and on about data, more data, and correlations in data. Alright then, we get the message, but what about causation? What is it? How do we quantify it? As humans, do we know exactly what *why* means? We conceptualize cause and effect intuitively, even at eight months old. I actually think we function more on a natural and intuitive level in the world of *why* than in the world of association. *Why* then (see?) do our machines, which we expect that at some point will be able to reason like us, only function at an association and regression level? This is the point that mathematician and philosopher Judea Pearl argues for in the field of AI, and in his wonderful book *The Book of Why* (Basic Books, 2020). My favorite quote from this book: “Noncausal correlation violates our common sense.” The idea is that we need to both articulate and quantify which correlations are due to causation and which are due to some other factors.

Pearl builds his mathematical causality models using diagrams (graphs) that are similar to Bayesian network graphs, but endowed with a probabilistic reasoning scheme based on the *do calculus*, or computing probabilities *given the do* operator, as opposed to computing probabilities *given the observe* operator, which is very familiar from noncausal statistical models. The main point is this:

Observing is not the same as doing. In math notation, $Prob(\text{number of bus riders} | \text{color-coded routes})$ is not the same as $Prob(\text{number of bus riders} | do \text{ color-coded routes})$.

We can infer the first one from the data. Look for the ridership numbers given that the bus routes in a certain city are color-coded. This probability does not tell us the *effect* color-coded routes have on the number of riders. The second probability, *with the do operator*, is different, and the data alone, without a causal diagram, cannot tell us the answer. The difference is when we invoke the *do* operator, then we are deliberately changing the bus routes to color-coded, and we want to assess the effect of that change on bus ridership. If the ridership increases after this deliberate *doing*, and given that we drew the correct graph, including the variables and how they talk to each other, then we can assert that using color-coded bus routes *caused* the increase in ridership. When we *do* instead of *observe*, we manually block all the roads that could naturally lead to colored bus routes, such as a change in leadership or a time of the year that might affect ridership. We cannot block these roads if we were simply observing the data. Moreover, when we use the *do* operator, we intentionally and manually *set the value* of bus routes to color-coded (as opposed to numbered, etc.)

Actually, the bus routes example is not made up. I am currently collaborating with the department of public transportation in Harrisonburg, Virginia, with the goals

of increasing their ridership, improving their efficiency, and optimizing their operations given both limited resources and a drastic drop in the city's population when the university is not in session. In 2019, the transportation department *deliberately* changed its routes from a number system to a color-coded system, and at the same time *deliberately* changed its schedules from adaptive to the university's class schedule to fixed schedules. Here is what happened: their ridership increased a whopping 18%. You bet my students who are working on the project this summer (2022) will soon be drawing causal diagrams and writing probabilities that look like:

$$P(\text{ridership} | \text{do color coded routes, do fixed schedules})$$

A machine that is so good at detecting a pattern and acting on it—like a lizard observing a bug fly around, learning its pattern, then catching it and eating it—has a very different level of *intellect* than a machine that is able to reason on two higher levels than mere detection of patterns:

1. *If I deliberately take this action, what will happen to [insert variable here]?*
2. *If I didn't take this action, would [the variable taking a certain value] still have happened?* If Harrisonburg did not move to color-coded routes and to fixed schedules, would the ridership still have increased? What if only one of these variables changed instead of both?

The data alone cannot answer these questions. In fact, carefully constructed causal diagrams help us tell apart the times when we can use the data alone to answer these questions, and when we cannot answer them *irrespective of how much more data we collect*. Until our machines are endowed with graphs representing causal reasoning, our machines have the same level of intellect as lizards. Amazingly, humans do all these computations instantaneously, albeit arriving at the wrong conclusions many times and arguing with each other about causes and effects for decades. We still need math and graphs to settle matters. In fact, the graph guides us and tells us which data we must look for and collect, which variables to condition on, and which variables to apply the *do* operator on. This intentional design and reasoning is very different than the culture of amassing big volumes of data, or aimlessly conditioning on all kinds of variables.

Now that we know this, we can draw diagrams and design models that can help us settle all kinds of causal questions. Did I heal because of the doctor's treatment or did I heal because time has passed and life has calmed down? We would still need to collect and organize data, but this process will now be intentional and guided. A machine endowed with these ways of reasoning—a causal diagram model, along with the (very short) list of valid operations that go with the causal diagram model—will be able to answer queries on all three levels of causation:

- Are variables A and B correlated? Are ridership and labeling of bus routes correlated?
- If I set variable A to a specific value, how would variable B change? If I deliberately set color-coded routes, would ridership increase?
- If variable A did not take a certain value, would variable B have changed? If I did not change to color-coded bus routes, would ridership still have increased?

We still need to learn how to deal with probability expressions that involve the *do* operator. We have established that seeing is not the same as doing: *seeing* is in the data, while *doing* is deliberately running an experiment to assess the causal effect of a certain variable on another. It more costly than just counting proportions seen in the data. Pearl establishes three rules for manipulating probability expressions that involve the *do* operator. These help us move from expressions with *doing* to others with only *seeing*, where we can get the answers from the data. These rules are valuable because they enable us to quantify causal effects by *seeing*, bypassing *doing*. We go over these in [Chapter 11](#) on probability.

A Brief History of Graph Theory

We cannot leave this chapter without a good overview of graph theory and the current state of the field. This area is built on such simple foundations, yet it is beautiful, stimulating, and has far-reaching applications that made me reassess my whole mathematical career path and try to convert urgently.

The vocabulary of graph theory includes graphs, nodes, edges, degrees, connectivity, trees, spanning trees, circuits, fundamental circuits, vector space of a graph, rank and nullity (like in linear algebra), duality, path, walk, Euler line, Hamiltonian circuit, cut, network flow, traversing, coloring, enumerating, links, and vulnerability.

The timeline of the development of graph theory is enlightening, with its roots in transportation systems, maps and geography, electric circuits, and molecular structures in chemistry:

- In 1736, Euler published the first paper in graph theory, solving the [Königsberg bridge problem](#). Then nothing happened in the field for more than a hundred years.
- In 1847, Kirchhoff developed the theory of trees while working on electrical networks.
- Shortly after, in the 1850s, Cayley discovered trees as he was trying to enumerate the isomers of saturated hydrocarbons C_nH_{2n+2} . Arthur Cayley (1821–1895) is one of the founding fathers of graph theory. We find his name anywhere there is graph data. More recently, [CayleyNet](#) uses complex rational functions called

Cayley polynomials for a spectral domain approach for deep learning on graph data.

- During the same time period, in 1850, Sir William Hamilton invented the game that became the basis of Hamiltonian circuits, and sold it in Dublin. We have a wooden, regular polyhedron with 12 faces and 20 corners; each face is a regular pentagon and 3 edges meet at each corner. The 20 corners have the names of 20 cities, such as London, Rome, New York, Mumbai, Delhi, Paris, and so on. We have to find a route along the edges of the polyhedron, passing through each of the 20 cities exactly once (a Hamiltonian circuit). The solution of this specific problem is easy, but until now we've had no necessary and sufficient condition for the existence of such a route in an arbitrary graph.
- Also during the same time period, at a lecture by Möbius (1840s), a letter by De Morgan (1850s), and in a publication by Cayley in the first volume of the Proceedings of the Royal Geographic Society (1879), the most famous problem in graph theory (solved in 1970), the four color theorem, came to life. This has occupied many mathematicians since then, leading to many interesting discoveries. It states that four colors are sufficient for coloring any map on a plane such that the countries with common boundaries have different colors. The interesting thing is that if we give ourselves more space by moving out of the flat plane, for example to the surface of a sphere, then we do have solutions for this conjecture.
- Unfortunately, nothing happened for another 70 years or so, until the 1920s when König wrote the first book on the subject and published it in 1936.
- Things changed with the arrival of computers and their increasing ability to explore large problems of combinatorial nature. This spurred intense activity in both pure and applied graph theory. There are now thousands of papers and dozens of books on the subject, with significant contributors such as Claude Berge, Oystein Ore, Paul Erdős, William Tutte, and Frank Harary.

Main Considerations in Graph Theory

Let's organize the main topics in graph theory and aim for a bird's-eye view without diving into details:

Spanning Trees and Shortest Spanning Trees

These are of great importance and are used in network routing protocols, shortest path algorithms, and search algorithms. A spanning tree of a graph is a subgraph that is a tree (any two vertices can be connected using one unique path) including all of the vertices of the graph. That is, spanning trees keep the vertices of a graph together. The same graph can have many spanning trees.

Cut Sets and Cut Vertices

We can break any connected graph apart, disconnecting it, by cutting through enough edges, or sometimes by removing enough vertices. If we are able to find these cut sets in a given graph, such as in a communication network, an electrical grid, a transportation network, or others, we can cut all communication means between its disconnected parts. Usually we are interested in the smallest or minimal cut sets, which will accomplish the task of disconnecting the graph by removing the least amount of its edges or vertices. This helps us identify the weakest links in a network. In contrast to spanning trees, cut sets separate the vertices, as opposed to keeping all of them together. Thus, we would rightly expect a close relationship between spanning trees and cut sets. Moreover, if the graph represents a network with a source of some sort, such as fluid, traffic, electricity, or information, and a sink, where each edge allows only a certain amount to flow through it, then there is a close relationship between the maximum flow that can move from the source to the sink and the cut through the edges of the graph that disconnects the source from the sink, with minimal total capacity of the cut edges. This is the *max-flow min-cut theorem*, which states that in a flow network, the maximum amount of flow passing from the source to the sink is equal to the total weight of the edges in a minimal cut. In mathematics, when a maximization problem (max flow) becomes equivalent to a minimization problem (min cut) in a nontrivial way (for example, by not just flipping the sign of the objective function), it signals duality. Indeed, the max-flow min-cut theorem for graphs is a special case of the duality theorem from linear optimization.

Planarity

Is the geometric representation of a graph planar or three-dimensional? That is, can we draw the vertices of the graph and connect its edges, all in one plane, without its edges crossing each other? This is interesting for technological applications such as automatic wiring of complex systems, printed circuits, and large-scale integrated circuits. For nonplanar graphs, we are interested in properties such as the thickness of these graphs and the number of crossings between edges. An equivalent condition for a planar graph is the existence of a *dual graph*, where the relationship between a graph and its dual becomes clear in the context of the vector space of a graph. Linear algebra and graphs come together here, where algebraic and combinatoric representations answer questions about geometric figures and vice versa. For the planarity question, we need to consider only simple, nonseparable graphs whose vertices all have three degrees or more. Moreover, any graph with a number of edges larger than three times the number of its vertices minus six is nonplanar. There are many unsolved problems in this field of study.

Graphs as Vector Spaces

It is important to understand a graph as both a geometric object and an algebraic object, along with the correspondence between the two representations. This is the case for graphs. Every graph corresponds to an e dimensional vector space over the field of integers modulo 2, where e is the number of edges of the graph. So if the graph only has three edges $edge_1, edge_2, edge_3$, then it corresponds to the three-dimensional vector space containing the vectors $(0,0,0), (1,0,0), (0,1,0), (1,1,0), (1,0,1), (0,1,1), (0,0,1), (1,1,1)$. Here $(0,0,0)$ corresponds to the null subgraph containing none of the three edges, $(1,1,1)$ corresponds to the full graph containing all three edges, $(0,1,1)$ corresponds to the subgraph containing only $edge_2$ and $edge_3$, and so on.



Field of Integers Modulo 2

The field of integers modulo 2 only contains the two elements 0 and 1, with the operations $+$ and $\times$ both happening modulo 2. These are in fact equivalent to the logical operations *xor* (exclusive or operator) and *and* in Boolean logic. A vector space has to be defined over a field and has to be closed under multiplication of its vectors by *scalars* from that field. In this case, the scalars are only 0 and 1, and the multiplication happens modulo 2. Graphs are therefore nice examples of vector spaces over finite fields, which are different than the usual real or complex numbers. The dimension of the vector space of a graph is the number of edges e of the graph, and the total number of vectors in this vector space is 2^e . We can see here how graph theory is immediately applicable to switching circuits (with on and off switches), digital systems, and signals, since all operate in the field of integers modulo 2.

With this simple correspondence, and backed by the whole field of linear algebra, it is natural to try to understand cut sets, circuits, fundamental circuits, spanning trees, and other important graph substructures, and the relationships among them, in the context of vector subspaces, basis, intersections, orthogonality, and dimensions of these subspaces.

Realizability

We have already used the adjacency matrix and the incidence matrix as matrix representations that completely describe a graph. Other matrices describe important features of the graph, such as the circuit matrix, the cut set matrix, and the path matrix. Then, of course, the relevant studies have to do with how all these relate to each other and interact.

Another very important topic is that of *realizability*: what conditions must a given matrix satisfy so that it is the circuit matrix of some graph?

Coloring and Matching

In many situations we are interested in assigning labels, or colors, to the nodes, edges of a graph, or even regions in a planar graph. The famous graph coloring problem is when the colors we want to assign to each node are such that no neighboring vertices get the same color; moreover, we want to do this using the minimal amount of colors. The smallest number of colors required to color a graph is called its *chromatic number*. Related to coloring are topics such as node partitioning, covering, and the chromatic polynomial.

A matching is a set of edges where no two are adjacent. A maximal matching is a maximal set of edges where no two are adjacent. Matching in a general graph and in a bipartite graph have many applications, such as matching a minimal set of classes to satisfy graduation requirements, or matching job assignments to employee preferences (this ends up being a max-flow min-cut problem). We can use random walk based algorithms to find perfect matchings on large bipartite graphs.

Enumeration

Cayley in 1857 was interested in counting the number of isomers of saturated hydrocarbon C_nH_{2n+2} , which led him to count the number of different trees with n nodes, and to his contributions to graph theory. There are many types of graphs to be enumerated, and many have been the topics of their own research papers. Examples include enumerating all rooted trees, simple graphs, simple digraphs, and others possessing specific properties. Enumeration is a huge area in graph theory. One important enumeration technique is Pólya's counting theorem, where one needs to find an appropriate permutation group and then obtain its cycle index, which is nontrivial.

Algorithms and Computational Aspects of Graphs

Algorithms and computer implementations are of tremendous value for anyone working with graph modeling. Algorithms exist for traditional graph theoretical tasks such as:

- Find out if graph is separable.
- Find out if a graph is connected.
- Find out the components of a graph.
- Find the spanning trees of a graph.
- Find a set of fundamental circuits.

- Find cut sets.
- Find the shortest path from a given node to another.
- Test whether the graph is planar.
- Build a graph with specific properties.

Graph neural networks nowadays come with their own open source packages. As always, for an algorithm to be of any practical use it must be efficient. Its running time must not increase factorially or even exponentially with the number of nodes of the graph. It should be polynomial time, proportional to n^k , where k is preferably a low number.

For anyone wishing to enter the field of graph modeling, it is of great use to familiarize yourself with both the theory and the computational aspects of graphs.

Summary and Looking Ahead

This chapter was a summary of various aspects of graphical modeling, with emphasis on examples, applications, and building intuition. There are many references for readers aiming to dive deeper. The main message is not to get lost in the weeds (and they are very thick) without an aim or an understanding of the big picture, the current state of the field, and how it relates to AI.

We also introduced random walks on graphs, Bayesian networks, and probabilistic causal models, which shifted our brain even more in the direction of probabilistic thinking, the main topic of [Chapter 11](#). It was my intention all along to go over all kinds of uses for probability in AI before going into a math chapter on probability ([Chapter 11](#)).

We leave this chapter with this very nice read: “[Relational Inductive Biases, Deep Learning, and Graph Networks](#)” (Battaglia et al. 2018), which makes the case for the deep learning community to adopt graph networks:

We present a new building block for the AI toolkit with a strong relational inductive bias—the graph network—which generalizes and extends various approaches for neural networks that operate on graphs, and provides a straightforward interface for manipulating structured knowledge and producing structured behaviors. We discuss how graph networks can support relational reasoning and combinatorial generalization, laying the foundation for more sophisticated, interpretable, and flexible patterns of reasoning.

Operations Research

Many scientists owe their greatness not to their skill in solving problems but to their wisdom in choosing them.

—E. Bright Wilson (1908–1992), American chemist

In this chapter, we explore the integration of AI into the field of operations research, leveraging the best of both worlds for more efficient and more informed decision making. Although this introductory statement sounds like an ad, it is precisely what operations research is all about. Advances in machine learning can only help move the field forward.

Operations research is one of the most attractive and stimulating areas of applied mathematics. It is the science of balancing different needs and available resources in the most time- and cost-efficient ways. Many problems in operations research reduce to searching for an optimal point, the holy grail, at which everything functions smoothly and efficiently: no backups, no interruptions to timely services, no waste, balanced costs, and good revenues for everyone involved. A lot of applications never find the holy grail, but many operations research methods allow us to come very close, at least for the simplified models of the complex reality. Constrained mathematical optimization penetrates every industry, every network, and every aspect of our lives. Done properly, we enjoy its benefits; done improperly, we suffer its impact: global and local economies are still experiencing the ramifications of COVID-19, the war on Ukraine, and the interruptions to the supply chain.

Before exploring how machine learning is starting to make its way into operations research, we highlight a few ideas that an interested person must internalize if they want to get involved in the field. Since we only have one chapter to spend on this beautiful topic, we must distill it into its essence:

The no free lunch theorem

This makes us shift our attention into devising and analyzing methods that work best for the special case scenario at hand, as opposed to looking for the most general and most widely applicable methods, like many mathematicians are naturally inclined to do. It essentially asks all these mathematicians to pretty much *chill*, and be satisfied with specialized solutions for specific types of problems.

Complexity analysis of problems and asymptotic analysis of algorithms

Asymptotic analysis tells us that even if the algorithm is ultra innovative and genius, it is useless if its computational requirements skyrocket with the size of the problem. Operations research solutions need to scale to big scenarios with many variables. *Complexity analysis*, on the other hand, addresses the *level of difficulty* of the problems themselves rather than the algorithms devised to tackle them. Combinatorial problems, which are $O(n!)$, are ultra bad: $n!$ is bigger than k^n for n large enough, but an exponential k^n complexity would already be very bad!

Important topics and applications in operations research

These we can find in any good book on operations research. We always need to keep one at hand. Moving from a specific application and business objectives to a mathematical formulation is the skill that cannot be stressed enough in order to thrive this field.

Various types of optimization methods and algorithms

This is the workhorse of operations research solutions and software packages.

Software packages

The wide availability of these, along with the limited number of pages, are my excuse not to elaborate on anything algorithmic or computational in this chapter.

To sum up operations research in six words: *mathematical formulation, optimization, algorithms, software, and decisions*.

When reading through this chapter, it is helpful to think of the concepts in the context of how the companies that we interact with in our daily lives manage their operations. Consider, for example, Amazon's logistics. Amazon is the largest ecommerce company in the world. Its share of the US ecommerce market in 2022 is 45%, selling and delivering millions of units of merchandise every day with around \$5,000 in sales every second. How does Amazon succeed at doing this? How does the company manage its inventory, warehouses, transportation, and extremely efficient delivery system? How does Amazon formulate its subproblems and integrate them into one big successful operation? Same with transportation logistics, such as Uber. Every day, Uber provides up to 15 million shared rides worldwide, matching available drivers with nearby riders, routing and timing pickups and drop-offs, pricing trips,

predicting driver revenues and supply-and-demand patterns, and performing countless analytics.

The complex and highly interconnected optimization problems that allow such massive systems to run relatively smoothly are typical to operations research. Moreover, a lot of the involved problems are NP-hard (in computational complexity, this means they have a nondeterministic polynomial time level of hardness; in English, *very expensive to compute*). Add to that their stochastic nature, and we have interesting math problems that need to be solved.

Overall, the mathematical methods and algorithms of operations research save the world billions of dollars annually. A survey of the largest 500 companies in the United States showed that 85% use linear programming (which is another name for linear optimization, a massive part of operations research, and a reason we spend some decent time on the simplex method and duality in this chapter). Coupled with tools from the AI industry, now is the perfect time to get into the field. The rewards will be on many levels: intellectual, financial, and a meaningful contribution to the greater good of humanity. So in no way should the few selected topics for this chapter dim the significance of other equally important topics in the field.

To dive deeper into operations research (after reading this chapter, of course), the best way is to learn from the best:

- Browse through the winner and finalist projects of the [Franz Edelman Award for Achievement in Advanced Analytics, Operations Research, and Management Science](#).
- Keep the book *Introduction to Operations Research*, by Frederick Hillier and Gerald Lieberman (McGraw Hill, 2021) close to your heart.

No Free Lunch

The *no free lunch theorem for optimization* states that there is no one particular optimization algorithm that works best for every problem. All algorithms that look for an optimizer of an objective function (cost function, loss function, utility function, likelihood function) have similar performance when averaged over all possible objective functions. So if some algorithm performs better than another on some class of objective functions, there are other objective functions where the other algorithm performs better. There is no superior algorithm that works for all kinds of problems. Therefore, picking an algorithm should be problem (or domain) dependent. Depending on our application area, there is plenty of information on which algorithms practitioners use, their justifications for why these are their chosen ones, their comparisons with others on both high-dimensional and reasonable-dimension

problems, and their constant attempts for better performance, based mostly on two criteria: speed (computationally not expensive), and accuracy (gives good answers).

Complexity Analysis and $O()$ Notation

Many times, the problem of *efficiently allocating limited resources under various constraints* boils down to devising efficient algorithms for *discrete* optimization. Linear programming, integer programming, combinatorial optimization, and optimization on graph structures (networks) are all intertwined (sometimes these are nothing more than two different names for the same thing) and deal with one objective: finding an optimizer from a *discrete and finite set* of valid options—the *feasible set*. If the feasible set is not discrete to start with, sometimes we can reduce it to a discrete set if we are to take advantage of the wealth of tools developed for this field. Here is the main issue: exhaustive search is usually not tractable. This means that if we list all the available options in the feasible set and evaluate the objective function at each of them, we would spend an ungodly amount of time to find the point(s) that give the optimal answer. No one said that a finite feasible means that it is not enormous. We need specialized algorithms that efficiently rule out large swaths of the search space. Some algorithms pinpoint the exact solution for some problems, while others can only find approximate solutions, which we have no option but to settle for.

Let's now make the following distinctions up front, since this confuses a lot of people:

Complexity analysis is for the problems that we want to solve (routing, traveling salesman, knapsack, etc.)

The intrinsic complexity of a problem is independent of the algorithms used to tackle it. In fact, it sometimes tells us that we cannot hope for a more efficient algorithm for such kinds of problems, or whether we can do better in other cases. In any case, complexity analysis for problems is a rich science on its own, and the field of operations research provides a wealth of *complex* problems to ponder on. This is where the following terms appear: polynomial problem, nondeterministic polynomial problem, nondeterministic polynomial complete problem, nondeterministic polynomial time hard problem, complement nondeterministic polynomial problem, and complement nondeterministic polynomial complete problem. Those terms are so confusing that someone seriously needs to reconsider their nomenclature. We will not define each here (mainly because the theory is not yet set on the boundaries between these classes of problems), but we will make the following divide: problems that can be solved in polynomial time or less, versus problems for which we cannot find an exact solution in polynomial time, no matter what algorithm is used, in which case we have to settle for approximation algorithms (for example, the traveling salesman problem). Note that sometimes polynomial time problems might not be such a great thing, because, for example, $O(n^{2000})$ is not so fast after all.

Asymptotic analysis is for the algorithms that we design to solve these problems

This is where we attempt to estimate the number of operations that the algorithm requires and quantify it relative to the size of the problem. We usually use the big O notation.



Big $O()$ Notation

A function $g(n)$ is $O(f(n))$ when $g(n) \leq cf(n)$ for some constant c , and for all $n \geq n_0$.

For example, $2n+1$ is $O(n)$, $5n^3 - 7n^2 + 1$ is $O(n^3)$, $n^2 2^n - 55n^{100}$ is $O(n^2 2^n)$, and $15n \log(n) - 5n$ is $O(n \log(n))$.

Do not forget the constant asymptotics case $O(1)$, where the operation count of an algorithm is independent of the size of the problem (awesome thing, because this means that it scales without any worries of enormous problems).

For some algorithms, we can count the exact number of operations; for example, to compute the scalar product (dot product) of two vectors of length n , a simple algorithm uses exactly $2n-1$ multiplications and additions, which makes it $O(n)$. For multiplying two matrices each of size $n \times n$, a simple algorithm computing the dot product of each row from the first matrix with each column from the second matrix requires exactly $(2n-1)n^2$ operations, so this will be $O(n^3)$. Matrix inversion is also usually $O(n^3)$.

For anyone interested in asymptotic analysis for algorithms, it quickly becomes obvious that it is slightly more involved than operation counts, because sometimes we have to make estimates or averages on the size of input (what does n stand for?), how to count the operations in an algorithm (by each line of code?), and we cannot ignore the fact that doing computations on large numbers is more consuming in time and memory than doing operations on smaller numbers. Finally, we prefer algorithms that run in polynomial time or less and not in exponential time or more. Let's demonstrate with a very simple example.

Polynomial Algorithm $O(n^k)$ Versus Exponential Algorithm $O(k^n)$

Suppose we are working on a machine that is able to execute 10^7 operations per second (10 million). Let's run it for 1,000 seconds, which is around 16 minutes on 2 different algorithms, one exponential in the size of the problem, say, $O(2^n)$ and the other polynomial $O(n^3)$. The size of the problem is n in this case, and refers to a measure of the dimensionality of the input, for example, the number of nodes of a graph, the number of entries of a matrix, the number of features of a data set, or

the number of instances. What is the largest size of problem that each algorithm can handle for 16 minutes on this machine?

For the exponential time algorithm, the number of operations it requires is at most (worst-case scenario) $c2^n = 10^7 * 1000$ for some preferably small c . Thus, the size of the problem it can run for 1,000 seconds with 10 million operations per second is $n = 10 \log_2(10) - \log_2(c) \approx 33$.

Now contrast this with the polynomial time algorithm, whose worst case is $cn^3 = 10^7 * 1000$, so $n = \frac{1}{\sqrt[3]{c}} \sqrt[3]{10^{10}} \approx 2100$. This is almost two orders of magnitude larger than the exponential time algorithm.

The conclusion is that given the same hardware and amount of time, polynomial time algorithms $O(n^k)$ allow us to solve much larger problems than exponential time algorithms $O(2^n)$. Combinatorial time algorithms $O(n!)$ are hopeless. Moreover, we *always* want k to be small. The smaller the better.

A person who is used to operating in *exact* realms and not in *approximate* or *asymptotic* realms might be troubled by this discussion, because sometimes, some higher-order algorithms are better for smaller size problems than lower order ones. For example, suppose the exact operation count of an $O(n)$ algorithm is $20n - 99$, and that of an $O(n^2)$ is $n^2 + 1$, then it is true that asymptotically (or for large enough n), the $O(n)$ algorithm is better than the $O(n^2)$, but that is not the case if n is smaller than 10, because in this case, $n^2 + 1 < 20n - 99$. This is OK for small enough problems, but never for larger problems.

Two optimization methods that we will soon mention in this chapter are the *simplex* method and the *interior point* method for linear optimization (optimization where both the objective function and the constraints are linear). The interior point method is a polynomial time algorithm and the simplex method is exponential time, so you would expect that everyone would use the cheaper interior point and abandon simplex, but this is not true. The simplex method (and the dual simplex) is still widely used for linear optimization instead of interior point because that exponential time is a worst-case scenario and most applications are not worst-case. Moreover, there are usually trade-offs between algorithms in terms of computational effort per iteration, number of iterations required, the effect of better starting points, whether the algorithm converges or will need extra help near the end, how much computation this extra help would require, and can the algorithm take advantage of parallel processing? For this reason, computer packages for linear optimization have efficient implementations of both the simplex and the interior point methods (and many other algorithms as well). Ultimately, we choose what works best for our use cases.

Optimization: The Heart of Operations Research

We found our way back to optimization. In machine learning, optimization is about minimizing the loss function for models that learn deterministic functions, or maximizing the likelihood function for models that learn probability distributions. We do not want a solution that matches the data exactly, since that would not generalize well to unseen data. Hence the regularization methods, early stopping, and others. In machine learning, we use the available data to learn the model: the deterministic function or the probability distribution that is the source of the data (the data-generating rule or process), then we use this learned function or distribution to make inferences. Optimization is just one step along the way: minimize the loss function, with or without regularization terms. The loss functions that appear in machine learning are usually differentiable and nonlinear, and the optimization is unconstrained. We can add constraints to *guide* the process into some desired realm, depending on the application.

Methods for optimization can either include computing derivatives of the objective function $f(\vec{x})$, such as machine learning's favorite gradient descent (stochastic gradient descent, ADAM, etc.), or not. There are optimization algorithms that are derivative free. These are very useful when the objective function is not differentiable (such as functions with corners) or when the formula of the objective function is not even available. Examples of derivative-free optimization methods include Bayesian search, Cuckoo search, and genetic algorithms.

Optimization, in particular *linear optimization*, has been at the heart of operations research since the Second World War, when methods for linear optimization such as the *simplex method* were developed to aid in military logistics and operations. The goal, as always, is to minimize an objective function (cost, distance, time, etc.) given certain constraints (budget, deadlines, capacity, etc.):

$$\min_{\text{constraints}} f(\vec{x})$$

To learn optimization for operations research, a typical course usually spends a lot of time on linear optimization, integer optimization, and optimization on networks (graphs), since many real-life logistics and resource allocation problems fit perfectly into these formulations. To become thriving operations researchers, we need to learn:

Linear optimization

This is where both the objective function and the constraints are linear. Here we learn about the simplex method, duality, **Lagrangian relaxation**, and sensitivity analysis. In linear problems, the boundaries of our world are flat, made of lines, planes, and hyperplanes. This (hyper)polygonal geometry, or *polyhedron*, usually has corner points that are candidates for being optimizers, so we devise

systematic ways to sift through these points and test them for optimality (this is what the simplex method and the dual simplex method do).

Interior point methods

For large-scale linear optimization problems that could be beyond the reach of the simplex method. In short, the simplex method goes around the *boundary* of the feasible search space (the edges of the polyhedron), checks each corner it arrives at for optimality, then moves to another corner at the boundary. The interior point method, on the other hand, goes *through the interior* of the feasible search space, arriving at an optimal corner from the inside of the feasible search space, as opposed to from the boundary.

Integer programming

Optimization where the entries of the optimizing vector must all be integers. Sometimes they can only be zero or one (send the truck to the warehouse in Ohio or not). The **knapsack problem** is a very simple prototype example. Here we learn about the branch and bound method for large integer programming problems.

Optimization on networks

We can reformulate many network problems as linear optimization problems where the simplex methods and specialized versions of it work, but it is much better to exploit the network structure and tap into useful results from graph theory, such as the max-flow min-cut theorem, for more efficient algorithms. Many problems on networks boil down to optimizing for one of the following: shortest path on the network (path from one node to another with minimum distance or minimum cost), minimum spanning tree of a network (this is great for optimizing *the design* of networks), maximum flow (from origin to destination or from source to sink), minimum cost flow, multicommodity flow, or traveling salesman (finding the minimum cost [or distance or weight] cyclic route that passes through all the network's nodes only once [Hamiltonian circuit]).

Nonlinear optimization

The objective function and/or the constraints are nonlinear. One recurring example throughout this book is minimizing nonlinear loss functions for machine learning models. These are always nonlinear, and we commonly use gradient descent-type algorithms. For smaller problems we can use Newton type algorithms (second derivatives). In operations research, nonlinearities in the objective function and/or constraints might appear because the cost of shipping goods from one location to another might not be fixed (for example, depends on the distance or on the quantity), or a flow through a network might include losses or gains. A special type of nonlinear optimization that we know a lot about is *quadratic* optimization with linear constraints. This appears in applications such as network equations for electric circuits, and elasticity theory for structures

where we consider displacements, stresses, strains, and balance of forces in a structure. Think of how easy it is to find the minimum of the quadratic function $f(x) = sx^2$, where s is a positive constant. This ease translates nicely to higher dimensions, where our objective function looks like $f(\vec{x}) = \vec{x}^t S \vec{x}$, where S is a positive semidefinite matrix, playing the same role for high dimensions as a positive constant for one dimension. Here we even have duality theory that we can take advantage of, similar to the linear optimization case. In optimization, when we lose linearity, we hope our functions are quadratic and our constraints are linear. When we lose that, we hope our functions and/or feasible set are convex. When we lose convexity, we are on our own, hoping our methods don't get stuck at the local minima of high-dimensional landscapes, and somehow find their way to optimal solutions.

Dynamic programming and Markov decision processes

Dynamic programming has to do with projects with multiple stages, where decisions have to be made at each stage, and each decision generates some immediate cost. The decision at each stage has to do with the current state, together with a policy to transition to the next state (choose the next state via a minimization of a deterministic function or a probability). Dynamic programming is all about devising efficient ways, usually recursive methods, to finding the *optimal sequence of interrelated decisions* to fulfill a certain goal. The idea is to avoid having to list all the options for each stage of the decision process, then selecting the best combination of decisions. Such an exhaustive search is extremely expensive for problems with many decision stages, each having many states. Now if the transition policy from one stage to the other is probabilistic rather than deterministic, and if the stages of the decision process continue to recur indefinitely, meaning if the project has an infinite number of stages, then we have a Markov decision process (or Markov chain) on our hands. This is a process that evolves over time in a probabilistic manner. A very special property of a Markov decision process is that the probabilities involving how the process evolves in the future are independent of past events, and depend only on the system's *current state*. Both discrete time and continuous time Markov chains model important systems, such as **queuing systems**, dynamic traffic light control to minimize car waiting time, and flexible call center staffing. The important math objects are the transition matrices, and we solve for the steady state probabilities. They end up having to compute the eigenspace of the transition matrix.

Stochastic algorithms

Dynamic programming with probabilistic transition policy and Markov chain are both examples of stochastic algorithms. So are stochastic gradient descent and random walks on graphs. Any algorithm that involves an element of randomness is stochastic. The mathematics transitions to the language of

probabilities, expectations, stationary states, convergence, etc. Another example where stochastic algorithms and analysis of processes appear is *queuing theory*, such as queues at a hospital emergency room or at a ship maintenance yard. This builds on probability distributions of arrival times of customers and service times by the service facility.

Metaheuristics

For many optimization problems, finding the optimal solution might be impractical, so we (who still need to make decisions) resort to *heuristic methods* to find an *answer* (I will not call it a solution), which is not necessarily optimal but is good enough for the problem at hand. Metaheuristics are general solution methods that provide strategy guidelines and general frameworks for developing heuristic methods to fit certain families of problems. We cannot guarantee the optimality of an answer from a heuristic method, but heuristics do speed up the process of finding satisfactory solutions where optimal solutions are too expensive to compute or are out of reach. There is also the topic of *satisfiability*. Since problems in operations research are almost always constrained, the natural question is: are the constraints satisfiable? Meaning, is the feasible set nonempty? Some operations research problems get reformulated as satisfiability problems.

In real-world problems, a big part of the work of operations research departments is formulating their specific use cases and objectives in a way that can fit into one of these optimization frameworks. Here it is important to recognize special structures (such as sparsity in the involved matrices) or substructures that we can exploit for more efficient algorithms. This is crucial for complicated and large-scale systems.

Thinking About Optimization

When we encounter an optimization problem in mathematics:

$$\min_{\vec{x} \in \text{some feasible set}} f(\vec{x})$$

where the feasible set is defined by some constraints that the vector $\vec{x}$ must satisfy (or it could be totally unconstrained), we usually pause and brainstorm a little:

- Is $f(\vec{x})$ linear?
- Is $f(\vec{x})$ convex? Bounded below?
- Is the minimum value finite, or does it $\rightarrow -\infty$?
- Is the feasible set nonempty? Meaning are there $\vec{x}$'s that actually satisfy the constraints?

- Is the feasible set convex?
- Does a minimizer exist?
- Is a minimizer unique, or are there others?
- How do we find the minimizer?
- What is the value of the minimum?
- How much does the minimizer and the value of the minimum change if something changes in our constraints or in our objective function?

Depending on the type of problem at hand, we might be able to answer these questions independently, meaning sometimes we can answer only some of them and not others. This is fine because any information about the optimizer and the value of the optimum is valuable.

Let's explore common types of optimization problems.

Optimization: Finite Dimensions, Unconstrained

This is similar to the optimization that we do in calculus classes, and the optimization we do when training a machine learning model, minimizing the loss function. The objective function $f(\vec{x})$ is differentiable:

$$\min_{\vec{x} \in \mathbb{R}^d} f(\vec{x})$$

In unconstrained and differentiable optimization, the minimizer $\vec{x}^*$ satisfies $\nabla f(\vec{x}) = 0$. Moreover, the Hessian (matrix of second derivatives) is positive semi-definite at $\vec{x}^*$. When discussing optimization for machine learning, we settled on stochastic gradient descent and its variants for very high-dimensional problems. For smaller problems, Newton-type (working with second derivatives, not only first ones) methods work as well. For very few problems, such as the mean squared error loss function for linear regression, we can get analytical solutions. Examples where we can get analytical solutions are usually carefully constructed (such as all the examples in our calculus books), and very low dimensional.

Optimization: Finite Dimensions, Constrained Lagrange Multipliers

Let's think of the case where we only have one constraint $g(\vec{x}) = b$. This explains what we need rather well. The minimization problem looks like:

$$\min_{\substack{g(\vec{x}) = b \\ x \in \mathbb{R}^d}} f(\vec{x})$$

If $f(\vec{x})$ and $g(\vec{x})$ are differentiable functions from $\mathbb{R}^d \rightarrow \mathbb{R}$, we can introduce Lagrange multipliers (a method from 1797) to change our problem into an *unconstrained* one, but in higher dimensions (corresponding to the new Lagrange multipliers that we introduce to the optimization problem). Nothing is free. In this case, we add a multiple of our constraint to the objective function, then minimize, which means look for the points where the gradient is zero. The new objective function for the unconstrained problem is called the Lagrangian, and it is a function of both the decision vector $\vec{x}$ and the new variable λ , which we multiplied by our constraint, called the *Lagrange multiplier*:

$$\mathcal{L}(\vec{x}; \lambda) = f(\vec{x}) + \lambda(b - g(\vec{x}))$$

If we have more than one constraint, say five constraints, then we introduce a Lagrange multiplier for each, adding five extra dimensions to our optimization problem to move it from the constrained regime to the unconstrained one.

The optimizer $(\vec{x}^*, \lambda^*)$ of the unconstrained problem must satisfy: $\nabla \mathcal{L}(\vec{x}; \lambda) = 0$. We go about finding it the same way we go about general unconstrained problems (see the previous case). The $\vec{x}^*$ from $(\vec{x}^*, \lambda^*)$ is the solution of the constrained problem that we were originally searching for. This means that it is the point on the hypersurface defined by the constraint $g(\vec{x}) = b$ where the value of f is smallest.

If the problem has a special structure that we can exploit, such as if f is quadratic and the constraint g is linear, or if both f and g are linear, then we have more convenient methods to go about this constrained optimization, both if we decide to use Lagrange multipliers (which introduce duality) and without using Lagrange multipliers. Luckily, optimization problems with simple structures are very well studied, not only because they make the mathematics and computations easier, but also because they appear all the time in science and in real-life applications, which gives some credibility to my theory that nature is simpler than mathematicians think it is. We will revisit Lagrange multipliers for constrained problems in the section on duality, where we focus solely on fully linear problems or quadratic problems with linear constraints.

The meaning of Lagrange multipliers

The nice thing that we should make a permanent mental note of is that the Lagrange multiplier λ is not some worthless auxiliary scalar that helps us change a constrained problem into an unconstrained one. It has a meaning that is very helpful for sensitivity analysis, for finance and operations research applications, and for duality theory (which are all related to each other). Mathematically, by observing the formula of the Lagrangian $\mathcal{L}(\vec{x}; \lambda) = f(\vec{x}) + \lambda(b - g(\vec{x}))$, λ is the rate of change of the Lagrangian as a function of b , if we were allowed to vary b (the value of the constraint in applications we care about the effect of pushing or relaxing the constraints). That is:

$$\frac{\partial \mathcal{L}(\vec{x}; \lambda, b)}{\partial b} = \frac{\partial f(\vec{x}) + \lambda(b - g(\vec{x}))}{\partial b} = \frac{\partial f(\vec{x})}{\partial b} + \frac{\lambda(b - g(\vec{x}))}{\partial b} = 0 + \lambda = \lambda$$

Moreover, we can interpret the optimal value λ^* corresponding to the optimizer $\vec{x}^*$ as the *marginal effect of b* on the optimal attainable value of the objective function $f(\vec{x}^*)$. Hence, if $\lambda^* = 2.1$, then increasing b by one unit (pushing the constraint by one unit) will increase the optimal value of f by 2.1 units. This is very valuable information for applications in finance and operations research. Let's see why this is the case. We want to prove that:

$$\frac{df(\vec{x}^*(b))}{db} = \lambda^*$$

Note that two things happen at the optimizer $\vec{x}^*(b)$, which we get when we set the gradient of the Lagrangian to zero: $\nabla f(\vec{x}^*(b)) = \lambda^* \nabla g(\vec{x}^*(b))$, and $g(\vec{x}^*(b)) = b$. Using this information and the chain rule for derivatives (go back to your calculus book and master the chain rule, we use it all the time), we now have:

$$\begin{aligned} \frac{df(\vec{x}^*(b))}{db} &= \nabla f(\vec{x}^*(b)) \cdot \frac{d\vec{x}^*(b)}{db} = \lambda^* \nabla g(\vec{x}^*(b)) \frac{d\vec{x}^*(b)}{db} = \lambda^* \frac{dg(\vec{x}^*(b))}{db} = \lambda^* \frac{db}{db} = \lambda^* \times 1 \\ &= \lambda^* \end{aligned}$$

In other words, the Lagrange multiplier λ^* is the rate of change of the optimal cost (value of the objective function) due to the relaxation of the corresponding constraint. In economics, λ^* is called the *marginal cost* with respect to the constraint, or the *shadow price*. When we discuss duality later in this chapter, we use the letters p for the decision variables of the dual problem for this *price* reason.

Optimization: Infinite Dimensions, Calculus of Variations

The field of calculus of variations is an optimization field, but instead of searching for optimizing *points* in finite dimensional spaces, we are searching for optimizing *functions* in infinite dimensional spaces.

A finite dimensional function from $\mathbb{R}^d \rightarrow \mathbb{R}$ looks like $f(\vec{x}) = f(x_1, x_2, \dots, x_d)$, and its gradient (which is always important for optimization) is:

$$\nabla f = \begin{pmatrix} \frac{\partial f}{\partial x_1} \\ \frac{\partial f}{\partial x_2} \\ \vdots \\ \frac{\partial f}{\partial x_d} \end{pmatrix}$$

Its directional derivative measures the change of f , or its *variation*, in the direction of some vector $\vec{n}$:

$$\frac{\partial f(\vec{x})}{\partial \vec{n}} = \lim_{h \rightarrow 0} \frac{f(\vec{x} + h\vec{n}) - f(\vec{x})}{h} = \nabla f \cdot \vec{n}$$

Now if we allow $\vec{x}$ to depend on time, then we have:

$$f'(\vec{x}(t)) = \frac{df(\vec{x}(t))}{dt} = \nabla f \cdot \frac{d\vec{x}(t)}{dt} = \nabla f \cdot \vec{x}'(t)$$

That expression comes in handy when calculating *variations* of *infinite dimensional functionals*. A functional is a function whose input is a function and whose output is a real number. Thus, for an infinite dimensional functional $E(u)$: some function space $\rightarrow \mathbb{R}$ maps a function u that lives in *some function space* to a real number.

One example of a functional is the integral of a continuous function on the interval $[0,1]$. Another popular example is this integral:

$$E(u(x)) = \int_0^1 u(x)^2 + u'(x)^2 dx$$

For instance, this functional maps the function x^2 to the number $5/3$, which is the value of the integral.

The analog for that finite dimensional time derivative expression (when we allow dependence on time), is now:

$$\frac{dE(u(t))}{dt} = \nabla E \cdot \frac{du(t)}{dt} = \nabla E \cdot u'(t) = \langle \nabla E, u'(t) \rangle_{\text{some function space}}$$

This is very useful. It usually helps us pinpoint the infinite dimensional gradient ∇E , if in our calculations we manage to isolate the quantity multiplied by $u'(t)$. These calculations usually involve integral expressions, and the *product* is usually defined in some *infinite dimensional* sense, which also involves integral expressions. This is why we use the product notation of $\langle \nabla E, u'(t) \rangle$ instead of the usual dot.

We will work through one example to demonstrate a product for functions that live in the infinite dimensional space $L^2(D)$, which contains all the functions $u(x)$ with a finite $\int_D |u(x)|^2 dx$, and the gradient of a functional in the $L^2(D)$ sense.

Analogy between optimizing functions and optimizing functionals

It is good to keep in mind the analogy between finite and infinite dimensional formulations, as everything in math usually ties neatly together. At the same time we must be cautious, as the transition to the infinite is massive, and many finite dimensional properties and methods do not make it through.

In finite dimensions, the *optimizing point* or points satisfy an equation based on setting the gradient of the *objective function* (alternatively in this book: loss function, cost function, or utility function) equal to zero.

In infinite dimensions, the *optimizing function* or functions satisfy a *differential equation* based on setting the gradient of the *objective functional* equal to zero, that is, given that we somehow manage to define the gradient of a *functional*. To find the optimizer, we either have to solve this differential equation, called the *Euler-Lagrange equation*, or follow some optimization scheme on the landscape of the functional. It is *impossible to visualize* the landscape of an infinite dimensional functional, so ironically, we end up visualizing it as only one-dimensional, where the x-axis represents the function space u , and the y-axis represents $E(u)$.

The gradient descent that we use extensively in finite dimensional machine learning is an example of an optimization scheme. The same idea of the gradient descent applies to infinite dimensions: follow the direction of the steepest increase (if maximizing) or decrease (if minimizing).

Of course, we need to define what the gradient means for functionals defined on infinite dimensional spaces. It turns out there are many ways we can define gradients, depending on what spaces the involved functions live in (such as space of all continuous functions, space of all functions that have one continuous derivative, space of

functions whose integral of their square is finite, and many others). The meaning of the gradient remains the same: it measures the *variation* of the functional in a certain sense, just like the gradient of a finite dimensional function measures the variation (change) of the function in a certain direction.



You can skip the rest of this section if you are not interested in differential equations, as this section is not essential to operations research. The only purpose of it is to explore transitioning into infinite dimensions, and to see how we can obtain differential equations when we minimize formulas containing integrals (the functional formulas). For more details on the following examples, check the PDF file on calculus of variations on this [book's GitHub page](#).

Example 1: Harmonic functions, the Dirichlet energy, and the heat equation

A *harmonic function* is a function whose sum of all of its second derivatives is zero, for example, in two dimensions: $\Delta u = u_{xx} + u_{yy} = 0$. Example functions include $e^x \sin(y)$ and $x^2 - y^2$. In real life, these types of functions appear in electrostatics when modeling electrostatic potentials and charge density distributions, or when modeling rhythmic motions, such as a string undergoing a rhythmic periodic motion, or a frictionless pendulum oscillating indefinitely.

A harmonic function minimizes the Dirichlet energy. Instead of trying to look for functions whose second derivatives add up to zero (there are very well-established ways to do this for $\Delta u = 0$) and satisfying certain boundary conditions, there is another nice way to think about a harmonic function as the minimizer of an energy functional, called the Dirichlet energy functional:

$$E(u(x)) = \int_D \frac{1}{2} |\nabla u(x)|^2 dx$$

Here, $u(x)$ belongs to an appropriate function space that guarantees that the integral is finite, and $u(x)=h(x)$ is specified on the boundary ∂D of the domain D .

When we set the gradient of this energy functional equal to zero, to find its minimizer, we get the (Euler-Lagrange) equation $\Delta u = 0$, $u=h(x)$ on ∂D , which is exactly the differential equation a harmonic function satisfies. Let's see how this happens:

$$\begin{aligned}
E'(u(x)) &= \int_D \frac{1}{2} (|\nabla u(x)|^2)' dx \\
&= \int_D \nabla u(x) \nabla u'(x) dx \\
&= - \int_D \Delta u(x) u'(x) dx + \int_D \nabla u(x) \cdot \vec{n} u'(x) ds \\
&= - \int_D \Delta u(x) u'(x) dx + \int_D \frac{\partial u(x)}{\partial \vec{n}} u'(x) ds
\end{aligned}$$

We obtained the third equality using integration by parts, which moves the derivative from one factor in the integral to the other, picking up a negative sign and a boundary term in the process. The boundary term contains the two factors in the original integral, but without the derivative that was moved from one to the other.

The previous expression is true for any $u'(x)$ in our function space; in particular, it will be true for those with $u'(x) = 0$ on the boundary of the domain, which kills the integral term at the boundary and leaves us with:

$$E'(u(x)) = - \int_D \Delta u(x) u'(x) dx = \langle -\Delta u(x), u'(x) \rangle_{L^2(D)}$$

We just defined the *product* in the $L^2(D)$ function space as the integral of the usual product of the functions over the domain.

Now note that, analogous to the finite dimensional case, we have:

$$E'(u(x)) = \langle \nabla_{L^2(D)} E(u(x)), u'(x) \rangle_{L^2(D)}$$

Comparing the last two expressions, we notice that the gradient of the Dirichlet energy $E(u)$ in the $L^2(D)$ sense is $-\Delta u(x)$, and it is zero exactly for harmonic functions.

The heat equation does gradient descent for the Dirichlet energy functional. There is a bit more to this story. In nature, when a system happens to be evolving in time, the first question usually is: what's driving the evolution? A neat and intuitive way to answer this is that the system evolves in a way that *decreases some energy in the most efficient way*. It is usually hard to *discover* the formulas for these energy functionals, but if we happen to discover one, we might get a Ph.D. degree because we did, like what happened with me.

A simple example is the heat equation $u_t = \Delta u$, with $u(x,t)=0$ on ∂D and some initial condition $u(x,0)=g(x)$. This models the diffusion of heat, smoke, atoms on material surfaces, etc. It has time dependence naturally built into it. If we follow the

evolution in time of the solution $u(x,t)$ of the heat equation (which might represent temperature, solute concentration, gas in a room, etc.), then we notice that we are sliding on the landscape of the Dirichlet energy functional $E(u) = \frac{1}{2} \int_D |\nabla u(x,t)|^2 dx$, in the steepest descent direction, in the $L^2(D)$ sense, because, as we discussed:

$$u_t = \Delta u = -\nabla_{L^2(D)} E(u)$$

This means that starting initially at some $u(x,0) = g(x)$, the fastest way to arrive to the minimizer of the Dirichlet energy, which is the harmonic function, on the infinite dimensional landscape of the Dirichlet energy, is through solving the heat equation: follow the path of the initial $g(x)$ as it evolves with time.

In this sense, the heat equation gives a viable route to formulate a minimizing scheme for the problem:

$$\min_{u=0 \text{ on } \partial D} \frac{1}{2} \int_D |\nabla u(x,t)|^2 dx$$

Example 2: The shortest path between two points is along the straight line connecting them

The shortest path between two points in $\mathbb{R}^2$ is along the straight line connecting them. To do this, we minimize the arc length of the curve connecting two points (x_1, y_1) and (x_2, y_2) , namely:

$$\min_{y(x_1) = y_1 \text{ and } y(x_2) = y_2} \int_{x_1}^{x_2} \sqrt{1 + y'(x)^2} dx$$

Similar to the previous example, this problem is a minimization problem of a functional, which contains integral expressions of some functions and their derivatives.

To solve this, we write the Euler-Lagrange equation by setting the gradient of the functional equal to zero. This leads us to the minimizing function $y(x)=mx+b$, where m and b are, respectively, the slope and the y -intercept of the straight line connecting the two given points. The details of this example are in the PDF file on calculus of variations on this [book's GitHub page](#).

Other introductory examples to the calculus of variations

Other introductory examples to calculus of variations, which we can solve via minimizing an appropriate energy functional (via a variational principle) include the minimal surface problem and the isoperimetric problem.

Optimization on Networks

I wanted to start with optimization on networks *before* the simplex method for linear optimization because more people are used to thinking in terms of algebraic forms (equations and functions) than in terms of graph or network structures, despite the abundance of network structures in nature and operations research applications. We need to become very comfortable with graph models. Optimization problems on network structures tend to be combinatorial in nature, $O(n!)$, which is no bueno, so we need algorithms that somehow circumvent this and efficiently sift through the search space. (Remember, the order of a problem is usually a worst-case scenario, and in worst cases we suffice ourselves with approximate solutions,)

We discuss typical network problems, which happen to capture a wide variety of real-life applications. The traveling salesman problem is one of the oldest and most famous, so we start there. We live in an age where we have open source software packages and cloud computing resources that include powerful algorithms for solving all the problems mentioned in this chapter, so in this section we focus on understanding the type of the network problem and its applications instead of the algorithms devised to solve them.

Traveling Salesman Problem

This is a famous problem in operations research that fits into many real-world situations. A salesman is required to visit a number of cities during a trip. Given the distances between the cities, in what order should he travel so as to visit every city precisely once and return home, with the objective of keeping the distance traveled at a minimum (Figure 10-1)?

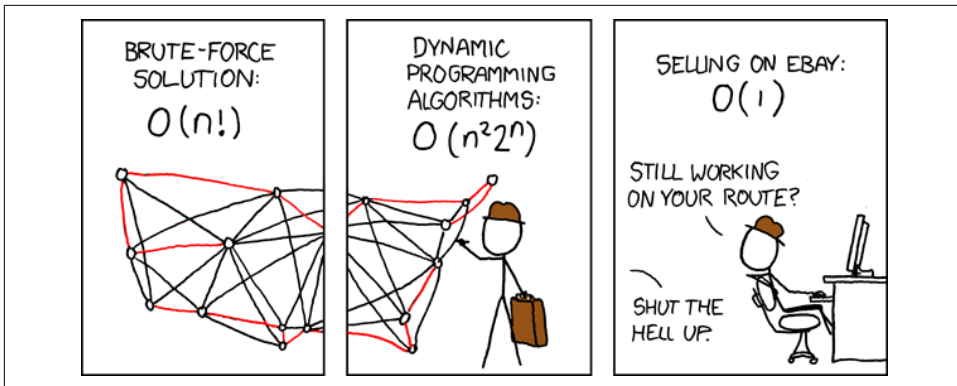


Figure 10-1. Traveling salesman problem ([image source](#))

Applications are numerous: a delivery truck leaving a warehouse must deliver packages to every address in the least costly way (measured by time or distance); or we

must find the most efficient hole sequence to drill on a printed circuit board when manufacturing electronic chips.

We represent the traveling salesman problem as an optimization problem on a graph: the cities are the nodes, there are edges between each pair of cities (making the graph complete), and each edge has a weight (or attribute or feature) representing the distance between the two cities. This graph has many paths passing through all the cities only once and returning to the one we started with (a *Hamiltonian circuit*), but we want the one with the smallest sum of distances.

Let's think of the complexity of this problem. The total number of different Hamiltonian circuits in a complete graph of n nodes is $(n-1)!/2$. Starting at any node, we have $n-1$ edges to choose from to pick the next city to visit, then $n-2$ options from the second city, $n-3$ from the third city, and so on. These choices are independent, so we have a total of $(n-1)!$ choices. We must divide by 2 to account for symmetry, in the sense that we can traverse the same Hamiltonian circuit forward or backward and still get the exact same total distance traveled. This counting problem is a circular permutation with symmetry. An exhaustive solution of the traveling salesman would list all $(n-1)!/2$ Hamiltonian circuits, adding up the distance traveled in each, then choosing the one with the shortest distance. Even for a reasonable value of n , it is too expensive; for example, to visit all 50 US state capitals (say we want to minimize total trip cost), we would need to try $(50 - 1)!/2 = 3.04 \times 10^{62}$ options! We do not have an efficient algorithm for problems of arbitrary size. Heuristic methods can provide excellent approximate solutions. Moreover, great algorithms based on an approach called *branch and cut* have solved this problem to optimality for very large numbers of cities.

Minimum Spanning Tree

I put the minimum spanning tree problem right after the traveling salesman because sometimes people confuse the two. This is a good place to clear the confusion. Here, we have a fully connected network with positive weights associated with each edge, which again can represent distance, time, capacity, or cost of connecting infrastructure such as water, electric, or phone lines. Similar to the traveling salesman problem, we want to find the set of edges that includes all the nodes of the graph and minimizes the total weight. The requirement here that is different than the traveling salesman is that we want to make sure we choose the set of edges in a way that provides a path between any two pairs of nodes, meaning we can reach any node in the graph from any other node. In the traveling salesman problem, we need to visit every city only once, then return to the starting city, which means that each node cannot get more than two edges (no such requirement for a spanning tree). The fact that we return to the last city in the traveling salesman problem means that we have an extra circuit closing edge that we do not need for spanning trees. If we remove that last edge of a traveling salesman solution, then we definitely get a spanning tree;

however, there is no guarantee that it is the one with minimal cost. **Figure 10-2** shows minimum spanning tree and traveling salesman solutions of the same graph.

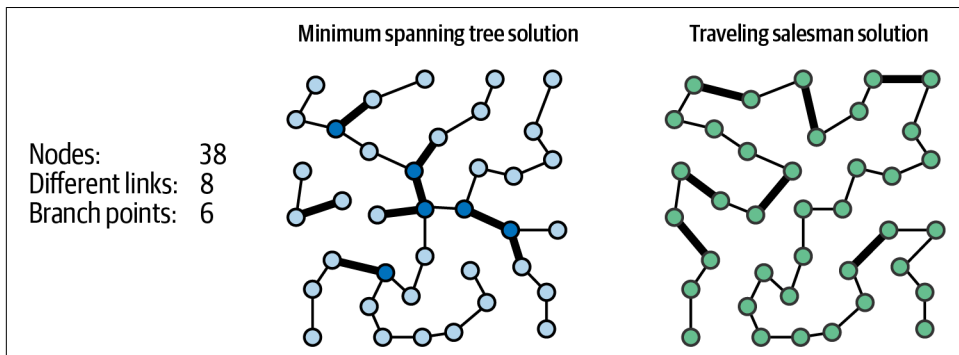


Figure 10-2. Minimum spanning tree and traveling salesman solutions of the same graph

Note that for any network, if we have n nodes then we only need $n-1$ edges so that we have a path between every two nodes, so we should never use more than $n-1$ edges for a minimal spanning tree because that would increase our cost. We need to choose the set of edges that minimizes the cost.

We have already mentioned some applications, such as designing telecommunication networks, routing and transportation networks, electric networks, and infrastructure networks (pipelines). These networks are expensive to develop, and designing them optimally saves millions of dollars.

Shortest Path

The simplest version of the shortest path problem is that we have two nodes on a graph and we want to connect them with a set of edges so that the total sum of the edge weights (distance, time) is minimal. This is different than the traveling salesman and the minimal spanning tree problems because we don't care about covering all the nodes of the graph. All we care about is getting ourselves from the origin to the destination in the least costly way.

One obvious application is travel from one destination to another with minimal distance, cost, time, etc. Other applications that are not immediately obvious but nevertheless are ultra-important are activity networks. Instead of an origin and a destination, we might have a beginning of a project and an end. Each node represents an activity, and each edge weight represents the cost or the time incurred if activity i is adjacent to activity j (if we have a directed graph, then it would be the cost or time incurred if activity i happens after activity j). The goal is to choose the sequence of activities that minimizes the total cost.

Other versions of the shortest path problem include finding the shortest path from an origin to *all other nodes*, or finding the shortest paths between *all pairs of nodes*.

Many vehicle routing algorithms and network design algorithms include shortest path algorithms as subroutines.

We can also formulate the shortest path problem as a linear optimization problem and use the methods available for linear optimization.

Max-Flow Min-Cut

Here we also have an origin and a destination, each *directed* edge has a capacity of some sort (max number of vehicles allowed on a route, max number of commodities shipped on a route, max amount of material or natural resource, such as oil or water, that a pipeline can handle), and we want to find the set of edges that maximizes the *flow* from the origin to the destination. Note that all edges point away from the origin and point toward the destination.

A very important theorem from graph theory plays a crucial role in determining the optimality (max flow) of a set of edges connecting the origin to the destination:

The *max-flow min-cut* theorem says the maximum flow from the origin to the destination through the directed network is equal to the minimal sum of weights of the edges required to cut any communication between the origin and the destination. That is, we can cut through the network to prevent communication between the origin and the destination in more than one way. The set of edges that cuts communication *and* has the least weight is the minimal cut set. The value of this minimal cut set is equal to the value of the maximum flow possible in the network. This result is pretty intuitive: what's the most that we can send through the edges of the network? This is bounded from above by the capacities of the edges crucial for connecting the origin to the destination.

We can reformulate the max flow problem as a linear optimization problem, and of course, the min cut problem will be its dual, so of course they have the same solution! We will see this soon in this chapter.

Finally, suppose we have more than one origin and more than one destination, similar to a distribution network. Then we can still maximize the flow through the network by solving the exact same problem, except now we add a fictional *super origin* pointing to all the real origins, and another fictional *super destination* that all the real destinations point toward, with infinite capacities, then do business as usual, solving for the max flow on this new graph with two new fictional *super nodes*.

Max-Flow Min-Cost

This is similar to the max flow problem, except that now we have a cost associated with sending a flow through each edge proportional to the number of units of flow. The goal is obviously to minimize the cost while satisfying the supply from all the origins to all the destinations. We can formulate this problem as a linear optimization problem and solve it using a simplex method optimized for networks. Applications are ubiquitous and so important: all kinds of distribution networks, with supply nodes, trans-shipment nodes, demand nodes, supply chains (of goods, blood, nuclear materials, food), solid waste management networks, coordinating the types of products to produce or spend resources to satisfy the market, cash flow management, and assignment problems, such as assigning employees to tasks, time slots to tasks, or job applicants to available jobs.



The Assignment Problem

The assignment problem is also called the matching problem. The number of assignees should be the same as the number of tasks, each can be assigned only one task, and each task can be performed by only one assignee. There is a cost to assigning task i to assignee j . The objective is to choose the matching between tasks and assignees that minimizes the total cost. The graph of such a problem is a special type called a bipartite graph. Such a graph can be divided into two parts, where all the edges go from one node in the first part to one node in the second part. An assignment problem where all the weights are the same is a max flow problem on a bipartite graph. All we have to do is assign a fictional super origin and another fictional super destination and solve the problem the same way we solve the max flow problem in the upcoming section on linear optimization and duality. There are many efficient algorithms for these problems.

The Critical Path Method for Project Design

The critical path method (CPM) is an optimization method on a network representing all the involved activities in a project, the total budget, the total time constraint, which ones need to happen before others, how much time and cost each activity incurs, and which activities can happen simultaneously. Think, for example, of a house construction project from start to finish. The critical path method for time and cost trade-offs is a great tool to aid in designing a project that incorporates trade-offs between time and cost, and make sure the project meets its deadlines at a minimal total cost. Similar to the critical path method is the Program Evaluation Review Technique (PERT), a project management planning tool that computes the amount

of time it will take to complete a project. Both methods provide three timelines: a shortest possible timeline, a longest possible timeline, and a most probable timeline.

The n-Queens Problem

Before moving on to linear optimization, the simplex method, and duality, we make a tiny detour and mention an interesting combinatorial problem that has puzzled mathematicians for 150 years, mainly because of its utter lack of structure: the n -queens problem, such as the one in [Figure 10-3](#). Michael Simkin [has finally \(July 2021\) answered the 150-year-old chess-based \$n\$ -queens problem](#). Here is an edited part of the abstract of his solution paper, titled “[The Number of n-Queens Configurations](#)”:

The n -queens problem is to determine the number of ways to place n mutually nonthreatening queens on an $n \times n$ [chess] board. We show that there exists a constant $\alpha = 1.942 \pm 3 \times 10^{-3}$ such that [the number of ways to place the mutually nonthreatening queens on the board is] $(1 \pm o(1))ne^{-\alpha})^n((1 \pm o(1))ne^{-\alpha})^n$. The constant α is characterized as the solution to a convex optimization problem in $P([-1/2, 1/2]^2)$, the space of Borel probability measures on the square.

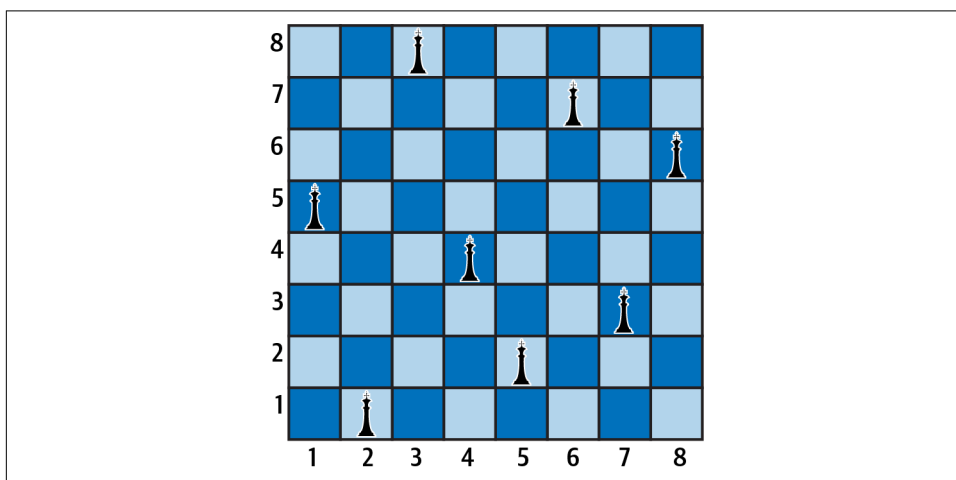


Figure 10-3. Eight queens in mutually nonthreatening positions on an 8×8 chessboard

This [web page](#) has an easy *backtracking* algorithm for solving the n -queens problem. Note that the solution by Simkin quantifies the total number of viable queen configurations, while algorithms only find one or some of these configurations.

Linear Optimization

Any optimization problem in finite dimensions, whether *linear* or *nonlinear*, looks like:

$$\begin{aligned} \min_{\vec{x}} \quad & f(\vec{x}) \\ g_1(\vec{x}) \leq & 0 \\ g_2(\vec{x}) \leq & 0 \\ \dots & \\ g_m(\vec{x}) \leq & 0 \end{aligned}$$

A point $\vec{x}$ that happens to satisfy all the constraints is a *feasible point*. We have the following cases:

There is only one optimal solution

Think that the landscape of the objective function has only one lowest point.

There are multiple optimal solutions

In this case, the set of optimal solutions can be bounded or unbounded.

The optimal value goes to $-\infty$

The landscape of the objective function goes downhill indefinitely, so no feasible point is optimal.

The feasible set is empty

We do not care about the objective function and its low values, since there are no points that satisfy all the constraints at the same time. The minimization problem has no solution.

The optimal value is finite but not attained

There is no optimizer, even when the feasible set is nonempty. For example, $\inf_{x \geq 0} \frac{1}{x}$ is equal to zero but there is no finite x such that $1/x = 0$. This never happens for linear problems.

For an optimization problem to be linear, both the objective function f and all the constraints g must be linear functions. Linear optimization gets the lion's share in operations research, since we can model many operations research problems as a minimization of a linear function with linear constraints that can either be equalities or inequalities.

The General Form and the Standard Form

Linearity is such a great thing, as it opens up all tools of linear algebra (vector and matrix computations). There are two forms of linear optimization problems that people usually work with:

The general form

This is convenient for developing the theory of linear programming. Here, there is no restriction on the signs of the decision variables (the entries of the vector $\vec{x}$):

$$\min_{A\vec{x} \geq \vec{b}} \left(\vec{c} \cdot \vec{x} \right)$$

The feasible set $A\vec{x} \geq \vec{b}$ is a **polyhedron**, which we can think of as the intersection of a finite number of half-spaces with flat boundaries. This polyhedron can be bounded or unbounded. We will see examples shortly.

The standard form

This is convenient for computations and developing algorithms, like the simplex and interior point methods. The decision variables must be nonnegative, so we are only searching for optimizers in the *first hyperoctant*, the high-dimensional analog of the first quadrant, where all the coordinates are nonnegative. Moreover, the constraints must always be equalities, not inequalities, so we are on the boundary of the polyhedron, not in the interior. This is a linear optimization problem written in standard form:

$$\begin{array}{l} \min_{\substack{\vec{x} \\ A\vec{x} = \vec{b} \\ \vec{x} \geq 0}} \left(\vec{c} \cdot \vec{x} \right) \end{array}$$

There is an easy way for us to intuitively understand a linear problem in standard form: synthesize the vector $\vec{b}$ from the columns of A in a way that minimizes the cost $\vec{c} \cdot \vec{x}$.

We can easily go back and forth between the standard form and general form of a linear optimization problem. For example, we can introduce surplus and slack variables to convert a general linear optimization problem to standard form, but note that in the process of doing that, we end up with the same problem in different dimensions. When we introduce a variable to change an inequality into an equality, such as introducing s_1 to convert the inequality $x_1 - 3x_2 \geq 4$ to the equality $x_1 - 3x_2 - s_1 = 4$, we increase the dimension (in this example from two to three). That is fine. It is

actually one of the nice things about math that we can model an unlimited amount of dimensions even though we only live in a three-dimensional world.

Visualizing a Linear Optimization Problem in Two Dimensions

Let's visualize the following two-dimensional problem, which is neither in general form nor in standard form (but we can easily convert it to either form, which we do not need to do for this simple problem, as we can extract the minimum by inspecting the graph):

$$\begin{aligned} \min \quad & (-x - y) \\ \text{s.t.} \quad & x + 2y \leq 3 \\ & 2x + y \leq 3 \\ & x \geq 0 \\ & y \geq 0 \end{aligned}$$

Figure 10-4 shows the boundaries of all the constraints (straight lines) of this linear optimization problem, along with the resulting feasible set. The optimal value of the objective function $-x - y$ is -2 , attained at the point $(1,1)$, which is one of the corners of the feasible set.

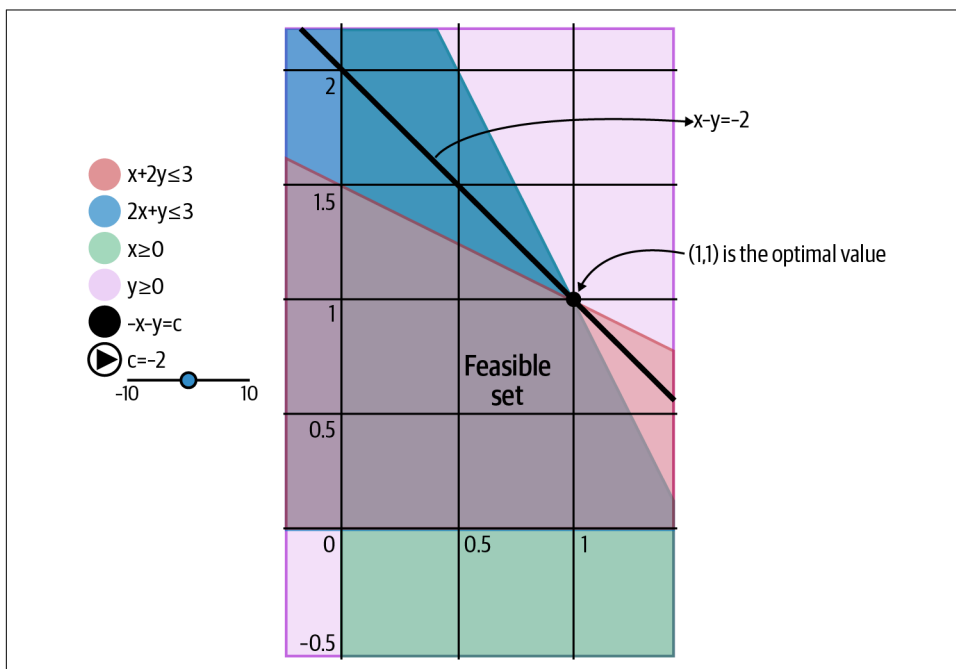


Figure 10-4. Feasible set and the optimal value of $-x - y$ is -2 attained at the corner point $(1,1)$

If this was an unconstrained problem, then the infimum of $-x - y$ would instead be $-\infty$. Constraints make a huge difference. The fact that the optimal value is at one of the corners of the polygon (two-dimensional polyhedron) is not a coincidence. If we draw the straight line $-x - y = c$ for some c that places part of the line inside the feasible set, then move in the direction of the negative of the gradient vector (recall that this is the direction of fastest descent), the line would move in the direction of the vector $-\nabla (-x - y) = -(-1, -1) = (1, 1)$ (it is definitely a coincidence that the gradient vector has the same coordinates as the optimizing point, as these two are completely unrelated). As long as the line has parts of it inside the feasible set, we can *keep pushing* and making c smaller until we can't push anymore, because if we did, we would exit the feasible set, become infeasible, and lose all our pushing work. This happens exactly when the whole line is outside the feasible set and barely hanging at the point $(1,1)$, which is still in the feasible set.

We found our optimizer, the point that makes the value of $-x - y$ smallest. We will get back to moving through the corners of feasible sets of linear problems soon, because that's where the optimizers are.

Convex to Linear

Even when the objective function is nonlinear, in many cases we may be lucky enough to reformulate the problem as a linear problem, then use linear optimization techniques to either obtain an exact solution or an approximation to the exact solution. One such case is when the objective function is convex. In optimization problems, after linearity, convexity is the next desirable thing, because we wouldn't worry about getting stuck at a local minimum: a local minimum for a convex function is also a global minimum.

We can always approximate a convex (and differentiable) function by a piecewise linear convex function, as in [Figure 10-5](#). Afterward, we can turn the optimization problem with a piecewise linear objective function into one with a linear objective function. This process, however, makes us lose differentiability in the first step (the function stops being smooth) and increase the dimension in the second step. Nothing is free.

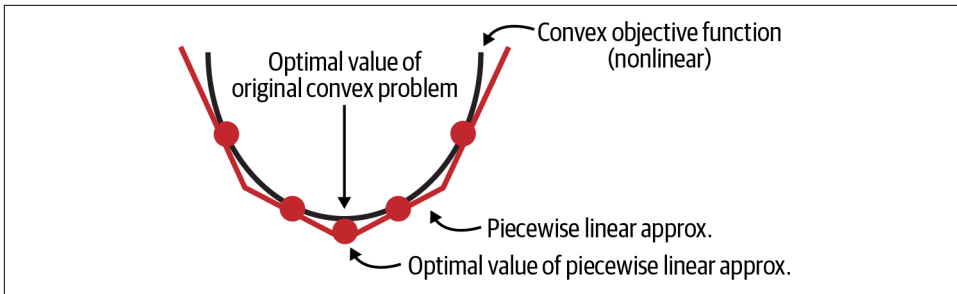


Figure 10-5. Approximating a convex function by piecewise linear functions

A *convex optimization* problem has a convex objective function and a convex feasible set. Convex optimization is a whole field of its own.



Convex Function

A function $f: \mathbb{R}^n \rightarrow \mathbb{R}$ is convex if and only if $f(\lambda x + (1 - \lambda)y) \leq \lambda f(x) + (1 - \lambda)f(y)$, for all $x, y \in \mathbb{R}^n$ and $0 \leq \lambda \leq 1$. This means that the segment connecting any two points on the graph of f lies above the graph of f .

Helpful facts about convex functions:

- A convex function cannot have a local minimum that fails to be a global minimum.
- If the functions $f_1, f_2, \dots, f_m: \mathbb{R}^n \rightarrow \mathbb{R}$ are convex functions, then the function $f(x) = \max_i f_i(x)$ is also convex. f may lose smoothness in this case, so optimization methods would not be able to use derivatives.
- The function $f(x) = \max \{m_1x + d_1, m_2x + d_2, \dots, m_nx + d_n\}$, or more compactly, $f(x) = \max_{i=1,2,\dots,n} \{m_ix + d_i\}$, is piecewise linear, as in [Figure 10-6](#). This is a convex function, since each $m_ix + d_i$ is convex (linear functions are convex and concave at the same time), and the maximum of convex functions is also convex.

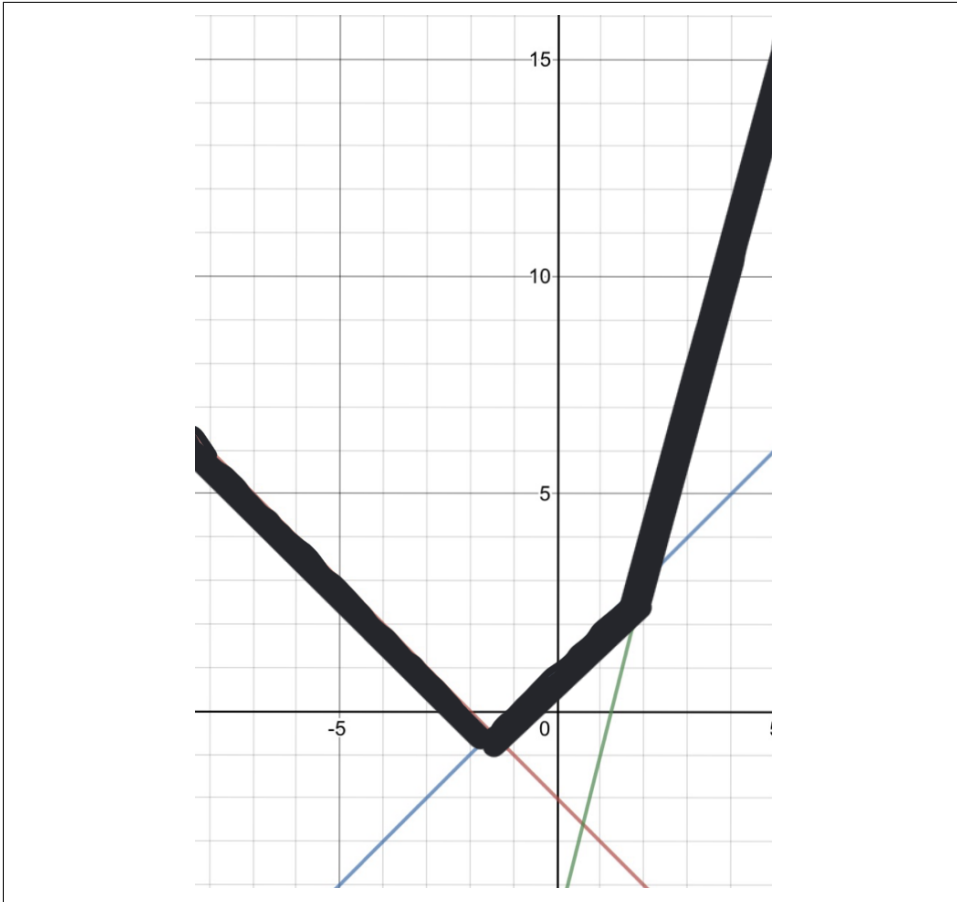


Figure 10-6. The maximum of linear functions is piecewise linear and convex

Now we can reformulate optimization problems with piecewise linear convex objective functions as linear optimization problems:

$$\min_{Ax \geq b} (\max_i m_i x + d_i) \leftrightarrow \min_{\substack{Ax \geq b \\ z \geq m_i x + d_i}} z$$

Note that we increased the dimension when we added a new decision variable z .

For example, the absolute value function $f(x) = |x| = \max\{x, -x\}$ is piecewise linear and convex. We can reformulate an optimization problem where the objective function includes absolute values of the decision variables as a linear optimization problem in two ways (here the c_i 's in the objective function must be nonnegative, otherwise the objective function might be nonconvex):

$$\min_{Ax \geq b} \sum_{i=1}^n c_i |x_i| \leftrightarrow \min_{\substack{Ax \geq b \\ z_i \geq x_i \\ -z_i \geq x_i}} \left(\sum_{i=1}^n c_i z_i \right) \leftrightarrow \min_{\substack{Ax^+ - Ax^- \geq b \\ x^+, x^- \geq 0}} \left(\sum_{i=1}^n c_i (x_i^+ + x_i^-) \right)$$

The Geometry of Linear Optimization

Let's think of the geometry of a linear optimization problem in standard form, as this is the form that is most convenient for algorithms searching for the minimizer. Geometry is all about the involved shapes, lines, surfaces, points, edges, corners, etc. The problem in standard form:

$$\min_{\substack{\vec{A}\vec{x} = \vec{b} \\ \vec{x} \geq \vec{0}}} \left(\vec{c} \cdot \vec{x} \right)$$

involves linear algebraic equations. We want to understand the geometric picture associated with these equations, along with the minimization process. Recall that linear is flat, and when flat things intersect with each other, they create hyperplanes, lines, and/or corners.

The linear constraints of the minimization problem define a polyhedron. We are highly interested in the corners of this polyhedron. But how do we know that the polyhedron has corners? What if it is only a half-space? As we mentioned before, if we change from general form to standard form, we jump up in dimension. Moreover, we enforce nonnegativity on the decision variables. Therefore, even if a polyhedron has no corners in the general form, it will always have corners in its higher-dimensional standard form: the standard form polyhedron gets situated in the first hyperoctant, and hence cannot possibly contain full lines. This is good. We have theorems that guarantee that, for a linear optimization problem, either the optimal value is $-\infty$, or there exists a finite optimal value attained at one of the corners of the polyhedron. So we must focus our attention on these corners when searching for the optimizer. Since many polyhedra that are associated with real-world constraints have tens of thousands of corners, we need efficient ways to sift through them.

Intuitively, we can start with the coordinates of one corner of the polyhedron, then work our way to an optimal corner. But how do we find the coordinates of these corners? We use linear algebra methods. This is why it is convenient to express the constraints as a linear system $\vec{A}\vec{x} = \vec{b}$ (with $\vec{x} \geq \vec{0}$). In linear optimization language, a corner is called a *basic feasible solution*. This algebraic name indicates that the corner's coordinates satisfy all the constraints (feasible), and solve some linear equations

associated with some basis (extracted from the system $A\vec{x} = \vec{b}$, specifically, from m columns of A).

The interplay of algebra and geometry

Before discussing the simplex method, let's associate the algebraic equations (or inequalities for problems that are in general form) of the constraints with geometric mental images:

Polyhedron

The constraints as a whole form a polyhedron. Algebraically, a polyhedron is the set of points satisfying the linear system $\vec{x} \in \mathbb{R}^n$, such that $A\vec{x} \geq \vec{b}$ for some $A_{m \times n}$ and $\vec{b} \in \mathbb{R}^m$.

Interior of a half-space

Here we consider *only one* inequality from the constraints, not the whole system $A\vec{x} \geq \vec{b}$, namely, $\vec{a}_i \cdot \vec{x} > \vec{b}_i$ (the strict inequality part of one inequality constraint). This corresponds to all the points that lie on one side relative to one face of the polyhedron. The inequality is strict, so that we are in the interior of the half-space and not on the boundary.

Hyperplane

Here we consider only one equality constraint, $\vec{a}_i \cdot \vec{x} = \vec{b}_i$, or only the equality part of an inequality constraint. This is the boundary of the half-space $\vec{a}_i \cdot \vec{x} > \vec{b}_i$, or one face of the polyhedron.

Active constraints

When we plug the coordinates of a point $\vec{x}^*$ into a constraint $\vec{a}_i \cdot \vec{x} \geq \vec{b}_i$ and we get equality, that is, $\vec{a}_i \cdot \vec{x}^* = \vec{b}_i$, then the constraint is *active* at this point. Geometrically, this places $\vec{x}^*$ at the boundary of the half-space, not in the interior.

Corner of the polyhedron

Geometrically, the right number of hyperplanes have to meet to form a corner. Algebraically, the right number of constraints is active at a corner point. This is a *basic feasible solution*, and we will go over it while discussing the simplex method.

Adjacent bases to find adjacent corners

These are two subsets of the columns of the matrix A that share all but one column. We use these to compute the coordinates of corners that are adjacent. In the simplex method, we need to geometrically move from one corner to an

adjacent one, and these adjacent bases help us accomplish that in a systematic algebraic way. We will also go over this while discussing the simplex method.

Degenerate case

To visualize this, suppose that we have two lines intersecting in two dimensions, forming a corner. Now if a third line meets them at exactly the same point, or in other words, if more than two constraints are active at the same point in two dimensions, then we have a degenerate case. In n dimensions, the point $\vec{x}^*$ has more than n hyperplanes passing through it, or more than n active constraints. Algebraically, this is what happens to our optimization algorithm as a consequence of this degeneracy: when we choose another set of linearly independent columns of A to solve for *another* basic feasible solution (corner), we might end up with the same one we got before, leading to cycling in our algorithm!

The Simplex Method

Our goal is to devise an algorithm that finds an optimal solution for a linear optimization problem in standard form:

$$\begin{aligned} \min_{\vec{x}} \quad & \vec{c} \cdot \vec{x} \\ \text{s.t.} \quad & A\vec{x} = \vec{b} \\ & \vec{x} \geq \vec{0} \end{aligned}$$

A is $m \times n$ with m linearly independent rows (so $m \leq n$), $\vec{b}$ is $m \times 1$, and $\vec{c}$ and $\vec{x}$ are $n \times 1$. Without loss of generality, we assume that the m rows of A are linearly independent, which means that there is no redundancy in the constraints of the problem. This also guarantees the existence of at least one set of m linearly independent columns of A ($\text{rank}(A) = m$). We need these linearly independent columns, or basis, to initiate our search for the optimizer, at a certain corner of the polyhedron, and to move from one corner to the other using the simplex method.

The main idea of the simplex method

We start at a corner of the polyhedron (also called a basic feasible solution), move to another corner in a direction that is guaranteed to reduce the objective function, or the cost, until we either reach an optimal solution or discover that the problem is unbounded and the optimal cost is $-\infty$ (these we know using certain optimality conditions, and become the termination criteria for our algorithm). There is a chance of cycling in the case of degenerate problems, but we can avoid this by making smart choices (systematic way of choosing) when there are ties in the process.

The simplex method hops around the corners of the polyhedron

The following is a linear optimization problem in three dimensions:

$$\begin{aligned} \min \quad & (-x_1 + 5x_2 - x_3) \\ \text{s.t.} \quad & x_1 \leq 2, x_3 \leq 3 \\ & 3x_2 + x_3 \leq 6, x_1 + x_2 + x_3 \leq 4 \\ & x_1, x_2, x_3 \geq 0 \end{aligned}$$

Figure 10-7 shows the polyhedron corresponding to its seven linear constraints.

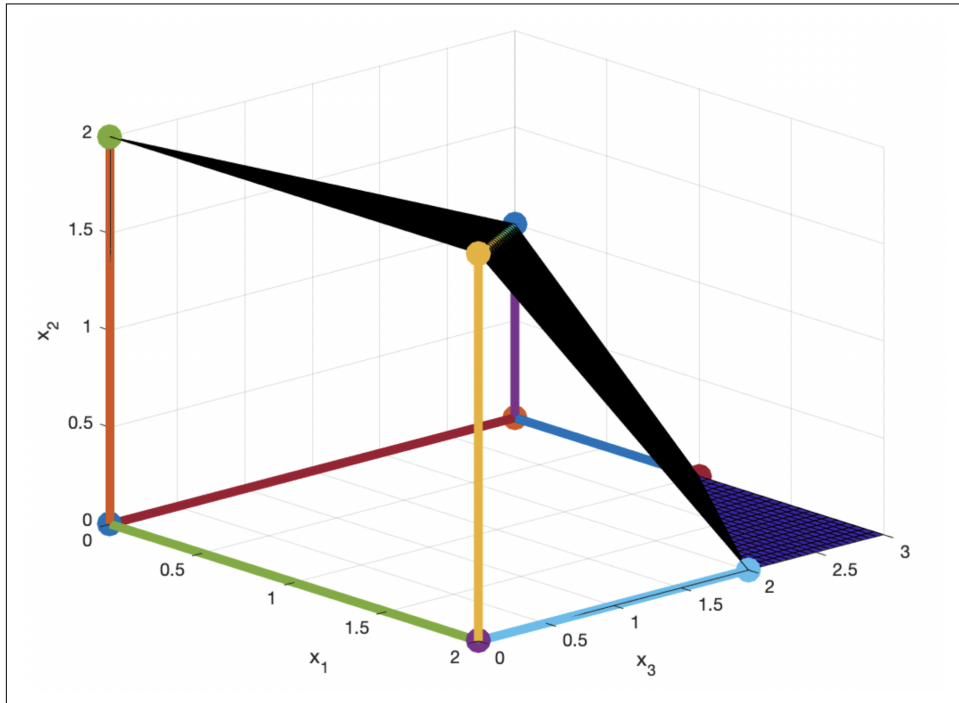


Figure 10-7. The simplex method moves from one corner of the polyhedron to the next until it finds an optimizing corner

Note that the problem is not in standard form. If we convert it to standard form, then we would gain four extra dimensions, corresponding to the four new variables that are required to convert the four inequality constraints to equality constraints. We cannot visualize the seven-dimensional polyhedron, but we do not need to. We can work with the simplex algorithm in seven dimensions and keep track of the important variables in three dimensions: (x_1, x_2, x_3) . This way, we can trace the path of the simplex method as it moves from one corner of the polyhedron to the next one,

reducing the value of the objective function $-x_1 + 5x_2 - x_3$ at each step, until it arrives at a corner with a minimal value.



You can skip the rest of this section and go straight to “**Transportation and Assignment Problems**” on page 386, unless you are interested in the details of the simplex method and its different implementations.

Steps of the simplex method

For a linear optimization problem in standard form, the simplex method progresses like this:

1. Start at a corner of the polyhedron (basic feasible solution $\vec{x}^*$). How do we find the coordinates of this basic feasible solution? Choose m linearly independent columns $A_1, \dots, A_m$ of A . Put them in a matrix B (basis matrix). Solve $B\vec{x}_b = \vec{b}$ for $\vec{x}_b$. If all the $\vec{x}_b$ are nonnegative, we have found a basic feasible solution $\vec{x}^*$. Place the entries of $\vec{x}_b$ in the corresponding positions in $\vec{x}^*$, and make the rest zero. Alternatively, we can solve $A\vec{x} = \vec{b}$ for $\vec{x}$, where the entries that correspond to the chosen columns are the unknown entries and the rest are zero. Therefore, a basic feasible solution $\vec{x}^*$ has zero nonbasic coordinates and basic coordinates $\vec{x}_b = B^{-1}\vec{b}$.

For example, if $A = \begin{pmatrix} 1 & 1 & 2 & 1 & 0 & 0 & 0 \\ 0 & 1 & 6 & 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 1 \end{pmatrix}$, and $\vec{b} = \begin{pmatrix} 8 \\ 12 \\ 4 \\ 6 \end{pmatrix}$, then we can choose

A_4, A_5, A_6, A_7 as a set of basic columns, giving $\vec{x} = (0, 0, 0, 8, 12, 4, 6)^t$ as a basic feasible solution (coordinates of one vertex of the polyhedron). We can alternatively choose A_3, A_5, A_6, A_7 as another set of basic columns, giving $\vec{x} = (0, 0, 4, 0, -12, 4, 6)^t$ as a basic solution but *not* basic feasible, because it has a negative coordinate.

2. Move from $\vec{x}^*$ to another corner $\vec{y}^* = \vec{x}^* + \theta^* \vec{d}$. We must find a direction $\vec{d}$ that keeps us in the polyhedron (feasible), increases only one nonbasic variable x_j from zero to a positive number, and keeps the other nonbasic variables at zero. At the same time, when we move from $\vec{x}^*$ to $\vec{y}^* = \vec{x}^* + \theta^* \vec{d}$, we must reduce the value of the objective function. That is, we want $\vec{c} \cdot \vec{y}^* \leq \vec{c} \cdot \vec{x}^*$. When we increase the value of x_j from zero to a positive number, the difference in the objective

function is $\bar{c}_j = c_j - \vec{c}_B \cdot B^{-1} A_j$, so we must choose a coordinate j for which this quantity is negative. To make all of this work, the coordinates of $\vec{d}$ end up being $d_j = 1$ (because we introduced x_j), $d_i = 0$ if $i \neq j$ or if i is nonbasic, and $\vec{d}_B = -B^{-1} A_j$; and the value of θ^* ends up being:

$$\theta^* = \min_{\text{all basic indices for which } d_{B(i)} < 0} \left\{ -\frac{x_{B(i)}}{d_{B(i)}} \right\} = -\frac{x_{B(l)}}{d_{B(l)}};$$

3. Now column $A_{B(l)}$ exits the basis B , and column A_j replaces it.
4. Repeat this process until we either reach a finite optimal solution (when no A_j from all the available columns of A gives us a negative c_j), or discover that the problem is unbounded and the optimal cost is $-\infty$. This happens when we have $\vec{d} \geq \vec{0}$, so $\vec{y} = \vec{x} + \theta \vec{d} \geq \vec{0}$, making it feasible no matter how large θ gets; thus pushing θ to ∞ will keep reducing the cost $\vec{c} \cdot \vec{y} = \vec{c} \cdot \vec{x} + \theta (\vec{c}_j - \vec{c}_B \cdot B^{-1} A_j)$ all the way to $-\infty$.

Notes on the simplex method

Some things to remember for the simplex method:

- Step 4 gives the two termination criteria for the simplex algorithm: no negative reduced cost $\bar{c}_j$, or all the coordinates of a feasible set reducing cost direction $\vec{d}$ are nonnegative.
- If the feasible set is nonempty and every basic feasible solution is nondegenerate, then the simplex method is guaranteed to terminate after finitely many iterations, with either a finite optimal solution or $-\infty$ optimal cost.
- Suppose some of the basic feasible solutions are degenerate (some of the *basic* variables are also zero) and we end up at one of them. In this case, there is a chance that when we change the basis by introducing A_j and make $A_{B(l)}$ exit, we stay at the same corner $\vec{y} = \vec{x} + 0\vec{d}$ (this happens when $x_{B(l)} = 0$, so $\theta^* = -\frac{x_{B(l)}}{d_{B(l)}} = 0$). In this case, choose a new A_j until you actually move from $\vec{x}$ to $\vec{y} = \vec{x} + \theta \vec{d}$, $\theta^* > 0$. One really bad thing that could happen here is that after we stop at $\vec{x}$ and keep changing basis (stalling for a little while at $\vec{x}$) until we find one that actually moves us away from $\vec{x}$ to $\vec{y} = \vec{x} + \theta^* \vec{d}$ in a cost-reducing direction, we might end up with the same basis we started the algorithm with! This will lead to cycling, and the algorithm may loop indefinitely. Cycling can be avoided by making smart choices for which columns of A will enter and exit the

basis: a systematic way of choosing A_j and later $B(l)$ in θ^* when there are ties in the process.

- When there are ties in the process (we have more than one reducing cost option A_j that gives $\bar{c}_j < 0$, and/or more than one minimizing index $B(l)$ for θ^*), we can devise rules to choose entering A_j and/or exiting $A_{B(l)}$ at a step with such a tie. The rules we decide to follow when there are such ties are called *pivoting rules*.
- A very simple and computationally inexpensive pivoting rule is *Bland's rule*: choose A_j with the smallest index j for which $\bar{c}_j < 0$ to enter the basis, and choose $A_{B(l)}$ with the smallest eligible index $B(l)$ to exit the basis. This smallest subscript pivoting rule helps us avoid cycling. There are other pivoting rules as well.
- If $n - m = 2$ (so A has only two more columns than rows), then the simplex method will not cycle no matter which pivoting rule is used.
- For problems that did not originate from a general form problem, especially those with a large number of variables, it might not always be obvious how to choose the initial basis B and associated basic feasible solution x (because it would not be clear which m columns of A are linearly independent). In this case, we introduce *artificial variables* and solve an *auxiliary linear programming problem* to determine whether the original problem is *infeasible* and hence there is no solution; or, if the problem is feasible, drive the artificial variables out of the basis and obtain an initial basis and an associated basic feasible solution for our original problem. This process is called *Phase I* of the simplex method. The rest of the simplex method is called *Phase II*.
- The big-M method combines Phase I and Phase II of the simplex method. Here we use the simplex method to solve:

$$\min_{\substack{\vec{x} \\ A\vec{x} = \vec{b} \\ \vec{x} \geq 0, \vec{y} \geq 0}} \left(\vec{c} \cdot \vec{x} + M(y_1 + y_2 + \dots + y_m) \right)$$

For a sufficiently large choice of M , if the original problem is feasible and its optimal cost is finite, all of the artificial variables $y_1, y_2, \dots, y_m$ are eventually driven to zero, which takes us back to our original problem. We can treat M as an undetermined parameter and let the reduced costs be functions of M , and treat M as a very large number when determining whether a reduced cost is negative.

The revised simplex method

The revised simplex method is a computationally less expensive implementation of the simplex method. It provides a cheaper way to compute $\bar{B}^{-1}$ by exploiting the relationship between the old basis B and the new basis $\bar{B}$: they only have one different

column (the two vertices involved are adjacent). So we can obtain the new $\bar{B}^{-1}$ from the previous B^{-1} .

The following is a typical iteration of the revised simplex algorithm. This will also help reinforce the simplex method steps from the previous section. Note that for simplicity, we suppress the vector notation for the x 's, y 's, b 's, and d 's:

1. Start with a B consisting of m basic columns from A and the associated basic feasible solution x with $x_B = B^{-1}b$, and $x_i = 0$ otherwise.
2. Compute B^{-1} (it is B^{-1} not B that appears in the simplex method computations).
3. For j nonbasic, compute the reduced costs $\bar{c}_j = c_j - \vec{c}_B B^{-1} A_j$ (this will give you $n - m$ reduced costs).
4. If all the $\bar{c}_j$ are nonnegative, the current basic feasible solution x is optimal, and the algorithm terminates with x as the optimizer and $c.x$ as the optimal cost (there is no A_j that could enter the basis and reduce the cost even more).
5. *Else*, choose a j for which $\bar{c}_j < 0$ (Bland's pivoting rule tells us to choose the smallest such j). Note that this makes A_j enter the basis.
6. Compute a feasible direction d : $d_j = 1$, $d_B = -B^{-1}A_j$, and $d_i = 0$ otherwise.
 - If all the components of d_B are nonnegative, the algorithm terminates with optimal cost $-\infty$ and no optimizer.
 - *Else*, choose the components of $\vec{d}_B$ that are negative, and let:

$$\theta^* = \min_{\text{all basic indices for which } d_{B(i)} < 0} \left\{ -\frac{x_{B(i)}}{d_{B(i)}} \right\} = -\frac{x_{B(l)}}{d_{B(l)}}$$

This step computes θ^* and assigns $B(l)$ as the index of the exiting column.

7. Compute the new basic feasible solution $y = x + \theta^* d$ (this new basic feasible solution corresponds to the new basis $\bar{B}$, which has A_j replace $A_{B(l)}$ in B).
8. This step computes the new $\bar{B}^{-1}$ for the next iteration without forming the new basis $\bar{B}$ then inverting it: form the $m \times m + 1$ augmented matrix $(B^{-1} \mid B^{-1} A_j)$. Perform row operations using the l th row (add to each row a multiple of the l th row) to make the last column the unit vector e_l , which is zero everywhere except for 1 in the l th coordinate. The first m columns of the result is your new $\bar{B}^{-1}$.

Justification

Let $u = B^{-1}A_j$ and note that $B^{-1}\bar{B} = (e_1 e_2 \cdots u \cdots e_m)$, where e_i is the unit column vector with 1 in the i th entry and zero everywhere else, and u is the l th column. The matrix becomes the identity matrix if we perform row

operations using the l th row and transform u into e_l . All row operations can be bundled together in an invertible matrix Q applied from the left: $QB^{-1}\bar{B} = I$. Now right-multiply by $\bar{B}^{-1}$ to get $QB^{-1} = \bar{B}^{-1}$. This means that to obtain $\bar{B}^{-1}$, perform on B^{-1} the same row operations that will transform u to e_l .



Calculating $\bar{B}^{-1}$ Using B^{-1} in the Revised Simplex

This method does not start from the original m columns of A and finds the inverse; instead, it does row operations on the previously calculated B^{-1} , which could include roundoff errors. Doing this over many iterations will accumulate these errors, so it will be better to compute $\bar{B}^{-1}$ straight from the columns of A every now and then to avoid error accumulation.

The full tableau implementation of the simplex method

The full tableau implementation of the simplex method has the advantage of only storing and updating one matrix. Here, instead of maintaining and updating B^{-1} , maintain and update the $m \times n + 1$ matrix $x_B | B^{-1}A = (B^{-1}b | B^{-1}A)$. The column $u = B^{-1}A_j$ corresponding to the variable entering the basis is called the *pivot column*. If the l th basic variable exits the basis, then the l th row is called the *pivot row*. The element belonging to both the pivot row and the pivot column is called the *pivot element*. Now add a *zeroth row* on top of your *tableau* that keeps track of the negative of the current cost $-c \cdot x = -c_B \cdot x_B = c_B \cdot B^{-1}b$ and the reduced costs $c - c_B \cdot B^{-1}A$. So the tableau looks like:

$$\left(\begin{array}{cc} -c_B \cdot B^{-1}b & c - c_B \cdot B^{-1}A \\ B^{-1}b & B^{-1}A \end{array} \right)$$

or more expanded:

$$\left(\begin{array}{cc} -c_B \cdot x_B & \bar{c}_1 \cdots \bar{c}_n \\ x_{B(1)} & |\cdots| \\ \vdots & B^{-1}A_1 \cdots B^{-1}A_n \\ x_{B(m)} & |\cdots| \end{array} \right)$$

One helpful thing that is nice to master: extracting B^{-1} and B easily from a given simplex tableau (like in the movie *The Matrix*).

The most efficient implementation of the simplex method is the revised simplex (memory usage is $O(m^2)$, worst-case time for single iteration is $O(mn)$, best-case time for single iteration is $O(m^2)$, while all of the above measures for the full tableau method are $O(mn)$), but everything depends on how sparse the matrices are.

Transportation and Assignment Problems

Transportation and assignment problems are linear optimization problems that we can formulate as min cost network flow problems.

Transportation problem

Allocate products to warehouses, minimize costs.

Assignment problem

Allocate assignees to tasks, number of assignees is equal to the number of tasks, and each assignee performs one task. There is a cost when assignee i performs task j . The objective is to select an assignment that minimizes the cost. One example is assigning Uber drivers to customers, or machines to tasks.

We exploit the fact that the involved matrices are sparse so we don't have to do a full implementation of the simplex algorithm, only a special *streamlined* version that solves both the assignment and transportation problems. This is related to the *network simplex method* that solves any minimum cost flow problem, including both transportation and assignment problems. The transportation and assignment problems are special cases of the minimum flow problem. The Hungarian method is special for the assignment problem. Since it is specialized for this, it is more efficient. These special-purpose algorithms are included in some linear programming software packages.

Duality, Lagrange Relaxation, Shadow Prices, Max-Min, Min-Max, and All That

We hinted and danced around the idea of *duality* earlier in this chapter when discussing finite dimensional constrained optimization and relaxing the constraints using Lagrange multipliers. Duality is really helpful when our constrained problems are linear, or quadratic with linear constraints. It gives us the option to solve either the optimization problem at hand (the *primal*) or another related problem (its *dual*), whichever happens to be easier or less expensive, and get the same solution. Usually, having more decision variables (dimensions of the problem) is not as strenuous for an algorithm as having more constraints. Since the dual problem flips the roles of decision variables and constraints, then solving it instead of the primal problem

makes more sense when we have too many constraints (another way here is using the dual simplex method to solve the primal problem, which we will talk about soon). Another way the dual problem helps is that it sometimes provides shortcuts to the solution of the primal problem. A feasible vector $\vec{x}$ to the primal problem will end up being the optimizer if there happens to be a feasible vector $\vec{p}$ to the dual problem, such that $\vec{c} \cdot \vec{x} = \vec{p} \cdot \vec{b}$.

When learning about duality in the next few paragraphs, think of it in the same way you see [Figure 10-8](#): something is happening in the primal realm, some form of related shadow or echo is happening in the dual realm (some alternate universe), and the two meet at the optimizer, like a gate where the two universes touch.

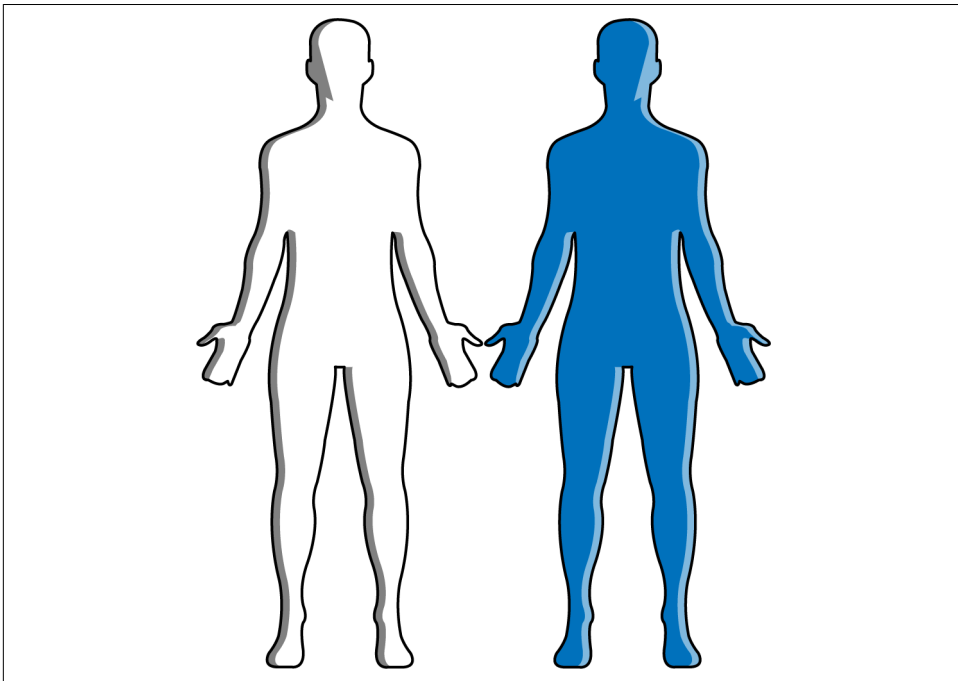


Figure 10-8. Duality, shadow problems, shadow prices

So if we are maximizing in one universe, we are minimizing in the other; if we are doing something with the constraints in one universe, we do something to the decision variables in the other, and vice versa.

Motivation for duality-Lagrange multipliers

For any optimization problem (linear or nonlinear):

$$\min_{\vec{x} \in \text{feasible set}} f(\vec{x})$$

Instead of finding the minimizer $\vec{x}^*$ by setting the gradient equal to zero, look for an upper bound of $f(\vec{x}^*)$ (easy by plugging any element of the feasible set into $f(\vec{x})$) and for a lower bound of $f(\vec{x}^*)$ (this is a harder inequality and usually requires clever ideas). Now we would have $\text{lower bound} \leq f(\vec{x}^*) \leq \text{upper bound}$, so we *tighten* these bounds to get closer to the actual solution $f(\vec{x}^*)$. We tighten the bounds by minimizing the upper bounds (this brings us back to the original minimization problem) and *maximizing the lower bounds* (this establishes the dual problem).

Now for a linear minimization problem in *any form* (standard form, general form, or neither):

$$\min_{\text{linear constraints on } \vec{x}} \vec{c} \cdot \vec{x}$$

What is the clever idea that gives us lower bounds for $f(\vec{x}) = \vec{c} \cdot \vec{x}$? We look for lower bounds for $f(\vec{x}) = \vec{c} \cdot \vec{x}$ made up of a *linear combination* of the problem constraints. So we multiply each of our constraints by multipliers p_i (Lagrange multipliers), choosing their signs in a way such that the constraint inequality is in the $\geq$ direction. How so? Well, the linear constraints are linear combinations of the entries of $\vec{x}$, the objective function $\vec{c} \cdot \vec{x}$ is also a linear combination of the entries of $\vec{x}$, a linear combination of a linear combination is still a linear combination, so we can totally pick a linear combination of the constraints that we can compare to $\vec{c} \cdot \vec{x}$.

Namely, if we have m linear constraints, we need:

$$p_1 b_1 + p_2 b_2 + \cdots + p_m b_m \leq \vec{c} \cdot \vec{x}$$

The sign of a multiplier p_i would be free if the constraint has an equality. Once we have these lower bounds, we tighten them by maximizing on p_i , which gives us the dual problem.

Finding the dual linear optimization problem from the primal linear optimization problem

It is important to get the sizes of the inputs to a linear optimization problem right. The inputs are: A , which is $m \times n$, $\vec{c}$, which is $n \times 1$, and $\vec{b}$, which is $m \times 1$. The decision variables in the primal problem are in the vector $\vec{x}$, which is $n \times 1$. The decision variables in the dual problem are in the vector $\vec{p}$, which is $m \times 1$.

In general, if A appears in the primal problem, then A^t appears in the dual problem. So in the primal problem, we have the dot product of the *rows* of A and $\vec{x}$. In the dual problem, we have the dot product of the *columns* of A and $\vec{p}$. If the linear optimization problem is in *any form*, it's easy to write its dual following this process:

- If the primal is a minimization, then the dual is a maximization and vice versa.
- The primal cost function is $\vec{c} \cdot \vec{x}$, and the dual cost function is $\vec{p} \cdot \vec{b}$.
- In a *minimization* primal problem, we separate the constraints into two types:

Type one

Constraints telling us about the sign of the decision variable, for example:

- $x_3 \geq 0$, then in the dual this will correspond to $A_3 \cdot \vec{p} \leq c_3$, where A_3 is the third column of A and c_3 is the third entry of $\vec{c}$.
- $x_{12} \leq 0$, then in the dual this will correspond to $A_{12} \cdot \vec{p} \geq c_{12}$, where A_{12} is the 12th column of A and c_{12} is the 12th entry of $\vec{c}$.
- x_5 is free, meaning has no specified sign. Then in the dual this will correspond to $A_5 \cdot \vec{p} = c_5$, where A_5 is the fifth column of A and c_5 is the fifth entry of $\vec{c}$.

Type two

Constraints of the form $a_i \cdot x \geq \leq = b_i$, where a_i is the i th row of A . In the dual these will correspond to constraints on the sign of p_i , for example:

- $a_2 \cdot x \geq b_2$, then in the dual this will correspond to $p_2 \geq 0$.
- $a_7 \cdot x \leq b_7$, then in the dual this will correspond to $p_7 \leq 0$.
- $a_8 \cdot x = b_8$, then the sign of p_8 is free.

In particular, if the linear optimization problem is in standard form:

$$\begin{aligned} \min \quad & \vec{c} \cdot \vec{x}, \\ \text{s.t.} \quad & A\vec{x} = \vec{b} \\ & \vec{x} \geq \vec{0} \end{aligned}$$

then its dual is:

$$\begin{aligned} & \max_{\vec{p} \text{ is free}} \vec{p} \cdot \vec{b} . \\ & A^T \vec{p} \leq \vec{c} \end{aligned}$$

If the linear optimization problem is in general form:

$$\begin{aligned} & \min_{\vec{x} \geq \vec{0}} \vec{c} \cdot \vec{x} \\ & \vec{x} \text{ is free} \end{aligned}$$

then its dual is:

$$\begin{aligned} & \max_{\vec{p} \geq \vec{0}} \vec{p} \cdot \vec{b} \\ & A^T \vec{p} = \vec{c} \end{aligned}$$

How to solve the dual problem? The simplex method solves the dual problem; however, now you move to basic feasible solutions that *increase* the cost rather than *decrease* the cost.

Derivation for the dual of a linear optimization problem in standard form

There is another way to think about deriving the dual problem, but for this one the linear problem has to be in its standard form. Here's the idea of it: we relax the constraint $A\vec{x} = \vec{b}$ but introduce Lagrange multipliers $\vec{p}$ (pay a penalty $\vec{p}$ when the constraint is violated). So:

$$\begin{aligned} & \min_{\vec{x} \geq \vec{0}} \vec{c} \cdot \vec{x} \\ & A\vec{x} = \vec{b} \end{aligned}$$

becomes:

$$\min_{\vec{x} \geq \vec{0}} \vec{c} \cdot \vec{x} + \vec{p} \cdot (\vec{b} - A\vec{x}) = g(\vec{p}).$$

Now prove that $g(\vec{p})$ is a lower bound for the original $\vec{c} \cdot \vec{x}^*$ (this is the weak duality theorem), then maximize over p . The dual problem appears in the process.

The *strong duality theorem* says that the min of the primal problem and the max of the dual problem are equal. Note that if the primal problem is unbounded, then the dual problem is infeasible; and if the dual problem is unbounded, then the primal problem is infeasible.

Farkas' lemma is at the core of duality theory and has many economical and financial applications.

Dual simplex method

The dual simplex method solves the primal problem (not the dual problem) using duality theory. The main difference between the simplex method and the dual simplex method is that the regular simplex method starts with a basic feasible solution that is not optimal and moves toward optimality, while the dual simplex method starts with an infeasible solution that is optimal and works toward feasibility. The dual simplex method is like a mirror image of the simplex method.

First, note that when we solve the primal problem using the simplex method, we obtain the optimal cost for the dual problem for free (equal to the primal optimal cost), but also, *we can read off the solution (optimizer) to the dual problem from the final tableau for the primal problem*. An optimal dual variable is nonzero only if its associated constraint in the primal is binding. This should be intuitively clear, since the optimal dual variables are the shadow prices (Lagrange multipliers) associated with the constraints. We can interpret these shadow prices as values assigned to the scarce resources (binding constraints), so that the value of these resources equals the value of the primal objective function. The optimal dual variables satisfy the optimality conditions of the simplex method. In the final tableau of the simplex method, the reduced costs of the basic variables must be zero. The optimal dual variables must be the shadow prices associated with an optimal solution.

Another way is to think of the dual simplex method as a disguised simplex method solving the dual problem. However, we do so without explicitly writing the dual problem and applying the simplex method to maximize.

Moreover, the simplex method produces a sequence of primal basic feasible solutions (corners of the polyhedron); as soon as it finds one that is also dual feasible, the method terminates. On the other hand, the dual simplex method produces a sequence of dual basic feasible solutions; as soon as it finds one that is also primal feasible, the method terminates.

Example: Networks, linear optimization, and duality

Consider the network in **Figure 10-9**. The numbers indicate the edge capacity, which is the maximum amount of flow that each edge can handle. The *max flow* problem is to send the maximum flow from the origin node to the destination node. Intuitively, the maximum flow through the network will be limited by the capacities that the edges can transmit. In fact, this observation underlies a dual problem that takes place: maximizing the flow through the network is equivalent to minimizing the total capacities of the edges such that if we cut, then we cannot get from the origin to the destination. This is the *max-flow min-cut theorem*.

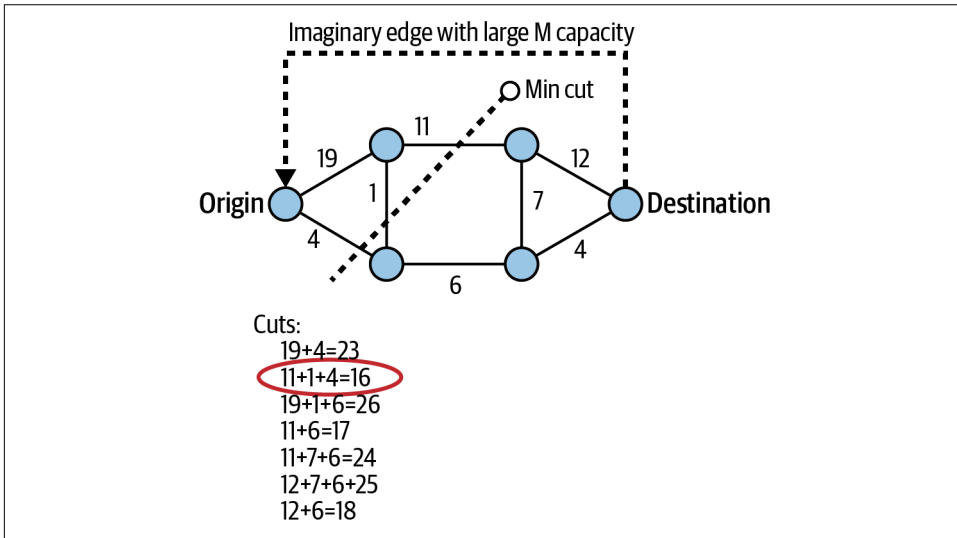


Figure 10-9. Duality: maximum flow through the network is equal to the smallest cut capacity

Figure 10-9 shows the values of all the cuts (the set of edges that if we cut together we would not be able to get from the origin to the destination) through the network, along with the cut of minimal total edge capacity, which is 16. By the max-flow min-cut theorem, the max flow that we can send through the network would be 16: send $y_1 = 12$ units through the edge with capacity 19, and $y_2 = 4$ units through the edge with capacity 4. Of those, $y_3 = 1$ unit will flow through the edge with capacity 1, $y_4 = 11$ units will flow through the edge with capacity 11, and $y_5 = 1 + 4 = 5$ units will flow through the bottom edge with capacity 6. All 12 units will make their way to the destination through the last 2 edges connected to it, with $y_6 = 0$ (no units need to flow through the vertical edge with capacity 7), $y_7 = 11$ units flow through the rightmost edge with capacity 12, and $y_8 = 5$ units flow through the rightmost edge with capacity 6. The solution to the max flow problem is now $(y_1, y_2, y_3, y_4, y_5, y_6, y_7, y_8) = (12, 4, 1, 11, 5, 0, 11, 5)$.

To formulate this network problem as a linear optimization problem (which we just solved graphically using our knowledge of the value of the minimal cut, which is the solution of the dual problem), we need to add one more fictional edge with flow value y_9 that connects the destination to the origin, and assume that the flow that gets to the destination fictionally finds its way back to the origin. In other words, we *close the circuit*, and apply *Kirchhoff's current law*, which says *the sum of currents in a network of conductors meeting at a point is zero*, or the flow into a node is equal to the flow out of it. The linear maximization problem now becomes:

$$\begin{aligned} & \max_{\vec{y}} y_9 \\ & A\vec{y} = \vec{0} \\ & |y_i| \leq M_i \end{aligned}$$

where A (Figure 10-10) is the incidence matrix of our network, $\vec{y} = (y_1, y_2, y_3, y_4, y_5, y_6, y_7, y_8, y_9)^t$ is the vector of *signed* maximum flow (we allow the y values to be negative so that the flow in would cancel the flow out) that we can send through each edge and that we need to solve for (we just found its solution without the signs by inspection using the minimal cut intuition), M_i is the max capacity of each edge in the network, and the condition $A\vec{y} = \vec{0}$ guarantees that the flow into a node is equal to the flow out of a node. Of course, in this case the network will have directed edges showing in which direction the optimal flow will go through each edge.

Nodes \ Edges	①	②	③	④	⑤	⑥	⑦	⑧	⑨
	1	2	3	4	5	6	7	8	9
1	1	1	0	0	0	0	0	0	1
2	1	0	1	1	0	0	0	0	0
3	0	1	1	0	1	0	0	0	0
4	0	0	0	1	0	1	1	0	0
5	0	0	0	0	1	1	0	1	0
6	0	0	0	0	0	0	1	1	1

Figure 10-10. Incidence matrix of the network in Figure 10-9

Now that we have a linear formulation of the max flow problem, we can write its dual easily using the methods we learned in this chapter (the minimum cut problem), and solve either the primal or the dual. Note that all we need for this formulation is the

incidence matrix of the network, the edge capacities, and the Kirchhoff's condition that the flow into a node is equal to the flow out of the node.

Example: Two-person zero-sum games, linear optimization, and duality

Another relevant setting where duality and linear optimization are built in is the two-person zero-sum game from game theory. A gain to one player is a loss to the other (hint: duality). To articulate the problem mathematically, we need the *payoff matrix* for all the options in the game for each of $player_1$ and $player_2$. Each player wants to devise a strategy that maximizes their pay given their options (no one said that the payoff matrices for the games have to be fair). We need to solve for the optimal strategy for each player. If we set up the optimization problem for $player_1$, we do not start from scratch to get the optimization problem for the strategy of $player_2$: we just write its dual. The sum expected payoff of the game will be the same for both players, assuming that both of them act rationally and follow their optimal strategies.

Consider, for example, the payoff matrix in **Figure 10-11**. The game goes like this: $player_1$ chooses a row and $player_2$ chooses a column at the same time. $player_1$ pays the number in the chosen row and column to $player_2$. Therefore, $player_1$ wants to minimize and $player_2$ wants to maximize. The players repeat the game many times.

What is the optimal strategy for each of $player_1$ and $player_2$ and what is the expected payoff of the game?

	y_1	y_2	y_3	
x_1	1	0	4	y_1+4y_3
x_2	3	-1	2	$3y_1-y_2+2y_3$
	x_1+3x_2	$-x_2$	$4x_1+2x_2$	

Figure 10-11. Payoff matrix

To find the optimal strategy, suppose $player_1$ chooses row_1 , with probability x_1 and row_2 with probability x_2 . Then $x_1 + x_2 = 1$, $0 \leq x_1 \leq 1$, and $0 \leq x_2 \leq 1$. $player_1$ rationalizes that if they use an (x_1, x_2) mixed strategy, there would be another row in the payoff matrix corresponding to this new strategy (see **Figure 10-11**). Now $player_1$ knows that $player_2$ wants to choose the column that maximizes their payoff, so $player_1$ must choose (x_1, x_2) that makes the worst payoff (maximum of the third row) as small as possible. Therefore, $player_1$ must solve the *min-max* problem:

$$\begin{aligned}
& \min_{0 \leq x_1 \leq 1} \max \{x_1 + 3x_2, -x_2, 4x_1 + 2x_2\} \\
& 0 \leq x_2 \leq 1 \\
& x_1 + x_2 = 1
\end{aligned}$$

Recall that the maximum of linear functions is a convex piecewise linear function. We can easily change such a *min-max(linear functions)* problem to a linear minimization problem:

$$\begin{aligned}
& \min z \\
& z \geq x_1 + 3x_2 \\
& z \geq -x_2 \\
& z \geq 4x_1 + 2x_2 \\
& 0 \leq x_1 \leq 1 \\
& 0 \leq x_2 \leq 1 \\
& x_1 + x_2 = 1
\end{aligned}$$

Figure 10-12 shows the formulation of the dual of this problem, and Figure 10-13 shows that this is exactly the problem that *player*₂ is trying to solve.

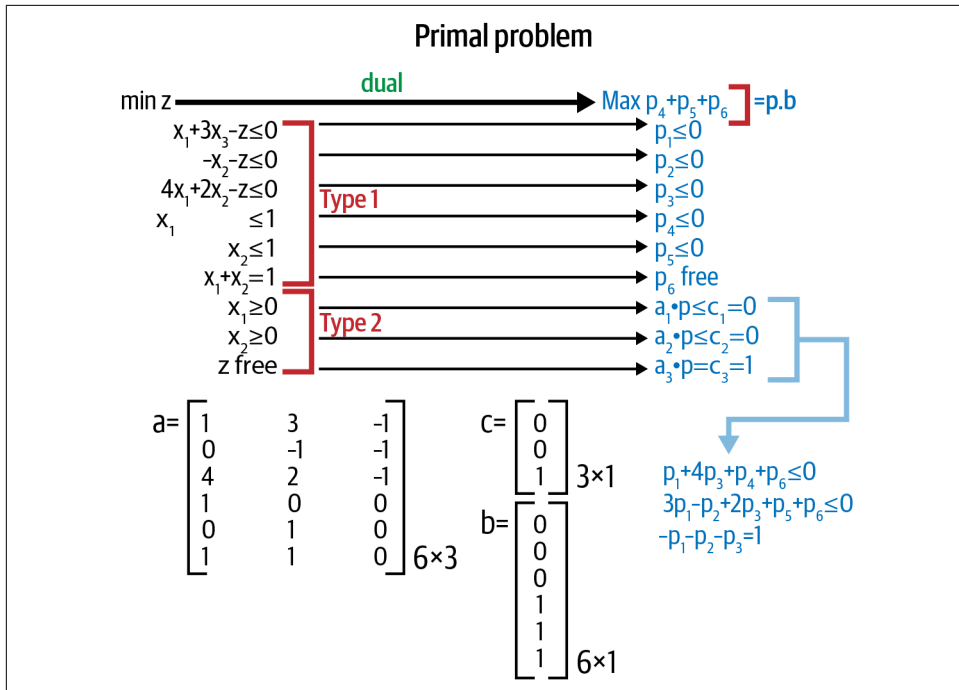


Figure 10-12. The dual of *player*₁'s problem

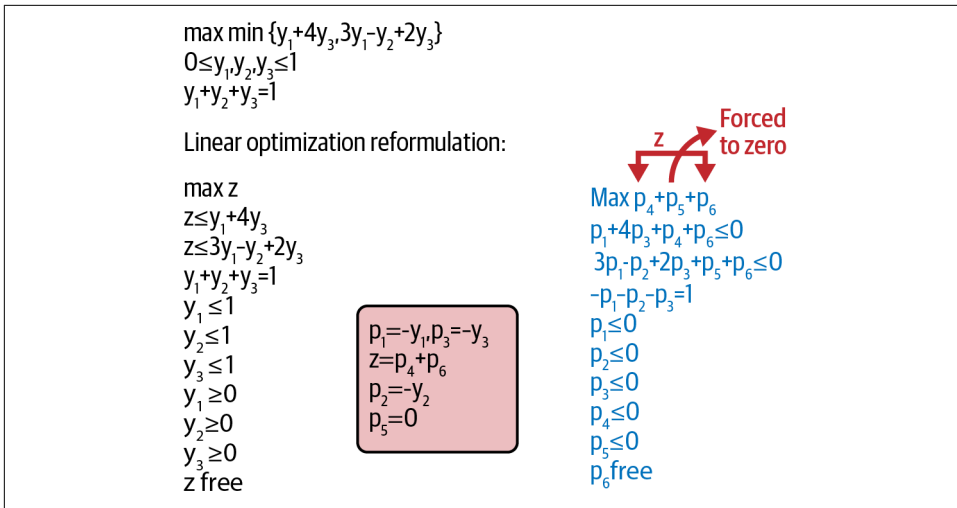


Figure 10-13. The dual of *player*₁'s min-max problem is the same as *player*₂'s max-min problem

Note that the constraints $y_1 \leq 1$, $y_2 \leq 1$, and $y_3 \leq 1$ are redundant since all the y 's are nonnegative and they add up to 1. Similarly with the constraints $x_1 \leq 1$ and $x_2 \leq 1$. This happens a lot when formulating linear optimization problems.

Solving either the primal or the dual problem, we find each player's optimal strategy: *player*₁ must go with the first row $x_1 = 0.25$ of the time, and with the second row $x_2 = 0.75$ of the time, for an expected payoff of 2.5, which means *player*₁ expects to lose no more than 2.5 with this strategy. *player*₂ must go with the first column $y_1 = 0.5$ of the time, and with the third column $y_3 = 0.5$ of the time (never with the second column $y_2 = 0$), for an expected payoff of 2.5, which means *player*₂ expects to gain no less than 2.5 with this strategy.

Quadratic optimization with linear constraints, Lagrangian, min-max theorem, and duality

A *nonlinear* optimization problem that has a nice structure, appears in all kinds of applications, and has a lot to teach us on how things tie together is a quadratic problem with linear constraints:

$$\min_{\vec{x}} \frac{1}{2} \vec{x}^t S \vec{x}$$

$A\vec{x} = b$

Here, S is a symmetric and positive semidefinite matrix, which means that its eigenvalues are nonnegative. For high dimensions, this plays the role of keeping the objective function convex and bounded below, or shaped like the bowl of the one-dimensional function $f(x) = x^2$.

For example, this is a two-dimensional quadratic optimization problem with one linear constraint:

$$\min_{a_1x_1 + a_2x_2 = b} \frac{1}{2}(s_1x_1^2 + s_2x_2^2)$$

Here, $S = \begin{pmatrix} s_1 & 0 \\ 0 & s_2 \end{pmatrix}$ where the s entries are nonnegative, and $A = (a_1 \ a_2)$. Inspecting this problem, we are searching for the point (x_1, x_2) on the straight line $a_1x_1 + a_2x_2 = b$ that minimizes the quantity $f(\vec{x}) = s_1x_1^2 + s_2x_2^2$. The level sets of the objective function $s_1x_1^2 + s_2x_2^2 = k$ are concentric ellipses that cover the whole $\mathbb{R}^2$ plane. The winning ellipse (the one with the smallest level set value) is the one that is tangent to the straight line at the winning point (Figure 10-14). At this point, the gradient vector of the ellipse and the gradient vector of the constraint align, which is exactly what Lagrange multiplier formulation gives us: to formulate the Lagrangian, relax the constraint, but pay a penalty equal to the Lagrangian multiplier p times how much we relaxed it in the objective function, minimizing the unconstrained problem.

$$\mathcal{L}(\vec{x}; p) = f(\vec{x}) + p(b - g(\vec{x})) = s_1x_1^2 + s_2x_2^2 + p(b - a_1x_1 - a_2x_2)$$

When we minimize the Lagrangian, we set its gradient equal to zero, and that leads to $\nabla f(\vec{x}) = p\nabla g(\vec{x})$. This says that the gradient vector of the objective function is parallel to the gradient vector of the constraint at the optimizing point(s). Since the gradient vector of any function is perpendicular to its level sets, the constraint is in fact tangent to the level set of the objective function at the minimizing point(s). Therefore, to find the optimizing point(s), we look for the level sets of the objective function where it happens to be tangent to the constraint.

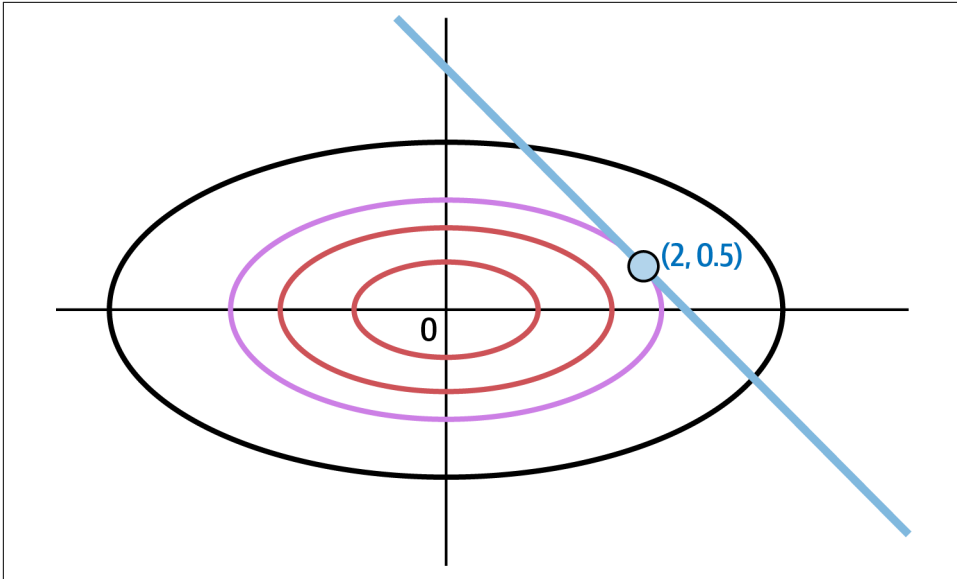


Figure 10-14. The level sets of the quadratic function $x_1^2 + 4x_2^2$ are concentric ellipses; each of them has a constant value. When we impose the linear constraint $x_1 + x_2 = 2.5$, we get the optimizer $(2, 0.5)$ at exactly the point where one of the level sets is tangent to the constraint. The value of the optimal level set is $x_1^2 + 4x_2^2 = 5$.

Another example that helps us visualize Lagrangian and the upcoming *min-max theorem* is a trivial one-dimensional example:

$$\min_{x=1} x^2$$

The Lagrangian is $\mathcal{L}(x; p) = x^2 - p(1 - x)$. We use this toy example whose optimizer is obviously $x = 1$ with minimal value 1 so that we can visualize the Lagrangian. Recall that the Lagrange formulation makes the dimension jump up; in this case we have one constraint, so the dimension increases from one to two, and in our limited three-dimensional world we can only visualize functions of two variables (x and p). **Figure 10-15** shows the landscape of our trivial Lagrangian function, which is now representative for Lagrangian formulations for quadratic optimization problems with linear constraints. The main thing to pay attention to in **Figure 10-15** is that the optimizers of these kinds of problems $(x^*; p^*)$ happen at *saddle points* of the Lagrangian. These are points where the *second derivative* is positive in one variable and negative in another, so the landscape of the Lagrangian function is convex in one variable (x) and concave in the other (p).

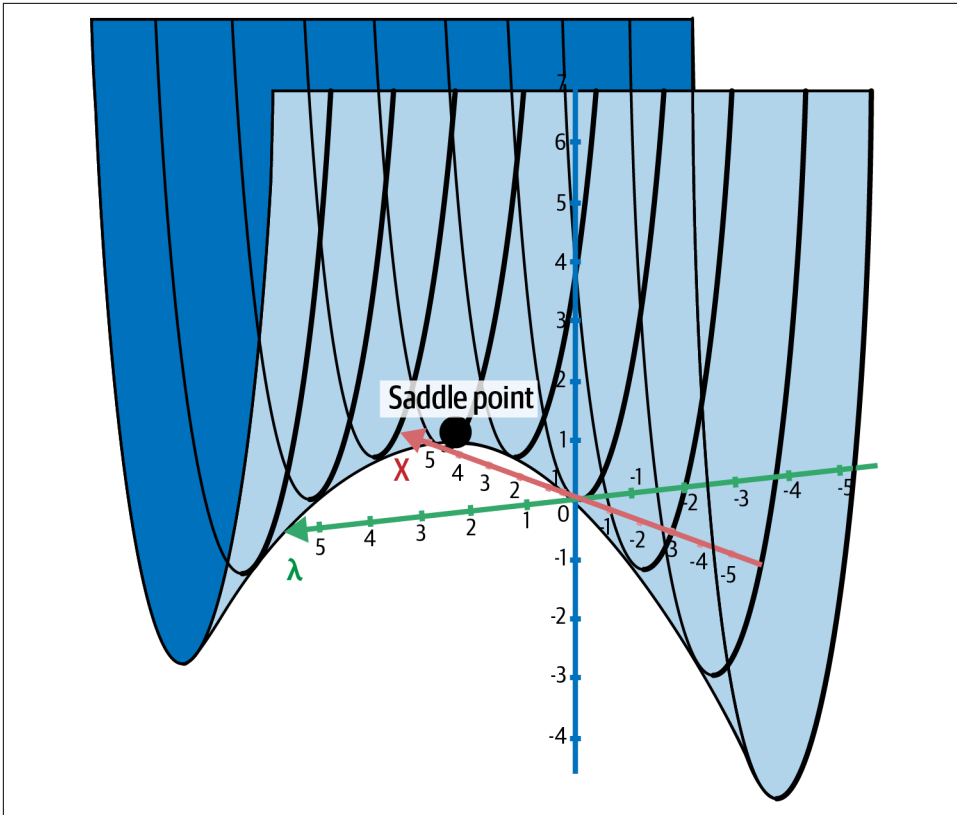


Figure 10-15. The optimizer of the constrained problem happens at the saddle point of the Lagrangian (note that the minimum of the Lagrangian itself is $-\infty$, but that is not what we care about because we care about the optimizer of the quadratic function with linear constraints)

One way to locate the saddle points of the Lagrangian (which give us the optimizers of the corresponding constrained problems) is to solve $\nabla \mathcal{L}(x; p) = \vec{0}$ for x and p , but that is the brute-force way that works for simple problems (like the trivial one at hand) or for small problems. Another way to find these saddle points is to minimize in x then maximize in p (Figure 10-16). Yet another way is to maximize in p then minimize in x . The *min-max theorem* says that these two paths are the same.

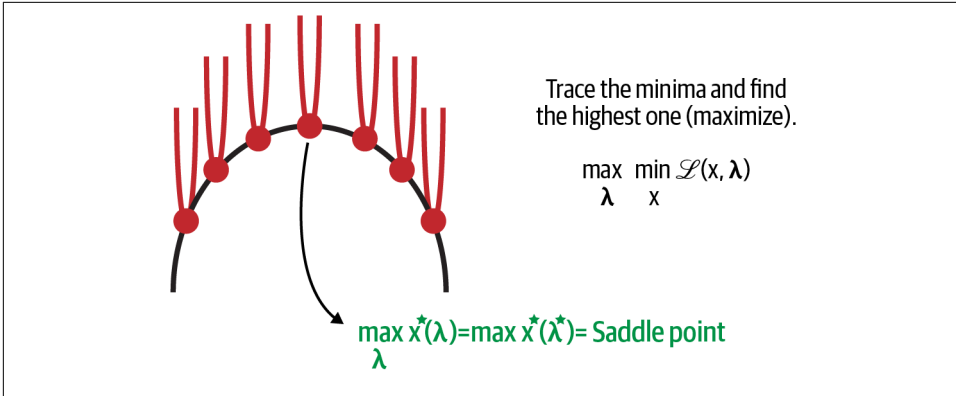


Figure 10-16. Minimizing over x , then maximizing over p gives $\max_p x^*(p) = x^*(p^*) =$ the saddle point. This will give the same answer as maximizing over p , then minimizing over x .

Therefore, at the saddle point (x^*, p^*) , we have $\nabla \mathcal{L}(x; p) = 0$ (which is the same as $\frac{\partial \mathcal{L}(x; p)}{\partial x} = 0$ and $\frac{\partial \mathcal{L}(x; p)}{\partial p} = 0$), and

$$\min_x \max_{p, \text{hold } x \text{ fixed}} \mathcal{L}(x; p) = \max_p \min_{x, \text{hold } p \text{ fixed}} \mathcal{L}(x; p)$$

We have gone full circle to yet again demonstrate that accompanying in a constrained minimization problem in x we have another constrained maximization problem in the Lagrange multipliers p . The interplay between Lagrange multipliers, duality, and constrained optimization is on full display.

Now that we have gone through the important ideas, let's go back and put them in the context of the higher-dimensional quadratic problem with linear constraints that we started this subsection with:

$$\min_{\vec{A}\vec{x} = \vec{b}} \frac{1}{2} \vec{x}^t S \vec{x}$$

where S is a symmetric and positive definite matrix. The Lagrange formulation with *relaxed* constraints is:

$$\min \mathcal{L}(\vec{x}; \vec{p}) = \min \frac{1}{2} \vec{x}^t S \vec{x} + \vec{p} \cdot \left(\vec{b} - A \vec{x} \right)$$

Solving this unconstrained problem, whether by setting $\nabla \mathcal{L}(\vec{x}; \vec{p}) = \vec{0}$, or by minimizing over $\vec{x}$ then maximizing over $\vec{p}$, or by maximizing over $\vec{p}$ then minimizing over $\vec{x}$, we get the same solution $(\vec{x}^*; \vec{p}^*)$, which happens at the *saddle point* of our high-dimensional Lagrangian, and gives the optimal value of the objective function (the advantage of this problem with simple structure is that we can solve it by hand):

$$\text{minimum cost } f = \frac{1}{2} \vec{b} \cdot (AS^{-1}A^t)^{-1} \vec{b}$$

Moreover, the optimal *shadow prices* are: $\vec{p}^* = \frac{df}{d\vec{b}} = (AS^{-1}A^t)^{-1} \vec{b}$.

The last thing we need to learn here is the characterization of saddle points in higher dimensions. For the one-dimensional constrained problem, the hallmark was having the second derivative of the Lagrangian (which was a function of x and p) negative in one variable and positive in the other. The high-dimensional analog is this: the eigenvalues of the Hessian matrix (matrix of second derivatives) is negative in one set of variables and positive in the other set of variables, so it is concave in one set of variables and convex in the other. Our discussion applies to optimizing any higher-dimensional objective function that is convex in one set of variables and concave in the other. This is the hallmark that the landscape has saddle points. For the Lagrangian function, the saddle point is exactly where the constrained problem attains its minimum.

Does this apply to linear optimization problems with linear constraints, which are everywhere in operations research? Yes, as long as we have the correct signs for all the coefficients in the problem, such as the max-flow min-cut and the two-person zero-sum game examples we saw in the previous subsections.

Sensitivity

Here we care about the sensitivity of the optimization problem and its solution with respect to changes in its input data. That is, what happens to the optimal solution $\vec{x}^*$ and the optimal cost $\vec{c} \cdot \vec{x}^*$ if we slightly change $\vec{c}$ or A or $\vec{b}$? Can we obtain the new optimal solution from the old one? Under what conditions can we do that? These are some important cases that sensitivity analysis addresses:

- We have already interpreted the optimal $\vec{p}$ in the dual problem as the vector of marginal prices. This is related to sensitivity analysis: the rate of change of the optimal cost with respect to the constraint value.

- If we add a new decision variable, we check its reduced cost, and if it is negative, we add a new column to the tableau and proceed from there.
- If an entry of $\vec{b}$ or $\vec{c}$ is changed by δ , we obtain an interval of values of δ for which the same basis remains optimal.
- If an entry of A is changed by δ , a similar analysis is possible. However, this case is somewhat complicated if the change affects an entry of a basic column.

In general, if we have a function and we want its sensitivity to variations with respect to one of its inputs, then that is similar to asking about its first derivative with respect to that input (at a certain state), or a discrete first derivative (finite difference) at that state. What makes sensitivity questions more interesting here is the fact that we are dealing with *constrained* problems and checking the effect of small variations in all kinds of inputs to the problem.

Game Theory and Multiagents

Game theory is so important for economics, politics, military operations, multiagent AI, and basically for modeling any environment where there are adversaries or competitors and we have to make decisions or strategize in these conditions. Our optimal strategies are heavily influenced by our adversaries' strategies, whether we know them or are only speculating about them.

The easiest and most well-understood game theory setting is that of *two-person zero-sum games*, which we saw when discussing duality. Here, there are two competing entities, where the loss of one entity is the win of the other, for example, two political campaigns or two competing firms. It has been a challenge to extend the theory to more complex real-life situations with many competitors with varying advantages and disadvantages over each other, varying degrees of cooperation, along with many interrelated strategies. There is still a gap between the situations that the theory can accurately portray and analyze and real-life situations. Progress is happening, and many researchers are on this case, due to the incredible benefits such a complete theory would bring to the world. Imagine being able to view the whole network of adversaries from above, with their movements, connections, possible strategies, and their consequences.

For multiagent environments, game theory models the rational behavior or decision-making process for each involved agent (player, firm, country, military, political campaign, etc.). In this sense, game theory for multiagents is similar to decision theory for a single agent.

The most important concept for noncooperative game theory (where the agents make their decisions independently) is that of the *Nash equilibrium*: a strategy outline for the game where each agent has no incentive to deviate from the outline's prescribed strategy. That is, the agent will be worse off if they deviate from the strategy, of course, assuming everyone is acting rationally.

As we saw in the section on duality, for two-person zero-sum games, we can model them as a min-max problem, and use the *min-max theorem*. We can also model them as a linear optimization problem, where one player is solving the primal problem, and the other is solving the dual problem. This means that we can either set up the optimization problem for the first player or for the second player. Both problems will end up with the same solution. Here we are given the payoff chart for all strategies in the game for both players, and the objective is to find the strategy combination that maximizes the pay (or minimizes the loss) for each player. Intuitively, we can see why duality is built into this problem. The two players are pushing against each other, and the optimal strategy for each player solves both the primal and the dual problems.

We can also use graphs and results from graph theory to analyze two-person games. This is similar to how we can formulate the max flow through a network as a linear optimization problem. Ultimately, many things in math connect neatly together, and one of the most satisfying feelings is when we understand these connections.

For multiagents, certain techniques are available for decision making, which include: voting procedures, auctions for allocating scarce resources, bargaining for reaching agreements, and contract net protocol for task sharing. In terms of mathematical modeling for multiagent games, we will discuss in [Chapter 13](#) (AI and partial differential equations) the *Hamilton-Jacobi-Bellman* partial differential equation. Here, to find the optimal strategy for each player, we have to solve a high-dimensional Hamilton-Jacobi-Bellman type partial differential equation for the game's value function. Before deep learning, these types of high-dimensional partial differential equations were intractable, and one had to make many approximations, or not consider all the participating entities. Recently (2018), a [deep learning technique](#) has been applied to solve these high-dimensional partial differential equations, once reformulated as a backward stochastic differential equation with terminal conditions (don't worry if you do not know what this means; it is not important for this chapter).

We encountered another two-person adversarial game theoretic setting earlier in this book when discussing generative adversarial networks in [Chapter 8](#) on probabilistic generative models.

Queuing

Queues are everywhere: computing jobs for machines, service queues at a shipyard, queues at the emergency room, airport check-in queues, and queues at the local Starbucks. Well-designed queue systems save different facilities and our entire economy an invaluable amount of time, energy, and money. They enhance our overall well-being.

Mathematical modeling of queues has the objective of determining the appropriate level of service to minimize waiting times. The model might include a priority discipline, which means that there are priority groups, and the order in which the members get serviced depends on their priority groups. It might also include different type of services that happen sequentially or in parallel, or some in sequence and others in parallel (for example, in a ship maintenance facility). Some models include multiple service facilities—a queuing network.

There are thousands of papers on queuing theory. It is important to recognize the basic ingredients of a queuing mathematical model:

- The members of the queue (customers, ships, jobs, patients) arrive at certain inter-arrival times. If the arrival process is random, then the math model must decide on a probability distribution that this inter-arrival time adheres to, either from the data or from mathematical distributions known to model such times. Some models assume constant arrival times. Others assume the exponential distribution (a Markovian process) as it facilitates the mathematical analysis and mimics the real-life process better. Others assume the *Erlang* distribution, which allows different exponential distributions for different time intervals. Others assume even more general distributions. The more general the distribution, the less easy the mathematical analysis. Numerical simulations are our best friend forever.
- The number of servers (parallel and sequential) available: an integer.
- The service times also follow a certain probability distribution that we must decide on. Common distributions are similar to those used for inter-arrival times.

In addition, the mathematical model must also keep track of the following:

- The initial number of members in the full queuing system (those waiting and those currently being serviced)
- The probability of having n members in the full queuing system at a given later time

Finally, the model wants to compute the *steady state of the queuing system*:

- The probability of having n members in the full queuing system
- The expected number of new members arriving per unit time
- The expected number of members completing their service per unit time
- The expected waiting time for each member in the system

Members enter the queue at a certain mean rate, wait to get serviced, get serviced at a certain mean rate, then leave the facility. The mathematical model must quantify these and balance them.

Inventory

With the current shortages in the supply chain, the symptoms are empty shelves at grocery stores, shortages of car repair parts, new cars, materials for home renovation, and many others. There is obviously a gap between supply and demand. The times between replenishing the supplies at stores have increased in a way that is causing backlogs, low productivity, and an overall slowed economy. Mathematical models for inventory management quantify the supply (stochastically or deterministically), the demand, and devise an optimal inventory policy for timing replenishing and deciding on the quantity required at each replenish. Ideally, the model must have access to an information processing system that gathers data on current inventory levels and then signals when and by how much to replenish them.

Machine Learning for Operations Research

For starters, what is extremely exciting nowadays in operations research, as opposed to 10 years ago, is the ability to solve massive operations research problems, sometimes involving tens of millions constraints and decision variables. We have to thank the computational power explosion and the continuing improvement of computer implementations of operations research algorithms for this.

Moreover, machine learning can help predict the values of many parameters that enter into operations research models using volumes of available data. If these parameters were hard to measure, modelers had to either remove them from the model or make assumptions about their values. This doesn't have to be the case anymore because of more accurate machine learning models that are able to take thousands of variables into account.

Finally, machine learning can help speed up searching through combinatorially large search spaces by *learning* which parts of the space to focus on or which subproblems to prioritize. This is exactly what the article “[Learning to Delegate for Large-scale Vehicle Routing](#)” (Li et al. 2021) does, speeding up vehicle routing 10 to 100 more times than the state-of-the-art available routing algorithms.

Similar research at the intersection of machine learning and operations research is booming with great progress and scalable solutions. The list of abstracts from the conference [Operations Research Meets Machine Learning](#) offers a great variety of relevant projects, such as real-time data synthesis and treatment from sensors in waste bins (tracking the volume) for more efficient waste collection operation (since this relies on real-time data, the team relies on dynamic routing). Another great example is a bike sharing system, where the objective is to predict the number of bikes needed at each location and allocate teams to distribute the required number of bikes efficiently. Here is the abstract:

Operators in a bike sharing system control room are constantly re-allocating bikes where they are most likely to be needed, this requires an insight on the optimum number of bikes needed in each station, and the most efficient way to distribute teams to move the bikes around. Forecasting engines and Decision Optimization is used to calculate the optimal number of bikes for each station at any given time, and plan efficient routes to help the redistribution of bikes accordingly. A solution delivered by DecisionBrain and IBM for the bike sharing system in London is the first application of its kind that uses both optimization and machine learning to solve cycle hire inventory, distribution and maintenance problems, and could easily be re-deployed for other cycle sharing systems around the world.

In fact, [DecisionBrain's projects](#) are worth browsing and thinking through.

Currently, my team and I are working on a problem with the Department of Public Transportation in my city. This is a perfect setting where machine learning meets operations research. Using historical ridership data, in particular daily boardings and alightings at each bus stop in the city, along with population density, demographics, vulnerability, city zoning data, car ownership, university enrollment, and parking data, we use neural networks to predict supply and demand patterns at each stop. Then we use this data and optimal network design from operations research to redesign the bus routes so that the bus stops, in particular those in the most socially vulnerable areas in the city, are adequately and efficiently serviced.

Hamilton-Jacobi-Bellman Equation

The fields of operations research, game theory, and partial differential equations intersect through dynamic programming and the *Hamilton-Jacobi-Bellman* partial differential equation. Richard Bellman (mathematician, 1920–1984) first coined the term *curse of dimensionality* in the context of dynamic programming. Now the curse of dimensionality has rendered real-life applications of this very useful equation limited and unable to incorporate all the players of a game (or competing markers, countries, militaries) and solve for their optimal strategies, or the thousands of variables that can be involved in operations research problems, such as for optimal resource allocation problems. The tides have turned with deep learning. The paper [“Solving High-Dimensional Partial Differential Equations Using Deep Learning”](#) (Han et al.

2018) presents a method to solve this equation and others for very high dimensions. We will discuss the idea of how the authors do it in [Chapter 13](#) on AI and partial differential equations.

Operations Research for AI

Operations research is the science of decision making based on optimal solutions. Humans are always trying to make decisions based on the available circumstances. Artificial intelligence aims to replicate all aspects of human intelligence, including decision making. In this sense, the decision-making methods that operations research employs automatically fit into AI. The ideas in dynamic programming, Markov chains, optimal control and the Hamilton-Jacobi-Bellman equation, advances in game theory and multiagent games, network optimization, and others have evolved along with AI throughout the decades. In fact, many startups market themselves as AI companies, while in reality they are doing good old (and awesome) operations research.

Summary and Looking Ahead

Operations research is the field of making the best decisions given the current knowledge and circumstances. It always comes down to finding clever ways to search for optimizers in very high-dimensional spaces.

One theme throughout this book is the curse of dimensionality, and all the effort researchers put in to find ways around it. In no field does this curse show up as broadly as in operations research. Here, the search spaces grow combinatorially with the number of players in a particular problem: number of cities on a route, number of competing entities, number of people, number of commodities, etc. There are very powerful exact methods and heuristic methods, but there is much room for improvement in terms of speed and scale.

Machine learning, in particular deep learning, provides a way to learn from previously solved problems, labeled data, or simulated data. This speeds up optimization searches if we identify the bottlenecks and are able to articulate the source of the bottleneck as a machine learning problem. For example, a bottleneck can be: *we have too many subproblems to solve but we do not know which ones to prioritize to quickly get us closer to the optimum*. To use machine learning to address this, we need a data set of already solved problems and subproblems, and have a machine learning model learn which subproblems should be prioritized. Once the model learns this, we can use it to speed up new problems.

Other uses of machine learning in operations research include *business as usual* type of machine learning: predict demand from available data, either real-time or historical data, then use operations research to optimize resource allocation. Here

machine learning helps make better predictions for demand and hence increases efficiency and reduces waste.

In this chapter, we gave a broad overview of the field of operations research and its most important types of problems. We especially emphasized linear optimization, networks, and duality. Powerful software packages are available for many useful problems. We hope that these packages keep integrating the latest progresses in the field.

Two topics that are usually not taught in an introductory operations research classes are the Hamilton-Jacobi-Bellman partial differential equation for optimal control and strategies of multiplayer games, and optimizing *functionals* using calculus of variations. These are usually considered advanced topics in partial differential equations. We discussed them here because they both tie naturally into optimization and operations research. Moreover, viewing them in this context demystifies their corresponding fields.

When doing operations research and optimizing for cost reduction, revenue increase, time efficiency, etc., it is important that our optimization models do not ignore the human factor. If the output of a scheduling model messes up low-wage workers' lives through erratic schedules to keep a certain *on-time* company performance, then that is not a good model and the quality of the lives and livelihoods of the workers that a company relies on needs to be quantified, then factored into the model. Yes, *the quality of life* needs to be quantified, since everything else is being factored in, and we cannot leave this out. Companies with hundreds of thousands of low-wage workers have a responsibility that their operations research algorithms do not end up trapping their workers in poverty.

We leave this chapter with this excerpt from a paper authored by [Charles Hitch](#), “[Uncertainties In Operations Research](#)” from 1960. Reading this (the brackets are my edits), one cannot help but ponder how far the operations research field has come since 1960:

No other characteristic of decision making is as pervasive as uncertainty. When, as operations researchers, to simplify a first cut at an analysis, we assume that the situation can be described by certainty equivalents, we may be doing violence to the facts and indeed the violence may be so grievous as to falsify the problem and give us a nonsense solution. How, for example, can we help the military make development decisions—decisions about which aircraft or missiles to develop when the essence of the problem is that no one can predict with accuracy how long it will take to develop any of the competing equipments, or to get them operational, how much they will cost, what their performance will be, or what the world will be like at whatever uncertain future date turns out to be relevant (if indeed, the world still exists then)? When I say “cannot predict with accuracy” I am not exaggerating. We find that typically, for example, the production costs of new equipment are underestimated in the early stages of development by factors of two to twenty (not 2 to 20 per cent, but factors of two

to twenty). Why they are always underestimated, never overestimated, I leave to your fertile imaginations. [...] Another thing that [an operations researcher] can frequently do, especially in problems involving research and development, is to ascertain the critical uncertainties and recommend strategies to reduce them—to buy information. If you do not know which of two dissimilar techniques for missile guidance will turn out to be better, your best recommendation is very likely to be: keep them both in development a while longer and choose between them when more and better information is available. Never mind the people who call you indecisive. You can prove that this kind of indecisiveness can save both money and time. Of course you can't afford to try everything. There isn't enough budget. There aren't enough resources. You remember when we used to say "If you gave the military services everything they asked for they'd try to fortify the moon!" (We'll have to change that figure of speech.) Actually, it is because of limitations on resources that operations research and operations researchers are important. There'd be no problems for us if there were no constraints on resources. It is our job and opportunity to find, or invent, within the constraints, some better pattern of adjusting to an uncertain world than our betters would find if we weren't here; or some better way, taking costs and pay-offs into account, to buy information to reduce the uncertainty.

Probability

Can we still expect anything, if chance is all there is?

—H.

Probability theory is one of the most beautiful subjects in mathematics, moving us back and forth between the stochastic and deterministic realms in what should be magic but turns out to be mathematics and its wonders. Probability provides a systematic way to quantify randomness, control uncertainty, and extend logic and reasoning to situations that are of paramount importance in AI: when information and knowledge include uncertainties, and/or when the agent navigates unpredictable or partially observed environments. In such settings, an agent calculates probabilities about the unobserved aspects of a certain environment, then makes decisions based on these probabilities.

Humans are uncomfortable with uncertainty, but are comfortable with approximations and expectations. They do not wake up knowing exactly how every moment of their day will play out, and they make decisions along the way. A probabilistic intelligent machine exists in a world of probabilities, as opposed to deterministic and fully predetermined truths and falsehoods.

Throughout this book, we have used probability terms and techniques as they came along and only when we needed them. Through this process, we now realize that we need to be well versed in joint probability distributions (for example, of features of data), conditioning, independence, Bayes' Theorem, and Markov processes. We also realize that we can get back to the deterministic world via computing averages and expectations.

One feature of the chapters in this book is that each needs its own book to have an in-depth and comprehensive treatment. This couldn't be more true than for a chapter

on probability, where we can include thousands of topics. I had to make choices, so I based the topics that I opted to cover in this chapter on three criteria:

1. What we already used in this book that has to do with probability
2. What confused me the most in probability as a student (like why do we need measure theory when computing probabilities?)
3. What else we need to know from probability theory for AI applications

Where Did Probability Appear in This Book?

Let's make a fast list of the places where we used probability or resorted to stochastic methods in this book. We consider this list as the *essential probability for AI*. Note that *prior* probabilities are unconditional, because they are prior to observing the data, or the evidence; and *posterior* probabilities are conditional, because their value is conditioned on observing the relevant data. It makes sense that our degree of belief about something changes after receiving new and related evidence. The joint probability distribution of all the involved variables is what we are usually after, but it is generally too large, and the information needed to fully construct it is not always available.

Here is the list:

- When minimizing the loss function of deterministic machine learning models (where the training function takes nonrandom inputs and produces nonrandom outputs), such as regression, support vector machines, neural networks, etc., we use stochastic gradient descent and its variants, randomly choosing a subset of training data instances at each gradient descent step, as opposed to using the whole training data set, to speed up computations.
- In [Chapter 9](#) on graph models, we utilized random walks on graphs on many occasions, implementing these walks via the weighted adjacency matrix of the graph.
- Specific probability distributions appeared in [Chapter 10](#) on operations research, such as probability distributions for inter-arrival and service times for customers in a queue.
- Dynamic decision making and Markov processes also appeared in [Chapter 10](#) on operations research and are fundamental for reinforcement learning in AI. They will appear again in this chapter, then once more in [Chapter 13](#) in the context of the Hamilton-Jacobi-Bellman equation.
- For two-person zero-sum games in [Chapter 10](#), each player had a probability of making a certain move, and we used that to compute the player's optimal strategy and expected payoff.

- Monte Carlo simulation methods are computational algorithms that rely on repeated random sampling to solve deterministic problems numerically. We illustrate an example of these in [Chapter 13](#) on AI and PDEs.
- We mentioned the universality theorem for neural networks many times, and we only refer to its proof in this chapter. The proof of this fundamental theorem is theoretical, and gives us a nice flavor of measure theory and functional analysis. We build the intuition for measure-theoretic probability later in this chapter.
- Probabilistic machine learning models learn the joint probability distribution of the data features $Prob(x_1, x_2, \dots, x_n, y_{target})$ instead of learning the deterministic functions of these features. This joint probability distribution encodes the likelihood of these features occurring at the same time. Given the input data features $(x_1, x_2, \dots, x_n)$, the model outputs the conditional probability of the target variable given the data features $Prob(y_{predict} | x_1, x_2, \dots, x_n)$, as opposed to outputting $y_{predict}$ as a deterministic function of the features: $y_{predict} = f(x_1, x_2, \dots, x_n)$.
- Random variables and the two most important quantities associated with them, namely the expectation (expected average value of the random variable) and variance (a measure of the spread around the average): we have been using these without formally defining them. We will define them in this chapter.
- The product rule or the chain rule for probability, namely:

$$Prob(x_1, x_2) = Prob(x_1 | x_2) Prob(x_2) = Prob(x_2 | x_1) Prob(x_1)$$

or for more than two variables, say three without loss of generality:

$$\begin{aligned} Prob(x_1, x_2, x_3) &= Prob(x_1 | x_2, x_3) Prob(x_2, x_3) \\ &= Prob(x_1 | x_2, x_3) Prob(x_2 | x_3) Prob(x_3) \end{aligned}$$

- The concepts of independence and conditional independence are fundamental. Two events are independent if the occurrence of one does not affect the probability of occurrence of the other. Independence of the considered features is tremendously simplifying. It helps us disentangle complex joint distributions of many variables, reducing them to simple products of fewer variables, and rendering many previously intractable computations tractable. This greatly simplifies the probabilistic interpretations of the world. Pay attention to the difference between independence of *only two events* ($Prob(x_1, x_2) = Prob(x_1) Prob(x_2)$) and independence of *many events*, which is a strong assumption where every event is independent of any intersection of the other events.
- For probabilistic generative models in [Chapter 8](#), we assumed a prior probability distribution, passed it through a neural network, and adjusted its parameters.

- Bayes' Theorem is essential when discussing joint and conditional probabilities. It helps us quantify an agent's beliefs relative to evidence. We use it in many contexts, which immediately illustrate its usefulness, such as:

$$Prob(disease|symptoms) = \frac{Prob(symptoms|disease)Prob(disease)}{Prob(symptoms)}$$

or

$$Prob(target|data) = \frac{Prob(data|target)Prob(target)}{Prob(data)}$$

or

$$Prob(target|evidence) = \frac{Prob(evidence|target)Prob(target)}{Prob(evidence)}$$

or

$$Prob(cause|effect) = \frac{Prob(effect|cause)Prob(cause)}{Prob(effect)}$$

Note that in the last formula, $Prob(cause|effect)$ quantifies the *diagnostic* direction, while $Prob(effect|cause)$ quantifies the *causal* direction.

- Bayesian networks are data structures that represent dependencies among variables. Here, we summarize the variable relationships in a directed graph and use that to determine which conditional probability tables we need to keep track of and update in the light of new evidence: we keep track of the probability of a child node conditional on observing its parents. The parents of a node are any variables that directly influence this node. In this sense, the Bayesian network is a representation of the joint probability distribution, with the simplification that we know how the involved variables relate to each other (which variables are the parents of which variables):

$$Prob(x_1, x_2, \dots, x_n) = \prod_{i=1}^n Prob(x_i | parents(X_i))$$

- In machine learning we can draw a line between regression models and classification models. In [Chapter 8](#) on probabilistic generative models, we encountered a popular probabilistic model for classification: Naive Bayes. In cause-and-effect language, the *naive* assumption is that some observed multiple effects are independent given a cause, so that we can write:

$$\begin{aligned}
 & Prob(cause|effect_1,effect_2,effect_3) \\
 & = P(cause)P(effect_1|cause)P(effect_2|cause)P(effect_3|cause)
 \end{aligned}$$

When that formula is used for classification given data features, the *cause* is the class. Moreover, we can draw a Bayesian network representing this setting. The cause variable is the parent node, and all the effects are children nodes stemming from the one parent node (Figure 11-1).

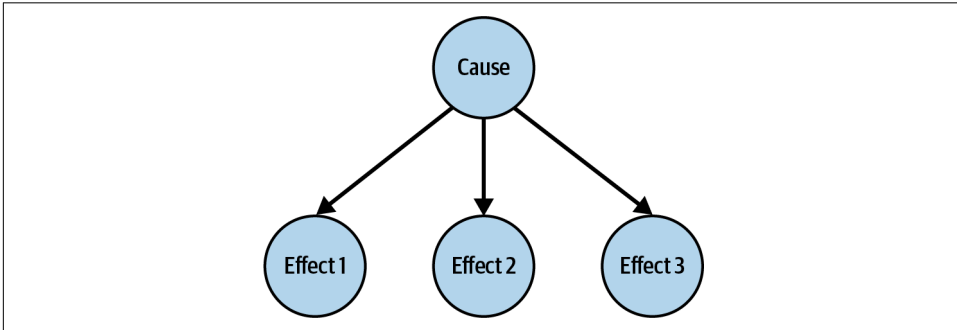


Figure 11-1. Bayesian network representing three effects having a common cause

What More Do We Need to Know That Is Essential for AI?

We need a few extra topics that have either not gotten any attention in this book or were only mentioned casually and pushed to this chapter for more details. These include:

- Judea Pearl's causal modeling and the *do* calculus
- Some paradoxes
- Large random matrices and high-dimensional probability
- Stochastic processes such as random walks, Brownian motion, and more
- Markov decision processes and reinforcement learning
- Theory of probability and its use in AI

The rest of this chapter focuses on these topics.

Causal Modeling and the Do Calculus

In principle, the arrows between related variables in a Bayesian network can point in any direction. They all eventually lead to the same joint probability distribution, albeit some in more complicated ways than others.

In contrast, causal networks are those special Bayesian networks where the directed edges of the graph cannot point in any direction other than the causal direction. For these, we have to be more mindful when constructing the connections and their directions. **Figure 11-2** shows an example of a causal Bayesian network.

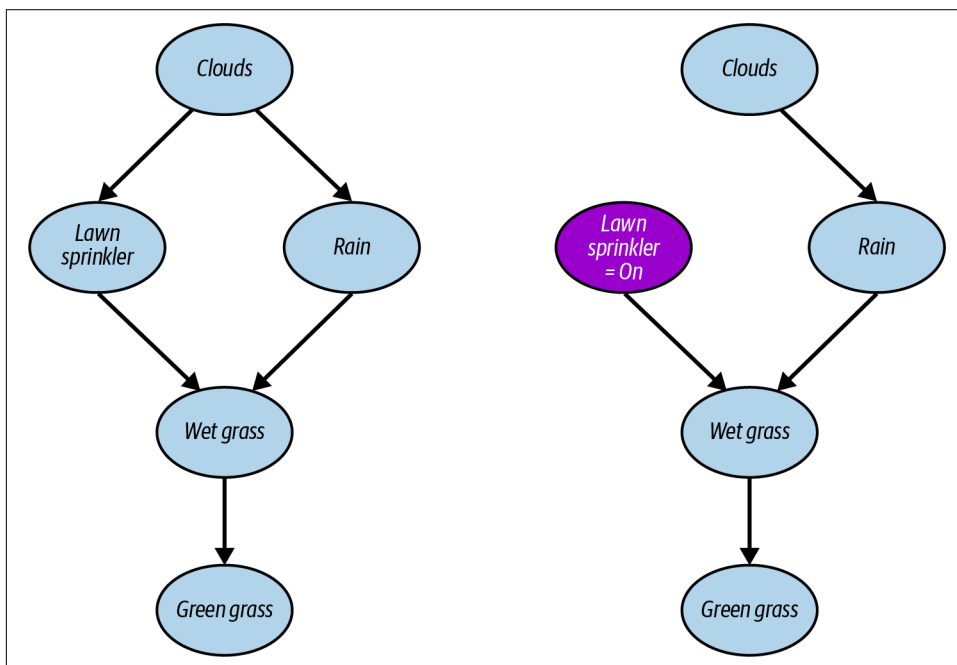


Figure 11-2. Causal Bayesian network

Note that both Bayesian networks and causal networks make strong assumptions on which variables listen to which variables.

Agents endowed with causal reasoning are, in human terms, *higher functioning* than those merely observing patterns in the data, then making decisions based on the relevant patterns.

The following distinction is of paramount importance:

- In Bayesian networks, we suffice ourselves with knowing only whether two variables are probabilistically dependent. Are fire and smoke probabilistically dependent?
- In causal networks, we go further and ask about *which variable responds to which variable*: smoke to fire (so we draw an arrow from fire to smoke in the diagram), or fire to smoke (so we draw an arrow from smoke to fire in the diagram)?

What we need here is a mathematical framework for *intervention* to quantify the effect of fixing the value of one variable. This is called the *do calculus* (as opposed to the statistical *observe and count* calculus). Let's present two fundamental formulas of the *do calculus*:

- The adjustment formula
- The backdoor criterion

According to **Judea Pearl**, the inventor of this wonderful way of causal reasoning (and whose *The Book of Why* (2020) inspires the discussion in this section and the next one), these *allow the researcher to explore and plot all possible routes up mount intervention, no matter how twisty*, and can save us the costs and difficulties of running randomized controlled trials, even when these are physically feasible and legally permissible.

An Alternative: The Do Calculus

Given a causal network, which we construct based on a combination of common sense and subject matter expertise, while at the same time throwing in extra unknown causes for each variable just to be sure that we are accounting for everything, the overarching formula is that of the joint probability distribution:

$$Prob(x_1, x_2, \dots, x_n) = \prod_{i=1}^n Prob(x_i | parents(X_i))$$

Then we *intervene*, applying $do(X_j = x^*)$. This severs any edges pointing to X_j , and affects all the conditional probabilities of the descendants of X_j , leading to a new joint probability distribution that wouldn't include the conditional probability for the intervened variable anymore. We already set its value to $X_j = x^*$ with probability one, and any other value would have probability zero. **Figure 11-2** shows how, when we set the sprinkler on, all arrows leading to it in the original network get severed.

Thus we have:

$$Prob_{intervened}(x_1, x_2, \dots, x_n) = \begin{cases} \prod_{i \neq j}^n Prob(x_i | parents(X_i)) & \text{if } X_j = x^* \\ 0 & \text{otherwise} \end{cases}$$

The adjustment formula

What we truly care about is how does setting $X_j = x^*$ affect the probability of every other variable in the network, and we want to compute these from the original unintervened network. In math words, without the *do* operator, since we can just observe the data to get these values, as opposed to running new experiments.

To this end, we introduce the *adjustment formula*, or *controlling for confounders* (possible common causes). This is a weighted average of the influence of X_j and its parents on X_i . The weights are the priors on the parent values:

$$\begin{aligned} \text{Prob}(x_i | \text{do}(X_j = x^*)) &= \text{Prob}_{\text{intervened}}(X_i = x_i) \\ &= \sum_{\text{parents}(X_j)} \text{Prob}(x_i | x^*, \text{parents}(X_j)) \text{Prob}(\text{parents}(X_j)) \end{aligned}$$

Note that this formula achieves our goal of eliminating the *do* operator and gets us back to finding our conditional probabilities by observing the data, rather than running some costly intervention experiments, or randomized control trials.

The backdoor criterion, or controlling for confounders

There is more to the causal diagrams story. We would like to know the effect of the intervention $\text{do}(X_j = x^*)$ on a certain *downstream* variable in the diagram X_{down} . We should be able to condition on the values of any variable in the diagram that is *another ancestor*. This also leads down to the downstream variable that we care about. In causal modeling, we call this process *blocking the back doors* or the *backdoor criterion*:

$$\begin{aligned} P(x_{\text{down}} | \text{do}(X_j = x^*)) &= P_{\text{intervened}}(X_{\text{down}} = x_{\text{down}}) \\ &= \sum_{\text{ancestor}(X_{\text{down}})} P(x_{\text{down}} | x^*, \text{ancestor}(X_{\text{down}})) P(\text{ancestor}(X_{\text{down}})) \end{aligned}$$

Controlling for confounders

The most common way for scientists and statisticians to predict the effects of an intervention so that they can make statements about causality, is to control for *possible common causes*, or confounders. **Figure 11-3** shows the variable Z as a confounder of the suspected causal relationship between X and Y.

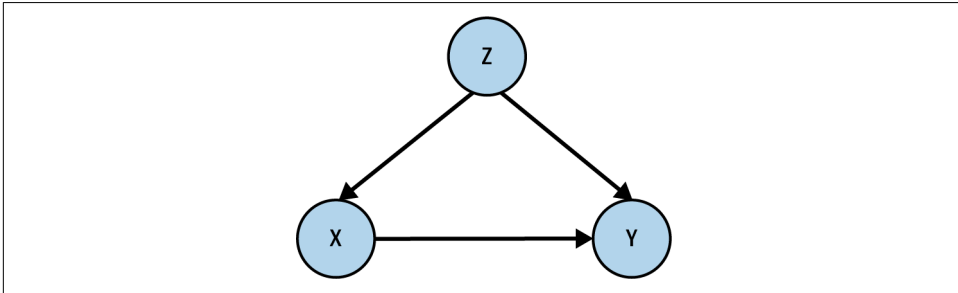


Figure 11-3. Z is a confounder of the suspected causal relationship between X and Y

This is because, in general, confounding is a main source of confusion between mere observation and intervention. It is also the source of the famous statement *correlation is not causation*. This is where we see some bizarre and entertaining examples: high temperature is a confounder for ice cream sales and shark attacks (but why would anyone study any sort of relationship between ice cream and sharks to start with?). The backdoor criterion and the adjustment formula easily take care of confounder obstacles to stipulating about causality.

We use the adjustment formula to control for confounders if we are confident that we have data on a sufficient set of *deconfounder* variables to block all the backdoor paths between the intervention and the outcome. To do this, we estimate the causal effect stratum by stratum from the data, then we compute a weighted average of those strata, where each stratum is weighted according to its prevalence in the population.

Now, without the backdoor criterion, statisticians and scientists have no guarantee that any adjustment is legitimate. In other words, the backdoor criterion guarantees that the causal effect in each stratum of the deconfounder is in fact the observed trend in this stratum.

Are there more rules that eliminate the *do* operator?

Rules that are able to move us from an expression with the *do* operator (intervention) to an expression without the *do* operator (observation) are extremely desirable, since they eliminate the need to intervene. They allow us to estimate causal effects by mere data observation. The adjustment formula and the backdoor criterion did exactly that for us.

Are there more rules? The more ambitious question is: is there a way to decide ahead of time whether a certain causal model lends itself to *do* operator elimination, so that we would know whether the assumptions of the model are sufficient to uncover the causal effect from observational data without any intervention? Knowing this is huge! For example, if the assumptions of the model are not sufficient to eliminate the *do* operator, then no matter how clever we are, there is no escape from running interventional experiments. On the other hand, if we do not have to intervene and still estimate causal effects, the savings are spectacular. These alone are worth digging more into probabilistic causal modeling and the *do* calculus.

To get the gist of Judea Pearl's *do* calculus, we always start with a causal diagram, and think of conditioning criteria leading to the deletion of edges pointing toward or out from the variable(s) of interest. Pearl's three rules give us the conditions under which:

1. We can insert or delete observations:

$$Prob(y \mid do(x), z, w) = Prob(y \mid do(x), w)$$

2. We can insert or delete interventions:

$$Prob(y|do(x),do(z),w) = Prob(y|do(x),w)$$

3. We can exchange interventions with observations:

$$Prob(y|do(x),do(z),w) = Prob(y|do(x),z,w)$$

For more details on the *do* calculus, see “[The Do-Calculus Revisited](#),” by Judea Pearl (Keynote Lecture, August 17, 2012).

Paradoxes and Diagram Interpretations

AI agents need to be able to handle paradoxes. We have all seen cartoons where a robot gets into a crazy loop or even physically self-dismantles with screws and springs flying all around when its logic encounters a paradox. We cannot let that happen. Furthermore, paradoxes often appear in very consequential settings, such as in the pharmaceutical and medical fields, so it is crucial that we scrutinize them under the lens of mathematics and carefully unravel their mysteries.

Let’s go over three famous paradoxes: *Monty Hall*, *Berkson*, and *Simpson*. We will view them in the light of diagrams and causal models: Monty Hall and Berkson paradoxes cause confusion due to colliders (two independent variables pointing to a third one), while Simpson paradoxes cause confusion due to confounders (one variable pointing to two others). An AI agent should be equipped with these diagrams as part of its data structure (or with the ability to construct them and adjust them) in order to reason properly.

Judea Pearl’s *The Book of Why* puts it perfectly:

Paradoxes reflect the tensions between causation and association. The tension starts because they stand on two different rungs of the Ladder of Causation [observation, intervention, counterfactuals] and is aggravated by the fact that human intuition operates under the logic of causation, while data conform to the logic of probabilities and proportions. Paradoxes arise when we misapply the rules we have learned in one realm to the other.

Monty Hall Problem

Suppose you’re on a game show, and you’re given the choice of three doors. Behind one door is a car, behind the others, goats. You pick a door, say #1, and the host, who knows what’s behind the doors, opens another door, say #3, which has a goat. He says to you, “Do you want to pick door #2?” Is it to your advantage to switch your choice of doors?

The answer is yes, switch doors, because without switching, your probability of getting the car is $1/3$, and after switching, it jumps up to $2/3$! The main thing to pay attention to here is that the host *knows* where the car is, and chooses to open a door that he knows does not have the car in it.

So why would the probability of winning double if we switch from our initial choice? Because the host offers new information that we would leverage only if we switch from our initial information-less choice:

Under the no-switch strategy

- If we initially chose the winning door (probability $1/3$), and we do not switch, then we win.
- If we initially chose a losing door (probability $2/3$), and we do not switch, then we lose.

This means that we would win only $1/3$ of the time under the no-switch strategy.

Under the switch strategy

- If we initially chose the winning door (probability $1/3$), and we switch from it, then we would lose.
- If we initially chose a losing door (probability $2/3$), new information comes *pointing to the other losing door*, and we switch, then we would win, because the only door left would be the winning door.

This means that we would win $2/3$ of the time under the switch strategy.

When we draw the diagram in **Figure 11-4** to represent this game, we realize that the door that the host chooses to open has two parents pointing toward it: the door you chose and the location of the car.

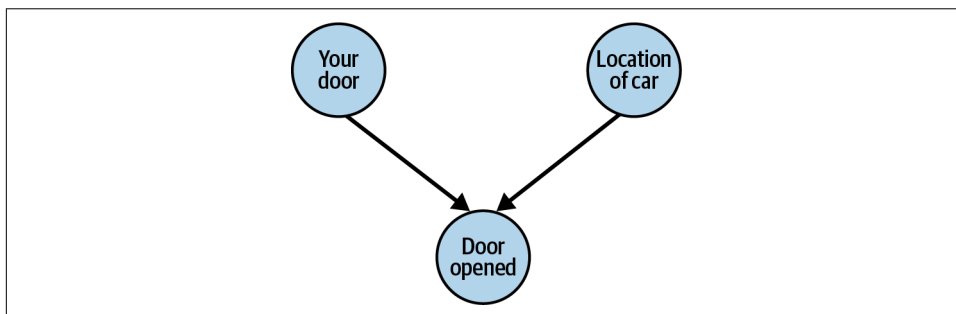


Figure 11-4. Causal diagram for the variables involved in the Monty Hall paradox

Conditioning on this *collider* changes the probabilities of the parents. It creates a spurious dependency between originally independent parents! This is similar to us changing our beliefs about the genetic traits of parents once we meet one of their children. These are causeless correlations, induced when we condition on colliders.

Now suppose that the host chooses their door *without knowing* whether it is a winning or a losing door. Then switching or nonswitching would not change the odds of winning the car, because in this case both you and the host have equal chances of winning $1/3$ of the time and losing $2/3$ of the time. Now when we draw the diagram for this totally random and no-prior-knowledge game, there is no arrow between the location of the car and the door that the host chooses to open, so your choice of the door and the location of the car *remain independent* even after conditioning on the host's choice.

Berkson's Paradox

In 1946, Joseph Berkson, a biostatistician at the Mayo Clinic, pointed out a peculiarity of observational studies conducted in a hospital setting: even if two diseases have no relation to each other in the general population, they can appear to be associated among patients in a hospital. In 1979, David Sackett of McMaster University, an expert on all sorts of statistical bias, provided strong evidence that Berkson's paradox is real. In one example, he studied two groups of diseases: respiratory and bone. About 7.5% of people in the general population have a bone disease, and this percentage is independent of whether they have respiratory disease. But for hospitalized people with respiratory disease, the frequency of bone disease jumps to 25%! Sackett called this phenomenon "admission rate bias" or "Berkson bias."

Similar to the Monty Hall case, the culprit for the appearance of the Berkson paradox is a collider diagram, where both originally independent diseases point to hospitalization: a patient with both diseases is much more likely to be hospitalized than a patient with only one of them. When we condition on hospitalization, which is the collider, a case of causeless correlation between the initially independent variables appears. We are getting used to collider bias now.

Simpson's Paradox

Imagine a paradox whose conclusion, if left to its own devices, is this absurd: *when we know the gender of the patient, then we should not prescribe the drug, because the data shows that the drug is bad for males and bad for females; but if the gender is unknown, then we should prescribe the drug, because the data shows that the drug is good for the general population.* This is obviously ridiculous, and our first instinct should be to protest: show me the data!

We recognize Simpson's paradox when a trend appears in several groups of the population but disappears or reverses when the groups are combined.

Let's first debunk the paradox. It is a simple numerical mistake of how to add fractions (or proportions). In summary, when we add fractions, we cannot simply add the respective numerators and the denominators:

$$\frac{A}{B} > \frac{a}{b} \text{ and } \frac{C}{D} > \frac{c}{d} \not\Rightarrow \frac{A+C}{B+D} > \frac{a+c}{b+d}$$

For example, suppose that the data shows that:

- 3/40 of the women who took the drug had a heart attack, compared to only 1/20 of the women who did not take the drug ($3/40 > 1/20$).
- 8/20 of the men who took the drug had a heart attack, compared to 12/40 of the men who did not take the drug ($8/20 > 12/40$).

Now when we merge the data for women and men, the inequality reverses direction: $3/40 > 1/20$ and $8/20 > 12/40$, but rightfully $(3 + 8)/(40 + 20) < (1 + 12)/(20 + 40)$. In other words: of the 60 men and women who took the drug, 11 had a heart attack, and of the 60 men and women who did not take the drug, 13 had a heart attack.

However, we committed a simple mistake with fractions when we merged the data this way. To solve Simpson's paradox, we should not merge the data by simply adding the numerators and the denominators and expect the inequality to hold. Note that of the 60 people who took the drug, 40 are women and 20 are men; while of the 60 people who did not take the drug, 20 are women and 40 are men. We are comparing apples and oranges and confounding that with gender. The gender affects *both* whether the drug is administered and whether a heart attack happens. The diagram in [Figure 11-5](#) illustrates this confounder relationship.

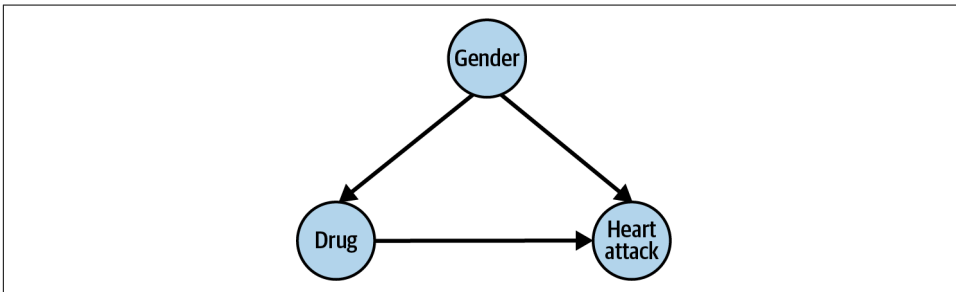


Figure 11-5. Gender is a confounder for both taking the drug and heart attack

Our strong intuition that something is wrong if we merge proportions naively is spot on. If things are fair on a local level everywhere, then they are fair globally; or if things act a certain way on every local level, then we should expect them to act that way globally.

It is no surprise that this mistake happens so often, as humans did not get fractions right until relatively recently. There are ancient texts with mistakes manipulating fractions in domains such as inheritance and trade. Our brains' resistance to fractions

seems to persist: we learn fractions in seventh grade, and that also happens to be the time to which we can trace the origin of many people's legendary hatred for math.

So what is the correct way to merge the data? Our grade-seven wisdom tells us to use the common denominator 40 and to condition on gender: for women $3/40 > 2/40$ and for men $16/40 > 12/40$. Now since in the general population men and women are equally distributed, we should take the average and rightfully conclude that $(3/40 + 16/40)/2 > (2/40 + 12/40)/2$; that is, the rate of heart attacks in the general population is 23.75% with the drug, and 17.5% without the drug. No magical and illogical reversal happened here. Moreover, this drug is pretty bad!

Large Random Matrices

Most AI applications deal with a vast amount of high-dimensional data (big data), organized in high-dimensional vectors, matrices, or tensors, representing data tables, images, natural language, graph networks, and others. A lot of this data is noisy or has an intrinsic random nature. To process such data, we need a mathematical framework that combines probability and statistics, which usually deal with *scalar* random variables, with linear algebra, which deals with *vectors and matrices*.

The mean and variance are still central ideas, so we find many statements and results containing the expectation and variance (uncertainty) of the involved high-dimensional random variables. Similar to the scalar case, the tricky part is controlling the variance, so a lot of work in the literature finds bounds (inequalities) on the tails of the random variables' distributions or how likely it is to find a random variable within some distances from its mean.

Since we now have matrix valued random variables, many results seek to understand the behaviors (distributions) of their spectra: eigenvalues and eigenvectors.

Examples of Random Vectors and Random Matrices

It is no wonder the study of large random matrices evolved into its own theory. They appear in all sorts of impactful applications, from finance to neuroscience to physics and the manufacture of technological devices. The following is only a sampling of examples. These have great implications, so there are large mathematical communities around each them.

Quantitative finance

One example of a random vector is an investment portfolio in quantitative finance. We often need to decide on how to invest in a large number of stocks, whose price movement is stochastic, for optimal performance. The investment portfolio itself is a large random vector that evolves with time. In the same spirit, the daily returns

of Nasdaq stocks (Nasdaq contains more than 2,500 stocks) is a time-evolving large random vector.

Neuroscience

Another example is from neuroscience. Random matrices appear when modeling a network of synaptic connections between neurons in the brain. The number of spikes fired by n neurons during t consecutive time intervals of a certain length is an $n \times t$ random matrix.

Mathematical physics: Wigner matrices

In mathematical physics, particularly in nuclear physics, physicist Eugene Wigner introduced random matrices to model the nuclei of heavy atoms and their spectra. In a nutshell, he related the spacings between the lines in the spectrum of a heavy atom's nucleus to the spacings between the eigenvalues of a random matrix.

The deterministic matrix that Wigner started with is the Hamiltonian of the system, which is a matrix describing all the interactions between the neutrons and protons contained in the nucleus. The task of diagonalizing the Hamiltonian to find the energy levels of the nucleus was impossible, so Wigner looked for an alternative. He abandoned exactness and determinism altogether and approached the question from a probabilistic perspective. Instead of asking *what precisely are the energy levels*, he asked questions like:

- What is the probability of finding an energy level within a certain interval?
- What is the probability that the distance between two successive energy levels is within a certain range?
- Can we replace the Hamiltonian of the system by a purely random matrix with the correct symmetry properties? For example, in the case of quantum systems invariant under time reversal, the Hamiltonian is a real symmetric matrix (of infinite size). In the presence of a magnetic field, the Hamiltonian is a complex, Hermitian matrix (the complex analog of a real symmetric matrix). In the presence of *spin-orbit coupling* (a quantum physics term), the Hamiltonian is symplectic (another special type of symmetric matrix).

Similarly, Wigner-type random matrices appear in condensed matter physics, where we model the interaction between pairs of atoms or pairs of spins using real symmetric Wigner matrices. Overall, Wigner matrices are considered *classical* in random matrix theory.

Multivariate statistics: Wishart matrices and covariance

In multivariate statistics, John Wishart introduced random matrices when he wanted to estimate sample covariance matrices of large random vectors. Wishart random matrices are also considered classical in random matrix theory. Note that a sample covariance matrix is an estimation for the population covariance matrix.

When dealing with sample covariance matrices, a common setting is that of n dimensional variables observed t times, that is, the original data set is a matrix of size $n \times t$. For example, we might need to estimate the covariance matrix of the returns of a large number of assets (using a smaller sample), such as the daily returns of the 2,500 Nasdaq stocks. If we use 5 years of daily data, given that there are 252 trading days in a year, then we have $5 \times 252 = 1,260$ data points for each of the 2,500 stocks. The original data set would be a matrix of size $2,500 \times 1,260$. This is a case where the number of observations is smaller than the number of variables. We have other cases where it is the other way around, as well as limiting cases where the number of observations and the number of variables are of drastically different scales. In all cases, we are interested in the law (probability distribution) for the eigenvalues of the sample covariance matrix.

Let's write the formulas for the entries of the covariance matrix. For one variable $\vec{z}_1$ (say one stock) with t observations whose mean (average) is $\bar{z}_1$, we have the variance:

$$\sigma_1^2 = \frac{(z_1(1) - \bar{z}_1)^2 + (z_1(2) - \bar{z}_1)^2 + \cdots + (z_1(t) - \bar{z}_1)^2}{t}$$

Similarly, for each of the n variables $\vec{z}_i$, we have their variance σ_i^2 . These sit on the diagonal of the covariance matrix. Now each off-diagonal σ_{ij} entry is the covariance of the corresponding pair of variables:

$$\sigma_{ij} = \frac{(z_i(1) - \bar{z}_i)(z_j(1) - \bar{z}_j) + (z_i(2) - \bar{z}_i)(z_j(2) - \bar{z}_j) + \cdots + (z_i(t) - \bar{z}_i)(z_j(t) - \bar{z}_j)}{t}$$

The covariance matrix is symmetric and positive definite (has positive eigenvalues). The randomness in a covariance matrix usually stems from noisy observations. Since measurement noise is inevitable, determining the covariance matrix becomes more involved mathematically. Another common issue is that often the samples are not independent. Correlated samples introduce some sort of redundancy, so we expect that the sample covariance matrix behaves as if we had observed fewer samples than we actually did. We must then analyze the sample covariance matrix in the presence of correlated samples.

Dynamical systems

Linearized dynamical systems are near equilibria ($\frac{d\vec{x}(t)}{dt} = A\vec{x}(t)$). In the context of chaotic systems, we want to understand how a small difference in initial conditions propagates as the dynamics unfolds. One approach is to linearize the dynamics in the vicinity of the unperturbed trajectory. The perturbation evolves as a product of matrices, corresponding to the linearized dynamics, applied on the initial perturbation.



Algorithms for Matrix Multiplication

Finding efficient algorithms for matrix multiplication is an essential, yet surprisingly difficult, goal. In matrix multiplication algorithms, saving on even one multiplication operation is worthy (saving on addition is not as much of a big deal). Recently, DeepMind developed [AlphaTensor](#) (2022) to automatically discover more efficient algorithms for matrix multiplication. This is a milestone because matrix multiplication is a fundamental part of a vast array of technologies, including neural networks, computer graphics, and scientific computing.

Other equally important examples

There are other examples. In number theory, we can model the distribution of zeros of the Riemann zeta function using the distribution of eigenvalues of certain random matrices. For those keeping an eye out for quantum computing, here's a historical note: before Schrödinger's equation, Heisenberg formulated quantum mechanics in terms of what he named *matrix mechanics*. Finally, we will encounter the *master equation for the evolution of probabilities* in [Chapter 13](#). This involves a large matrix of transition probabilities from one state of a system to another state.

Main Considerations in Random Matrix Theory

Depending on the formulation of the problem, the matrices that appear are either deterministic or random. For deterministic vectors and matrices, classical numerical linear algebra applies, but the extreme high dimensionality forces us to use randomization to efficiently do matrix multiplication (usually $O(n^3)$), decomposition, and computing the spectrum (eigenvalues and eigenvectors).

A substantial amount of matrix properties is encapsulated in their spectra, so we learn a great deal about matrices by studying these eigenvalues and eigenvectors. In the stochastic realm, when the matrices are random, these are random as well. So how do we compute them and find their probability distributions (or even only their means and variances or bounds on those)? These are the types of questions that the field of large random matrices (or randomized linear algebra, or high-dimensional probability) addresses. We usually focus on:

The involved stochastic math objects

Random vectors and random matrices. Each entry of a random vector or a random matrix is a random variable. These could either be static random variables or evolving with time. When a random variable evolves with time, it becomes a *random or stochastic process*. Obviously, stochastic processes are more involved mathematically than their static counterparts. For example, what can we say about variances that evolve with time?

Random projections

Our interest is always in projecting onto some lower-dimensional spaces while preserving essential information. These usually involve either multiplying matrices with vectors or factorizing matrices into a product of simpler matrices, such as the singular value decomposition. How do we do these when the data is large and the entries are random?

Adding and multiplying random matrices

Note that the sums and products of *scalar* random variables are also random variables, and their distributions are well studied. Similarly, the sums and products of *time evolving scalar* random variables, which are the foundation for Brownian motion and stochastic calculus, have a large body of literature supporting them. How does this theory transition to higher dimensions?

Computing the spectra

How do we compute the spectrum of a random matrix, and explore the properties of its (random) eigenvalues and eigenvectors?

Computing the spectra of sums and products of random matrices

How do we do this as well?

Multiplying many random matrices, as opposed to only two

This problem appears in many contexts in the technological industry, for example, when studying the transmission of light in a succession of slabs of different optical indices, or the propagation of an electron in a disordered wire, or the way displacements propagate in granular media.

Bayesian estimation for matrices

Bayesian *anything* always has to do with estimating the probability of something given some evidence. Here, the matrix we start with (the observations matrix) is a noisy version of the true matrix that we care for. The noise can be additive, so the observed matrix $E = \text{true matrix} + \text{a random noise matrix}$. The noise can also be multiplicative, so the observed matrix $E = \text{true matrix} \times \text{random noise matrix}$. In general, we do not know the true matrix, and would like to know the probability of this matrix given that we have observed the noisy matrix. That is, we have to compute $\text{Prob}(\text{true matrix} | \text{noisy matrix})$.

Random Matrix Ensembles

In most applications, we encounter (stochastic or deterministic) large matrices with no particular structure. The main premise that underlies random matrix theory is that we can replace such a large complex matrix by a typical element (expected element) of a certain ensemble of random matrices. Most of the time we restrict our attention to symmetric matrices with real entries, since these are the ones that most commonly arise in data analysis and statistical physics. Thankfully, these are easier to analyze mathematically.

Speaking of mathematics, we love polynomial functions. They are nonlinear, complex enough to capture enough complexities in the world around us, and are easy to evaluate and do computations with. When we study large random matrices, a special type of well-studied polynomials appears: orthogonal polynomials. An orthogonal polynomial sequence is a family of polynomials such that any two different polynomials in the sequence are orthogonal (their inner product is zero) to each other under some inner product, which is a generalized dot product. The most widely used orthogonal polynomials sequences are: Hermite polynomials, the Laguerre polynomials, and the Jacobi polynomials (these include the important classes of Chebyshev polynomials and Legendre polynomials). The famous names in the field of orthogonal polynomials, which was mostly developed in the late 19th century, are Chebyshev, Markov, and Stieltjes. No wonder these names are everywhere in probability theory, from Chebyshev inequalities to Markov chains and processes to Stieltjes transforms.

The following three fundamental random matrix ensembles are intimately related to orthogonal polynomials:

Wigner

This is the matrix equivalent of the Gaussian distribution. A 1×1 Wigner matrix is a single Gaussian random number. This is intimately related to Hermite orthogonal polynomials. The Gaussian distribution and its associated Hermite polynomials appear very naturally in contexts where the underlying variable is unbounded above and below. The average of the characteristic polynomials of Wigner random matrices obey simple recursion relations that allow us to express them as Hermite polynomials. Wigner ensemble is the simplest of all ensembles of random matrices. These are matrices where all elements are Gaussian random variables, with the only constraint that the matrix is real symmetric (the Gaussian orthogonal ensemble), complex Hermitian (the Gaussian unitary ensemble), or symplectic (the Gaussian symplectic ensemble).

Wishart

This is the matrix equivalent of the gamma distribution. A 1×1 Wishart is a gamma distributed number. This is intimately related to Laguerre orthogonal polynomials. Gamma distributions and Laguerre polynomials appear in problems where the variable is bounded from below (e.g., positive variables). The

average of the characteristic polynomials of Wishart random matrices obey simple recursion relations that allow us to express them as Laguerre polynomials.

Jacobi

This is the matrix equivalent of the beta distribution. A 1×1 Wishart is a beta distributed number. This is intimately related to Jacobi orthogonal polynomials. Beta distributions and Jacobi polynomials appear in problems where the variable is bounded from above and from below. A natural setting where Jacobi matrices appear is that of sample covariance matrices. They also show up in the simple problem of addition or multiplications of matrices with only two eigenvalues.

As for scalar random variables, we study the moments and the Stieltjes transform of random matrix ensembles. Moreover, since we are in the matrix realm, we study the joint probability distributions of the eigenvalues of these random matrices. For the ensembles mentioned previously, the eigenvalues are strongly correlated and we can think of them as particles interacting through pair-wise repulsion. These are called Coulomb repelling eigenvalues, and the idea here is borrowed from statistical physics (see, for example, “[Patterns in Eigenvalues](#)” by Persi Diaconis (2003) for a deeper dive into the behavior of the eigenvalues of matrices with special structures). It turns out that the most probable positions of the Coulomb gas problem coincide with the zeros of Hermite polynomials in the Wigner case, and of Laguerre polynomials in the Wishart case. Moreover, the eigenvalues of these ensembles fluctuate very little around their most probable positions.

Eigenvalue Density of the Sum of Two Large Random Matrices

Other than finding the joint probability distribution of the eigenvalues of random matrix ensembles, we care about the eigenvalue density (probability distribution) of sums of large random matrices, in terms of each individual matrix in the sum. *Dyson Brownian motion* appears in this context. It is an extension of *Brownian motion* from scalar random variables to random matrices. Moreover, a Fourier transform for matrices allows us to define the analog of the generating function for scalar independent and identically distributed random variables, and use its logarithm to find the eigenvalue density of sums of carefully constructed random matrices. Finally, we can apply Chernoff, Bernstein, and Hoeffding-type inequalities to the maximal eigenvalue of a finite sum of random Hermitian matrices.

Essential Math for Large Random Matrices

Before leaving the discussion of large random matrices, let's highlight the *must knows* if we want to dive deeply into this field. We touch on some of these in this chapter and leave the rest for your googling skills:

- Computing the spectrum: eigenvalues and eigenvectors of a matrix (solutions of $A\vec{v} = \lambda\vec{v}$)
- Characteristic polynomial of a matrix ($\det(\lambda I - A)$)
- Hermite, Laguerre, and Jacobi orthogonal polynomials
- The Gaussian, gamma, and beta probability distributions
- Moments and moment generating function of a random variable
- Stieltjes transform
- Chebyshev everything
- Markov everything
- Chernoff, Bernstein, and Hoeffding-type inequalities
- Brownian motion and Dyson Brownian motion

As of 2022, the fastest supercomputer is *Frontier*, the world's first exascale computer (1.102 exaFLOPS), at the Department of Energy's Oak Ridge National Laboratory. When matrices are very large, even on such a supercomputer, we cannot apply numerical linear algebra (for example, to solve systems of equations involving the matrix, to find its spectrum, or to find its singular value decomposition) as we know it. What we must do instead is *random sampling of the columns of the matrix*. It is best to sample the columns with probability that leads to the most faithful approximation, the one with the least variance. For example, if the problem is to multiply two large matrices A and B with each other, instead of sampling a column from A and a corresponding row from B uniformly, we choose the column from A and the corresponding row from B with a probability p_j proportional to $\text{norm}(\text{column } j \text{ of } A)\text{norm}(\text{row } j \text{ of } B)$. This means that we choose the columns and rows with large norms more often, leading to a higher probability of capturing the *important parts* of the product.

The column space of large and not large matrices is so important. Keep in mind the three best bases for the column space of a given matrix A :

- The singular vectors from the singular value decomposition
- The orthogonal vectors from the Gram Schmidt process (the famous QR decomposition of the matrix)
- Linearly independent columns directly selected from the columns of A

Stochastic Processes

Rather than thinking of a static (scalar or vector or matrix or tensor) random variable, we now think of a time-dependent random variable. Somehow the next step in math always ends up including time-evolving entities. On a side note, humans have not yet fully understood the nature of time or found a way to articulate its definition. We do, however, understand movement and change, a system transitioning from one state to another, and we associate time with that. We also associate probabilities for transitioning from one state to another. Hold this thought for Markov chains, coming up in a bit.

A *stochastic process* is an *infinite* sequence $X_0, X_1, X_2, \dots$ of random variables, where we think of the index t in each X_t as discrete time. So X_0 is the process at time 0 (or the value of a random quantity at a certain time 0), X_1 is the process at time 1 (or the value of a random quantity at a certain time 1), and so on. To formally define a random variable, which we have not done yet, we usually fix it over what we call a probability triple (*a sample space, a sigma algebra, a probability measure*). Do not fuss about the meaning of this triple yet; instead, fuss about the fact that all the random variables in one stochastic process $X_0, X_1, X_2, \dots$ live over the *same* probability triple, in this sense belonging to one family. Moreover, these random variables are often not independent.

Equally important (depending on the application) is a *continuous time* stochastic process, where X_t now encodes the value of a random quantity at *any* nonnegative time t . Moreover, this is easy to align with our intuitive perception of time, which is continuous.

Therefore, a stochastic process is a generalization of a finite dimensional multivariable distribution to infinite dimensions. This way of thinking about it comes in handy when trying to prove the *existence* of a stochastic process, since then we can resort to theorems that allow us to *extend* to infinite dimensions by relying on finite dimensional collections of the constituent distributions.

Examples of stochastic processes are all around us. We think of these whenever we encounter fluctuations: movement of gas molecules, electrical current fluctuations, stock prices in financial markets, the number of phone calls to a call center in a certain time period, or a gambler's process. And here's an interesting finding from [a microbiology paper discussing bacteria found in the gut](#): *the community assembly of blood, fleas, or torsalos is primarily governed by stochastic processes, while the gut microbiome is determined by deterministic processes.*

The stock market example is central in the theory of stochastic processes, because it is how Brownian motion (also called the Wiener stochastic process) got popularized, with L. Bachelier studying price changes in the Paris Bourse. The example of phone calls to a call center is also central in the theory, because it is how Poisson stochastic

process got popularized, with A. K. Erlang modeling the number of phone calls occurring in a certain period of time.

These two processes, Brownian and Poisson, appear in many other settings that have nothing to do with the previously mentioned examples. Maybe this tells us something deeper about nature and the unity of its underlying processes, but let's not get philosophical and stay with mathematics. In general, we can group stochastic processes into a few categories, depending on their mathematical properties. Some of these are discrete time processes, and others are continuous time. The distinction between which one is which is pretty intuitive.

To derive conclusions about Brownian and Poisson processes, and the other stochastic processes that we are about to overview, we need to analyze them mathematically. In probability theory, we start with establishing the *existence* of a stochastic process. That is, we need to explicitly define the probability triple (sample space, sigma algebra, probability measure) where the discrete time *infinite* sequence of random variables $X_0, X_1, X_2, \dots$ or the continuous time X_t process lives, and prove that we can find such a set of random variables satisfying its characterizing properties. We will revisit this later in this chapter, but a big name we want to search for when proving the existence of stochastic processes is A. Kolmogorov (1903–1987), namely, the *Kolmogorov existence theorem*. This ensures the existence of a stochastic process having the same *finite dimensional* distributions as our desired processes. That is, we can get our desired stochastic process (infinite process, indexed on discrete or continuous time) by specifying all finite dimensional distributions in some *consistent* way.

Let's survey the most prominent stochastic processes.

Bernoulli Process

This is the stochastic process mostly associated with repeatedly flipping a coin and any process in life that mimics that (at some airports, customs officials make us press a button: if the light turns green we pass, if it turns red we get searched). Mathematically, it is an infinite sequence of independent and identically distributed random variables $X_0, X_1, X_2, \dots$, where each random variable takes either the value zero with probability p or one with probability $1-p$. A sample realization of this process would look like 0,1,1,0,....

Poisson Process

We can think of the Poisson process as a stochastic process whose underlying random variables are counting variables. These count how many interesting events happen within a set period of time. These events are either independent or are weakly dependent, and each has a small probability of occurrence. They also happen at a set expected rate λ . This is the parameter that characterizes a Poisson random variable.

For example, in queuing theory, we use it to model the arrival of customers at a store, phone calls at a call center, or the occurrence of earthquakes in a certain time interval. This process has the natural numbers as its state space and the nonnegative numbers as its index set. The probability distribution underlying the random variables involved in a Poisson process has the formula:

$$Prob(X = n) = \frac{\lambda^n e^{-\lambda}}{n!}$$

The formula gives the probability of n interesting events occurring in a unit period of time. Clearly, within a fixed time interval it is not likely for many rare events to occur, which explains the rapid decay in the formula for a large n . The expectation and the variance of a Poisson random variable is λ .

A Poisson *process* $X_t; t \geq 0$, indexed by continuous time, has the properties:

- $X_0 = 0$
- The number of events (or points) in any interval of length t is a Poisson random variable with parameter λt

A Poisson process has two important features:

- The number of events in each finite interval is a Poisson random variable (has a Poisson probability distribution).
- The number of events in disjoint time intervals are independent random variables.

The Poisson process is an example of a Levy stochastic process, which is a process with stationary independent increments.

Random Walk

It is easy to think of the simplest random walk as someone taking steps on a road where they start somewhere, then move forward (add one to their position) with probability p , and backward (subtract one from their position) with probability $1 - p$. We can define the resulting discrete time stochastic process $X_0, X_1, \dots$ such that $X_0 = x_0$, $X_1 = X_0 + Z_1$, $X_2 = X_1 + Z_2 = X_0 + Z_1 + Z_2$, etc., where $Z_1, Z_2, \dots$ is a Bernoulli process. If $p = 0.5$, this is a symmetric random walk.

In [Chapter 9](#), we used random walks on graphs multiple times, where we start at a certain graph node and then transition to one of its adjacent nodes with given probabilities. A normalized adjacency matrix of the graph would define the transition probabilities at all nodes. This is a neat example of how random walks on graphs tie

to Markov chains, coming up soon. For more on this, check this nice set of notes on [random walks on graphs](#).

Wiener Process or Brownian Motion

We can think of a Wiener process or a Brownian motion as a random walk with infinitesimally small steps, so discrete movements become infinitesimally small fluctuations, and we get a *continuous* random walk. A Brownian motion is a continuous time stochastic process $X_t; t \geq 0$. The random variables X_t are real valued, have independent increments, and the difference between X_t and X_s at two separate times t and s is normally distributed (follows a Gaussian bell-shape distribution) with mean 0 and variance $t - s$. That is, $X_t - X_s$ are normally distributed based on the size of the increments.

The interesting thing about a real valued continuous time stochastic process is that it can move in continuous paths, giving rise to interesting random functions of time. For example, almost surely, a sample path of a Brownian motion (Wiener process) is continuous everywhere, but nowhere differentiable (too many spikes).

Brownian motion is fundamental in the study of stochastic processes. It is the starting point of stochastic calculus, lying at the intersection of several important classes of processes: it is a Gaussian Markov process, a Levy process (a process with stationary independent increments), and a martingale, discussed next.

Martingale

A discrete time martingale is a stochastic process $X_0, X_1, X_2, \dots$ where for any discrete time t :

$$\mathbb{E}(X_{t+1} | X_1, X_2, \dots, X_t) = X_t$$

That is, the expected value of the next observation, given all the previous observations, is equal to the most recent observation. This is a weird way of defining something (sadly, this is very common in this field), but let's give a few brief examples of some contexts within which martingales appear:

- An unbiased random walk is an example of a martingale.
- A gambler's fortune is a martingale if all the betting games that the gambler plays are fair. Suppose a gambler wins \$1 if a fair coin comes up heads, and loses \$1 if it comes up tails. If X_n is the gambler's fortune after n tosses, then the gambler's conditional expected fortune after the next coin toss, given the history, is equal to their present fortune.

- In an ecological community, where a group of species compete for resources, we can model the number of individuals of any particular species as a stochastic process. This sequence is a martingale under the **unified neutral theory of biodiversity and biogeography**.

Stopping times appear when discussing martingales. This is an interesting concept, capturing the idea that: *at any particular time t , you can look at the sequence so far and tell if it is time to stop*. A stopping time with respect to a stochastic process $X_1, X_2, X_3, \dots$ is a random variable S (for *stop*) with the property that for each t , the occurrence or nonoccurrence of the event $S = t$ depends only on the values of $X_1, X_2, X_3, \dots, X_t$. For example, the stopping time random variable models the time at which a gambler chooses to stop and leave a gambling table. This will depend on their previous winnings and losses, but not on the outcomes of the games that they haven't yet played.

Levy Process

We have mentioned the Poisson process and Brownian motion (Wiener process) as two of the most popular examples of a Levy process. This is a stochastic process with independent, stationary increments. It can model the motion of a particle whose successive displacements are random, in which displacements in pair-wise, disjoint time intervals are independent, and displacements in different time intervals of the same length have identical probability distributions. In this sense, it is the continuous time analog of a random walk.

Branching Process

A branching process randomly splits into branches. For example, it models a certain population's evolution (like bacteria, or neutrons in a nuclear reactor) where each individual in a given generation produces a random number of individuals in the next generation, according to some fixed probability distribution that does not vary from individual to individual. One of the main questions in the theory of branching processes is the probability of ultimate extinction, where the population dies out after a finite number of generations.

Markov Chain

Let's formally define a discrete time Markov chain, since it is one of the most important stochastic processes, and because it comes up in the context of reinforcement learning in AI. To define a Markov chain, we need:

- A discrete set of possible states S (finite or infinite). Think of this as the set of states that a particle or an agent can occupy. At each step, the Markov process randomly evolves from one state to another.

- An initial distribution prescribing the probability ν_i of each possible $state_i$. Initially, how likely is it for a particle to be at a certain location or for an agent to be in a certain state upon initialization?
- Transition probabilities p_{ij} specifying the probability that the particle or the agent transitions from $state_i$ to $state_j$. Note that we have, for each state i , the sum $p_{i1} + p_{i2} + \dots + p_{in} = 1$. Moreover, this process has no memory, because this transition probability depends only on $state_i$ and $state_j$, not on previously visited states.

Now, a Markov chain is a stochastic process $X_0, X_1, \dots$ taking values in S such that:

$$Prob(X_0 = state_{i_0}, X_1 = state_{i_1}, \dots, X_n = state_{i_n}) = \nu_{i_0} p_{i_0 i_1} p_{i_1 i_2} \dots p_{i_{n-1} i_n}$$

We can bundle up all the transition probabilities in a square matrix (a Markov matrix, with each row having nonnegative numbers adding up to 1), and multiply by that matrix. This matrix summarizes the probabilities of transitioning from any $state_i$ to $state_j$ in one step. The cool thing is that the powers of a Markov matrix are also Markov, so for example, the square of this matrix summarizes the probabilities of transitioning from any $state_i$ to $state_j$ in two steps, and so on.

Some fundamental notions related to Markov chains are transience, recurrence, and irreducibility. A $state_i$ is recurrent if we start from it, then we will certainly eventually return to it. It is called transient if it is not recurrent. A Markov chain is irreducible if it is possible for the chain to move from any state to any other state.

Finally, a stationary probability vector, defining a probability distribution over the possible states, is one that does not change when we multiply it by the transition matrix. This ties straight into eigenvectors in linear algebra with corresponding eigenvalue one. The *feel good* high that we get when the math we know connects together is an addictive feeling and is what keeps us trapped in a *love-hate* relationship with this field. Thankfully, the longer we stay, the more *love-love* it becomes.

Itô's Lemma

Let's tie a bit more math together. A stochastic process models a random quantity that evolves with time. A function of a stochastic process also evolves randomly with time. When deterministic functions evolve with time, the next question is usually, *how fast*? To answer that, we take its derivative with respect to time, and develop calculus around the derivatives (and integrals) of deterministic functions. The chain rule is of paramount importance, especially for training machine learning models.

Itô's lemma is the analog of the chain rule for functions of stochastic processes. It is the stochastic calculus counterpart of the chain rule. We use it to find the differential of a time-dependent function of a stochastic process.

Markov Decision Processes and Reinforcement Learning

In the AI community, Markov decision processes are associated with:

Dynamic programming and Richard Bellman

Bellman played a monumental role in the field, and his optimality condition is implemented in many algorithms.

Reinforcement learning

Finding an optimal strategy via a sequence of actions that are associated with positive or negative rewards (trials and errors). The agent has choices between several actions and transition states, where the transition probabilities depend on the chosen actions.

Deep reinforcement learning

Combining reinforcement learning with neural networks. Here, a neural network takes the observations as input, and outputs a probability for each possible action that the agent can take (a probability distribution). The agent then decides on the next action randomly, according to the estimated probabilities. For example, if the agent has two choices, turn left or turn right, and the neural network outputs 0.7 for turn left, then the agent will turn left with 70% probability, and turn right with 30% probability.

Reinforcement learning has great potential to advance us toward general intelligence. Intelligent agents need to make rational decisions when payoffs from actions are not immediate but instead result from a series of actions taken sequentially. This is the epitome of reasoning under uncertainty.

Examples of Reinforcement Learning

Examples of reinforcement learning are plentiful: self-driving cars, recommender systems, thermostat at home (getting positive rewards whenever it is close to the target temperature and saves energy, and negative rewards when humans need to tweak the temperature), and automatic investing in the stock market (the input is the stock prices, the output is how much of each stock to buy or sell, the rewards are the monetary gains or losses).

Perhaps the most famous example for deep reinforcement learning success is **DeepMind's AlphaGo**, the AI agent that in 2016 beat the world's best human player in the ancient Chinese game of Go. Thinking of reinforcement learning in terms of board games, such as chess or Go, is intuitive, because at each step we decide

on the sequence of actions that we must take, knowing very well that our current decision affects the whole outcome of the game. We better act optimally at each step. Moreover, at each step, our optimal strategy *evolves*, because it also depends on the actions of our opponent (who is solving the exact same problem but from their vantage point).

I am a bit biased against examples containing games, since nowadays my daughter is addicted to PlayStation 5. I prefer the investment market example. Our financial adviser operates in a market that changes every day, and needs to make decisions on buying/selling certain stocks at each time step, with the long-term goal of maximizing profit and minimizing losses. The market environment is stochastic and we do not know its rules, but we assume that we do for our modeling purposes. Now let's switch our human financial adviser to an AI agent, and let's see what kind of optimization problem this agent needs to solve at each time step in the constantly changing market environment.

Reinforcement Learning as a Markov Decision Process

Let's formulate reinforcement learning mathematically as a Markov decision process. The environment within which our agent exists is probabilistic, consisting of states and transition probabilities between these states. These transition probabilities *depend on the chosen actions*. Thus, the resulting Markov process that encodes the transitions from any state $\overrightarrow{state}$ to another state $\overrightarrow{state'}$ has an explicit dependency on the action $\overrightarrow{a}$ taken while in state $\overrightarrow{state}$.

The main assumption here is that we *know* this process, which means that we know the rules of the environment. In other words, we know the following probability for each $\overrightarrow{state}$, $\overrightarrow{state'}$, and action $\overrightarrow{a}$:

$$Prob\left(\text{next state} = \overrightarrow{state'} \mid \text{current state} = \overrightarrow{state}, \text{action taken} = \overrightarrow{a}\right)$$

We also know the reward system, which is:

$$Prob\left(\text{next reward value} \mid \text{current state} = \overrightarrow{state}, \text{action taken} = \overrightarrow{a}, \text{next state} = \overrightarrow{state'}\right)$$

This discussion now belongs in the realm of dynamic programming: we search for the *optimal policy* (sequence of good actions) leading to *optimal value* (maximal reward or minimal loss). This optimization problem is a bit more involved than the ones we have encountered so far, because it is a *sequence of actions* that leads

to the optimal value. Therefore, we must divide the problem into steps, looking for the action at each step that *shoots for* the optimal reward *multiple steps ahead in the future*. Bellman's *optimality equation* solves exactly this problem, simplifying it into the search for *only one optimal action* at the current state (as opposed to searching for all of them at once), given that we know what problem to optimize at each step. Bellman's huge contribution is the following assertion: the optimal value of the current state is equal to the average reward after taking *one* optimal action, *plus* the *expected optimal value* of all possible next states that this action can lead to.

The agent interacts with its environment via an iterative process. It starts with an initial state and a set of that state's possible actions (the probability distribution for taking an action given that state), then computes the following, iteratively:

1. The next optimal action to take (which transitions it to a new state with a new set of possible actions). This is called the *policy iteration*, and the optimization goal is to maximize future reward.
2. The expected value (rewards or losses) given that optimal action. This is called the *value iteration*.

The value function adds up the agent's expected future rewards given its current state and the optimal sequence of actions taken afterward:

$$Value(\overrightarrow{state}, \text{optimal sequence of actions}) = \mathbb{E}(\sum_k \gamma^k \text{reward}_k)$$

The *discount factor* γ is a number between 0 and 1. It is useful to encourage taking actions that result in *sooner rather than later* positive rewards. Putting this factor in the optimization problem adjusts the importance of rewards over time, giving less weight to future rewards (if γ is between 0 and 1, then γ^k is small for large k).

Let's make the optimization in the value function explicit (we are choosing the sequence of actions that maximizes the rewards given the current state):

$$Value(\overrightarrow{s}) = \max_{\text{actions and states}} \mathbb{E}\left(\sum_{k=0}^{\infty} \gamma^k (\text{reward})_k \mid \overrightarrow{state}_0 = \overrightarrow{s}\right)$$

Now we break this up to make sure the agent's current reward is explicit and separate from its future rewards:

$$Value(\overrightarrow{s}) = \max_{\text{actions and states}} \mathbb{E}\left(\text{reward}_0 + \sum_{k=1}^{\infty} \gamma^k (\text{reward})_k \mid \overrightarrow{state}_1 = \overrightarrow{s}'\right)$$

Finally, we find that the value function at the agent's current state depends on its current reward and a discounted value function at its future states:

$$Value(\vec{s}) = \max_{\text{actions and states}} \mathbb{E}(reward_0 + \gamma Value(\vec{s}'))$$

That statement allows us to solve our main optimization problem iteratively (backward in time). All the agent has to do now is to choose the action to get to the *next best state*. This expression for the value function is the powerful *Bellman's equation* or *Bellman's optimality condition*, which breaks up the original optimization problem into a recursive sequence of much simpler optimization problems, optimizing locally at each state (finding $Value(\vec{s}')$), then putting the result into the next optimization subproblem (finding $Value(\vec{s})$). The miracle is that working backward this way from the desired ultimate reward to deciding what action to take now gives us the overall optimal strategy along with the optimal value function at each state.

Reinforcement Learning in the Context of Optimal Control and Nonlinear Dynamics

In [Chapter 13](#) on PDEs, we revisit reinforcement learning in the context of nonlinear dynamics, optimal control, and the Hamilton-Jacobi-Bellman partial differential equation. Unlike the probabilistic Markov environment that our agent interacted with in the previous discussion, the dynamic programming approach to reinforcement learning (which leads to the Hamilton-Jacobi-Bellman partial differential equation) is deterministic.

Python Library for Reinforcement Learning

Finally, a helpful library for implementing reinforcement learning algorithms is the TF-Agents library (by Google, open sourced in 2018), a reinforcement learning library based on TensorFlow (Python).

Theoretical and Rigorous Grounds

Rigorous, or mathematically precise, probability theory needs measure theory. *But why?* you might rightfully protest. After all, we have managed to avoid this for the longest time.

Because we cannot avoid it any longer.

Let's write this but never admit to it out loud: it is measure theory that turns off many students from pursuing further studies in math, mostly, because its story is never told in chronological order about how and why it came to be. Moreover, a lot of work in

measure theoretic probability has to do with *proving* that a certain random variable *exists* (over some sample space, an event space or a sigma algebra, and a measure for each event or set in that sigma algebra), as if writing the random variable down and using it to model all sorts of random entities is not enough existence. This must be the reason why mathematicians and philosophers get along so well.

We have already flown through many concepts in this chapter at lightning speed (my students accuse me of this all the time), but we need to start over and give:

- A precise mathematical understanding of probabilities and sigma algebras
- A precise mathematical definition of a random variable and a probability distribution
- A precise mathematical definition of an expected value of a random variable, and its connection to integration
- An overview of probability inequalities (controlling uncertainty)
- An overview of the law of large numbers, the central limit theorem, and other convergence theorems

Alright, that is too ambitious. We cannot give a full course on rigorous probability theory in one section of one chapter. What we will do instead is make a convincing case for it, and leave with a decent understanding of the fundamental ideas.

We start with two major limitations of nonrigorous probability (other than the fact that each of its math objects has countless inconsistent names, notations, and fuzzy definitions).

Which Events Have a Probability?

Given a sample space (a set that we can sample randomly from), can any subset have a probability defined on it? What if we are sampling numbers uniformly from the real line and ask, what is the probability that we pick a rational number? An algebraic number (the solution to some polynomial equation with integer coefficients)? Or a member from some other complicated subset of the real line?

See how these questions are slowly drawing us into the details of set theory on the real line, which in turn pulls us straight into *measure theory*: the theory that addresses *which subsets of the real line we can measure, and which subsets we cannot*.

Defining a probability for a subset of a sample space is starting to sound a lot like defining a measure of that set, and it seems like only subsets that are *measurable* can have a probability defined for them. How about the other *nonmeasurable* subsets of the sample space? Too bad for them, we cannot define a probability for them. To reiterate, $Prob(A)$ does not make sense for every subset A of a sample space; instead, it only makes sense for measurable subsets of that space. So we must harness

all the measurable subsets together, abandon the rest and never think about them or their mathematics, and relax, because then we can act in a realm where all the events (subsets) that we harnessed have probabilities (measures) defined for them. The probability measure that we work with satisfies reasonable properties, in the sense that it is a nonnegative number in $[0,1]$, and probabilities of complementary events (subsets) add up to 1. This whole roundabout reveals the intricacies of the real line and its subsets, and more generally, the mysteries of the continuum and the wonders of the infinite.

Rigorous probability theory helps us appreciate the properties of both discrete and continuous spaces, revealed in examples as simple as constructing the discrete uniform distribution on a discrete set versus constructing the continuous uniform distribution on a given interval.

Can We Talk About a Wider Range of Random Variables?

The other limitation of nonrigorous probability, meaning the one that avoids measure theory as we just described it, is the restriction on the kinds of random variables that it allows. In particular, where exactly do we draw the line between a discrete and continuous random variable? Is there really such a line? How about random variables that have both discrete and continuous aspects? As a simple example, suppose a random variable's value is decided by a flip of a coin. It is Poisson distributed (discrete) if the coin comes up heads, and normally distributed (continuous) if the coin comes up tails. This new random variable is neither fully discrete nor fully continuous in the nonrigorous sense that we understand either type. Then what is it? The rigorous answer is this: of course there is no distinction between discrete and continuous random variables once we define the grounds that *any* random variable stands on. Here's the rigorous ground it must stand on. What set formulates the sample space? What subsets of this sample space are measurable? What is the probability measure? What is the *distribution* of the random variable? This is the common ground, or the starting point for *any* random variable. Once we specify this ground, then discrete, continuous, or anything in between becomes a small detail, as simple as answering: what set (or, say, product of sets) are we working with?

A Probability Triple (Sample Space, Sigma Algebra, Probability Measure)

It all starts with a probability triple (not really, but this is where rigor starts). We call this a probability measure space, with the understanding that the measure of the whole sample space is equal to one. That is, the probability of the sample space is one. We are now feeling very advanced, using the words probability and measure interchangeably. The comfort that the word measure provides is that it brings us back

to a deterministic realm. The sampling is random, but we can *measure* the likelihood of any occurrence (that is measurable).

The three objects making up a probability measure space are:

The sample space

The arbitrary nonempty set that we randomly pull samples from.

The sigma algebra

A set of subsets of the sample space that represent the allowed events (the events that we are allowed to talk to about their probability, because they are the only ones we are able to measure). A sigma algebra must contain the whole sample space, is closed under complements (meaning if a set is in the sigma algebra then so is its complement), and is closed under countable unions (meaning the union of countably many subsets of the sigma algebra is also a member of the sigma algebra). The corollary from the previous two properties and **De Morgan's laws** (which have to do with complements of unions and intersections) is that the sigma algebra is also closed under countable intersections.

The probability measure

A number between zero and one (inclusive) associated with *each subset* of the sigma algebra, that satisfies the reasonable properties that we associate with nonrigorous probability:

1. $\text{Prob}(\text{sample space}) = 1$
2. $\text{Prob}(\text{countable union of pair-wise disjoint sets}) = \text{countable sum of probabilities of each set}$

This is very good, because as long as we are able to articulate the sample space set, the sigma algebra, and a function with the previously mentioned properties mapping every member of the sigma algebra to its measure (probability), then we can start *building the theory* on solid grounds, defining all kinds of random variables, their expectations, variances, conditional probabilities, sums and products, limits of sequences, stochastic processes, time derivatives of functions of stochastic processes (Itô's calculus), and so on. We would not run into problems of what type of events have probabilities defined for them (all the members of the sigma algebra of the probability triple), or what type of random variables we can consider (any that we can rigorously define over a probability triple).

Where Is the Difficulty?

Note that the limitations of nonrigorous probability that we discussed *both* appear when we involve continuous variables, or when the sample space is in the continuum (uncountable). Had our world been only discrete, we wouldn't be going through all this trouble. When we move to rigorous probability and attempt to construct

probability triples for discrete sample spaces, we do not run into much trouble. The challenges appear in the continuum world, with uncountable sample spaces. Because suddenly we have to identify sigma algebras and associated probability measures on sets where the depth of the infinite continuum never ceases to fascinate. For example, this challenge appears even when we want to define a rigorous probability triple for the continuous uniform distribution on the interval $[0,1]$.

The *extension theorem* runs to our aid and allows us to construct complicated probability triples. Instead of defining a probability measure over a massive sigma algebra, we construct it on a simpler set of subsets, a *semialgebra*, then the theorem allows us to automatically extend the measure to a full sigma algebra. This theorem allows us to construct Lebesgue measure on $[0, 1]$ (which is exactly the continuous uniform distribution on $[0,1]$), product measures, the multidimensional Lebesgue measure, and finite and infinite coin tossing.

The worlds of set theory, real analysis, and probability have blended neatly together.

Random Variable, Expectation, and Integration

Now that we can associate probability triple with a sample space, defining probabilities for a large amount of subsets of the sample space (all the members of the associated sigma algebra), we can rigorously define a random variable. As we know very well from nonrigorous probability, a random variable assigns a numerical value to each element of the sample space. So if we think of the sample space as all the possible random outcomes of some experiment (heads and tails of flipping a coin), then a random variable assigns a numerical value to each of these outcomes.

To build on rigorous grounds, we must define how a random variable Y interacts with the whole probability triple associated with the sample space. The short answer is that: Y must be a *measurable* function from the sample space to the real line, in the sense that the set $Y^{-1}(-\infty, y)$ is a member of the sigma algebra, which in turn means that this set has a probability measure. Note that Y maps from the sample space to the real line, and Y^{-1} maps back from the real line to a subset of the sample space.

Just like a random variable from nonrigorous probability turns out to be a measurable function (with respect to a triple) in rigorous probability theory, the expectation $\mathbb{E}(Y)$ of a random variable turns out to be the same as the integral of the random variable (measurable function) with respect to the probability measure. We write:

$$\mathbb{E}(Y) = \int_{\Omega} Y dP = \int_{\Omega} Y(\omega) Prob(d\omega)$$



Understanding the Integral Notation in the Expectation Formula

It is easy to understand the integral with respect to a probability measure, such as the one in the previously mentioned formula, if we think of the meaning of the expectation of a random variable in a discrete setting, as the sum of the value of the random variable times the probability of the set over which it assumes that value:

$$\mathbb{E}(Y) = \sum_{i=1}^n y_i \text{Prob}(\omega \in \Omega \text{ such that } Y(\omega) = y_i)$$

Now compare this discrete expression to the continuum integral in the expectation formula:

$$\mathbb{E}(Y) = \int_{\Omega} Y dP = \int_{\Omega} Y(\omega) \text{Prob}(d\omega)$$

We rigorously build up the integral (expectation) the exact same way we build up the Lebesgue integral in a first course on measure theory: first for simple random variables (which we can easily break up into a discrete sum; integrals start from sums), then for nonnegative random variables, and finally for general random variables. We can easily prove basic properties for integrals, such as linearity and order preserving. Note that whether the sample space is discrete, continuous, or anything complicated, as long as we have our probability triple to build on, the integral makes sense (in a much wider range of setting than we ever imagined for our basic calculus Riemann-style integration). Once we encounter Lebesgue-style integration, we sort of never look back.

Now that we have the expectation, we can define the variance and covariance exactly the same way as nonrigorous probability theory.

Then we can talk about independence, and important properties such that if X and Y are independent, then $E(XY) = E(X)E(Y)$ and $\text{Var}(X + Y) = \text{Var}(X) + \text{Var}(Y)$.

Distribution of a Random Variable and the Change of Variable Theorem

The distribution of a random variable X is a *corresponding probability triple* $(\mathbb{R}, \mathcal{B}, \mu)$ defined on the real line, such that for every subset B of the Borel sigma algebra defined on the real line, we have:

$$\mu(B) = P(X \in B) = P(X^{-1}(B))$$

This is completely determined by the cumulative distribution function, $F_X(x) = P(X \leq x)$, of X .

Suppose we have a measurable real valued function f defined on the real line. Let X be a random variable on a probability triple $(\Omega, \sigma\text{-algebra}, P)$ with distribution μ . Note that for any real number x , $f(x)$ is a real number, and for the random variable X , $f(X)$ is a random variable.

The change of variable theorem says that the expected value of the random variable $f(X)$ with respect to the probability measure P on a sample space Ω is equal to the expected value of the function f with respect to the measure μ on $\mathbb{R}$. Let's write this first in terms of expectation and then in terms of integrals:

$$\mathbb{E}_P(f(X)) = \mathbb{E}_\mu(f)$$

$$\int_\Omega f(X(\omega))P(d\omega) = \int_{-\infty}^{\infty} f(t)\mu(dt)$$

A nice thing that comes in handy from this change of variables theorem is that we can switch between expectations, integrations, and probabilities. Let f be the indicator function of a measurable subset of $\mathbb{R}$ (which is one over the subset and zero otherwise), then the formula gives us:

$$\int_{-\infty}^{\infty} \mathbf{1}_B \mu(dt) = \mu(B) = P(X \in B).$$

Note that in [Chapter 8](#), we encountered another change of variables theorem from probability, which relates the probability distribution of a random variable with the probability distribution of a deterministic function of it, using the determinant of the Jacobian of this function transformation.

Next Steps in Rigorous Probability Theory

The next step in rigorous probability theory is to prove the famous inequalities (Markov, Chebyshev, Cauchy-Schwarz, Jensen's), introduce sums and products of random variables, the laws of large numbers, and the central limit theorem. Then we move to sequences of random variables and limit theorems.

Limit theorems

If we have a sequence of random variables that converges to some limit random variable, does it follow that the expectations of the sequence converge to the expectation of the limit? In integral language, when can we exchange the limit and the integral?

This is when we prove the monotone convergence, the bounded convergence, Fatou's lemma, the dominated convergence, and the uniformly integrable convergence theorems.

Finally we consider double or higher integrals, and conditions on when it is OK to flip integrals. Fubini's theorem answers that, and we can apply it to give a convolution formula for the distribution of a sum of independent random variables.

The Universality Theorem for Neural Networks

Rigorous measure theory (probability theory) helps us prove theorems for neural networks, which is an up-and-coming subfield of mathematics, aiming to provide theoretical grounds for many empirical AI successes.

The universality theorem for neural networks is a starting point. We have referred to it multiple times in this book. Here's the statement:

For any continuous function f on a compact set K , there exists a feed forward neural network, having only a single hidden layer, which uniformly approximates f to within an arbitrary $\epsilon > 0$ on K .

This [web page](#) has a nice and easy-to-follow proof.

Summary and Looking Ahead

In this chapter, we surveyed concepts in probability that are important for AI, machine learning, and data science. We zipped through topics such as causal modeling, paradoxes, large random matrices, stochastic processes, and reinforcement learning in AI.

Often when we learn about probability, we fall into the *frequentist* versus the *objectivist* positions regarding the definitions and the overall philosophy surrounding uncertainty. The following is a neat description of each viewpoint:

A frequentist position

Probabilities can only come from experiments and observing the results of repeated trials.

An objectivist position

Probabilities are real aspects of the universe: a real inclination or natural tendency to behave in a particular way. For example, a fair coin's propensity to turn up heads 50% of the time is an intrinsic property of the fair coin itself.

A frequentist is then only attempting to measure these natural inclinations via experiments. Rigorous probability theory unifies disparate views of probability. We swiftly introduced rigorous probability theory and established that it is in essence the same as measure theory in real analysis. We ended with the universal approximation theorem for neural networks.

We leave this chapter with a perfectly fitting tweet from **Yann LeCun**, which happens to touch on every topic we covered in this chapter:

I believe we need to find new concepts that would allow machines to:

- Learn how the world works by observing like babies.
- Learn to predict how one can influence the world through taking actions.
- Learn hierarchical representations that allow long-term predictions in abstract representation spaces.
- Properly deal with the fact that the world is not completely predictable.
- Enable agents to predict the effects of sequences of actions so as to be able to reason and plannable machines to plan hierarchically, decomposing a complex task into subtasks.
- All of this in ways that are compatible with gradient-based learning.

Mathematical Logic

Humans bend the rules.

—H.

Historically in the AI field, logic-based agents come before machine learning and neural network-based agents. The reason we went over machine learning, neural networks, probabilistic reasoning, graph representations, and operations research before logic is that we want to tie it all into one narrative of reasoning within an agent, as opposed to thinking of logic as old and neural networks as modern. We want to view the recent advancements as enhancing the way a logical AI agent represents and reasons about the world. A good way to think about this is similar to enlightenment: an AI agent used to reason using the rigid rules of handcoded knowledge base and handcoded rules to make inferences and decisions, then suddenly it gets enlightened and becomes endowed with more reasoning tools, networks, and neurons that allow it to expand both its knowledge base and inference methods. This way, it has more expressive power and can navigate more complex and uncertain situations. Moreover, combining all the tools would allow an agent the option to sometimes break the rules of a more rigid logic framework and employ a more flexible one, depending on the situation, just like humans. Bending, breaking, and even changing the rules are distinctive human attributes.

The dictionary meaning of the word *logic* sets the tone for this chapter and justifies its progression.



Logic

A framework that organizes the rules and processes used for sound thinking and reasoning. It is a framework that lays down the principles of validity under which to conduct reasoning and inference.

The most important words to pay attention to in this definition are *framework* and *principles for inference*. A logic system codifies within an agent the principles that govern reliable inference and correct proofs. Designing agents that are able to gather knowledge, reason logically with a flexible logic system that accommodates uncertainty about the environment that they exist in, and make inferences and decisions based on this logical reasoning lies at the heart of artificial intelligence.

We discuss the various systems of mathematical logic that we can program into an agent. The goal is to give the AI agent the ability to make inferences that enable it to act appropriately. These logical frameworks require knowledge bases to accompany the inference rules of varying sizes. They also have varying degrees of expressive and deductive powers.

Various Logic Frameworks

For each of the different logical frameworks (*propositional*, *first order*, *temporal*, *probabilistic*, and *fuzzy*) that we are about to highlight in this chapter, we will answer two questions about how they operate within an agent endowed with them:

1. What objects exist in the agent's world? Meaning, how does the agent perceive the composition of its world?
2. How does the agent perceive the objects' states? Meaning, what values can the agent assign to each object in its world under the particular logic framework?

It is easy to think about this if we liken our agent to an **ant and how it experiences the world**. Because of the ant's predetermined framework of perception and allowed movements, the ant experiences the world, along with its curvature, as two-dimensional. If the ant gets enhanced and endowed with a more expressive framework of perception and allowed movements (for example, wings), it will experience the three-dimensional world.

Propositional Logic

Here are the answers to our agent questions:

What objects exist in the agent's world?

Simple or complex statements, called *propositions*, hence the name *propositional logic*.

How does the agent perceive the objects' states?

True (1), false (0), or unknown. Propositional logic is also called Boolean logic because the objects in it can only assume two states. Paradoxes in propositional logic are statements that cannot be classified as true or false according to the logic framework's *truth table*.

These are examples of statements and their states:

- It is raining (can take true or false states).
- The Eiffel Tower is in Paris (always true).
- There is suspicious activity in the park (can take true or false states).
- This sentence is false (paradox).
- I am happy *and* I am sad (always false, unless you ask my husband).
- I am happy *or* I am sad (always true).
- If the score is 13, then the student fails (truth depends on failing thresholds, so we need a statement in the knowledge base that says: all students with a score below 16 fail, and set its value at true).
- $1 + 2$ is equivalent to $2 + 1$ (always true within an agent endowed with arithmetic rules).
- Paris is romantic (in propositional logic this has to be either true or false, but in fuzzy logic it can assume a value on a zero-to-one scale, for example, 0.8, which corresponds better to the way we perceive our world: on a scale as opposed to absolutes. Of course, I would assign the value true for this statement if I am programming an agent and confined to propositional logic, but someone who hates Paris would assign false. Oh well).

The objects in a propositional logic's world are simple statements and complex statements. We can form complex statements from simple ones using five allowed operators: *not* (negation), *and*, *or*, *implies* (which is the same as *if then*), and *equivalent to* (which is the same as *if and only if*).

We also have five rules to determine whether a statement is true or false:

1. The negation of a statement is true if and only if the statement is false.
2. $statement_1$ *and* $statement_2$ are true if and only if both $statement_1$ and $statement_2$ are true.
3. $statement_1$ *or* $statement_2$ is true if and only if either $statement_1$ or $statement_2$ are true (or if both are true).
4. $statement_1$ *implies* $statement_2$ is true except when $statement_1$ is true and $statement_2$ is false.
5. $statement_1$ is *equivalent to* $statement_2$ if and only if $statement_1$ and $statement_2$ are both true or both false.

We can summarize these rules in a *truth table* accounting for all the possibilities for the states of $statement_1$ and $statement_2$ and for their joining using the five

allowed operators. In the following truth table, we use S_1 for *statement*₁ and S_2 for *statement*₂ to save space:

S_1	S_2	not S_1	S_1 and S_2	S_1 or S_2	S_1 implies S_2	S_1 equivalent to S_2
F	F	T	F	F	T	T
F	T	T	F	T	T	F
T	F	F	F	T	F	F
T	T	F	T	T	T	T

We can compute the truth of any complex statement using this truth table by simple recursive evaluation. For example, if we are in a world where S_1 is true, S_2 is false, and S_3 is true, then we have the statement:

$$\text{not } S_1 \text{ and } (S_2 \text{ or } S_3) \iff \text{F and (F or T)} = \text{F and T} = \text{F}$$

To be able to reason and prove theorems using propositional logic, it is helpful to establish logical equivalences, meaning statements that have the exact same truth tables so they can replace each other in a reasoning process. The following are some examples of logical equivalences:

- Commutativity of *and*: $S_1 \text{ and } S_2 \iff S_2 \text{ and } S_1$
- Commutativity of *or*: $S_1 \text{ or } S_2 \iff S_2 \text{ or } S_1$
- Double negation elimination: $\text{not}(\text{not } S_1) \iff S_1$
- Contraposition: $S_1 \text{ implies } S_2 \iff \text{not}(S_2) \text{ implies not}(S_1)$
- Implication elimination: $S_1 \text{ implies } S_2 \iff \text{not}(S_1) \text{ or } S_2$
- De Morgan's law: $\text{not}(S_1 \text{ and } S_2) \iff \text{not}(S_1) \text{ or not}(S_2)$
- De Morgan's law: $\text{not}(S_1 \text{ or } S_2) \iff \text{not}(S_1) \text{ and not}(S_2)$

Let's demonstrate that $S_1 \text{ implies } S_2 \iff \text{not}(S_1) \text{ or } S_2$ by showing that they have the same truth table, since this equivalence is not so intuitive for some people:

S_1	not (S_1)	S_2	not(S_1) or S_2	S_1 implies S_2
F	T	F	T	T
F	T	T	T	T
T	F	T	T	T
T	F	F	F	F

One example that demonstrates how logical equivalences are useful is the *proof by contradiction* way of reasoning. To prove that the statement S_1 implies the statement S_2 , we can assume that we have S_1 but at the same time we do not have S_2 , then we

arrive at something false or absurd, which proves that we cannot assume S_1 without concluding S_2 as well. We can verify the validity of this way of proving that S_1 implies S_2 using propositional logic equivalences:

- $S_1 \text{ implies } S_2 = \text{true} \iff$
- $\text{not}(S_1 \text{ or } S_2) = \text{true}$ (implication removal) $\iff$
- $\text{not}(\text{not}(S_1 \text{ or } S_2)) = \text{not}(\text{true}) \iff$
- $S_1 \text{ and } \text{not}(S_2) = \text{false}$ (De Morgan and double negation)

We endow a propositional logic framework with *rules of inference*, so that we are able to reason sequentially from one statement (simple or complex) to the next and arrive at a desired goal or at a correct proof of a statement. These are some of the rules of inference that accompany propositional logic:

- If $S_1 \text{ implies } S_2$ is true, and we are given S_1 , then we can infer S_2 .
- If $S_1 \text{ and } S_2$ is true, then we can infer S_1 . Similarly, we can also infer S_2 .
- If S_1 is equivalent to S_2 , then we can infer $(S_1 \text{ implies } S_2)$, and $(S_2 \text{ implies } S_1)$.
- Conversely, if $(S_1 \text{ implies } S_2)$ and $(S_2 \text{ implies } S_1)$, then we can infer that $(S_1 \text{ is equivalent to } S_2)$.

We finally emphasize that propositional logic does not scale to large environments and cannot efficiently capture universal relationship patterns. However, propositional logic provides the foundation of first-order logic and higher-order logic, since those build on top of propositional logic's machinery.

From Few Axioms to a Whole Theory

The inference rules are *sound*. They allow us to prove only true statements, in the sense that given a true statement and if we can infer a sound inference rule with it, we arrive at a true statement. Therefore, the guarantee that sound inference rules provide is that they do not allow false statements to be inferred from true ones. We need slightly more than that guarantee.

A logical framework is *complete* when we are able to infer *all* possible true statements using only the system's knowledge base (axioms) and its inference rules. The idea of *completeness* of a system is very important. In all mathematical systems, such as number theory, probability theory, set theory, or Euclidean geometry, we start with a set of axioms (Peano axioms for number theory and mathematical analysis, and probability axioms for probability theory), then we deduce theorems from these axioms using the logical rules of inference. One main question in any math theory is whether the axioms along with the rules of inference ensure its completeness and its consistency.

No first-order theory, however, has the strength to uniquely describe a structure with an infinite domain, such as the natural numbers or the real line. Axiom systems that do fully describe these two structures (that is, categorical axiom systems) can be obtained in stronger logics such as second-order logic.

Codifying Logic Within an Agent

Before moving on to first-order logic, let's recap what we learned in the context of an AI agent endowed with propositional logic. The following process is important and will be the same for more expressive logics:

1. We program an initial knowledge base (axioms) in the form of true statements.
2. We program the inference rules.
3. The agent perceives certain statements about the current state of its world.
4. The agent may or may not have a goal statement.
5. The agent uses the inference rules to infer new statements and to decide what to do (move to the next room, open the door, set the alarm clock, etc.).
6. Completeness of the agent's system (knowledge base together with the inference rules) is important here, since it allows the agent to infer *any* satisfiable goal statement given enough inference steps.

How Do Deterministic and Probabilistic Machine Learning Fit In?

The premise of machine learning (including) neural networks is that we do not program an initial knowledge base into the agent, and we do not program inference rules. What we program instead is a way to represent the input data, the desired outputs, and a hypothesis function that maps the input to the output. The agent then learns the parameters of the function by optimizing the objective function (loss function). Finally, the agent makes inferences on new input data using the function it learned. So in this context, the knowledge base and the rules can be separated *during learning* or *during inference*. During learning, the knowledge base is the data and the hypothesis function, the goal is minimizing the loss, and the rules are the optimization process. After learning, the agent uses the learned function for inference.

We can think of probabilistic machine learning models in exactly the same way if we replace the deterministic hypothesis function by the joint probability distribution of the features of the data. Once learned, the agent can use it for inference. For example, Bayesian networks would play a similar role for uncertain knowledge as propositional logic for definite knowledge.

First-Order Logic

Let's answer the same questions for first-order logic:

What objects exist in the agent's world?

Statements, objects, and relations among them.

How does the agent perceive the objects' states?

True (1), false (0), or unknown.

Propositional logic is great for illustrating how knowledge-based agents work, and to explain the basic rules of a certain logic's *language* and rules of inference. However, propositional logic is limited in what knowledge it can represent and how it can reason about it. For example, in propositional logic, the statement:

All users who are older than 18 can see this ad.

is easy to express as an *implies* statement (which is the same as *if then*), since this kind of language exists in the propositional logic framework. This is how we can express the statement as an inference in propositional logic:

(User older than 18 *implies* see the ad) *and* (User older than 18 = T), then we can infer that (see the ad = T).

Let's now think of a slightly different statement:

Some of the users who are older than 18 click on the ad.

Suddenly the language of propositional logic is not sufficient to express the quantity *some* in the statement! An agent relying only on propositional logic will have to store the whole statement, as is, in its knowledge base, then not know how to infer anything useful out of it. Meaning, suppose the agent gets the information that a user is indeed older than 18, it cannot predict whether the user will click on the ad or not.

We need a language (or a logical framework) whose vocabulary includes *quantifiers* such as *there exist* and *for all*, so that we can write something like:

For all users who are older than 18, *there exists* a subset who clicks on the ad.

These two extra quantifiers are exactly what *first-order logic* framework provides. This increase in vocabulary allows us to be more economical in what to store in the knowledge base, since we are able to break down the knowledge into objects and relations between them. For example, instead of storing:

All the users who are older than 18 see the ad;

Some of the users who are older than 18 click on the ad;

Some of the users who are older than 18 buy the product;

Some of the users who click on the ad buy the product;

as four separate statements in the knowledge base of an agent with only propositional logic framework (which we still don't know how to infer anything useful from), we can store three statements in first-order logic:

For all users who are older than 18, see ad = T;

For all users with see ad = T, *there exists* a subset who clicks on the ad;

For all users who click on the ad, *there exists* a subset who buys the product.

Note that in both propositional and first-order logics, given only these statements we will not be able to infer whether a specific user who is older than 18 will click on the ad or buy the product, or even the percentage of those doing that, but at least in first-order logic we have the language to express the same knowledge more concisely, and in a way where we would be able to make some useful inferences.

The most distinctive feature of first-order logic from propositional logic is that it adds to its base language quantifiers such as *there exist* and *for all* on top of *not*, *and*, *or*, *implies*, and *is equivalent to* that already exist in propositional logic. This little addition opens the door to express objects separately from their descriptions and their relationships to each other.

The powerful thing about propositional and first-order logics is that their inference rules are independent from both the domain and its knowledge base or set of axioms. To develop a knowledge base for a specific domain, such as a math field, or circuit engineering, we must study the domain carefully, choose the vocabulary, then formulate the set of axioms required to support the desired inferences.

Relationships Between For All and There Exist

For all and *there exist* are connected to each other through negation. The following two statements are equivalent:

- All users who are above 18 see the ad.
- There exists no one above 18 who doesn't see the ad.

In propositional logic language, these two statements translate to:

- For all users such that user > 18 is true, see the ad is true.
- There exists no user such that user > 18 and see the ad is false.

These are the relationships:

- $\text{not}(\text{There exists an } x \text{ such that } P \text{ is true}) \iff \text{For all } x, P \text{ is false.}$
- $\text{not}(\text{for all } x, P \text{ is true}) \iff \text{There exists an } x \text{ such that } P \text{ is false.}$
- $\text{There exists an } x \text{ such that } P \text{ is true} \iff \text{not for all } x P \text{ is false.}$
- $\text{For all } x, P \text{ is true} \iff \text{There exists no } x \text{ such that } P \text{ is false.}$

We cannot leave this section without appreciating the expressive power we gained by moving to first-order logic. This logic framework is now sufficient for such assertions and inferences to make sense:

Universal approximation theorem for neural networks

Roughly speaking, the universal approximation theorem asserts that *for all* continuous functions, *there exists* a neural network that can approximate the function as closely as we wish. Note that this does not tell us how to construct such a network, it only asserts its existence. Still, this theorem is powerful enough to make us *unsurprised* about the success of neural networks in approximating all kinds of *input* to *output* functions in all kinds of applications.

Inferring relationships

Parents and children have inverse relationships to each other: if Sary is the child of Hala, then Hala is the mother of Sary. Moreover, the relationship is in one direction: Sary cannot be the mother of Hala. In first-order logic, we can assign two functions indicating the relationships: *mother of* and *child of*, variables that can be filled in by *Hala* and *Sary* or any other mother and child, and a relationship between the *functions* that holds for all their input variables:

For all x, y , if $\text{mother}(x, y) = \text{T}$ then $\text{mother}(y, x) = \text{F}$;
and *for all* x, y , $\text{mother}(x, y) \iff \text{child}(y, x)$.

Now if we equip an agent with this knowledge and tell it that Hala is the mother of Sary, or $\text{mother}(\text{Hala}, \text{Sary}) = \text{T}$, then it will be able to answer queries like:

- Is Hala the mother of Sary? T
- Is Sary the mother of Hala? F
- Is Sary the child of Hala? T
- Is Hala the child of Sary? F
- Is Laura the mother of Joseph? Unknown

Note that we will have to store each statement separately in a propositional logic world, which is outrageously inefficient.

Probabilistic Logic

What objects exist in the agent's world?

Statements.

How does the agent perceive the objects' states?

A probability value between 0 and 1 that a statement is true.

Probability is the extension of first-order logic that allows us to quantify our uncertainty about the truth of a statement. Rather than asserting whether a statement is true or false, we assign to the degree of our belief in the truth of the statement a score between zero and one. Propositional and first-order logics provide a set of inference rules that allow us to determine the truth of some statements, given the assumption that some other statements are true. Probability theory provides a set of inference rules that allow us to determine how likely it is that a statement is true, given the likelihood of truth of other statements.

This extension to dealing with uncertainty results in a more expressive framework than first-order logic. The axioms of probability allow us to extend traditional logic truth tables and inference rules. For example, $P(A) + P(\text{not } (A)) = 1$: if A is true, then $P(A) = 1$ and $P(\text{not } A) = 0$, which is consistent with first-order logic about a statement and its negation.

Viewing probability theory as a natural extension of first-order logic is satisfying to a mind that needs to connect things together as opposed to viewing them as disparate things. Viewing it this way also naturally leads to Bayesian reasoning about data, since we update an agent's prior distribution as we gather more knowledge and make better inferences. This binds all our subjects together in the most *logical* way.

Fuzzy Logic

What objects exist in the agent's world?

Statements with a degree of truth between $[0,1]$.

How does the agent perceive the objects' states?

A known interval value.

The worlds of propositional and first-order logic are black and white, true or false. They allow us to start with true statements and infer other true statements. This setting is perfect for mathematics where everything can either be right or wrong (true or false), or for a video game with very clear boundaries for its SIMS. In the real world, many statements can be vague about whether they are fully true (1) or fully false (0), meaning they exist on a *scale of truth* as opposed to at the edges: *Paris is romantic*; *She is happy*; *the movie The Dark Knight is good*. Fuzzy logic allows this and

assigns values to statements between 0 and 1 as opposed to strict 0 or strict 1: Paris is romantic (0.8); She is happy (0.6); the movie The Dark Knight is good (0.9).

How do we make inference in a vague world where truth comes on a sliding scale? It definitely is not as straightforward as inference in true-and-false worlds. For example, how true is the statement: “Paris is romantic *and* she is happy,” given the previous truth values? We need new rules to assign these values, and we need to know the context, or a domain. Another option is word vectors, which we discussed in [Chapter 7](#). These vectors carry the meaning of words in different dimensions, so we can compute the cosine similarity between the vector representing the word *Paris* and the vector representing the point *romantic*, and assign that as the truth value of the statement *Paris is romantic*.

Note that the degree of belief in probability theory is not the same as the scale of truth in fuzzy logic. In probabilistic logic, the statements themselves are unambiguous. What we want to infer is the probability that the unambiguous statement is true. Probability theory does not reason about statements that are not entirely true or false. We do not calculate the probability that Paris is romantic, but we calculate the probability that a person, when randomly asked whether Paris is romantic, would answer true or false.

One interesting thing about fuzzy logic is that it kicks two principles present in other logics to the curb: the principle that if a statement is true, then its negation is false; and the principle that two contradictory statements cannot be true at the same time. This actually opens the door to inconsistency and *open universe*. In a way, fuzzy logic doesn’t attempt to correct vagueness; instead, it embraces it and leverages it to allow functioning in a world where the boundaries are unclear.

Temporal Logic

There are other types of special purpose logics, where certain objects, such as *time* in this section, are given special attention, having their own axioms and inference rules, because they are central to the knowledge that needs to be represented and the reasoning about it. *Temporal logic* puts time dependence and the axioms and inference rules about time dependence at the forefront of its structure, as opposed to adding statements that include time information to the knowledge base. In temporal logic, statements or facts are true at certain times, which could be time points or time intervals, and these times are ordered.

What objects exist in the agent’s world?

Statements, objects, relations, times.

How does the agent perceive the objects’ states?

True (1), false (0), or unknown.

In temporal logic, we can represent statements such as:

- The alarm goes off when it is 7:00 a.m.
- Whenever a request is made to a server, access is eventually granted, but it can never be granted to two simultaneous requests.

Comparison with Human Natural Language

We spent the whole chapter going through logical systems that are able to express knowledge that humans' natural languages seem to do effortlessly. I just wrote a whole book about math using only the English language, as opposed to any other technical language. How do we do it? How do humans represent and expand their knowledge base, and what rules does natural language use for representation and reasoning that it is able to be so expressive? Moreover, the particular natural language used is not important: any multilingual speaker knows the thought but not necessarily which particular language they are using to express that thought. There is an internal nonverbal representation for what people know or want to express. How does it work, and how can we unlock its secrets and give them to our machines?

Similar to human language, if we represent the same knowledge in two different formal logics, then we can infer the same facts (assuming the logics have completeness inference rules). The only difference would be which logic framework provides an easier route for inference.

That said, human natural language allows for ambiguity on many occasions and cannot make absolute mathematical assertions without the formality of mathematics and the formal logic it employs. We cannot ask a human who has no access to a GPS system to predict the exact time it takes to drive from DC to NYC on a specific day, but we can ask a GPS machine for that kind of accuracy.

Machines and Complex Mathematical Reasoning

Mathematical reasoning is the distillation of human logic. When we prove a theorem using mathematical reasoning, we get one step closer to *the universal truth*. Teaching machines to prove mathematical theorems—and even more ambitiously, generating new ones—requires navigating infinite search spaces and symbolic reasoning. Once again, neural networks are proving useful for advancing intelligent machines. Researchers at Meta, Vrije Universiteit Amsterdam, and CERMICS École des Ponts ParisTech used a combination of deep learning, **online training**, transformers (large language models), and reinforcement learning for automated mathematical theorem proving. Their paper, *HyperTree Proof Search for Neural Theorem Proving* (2022) presents state-of-the-art results in this field.

Summary and Looking Ahead

An AI agent endowed with various types of logic can express knowledge about the world, reason about it, answer queries, and make inferences that are allowed within the boundaries of these logics.

We discussed various logic frameworks, including propositional logic, first-order logic, probabilistic logic, fuzzy logic, and temporal logic.

The next natural questions would be: what content should go into an agent's knowledge base? And how to represent facts about the world? In what framework should knowledge be represented and inference made?

- Propositional logic?
- First-order logic?
- Hierarchical task networks for reasoning about plans?
- Bayesian networks for reasoning with uncertainty?
- Causal diagrams and causal reasoning where an agent is allowed to selectively break the rules of logic?
- Markov models for reasoning over time?
- Deep neural networks for reasoning about images, sounds, or other data?

Another possible next step is to dive deeper into any of the logic frameworks that we discussed, learning their inference rules and existing algorithms for inference, along with their strengths, weaknesses, and which kinds of knowledge bases they apply to. A recurring theme in these studies is investigating inference rules that provide a complete proof system, meaning a system where the axioms or the knowledge base along with the rules allow one to prove *all* possible true statements. Such rules include the *resolution inference rule* for propositional logic and the *generalized resolution inference rule* for first-order logic, which work for special types of knowledge bases. These are all important for theory (proving mathematical theorems), and technology (verifying and synthesizing) software and hardware. Finally, some logics are *strictly more expressive* than others, in the sense that some statements that we can represent in the more expressive logic cannot be expressed by any finite number of statements using the language of the less expressive logic. For example, *higher-order logic* (which we did not discuss in this chapter) is strictly more expressive than first-order logic (which we did discuss in this chapter, and which is powerful enough to support entire math theories).

Artificial Intelligence and Partial Differential Equations

I want to model the whole world.

—H.

The first scene in the movie *Top Gun: Maverick* (2022) shows Maverick (Tom Cruise) manning an experimental military aircraft and pushing it to 10 times the speed of sound (10 Mach) before losing its stability at around 10.2 Mach. The fastest nonfictional manned aircraft so far can reach 6.7 Mach (Figure 13-1). Real speed or unreal (yet), it is mesmerizing to watch physics, math, and engineering come together to put these planes in the air, especially with their spectacular midair maneuvering.

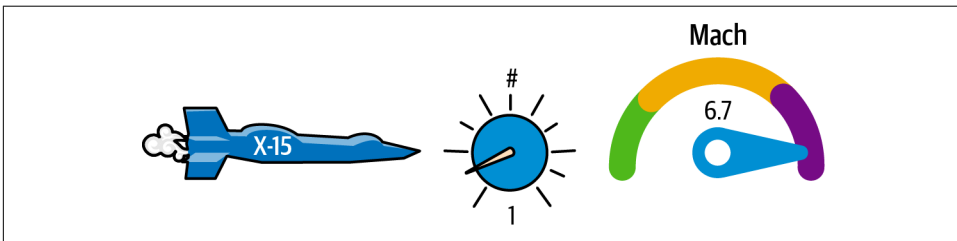


Figure 13-1. Fastest manned aircraft ever made (*image source*)

These are a few of the partial differential equations (PDEs) that come to mind while watching Maverick's awesome dogfight and 10 Mach scenes:

The wave equation for wave propagation

Think of the speed of sound, the propagation of a sound wave in the air, and the variations of the speed of sound at different altitudes due to the variations in temperature and air density.

Navier-Stokes equations for fluid dynamics

Think of the fluid flow, air tunnels, and turbulence.

The G-equation for combustion

Think of the combustion in the aircraft's engine and the flames coming out of the aircraft's exhausts.

Material elasticity equations

Think of the aircraft wing panel, the lift force, the buckling of the wing panel (the process of out-of-stress plane movement that happens under compression; see [Figure 13-2](#)) caused by loading, which in turn reduces the load-carrying capabilities of the wing. When load-carrying capabilities fall below the design limits, failure happens.



Figure 13-2. Buckling in an aircraft (image source)

PDE simulations also come to mind. Think of the flight path simulation and the crew chatting with Maverick as they watch his flight unfold in real time on their computer screens.

The list goes on. Are we claiming here that we made aircrafts fly because we wrote down and solved PDEs? No. Aviation museums tell the story of the Wright brothers, their experiments, and the evolution of aviation industry. Science and experimentation go hand in hand. What we want to claim instead is that we can invent, improve, and optimize all kinds of designs because of differential equations and math.

What Is a Partial Differential Equation?

A PDE is an equation, which means a lefthand side is equal to a righthand side, that involves a function of several variables along with any of its partial derivatives. A partial derivative of a function with respect to a certain variable measures the rate of change of the function with respect to that variable. Ordinary differential equations (ODEs) are those that involve functions of only one variable, such as only time, only

space, etc. (as opposed to several variables), and their derivatives. A dynamic system is a greatly important ODE, describing the evolution in time of the state of a system that we care for, such as a system of particles, or the state of a customer in a business setting. The ODE involves one derivative in time of the state of the system, and the dynamics are prescribed as a function of the system state, system physical parameters, and time. The ODE looks like $\frac{d\vec{x}(t)}{dt} = f(\vec{x}(t), a(t), t)$. We will visit dynamic systems multiple times in this chapter. Most of the time, if we are able to transform a PDE into a system of ODEs, or maybe into a dynamical system, it is more or less solved.

Nature gave us neither the deterministic functions nor the joint probability distributions that it uses to produce the world that we observe around us and can accurately measure. Until now, it has kept those secret. It did, however, give us ways to measure, assess, or make laws about *how things change* relative to each other, which is exactly what partial differential equations represent. Because *the ways things change* are nothing but derivatives.

The goal of solving a PDE is to undo the differential operator so that we can recover the function without any derivatives. So we search for an exact or an approximate inverse (or pseudoinverse) of the differential operator that the PDE represents. Integrals undo derivatives, so solution representations of PDEs often involve the integrals of some kernel functions against the input data of a PDE (its parameters, initial and/or boundary conditions). We will elaborate on this as the chapter evolves.

People usually classify ODEs and PDEs into *types*. My take on this is that we should not confuse ourselves with *classifications* unless we happen to be personally working with these special ODEs or PDEs and their solutions happen to have a direct and immediate impact on the future of humanity. When in this chapter you encounter a certain type, such as nonlinear parabolic or backward stochastic, accept the name, then move directly to understanding the point that I am trying to make. Don't even try to google these terms. It will be similar to googling your symptoms and finding that you will die tomorrow. Consider yourself warned.

Modeling with Differential Equations

Differential equations model countless phenomena in the real world: air turbulence, the motions of galaxies, the behavior of materials at the nanoscale, pricing financial instruments, games with adversaries and multiple players, and population mobility and growth. Typical courses on PDEs skip the modeling step, so the PDEs that we end up studying seem to come out of the blue, but that is not the case. Where PDEs come from is as important as trying to analyze them and solve them. Usually, PDEs express some conservation laws, such as conservation of energy, mass, momentum, etc., as they relate to our particular application. Many PDEs are an expression of a conservation statement that looks like:

rate of change of a quantity in time = gains – losses

Now, when we have a bounded domain, the PDE works in the *interior* of the domain, but we need to accompany it with *boundary conditions* that tell us exactly what is happening at the boundary of the domain. If the domain is unbounded, then we need *far field conditions* that tell us what is happening as $x \rightarrow \infty$. We write these conditions using limits notation. If the PDE has derivatives in time, then we need some initial time conditions, or end time conditions. How many of these conditions we need depends on the order of the PDE. Think of these as how many equations we need to solve for how many unknowns. The unknowns are the *integration constants* of the PDEs. When we solve a PDE, we seek information about the function, given information about its derivatives. To get rid of these derivatives and recover the function, we must *integrate* the PDE, getting integration constants along the way. We need the boundary and/or far field conditions to solve for these constants.

Models at Different Scales

Realistic models that mimic nature faithfully need to account for all the important variables along with their interactions, sometimes at varying scales of space and time. Some work goes into writing the equations for the mathematical models. Once formulated, they are elegant, condensing a whole wealth of information into a few lines of equations. These equations involve functions, their derivatives, and the model's parameters, and are usually harder to solve than to formulate. Moreover, if two models describe the same phenomenon at different scales, say one on the atomistic scale (rapidly wiggling molecules) and another at a bigger scale, say at the microscopic or macro scale (the one we observe), then the two models' equations would look very different, and they may even be relying on physical laws from different fields of science. Think, for example, about describing the motion of gases at the molecular level (particle velocity, position, forces acting on it, etc.) and how to relate that to the thermodynamics of the gaseous system observed at the macroscopic scale. Or think of the ways atoms bond together to form crystalline structures, and how those structures translate into material properties, such as conductivity, permeability, brittleness, etc. The natural question is then, can we reconcile such models when each operates and is successful to some degree at a different scale? More precisely, if we take the limit of one model to the regime of the other, would we get the same thing? These are the types of questions that analysts address. Reconciling different scale models validates them and unifies different areas of math and science.

The Parameters of a PDE

The PDEs that we write down for a model usually involve parameters. These parameters have to do with the properties of the physical system that we are modeling. For example, for the heat equation:

$$u_t(\vec{x}, t) = \alpha \Delta u(\vec{x}, t)$$

the parameter α is the *diffusion coefficient*, which is a physical constant depending on the properties of the diffusing substance and the properties of the medium it is diffusing into. We usually get these from reference tables obtained from experiments. Their values are very important for engineering purposes. When our equations model reality, we must use parameter values that are derived from this real experimental or observational data. But experimental and observational data are usually noisy, and have missing values, unexplained outliers, and all kinds of rough seas for mathematical models. Many times we don't even have experimental values for the parameters that are in our equations. The experiments can be expensive (think Large Hadron Collider) or even impossible. So one must *learn* the parameter values using indirect ways, based on accessible combinations of experimental, observational, and computer simulated values of some other variables. Historically, many of these parameter values were *hand tuned* to fit some desired outcome, which is not good! We should have clear justifications for the choices of the parameter values that go into simulations. We will see how machine learning helps PDEs here in learning parameter values from data.

Changing One Thing in a PDE Can Be a Big Deal

If you had a PDE class in college, think back to the simplest equation that you studied, perhaps the heat diffusion equation on a rod, such as those in [Figure 13-3](#). If you did not study PDEs, do not worry about its details. The formula of the heat equation is:

$$u_t(x, t) = \alpha \Delta u(x, t)$$

Here, $u(x, t)$ measures the temperature at the point x in the rod and at time t , and the operator Δ is the second derivative in x (so $\Delta u(x, t) = u_{xx}(x, t)$), since a rod is only one-dimensional if we ignore its thickness. In higher dimensions, the operator Δ is the sum of the second derivatives in each of the dimensions.

Now let's change the domain from a rod to a weird-shaped plate: not a square or a circle or ellipse, but something irregular; for example, compare [Figure 13-3](#) with [Figure 13-4](#). The formula for the true solution (which we learn in an introductory PDE class) that works for the rod does not work for the weird plate anymore. It gets even worse. Not only do we lose access to the analytical solution by changing the domain, when we try to numerically solve the differential equation with the new domain, the new geometry suddenly complicates matters.

Now we have to find a discrete mesh that accurately portrays the shape of the new domain with all its details, then we have to compute a numerical solution on top of

that mesh that satisfies the equation in the interior of the domain, and satisfies the boundary conditions along the weird-looking boundary.



Figure 13-3. Studying heat diffusion on a rod is easy (for people who studied PDEs) both analytically and numerically

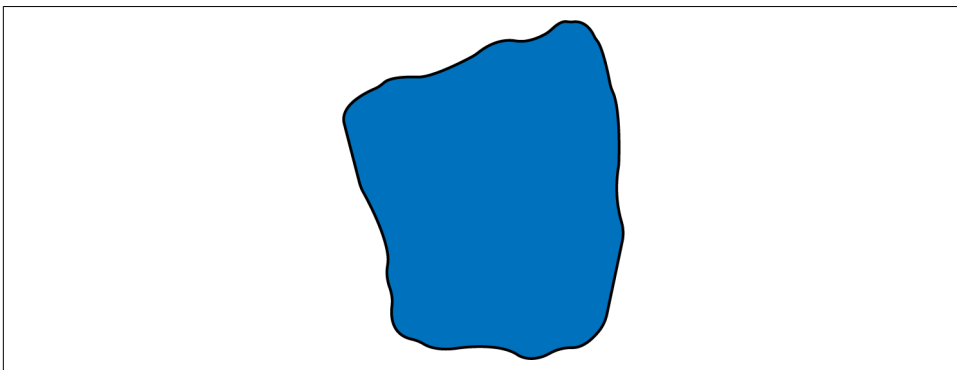


Figure 13-4. Studying heat diffusion on an irregular geometry is not as easy

This is normal in PDEs: Change one tiny thing and suddenly all the mathematical methods that we learned may not apply anymore. Such changes include:

- Change the shape of the domain
- Change the types of the boundary conditions
- Introduce a space or time dependence in the coefficients (the parameters)
- Introduce nonlinearities
- Introduce terms with more derivatives (higher order)
- Introduce more variables (higher dimension)

This frustrating aspect turns off many students from specializing in PDEs (no one wants to be an expert at only one equation, which could be very removed from modeling reality to start with). We don't want to be turned off. We want to see the big picture.

Natural phenomena are wonderfully varied, so we have to accept the variations in the PDEs and their solution methods as part of our quest to understand and predict nature. Moreover, PDEs are a large and old field. A lot of progress has been made on unifying methods for many families of both linear and nonlinear partial differential equations, and a lot of powerful analysis has been discovered along the way. The

status quo is that PDEs are a very useful field that does not have, and might never have, a unifying theory.

In general, nonlinear PDEs are more difficult than linear PDEs, higher-order PDEs are more difficult than lower-order ones, higher-dimensional PDEs are more difficult than lower-dimensional ones, systems of PDEs are more difficult than single PDEs, we cannot write explicit formulas for solutions for the majority of PDEs out there, and many PDEs are only satisfied in *weak* forms. Many PDEs have solutions that develop singularities as time evolves (think the wave equation and shock waves). Mathematicians who develop PDE theory spend their time proving the *existence* of solutions of PDEs, and trying to understand the *regularity* of these solutions, which means how nice they are in terms of actually possessing the derivatives that are involved in the PDE. These use a lot of advanced calculus methods, looking for estimates on integrals (inequalities for upper and lower bounds).

Can AI Step In?

Now, wouldn't it be great if we had methods that account for variations in the PDE, the geometry of the domain, the boundary conditions, and the parameter ranges, similar to the actual physical problems? Many sectors of the industry and areas of science have their eyes on AI and deep learning to address their long-standing problems or shed new light on them. The past decade's astronomical advancement in computing solutions of very high-dimensional problems has the potential to transform many fields held down by the curse of dimensionality. Such a transformation would be a sea change for PDEs and in turn for humanity as a whole because of the sheer amount of science that is unlocked by PDEs and their solutions.

For the rest of this chapter, we highlight the hurdles that the differential equations community encounters with the traditional approaches to finding solutions to their PDEs, and with fitting real and noisy data into their models. We then illustrate how machine learning is stepping in to help bypass or alleviate these difficulties. We also consider two questions:

- What can AI do for PDEs?
- What can PDEs do for AI?

We need to make sure that the machine learning hallmarks of *training function*, *loss function*, and *optimization* settings are clear when serving PDEs, along with the labels or targets for supervised learning. Fitting the well-established field of PDEs into a machine learning setting is not super straightforward. Ideally, we need to establish a map from a PDE to its solution. This requires some pausing and thinking.

Numerical Solutions Are Very Valuable

Writing a mathematical model that describes a natural phenomenon, in the form of equations describing how the involved variables interact with each other, is only a first step. We need to solve these equations.

Analytical solutions are harder than numerical solutions, since the more the model mimics nature, the more complex the equations tend to be. Even when analytical methods cannot provide formulas for the solutions, they still provide valuable insights into their important properties. Numerical solutions are easier than analytical solutions, because they involve discretizing the continuous equations, moving us from the realm of continuous functions to the realm of discrete numbers, or from infinite dimensional function spaces to finite dimensional vector spaces (linear algebra), which our machines are built to compute. Numerical solutions provide invaluable insights into the true analytical solutions of the models, and are easy to test against experimental observations, when available. They are also easy to tune, so they are great aids for experimental design.

We can devise numerical solutions at any scale, but the curse of dimensionality haunts us when we try to implement and compute our numerical schemes. In many situations, for a numerical simulation to mimic even one second of the natural evolution of a system requires a tremendous amount of computational power, so a lot of dimension reductions and simplification assumptions must happen, which moves us even farther from having a good approximation of the true solutions. The sad part is that this is the norm rather than the exception.

Continuous Functions Versus Discrete Functions

The function $f(x) = x^2 - 3$ is continuous over the whole real line $(-\infty, \infty)$. When we discretize it for a numerical scheme for a machine to process, first, the domain cannot be the whole real line anymore, because machines cannot yet conceptualize infinite domains. So our first approximation is slashing the domain dramatically to some finite $[-N, N]$ where N is a large number. Our second approximation is discretizing this finite domain, drastically reducing it one more time, from a continuum $[-N, N]$ to only a finite set of points. If we use many points, then our mesh will be finer and our approximation better, at the expense of increased computation cost. Say we use six points only to discretize the interval $[-5, 5]$: $-5, -3, -1, 1, 3, 5$, then our continuous function will be reduced to a vector with only six entries:

$$f(x) = x^2 - 3 \text{ is continuous on } (-\infty, \infty)$$

$$\xrightarrow{\text{discrete}} f = \begin{pmatrix} (-5)^2 - 3 \\ (-3)^2 - 3 \\ (-1)^2 - 3 \\ 1^2 - 3 \\ 3^2 - 3 \\ 5^2 - 3 \end{pmatrix} = \begin{pmatrix} 22 \\ 6 \\ -2 \\ -2 \\ 6 \\ 22 \end{pmatrix}$$

Figure 13-5 shows the continuous function and its insanely under-representative six point approximation.

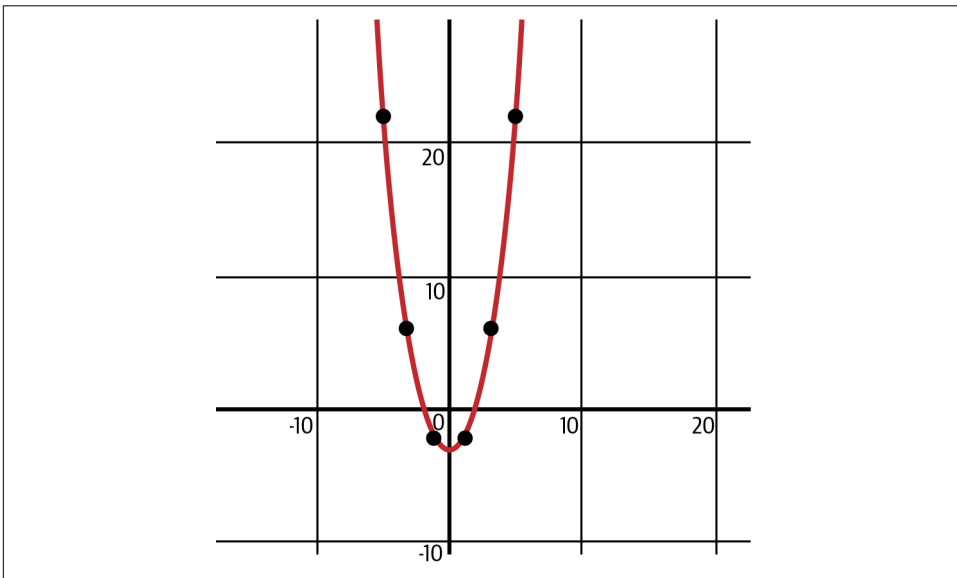


Figure 13-5. Discretizing the continuous function into a vector with only six points. We lose all the continuous wealth of information in between the points.



We Still Need to Discretize Derivatives

We can discretize a function $f(x)$ by selecting points in an interval, just like we did above. Differential equations contain *derivatives of functions* such as $f_x \Delta f$, not only functions. So we must discretize the derivatives, or find some other way to reduce a problem from functional spaces (such as continuous spaces) to vector spaces (so we can use linear algebra and compute using our machines). Finite differences and finite elements are two popular discretization methods for differential equations. We will go over them shortly, along with a probabilistic Monte Carlo method based on random walks.

One trade-off of the simplicity of numerical solutions is that when we discretize, we make an approximation, reducing an infinite continuum to a finite set of points, losing all the infinitely detailed information that is between the finite set of points. That is, we sacrifice high resolution. For certain equations, there are analytical methods that help us quantify exactly how much information we lose by discretization, and help move us back to an accurate analytical solution by taking the limit as the size of the discrete mesh goes to zero.

Discretizing continuous functions and the equations that involve them has advantages: easy access. We can teach high school students how to numerically solve the heat equation describing the diffusion of heat in a rod (soon in this chapter), but we cannot teach them how to solve it analytically until they finish their college calculus sequence and linear algebra. This is why we must teach children how to model and compute numerical solutions of real-life problems at a very young age. The simplicity of numerical solutions and the power of computation to aid in solving all kinds of human problems should make this a priority in our education system. I doubt that nature intended for us to build and unravel crazily complicated mathematical theories before *computing* how the world around us works. I also doubt that nature is as complicated as some mathematical theories happen to be (even though they are still interesting in their own right, if only as an exercise in how far the rules of logic and inference can lead us).

PDE Themes from My Ph.D. Thesis

The story of my Ph.D. thesis demonstrates the drastic difference between mathematical theory and numerical approaches. It is also a good prototype for some of the themes of this chapter. For my Ph.D., I worked on a mathematical model that describes the way atoms diffuse and hop between different levels of a stair-like surface of a thin crystal. This is useful for the materials science community and for the engineers involved in designing the mini things that go into our electronic devices. As

time evolves, the crystal's shape changes due to the movement of atoms on its surface. Eventually, the crystal relaxes into some stable shape.

Discretize right away and do a computer simulation

The moment I wrote down the equations, I was able to do a computer simulation that showed how the shape of the crystal evolves with time. This is one of the PDEs that I worked on (not that you should care about it or know what the function in it refers to):

$$u_t(h,t) = -u^2(u^3)_{hhhh} \text{ where } h \in [0,1], t \in [0,\infty)$$

For the trained eye, this is a highly nonlinear fourth order equation. The unknown function u appears both squared and cubed. Its cube appears with four derivatives in space, which we can think of as four degrees removed from the function that we want to evaluate. **Figure 13-6** shows my PDE's discretization in space using finite differences (we will discuss finite differences shortly) and its boundary conditions (the function values at points 0 and 1).

Discrete slope $u_i = \frac{1/N}{x_{i-1} - x_i}$ ODE: $\begin{cases} u_i = u_i^2 \Delta_i \Delta_i u^3 \quad i=1, \dots, N-1 \\ u_0 = u_N = 0 \\ \Delta_0 u^3 = \Delta_N u^3 = 0 \end{cases}$	Continuum slope $u(h, t)$ PDE: $\begin{cases} u_t = -u^2(u^3)_{hhhh} \\ u(0, t) = u(1, t) = 0 \\ u^3_{hh}(0, t) = u^3_{hh}(1, t) = 0 \end{cases}$
--	--

Figure 13-6. Discrete differential equations and their continuum analog

Usually, the more nonlinear an equation is, the more stubborn it is in submitting to standard analytical techniques. I still needed to do the mathematical analysis and prove that the shape that the numerical simulation showed is indeed what the equations want the solution to do, meaning that it is *the* analytical solution, and it is what nature chooses among possible others. I had to spend the next two years of my life doing only that. What I came up with was a tiny proof in a tiny case for a physically unrealistic *one-dimensional crystal*! I had to reduce my equations to only one dimension to be able to do any mathematical analysis with them.

The curse of dimensionality

One theme that is always present is the dimensionality of the problem. Even when I did the numerical simulation, which took less than one afternoon, I was only able to do it for the equation over the one-dimensional domain. When I tried to do a simulation to model a realistic thin film laboratory crystal that lies on a flat surface, meaning when I had to discretize a two-dimensional surface instead of a one-dimensional

segment, the number of discrete points jumped from 100 on the one-dimensional segment to $100^2 = 100,000$ on the two-dimensional surface. My computer at the time could not numerically solve the exact same equation that took only few seconds in the one-dimensional case. Granted, I was not sophisticated enough to compute on the university's server or use parallel computing (I do not know if distributed cloud computing had even been invented back then). Such is life with the curse of dimensionality. The computational expense rises exponentially with the number of dimensions. Now let's think of the equations whose domains have high dimensions to start with (even before discretization), like the Schrödinger equation for quantum particle systems, the Black-Scholes equation for pricing financial instruments, or the Hamilton-Jacobi-Bellman equation in dynamic programming that models multi-player games or resource allocation problems. Imagine then the magnitude of the curse of dimensionality.

The geometry of the problem

Another theme that we mentioned earlier but is worth repeating: the shape of the domain matters, for both analysis and numerical computation. In my unrealistic one-dimensional case, I used a segment as the domain for my equation. In the two-dimensional case, I had many more choices: a rectangle (has the advantage of a regular grid), a circle (has the advantage of radial symmetry), or any other realistic irregular shape that doesn't usually have a name. For analysis, the rectangle and the circle domains are the easiest (not for my particular equations but for other simpler equations like linear equations). For simulations, these are good too. But when the shape of the domain is not regular, which is the case for most realistic things, we need to place more discrete points in the parts where it is irregular if we want to capture the domain faithfully. The curse of dimensionality rears its unwelcome head again: more points mean longer vectors and larger input matrices for computations.

Model things that you care for

To finish my Ph.D. story, I never saw a real thin film crystal like the one I was working on until 10 years after I finished my degree, when my friend showed me a thin film crystal of gold in her lab. In retrospect, maybe I should have started there, by *seeing* exactly what it is in real life that I was trying to model. My priorities now are aligned differently, and I always start by asking whether I care for what it is I am trying to model, how closely the model I choose to work on mimics reality, and whether thinking about analytical solutions is even worth the time and the effort for this particular application.

Discretization and the Curse of Dimensionality

Mathematicians who study PDEs like the continuous world, but machines like the discrete world. Mathematicians like to analyze functions, but machines like to compute functions. To reconcile the two, and for machines to be of aid to mathematicians and vice versa, we can discretize our continuum equations. How? First, we discretize the domain of the equation, creating a discrete mesh. We choose the type of mesh (regular or irregular) and how fine or coarse it is. Then we discretize the differential equation itself, using one of four popular methods:

Finite differences

Deterministic, good for discretizing time, one-dimensional or relatively regular spatial geometries.

Finite elements

Deterministic, good for discretizing more complex spatial geometries, also spatial geometries that vary with time.

Variational or energy methods

This is similar to finite elements but works on a narrower set of PDEs. They should possess a *variational principle*, or an *energy formulation*, that is, the PDE itself should be equivalent to $\nabla E(u) = 0$ for some energy functional $E(u)$ (mapping functions to the real line). The reason I was able to get my Ph.D. is that I discovered such an energy functional for my PDE by pure luck. Just like the minimum of a calculus function happens at points where $\nabla f(\vec{x}) = 0$, the minimum of an energy functional happens at *functions* where $\nabla E(u) = 0$, but of course we need to define what it means to take the derivative of a *functional*.

Monte Carlo methods

Probabilistic, starts with discretizing the PDE, then uses that to devise an appropriate random walk scheme that enables us to *aggregate* the solution at a certain point in the domain.

The word *finite* in these methods stresses the fact that this process moves us from a continuum of infinite dimensional spaces of functions to finite dimensional spaces of vectors.

If the mesh we use for discretization is too fine, it captures more resolution, but we end up with high-dimensional vectors and matrices. Keep this curse of dimensionality in mind, and the following: one of the main reasons neural networks' popularity skyrocketed is that they seem to have a magical ability to overcome the curse of dimensionality. We will see how soon.

Finite Differences

We use finite differences to numerically approximate the derivatives of the functions that appear in PDEs. For example, a particle's velocity is the derivative in time of its position vector, and a particle's acceleration is two derivatives in time of its position vector.

In finite difference approximations, we replace the derivatives with linear combinations of function values at discrete points in the domain. Recall that one derivative measures a function's rate of change. Two derivatives measure concavity. Higher derivatives measure more stuff that some people in the sciences happen to use. The connection between a function's derivatives at a point and how its values near that point compare to each other is pretty intuitive.

The mathematical justification for these approximations rely on Taylor's theorem from calculus:

$$f(x) = f(x_i) + f'(x_i)(x - x_i) + \frac{f''(x_i)}{2}(x - x_i)^2 + \frac{f^{(3)}(x_i)}{3!}(x - x_i)^3 + \cdots \\ + \frac{f^{(n)}(x_i)}{n!}(x - x_i)^n + \text{error term}$$

where the error term depends on how nice the next order derivative $f^{(n+1)}(\xi)$ is near the x_i where we are attempting to use our polynomial approximation. Taylor's theorem approximates a nice enough function near a point with a polynomial whose coefficients are determined by the derivatives of the function at that point. The more derivatives the function has at a point, the nicer it is and the *more like a polynomial* it behaves near that point.

Now let's discretize a one-dimensional interval $[a, b]$, then write down finite difference approximations for the derivatives of a function $f(x)$ defined over this interval. We can discretize $[a, b]$ using $n + 1$ equally spaced points, so the mesh size is $h = \frac{b-a}{n}$. We can now evaluate f at any of these discrete points. If we care about the values near some point x_i , we define $f_{i+1} = f(x_i + h)$, $f_{i+2} = f(x_i + 2h)$, $f_{i-1} = f(x_i - h)$, etc. In the following, h is small, so an $O(h^2)$ method (or higher order in h) is more accurate than an $O(h)$ method:

1. Forward difference approximation of $O(h)$ accuracy for the first derivative (uses two points):

$$f'(x_i) \approx \frac{f_{i+1} - f_i}{h}$$

2. Backward difference approximation of $O(h)$ accuracy for the first derivative (uses two points):

$$f'(x_i) \approx \frac{f_i - f_{i-1}}{h}$$

3. Central-difference approximations of $O(h^2)$ accuracy for derivatives up to the fourth (uses two points, averages forward and backward differences):

$$f'(x_i) \approx \frac{f_{i+1} - f_{i-1}}{2h}$$

$$f''(x_i) \approx \frac{f_{i+1} - 2f_i + f_{i-1}}{h^2}$$

$$f'''(x_i) \approx \frac{f_{i+2} - 2f_{i+1} + 2f_{i-1} - f_{i-2}}{2h^3}$$

$$f^{(4)}(x_i) \approx \frac{f_{i+2} - 4f_{i+1} + 6f_i - 4f_{i-1} + f_{i-2}}{h^4}$$

4. Central-difference approximations of $O(h^4)$ accuracy for derivatives up to the fourth:

$$f'(x_i) \approx \frac{-f_{i+2} + 8f_{i+1} - 8f_{i-1} + f_{i-2}}{12h}$$

$$f''(x_i) \approx \frac{-f_{i+2} + 16f_{i+1} - 30f_i + 16f_{i-1} - f_{i-2}}{12h^2}$$

$$f'''(x_i) \approx \frac{-f_{i+3} + 8f_{i+2} - 13f_{i+1} + 13f_{i-1} - 8f_{i-2} + f_{i-3}}{8h^3}$$

$$f^{(4)}(x_i) \approx \frac{-f_{i+3} + 12f_{i+2} - 39f_{i+1} + 56f_i - 39f_{i-1} + 12f_{i-2} - f_{i-3}}{6h^4}$$

What does the $O(h^k)$ mean? It is the order of the numerical approximation in h . When we replace a derivative using its numerical approximation, we commit an error. The $O(h^k)$ tells us how much error we are committing. Obviously this depends on the size of the mesh h . The error should be smaller with finer meshes. To derive such error bounds, we use Taylor expansions of $f(x + h)$, $f(x - h)$, $f(x + 2h)$, $f(x - 2h)$, etc. and linear combinations of those to determine both the desired

derivative's approximation and the order of our finite difference approximation in terms of h . To be able to use Taylor expansions, we assume that we are dealing with functions that *indeed have the required number of derivatives* at the points where we are evaluating them. This means that we assume that our function is nice enough to allow these derivative evaluations. If the function has singularities near these points, then we need to find ways around that, such as using much finer meshes near the singularities.

Example: Solve $y''(x) = 1$ on $[0,1]$, with boundary conditions $y(0)=-1$ and $y(1)=0$

This is a second linear order ordinary differential equation on a bounded domain in one dimension. This example is trivial because the analytical solution is so easy. All we have to do is integrate the equation twice and recover the function without its derivatives $y(x) = 0.5x^2 + c_1x + c_2$, where the c 's are the constants of integration. We plug in the two boundary conditions to find the c 's and obtain the analytical solution $y(x) = 0.5x^2 + 0.5x - 1$. However, the point of this example is to show how to use finite differences to compute the *numerical solution*, not the analytical one, since analytical solutions are not available for many other differential equations, so we might as well get good at this. We first discretize the domain $[0,1]$. We can use as many points as we want. The more points, the higher dimension we have to deal with, but the resolution will be better. We'll use only eight points, so the mesh size is $h = 1/7$ (Figure 13-7). Our continuum $[0,1]$ interval is now reduced to the eight points $(0, 1/7, 2/7, 3/7, 4/7, 5/7, 6/7, 1)$.

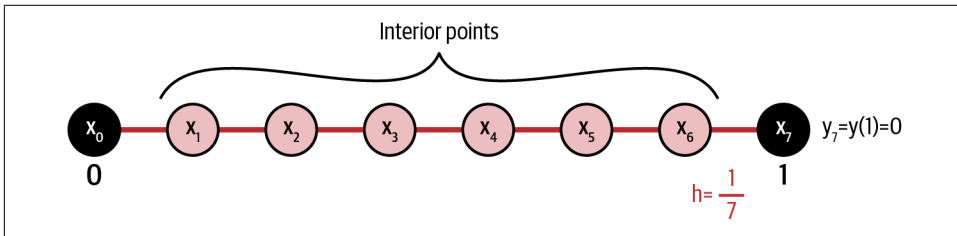


Figure 13-7. Discretizing the unit interval using eight discrete points, which is the same as seven intervals. The step size (or mesh size) is $h = 1/7$.

Next, we discretize the differential equation. We can use any finite difference scheme to discretize the second derivative. Let's choose the $O(h^2)$ central difference, so the discretized differential equation becomes:

$$\frac{y_{i+1} - 2y_i + y_{i-1}}{h^2} = 1 \text{ for } i = 1, 2, 3, 4, 5, 6$$

Note that the differential equation is only valid in the *interior* of the domain, that is why we do not include the points $i = 0$ and $i = 7$ when we write its discrete analog. We get the values at $i = 0$ and $i = 7$ from the boundary conditions: $y_0 = -1$ and $y_7 = 0$. Now, we have a system of six equations and six unknowns, $y_1, y_2, y_3, y_4, y_5, y_6$:

$$y_2 - 2y_1 - 1 = 1/49$$

$$y_3 - 2y_2 + y_1 = 1/49$$

$$y_4 - 2y_3 + y_2 = 1/49$$

$$y_5 - 2y_4 + y_3 = 1/49$$

$$y_6 - 2y_5 + y_4 = 1/49$$

$$0 - 2y_6 + y_5 = 1/49$$

So now we've moved from the continuum world to the linear algebra world:

$$\begin{pmatrix} -2 & 1 & 0 & 0 & 0 & 0 \\ 1 & -2 & 1 & 0 & 0 & 0 \\ 0 & 1 & -2 & 1 & 0 & 0 \\ 0 & 0 & 1 & -2 & 1 & 0 \\ 0 & 0 & 0 & 1 & -2 & 1 \\ 0 & 0 & 0 & 0 & 1 & -2 \end{pmatrix} \begin{pmatrix} y_1 \\ y_2 \\ y_3 \\ y_4 \\ y_5 \\ y_6 \end{pmatrix} = \begin{pmatrix} 1/49 + 1 \\ 1/49 \\ 1/49 \\ 1/49 \\ 1/49 \\ 1/49 \end{pmatrix}$$

Solving this system amounts to *inverting* that tridiagonal matrix, which is the discrete analog of our second-order derivative operator. In the continuum world we *integrate* the differential operator to recover $y(x)$, and in the discrete world we *invert* the discrete operator to recover the discrete values y_i . Keep the curse of dimensionality in mind when using more points to discretize the domain.

Obviously we must compare the discrete values y_i with their exact counterparts $y(x_i)$ to see how well our finite difference scheme with only eight discrete points performed (Figure 13-8). Figure 13-9 shows the graph of the numerical solution (using only four discrete points) against the exact analytical solution.

Numerical solution:	<i>Good or bad estimate?</i>	Correct analytical solution:
$y_0 = -1$	←→	$y(0) = -1$
y_1^*	←→	$y(\frac{1}{7}) = 0.5(\frac{1}{7})^2 + 0.5(\frac{1}{7}) - 1 =$
y_2^*	←→	$y(\frac{2}{7}) = 0.5(\frac{2}{7})^2 + 0.5(\frac{2}{7}) - 1 =$
y_3^*	←→	$y(\frac{3}{7}) = 0.5(\frac{3}{7})^2 + 0.5(\frac{3}{7}) - 1 =$
y_4^*	←→	$y(\frac{4}{7}) = 0.5(\frac{4}{7})^2 + 0.5(\frac{4}{7}) - 1 =$
y_5^*	←→	$y(\frac{5}{7}) = 0.5(\frac{5}{7})^2 + 0.5(\frac{5}{7}) - 1 =$
y_6^*	←→	$y(\frac{6}{7}) = 0.5(\frac{6}{7})^2 + 0.5(\frac{6}{7}) - 1 =$
$y_7 = 0$	←→	$y(1) = 0$

Figure 13-8. Comparing the numerical solution to the exact analytical solution at each discrete point

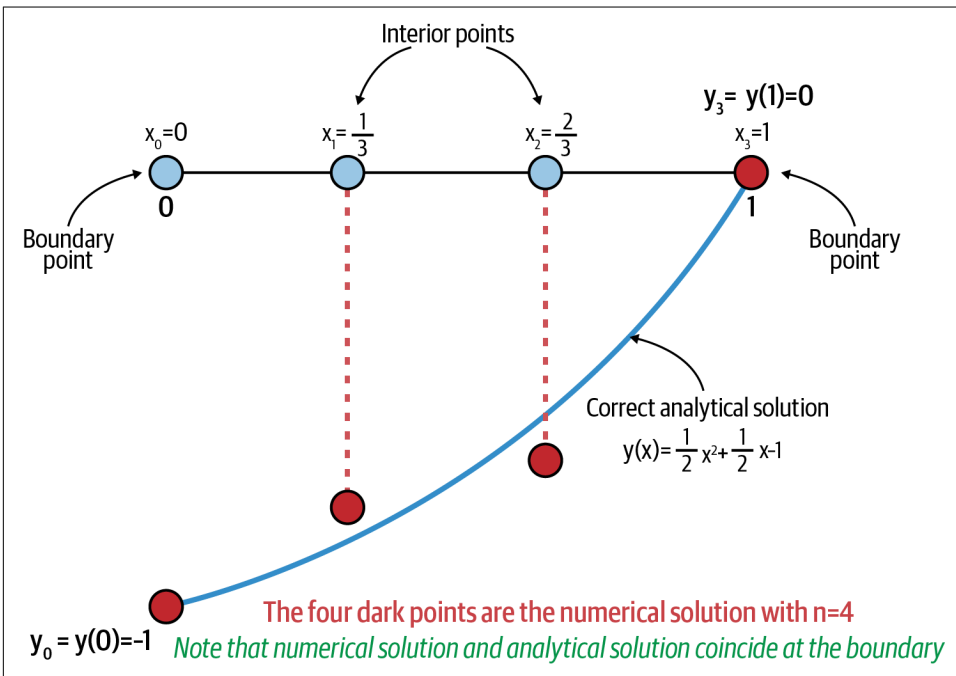


Figure 13-9. Graph of the numerical solution (using only four discrete points) against the exact analytical solution (solid line)

Now we can use finite differences to discretize *any* differential equation of any order or type, on a domain in any dimension. All we have to do is discretize the domain

and decide on finite difference schemes to approximate the derivatives at all the discrete points in the interior of the domain.

Example: Discretize the one-dimensional heat equation $u_t = \alpha u_{xx}$ in the interior of the interval $x \in (0,1)$

This is a second-order linear partial differential equation on a bounded spatial domain in one dimension. Here, $u = u(x,t)$ is a function of two variables, so our discretization scheme should address both coordinates. We can discretize only in space and keep time continuous, only in time and keep space continuous, or both in space and time. It is common to have more than one numerical route. Options are good. If we discretize in both space and time, then we end up with a system of algebraic equations. If we discretize only in space and not in time, then we end up with a system of ordinary differential equations. Since the PDE is linear, the discretized system is linear as well.

Let's write down a full discrete scheme. To discretize in space, let's use a second-order centered difference to approximate the second derivative. And to discretize in time, let's use a forward difference to approximate the first derivative:

$$\frac{u_{i,j+1} - u_{i,j}}{s} = \frac{u_{i+1,j} - 2u_{i,j} + u_{i-1,j}}{h^2} \text{ for } i = 1, 2, \dots, n \text{ and } j = 0, 1, 2, \dots$$

In such equations, $u(x,t)$ can be known at some initial time ($u(x,0) = g(x)$), and we want to know that $u(x,t)$ time evolves. In the numerical scheme, the subscript i stands for discrete space, and j stands for discrete time. Thus, the discrete analog of the initial condition is $u_{i,0} = g_i$, and we want to solve for the unknowns $u_{i,j+1}$ for $i = 1, 2, \dots, n$, and $j = 1, 2, \dots$. We can easily isolate $u_{i,j+1}$ in the previous numerical scheme:

$$u_{i,j+1} = \frac{s}{h^2} (u_{i+1,j} - 2u_{i,j} + u_{i-1,j}) + u_{i,j} \text{ for } i = 1, 2, \dots, n \text{ and } j = 0, 1, 2, \dots$$

Finally, we use this to find the values of $u_{i,1}, u_{i,2}, \dots$ (forward in time) for $i = 1, 2, \dots, n$. For example, we plug in $j=0$ to find the values of discrete u at the first time step:

$$\begin{aligned} u_{i,1} &= \frac{s}{h^2} (u_{i+1,0} - 2u_{i,0} + u_{i-1,0}) + u_{i,0} \text{ for } i = 1, 2, \dots, n \\ &= \frac{s}{h^2} (g_{i+1} - 2g_i + g_{i-1}) + g_i \text{ for } i = 1, 2, \dots, n \end{aligned}$$

Note that we know all the discrete values of g , so we now know discrete $u_{i,1}$ as well. Next, we plug in $j=1$ to find the values of discrete $u_{i,2}$ at the next time step, and so on.

Finite Elements

Finite element methods are different than finite difference methods in the sense that they operate on a *weak formulation* of the PDE as opposed to operating directly on the PDE. A weak formulation is *weighted and averaged*, so we are thinking integrals and integration by parts. We will come back to this shortly.

Before discussing the general idea of finite elements, let's observe [Figure 13-10](#) a little. This shows a finite element solution of a PDE on top of a circular domain. The discretization of the domain uses a triangular mesh, and the solution seems to be approximated by a piecewise linear function. We can use other polygonal shapes for the meshes, and we can use *smoother* functions than piecewise linear, such as piecewise quadratic or higher degree polynomials. The trade-off for more smoothness is more computation.

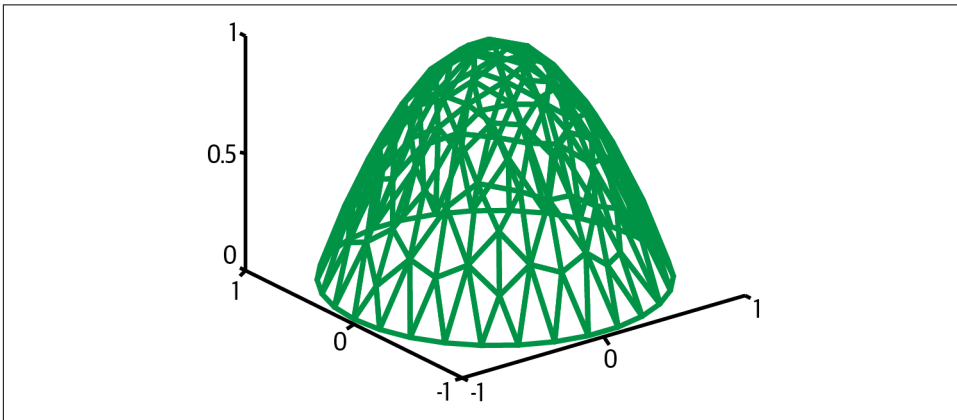


Figure 13-10. Finite elements solution over a circular domain ([image source](#))

Let's demonstrate how the finite element method gives a numerical approximation to the solution of the following PDE:

$$\begin{aligned} -\Delta u(x,y) &= f(x,y) \text{ for } (x,y) \in \Omega \subset \mathbb{R}^2 \\ u(x,y) &= 0 \text{ for } (x,y) \in \text{boundary}_\Omega \end{aligned}$$

This is a Poisson equation (appears in electrostatics). There is no time evolution. $f(x,y)$ is specified, and we are looking for an unknown function $u(x,y)$ that is zero at the entire boundary, and whose second derivatives u_{xx} and u_{yy} add up to $-f(x,y)$. This PDE is very well studied and we have formulas for its analytical solution, but we are only interested in the numerical approximation of the solution using the finite element method.

For this, we will produce an approximation of the unknown function $u(x,y)$ that lives in an *infinite* dimensional space using a known function that lives in a *finite* dimensional space. Finite dimensional spaces are spanned only by *finitely many linearly independent functions*. We get to choose these *basis functions*, so we make sure that our choice makes our computations very easy. We usually choose piecewise linear functions or piecewise polynomial functions, each *supported minimally on the mesh*. This means that the basis function is nonzero only on top of one or two adjacent elements of the mesh, and zero everywhere else. Thus, an integration involving this function on the whole domain of the PDE would reduce to an integration on only one or two elements of the mesh.

After choosing these *basis functions*, each supported on a few mesh elements, we approximate the true solution $u(x,y)$ by a linear combination of these easy and locally supported basis functions:

$$u(x,y) \approx u_1 \text{basis}_1(x,y) + u_2 \text{basis}_2(x,y) + \cdots u_n \text{basis}_n(x,y)$$

Now we must find the constants u_i of the linear combination. Therefore, we reduce our problem from solving for the unknown function $u(x,y)$ in the continuum to solving for the unknown vector of coefficients $(u_1, u_2, \dots, u_n)$. We must choose them so that the approximation $u_1 \text{element}_1(x,y) + u_2 \text{element}_2(x,y) + \cdots u_n \text{element}_n(x,y)$ satisfies the PDE, in some sense. We have n unknowns, so we must write n equations and solve a system of n equations, n unknowns. We get these from the PDE, or *its weak formulation*. To get a weak formulation of the PDE, we multiply it by a function $v(x,y)$, integrate over our domain, then use *integration by parts* to get rid of higher-order derivatives. Remember that the fewer derivatives we have, the closer to the unknown function we get. Let's do this step by step:

The original PDE is:

$$\begin{aligned} -\Delta u(x,y) &= f(x,y) \text{ for } (x,y) \in \Omega \subset \mathbb{R}^2 \\ u(x,y) &= 0 \text{ for } (x,y) \in \text{boundary}_\Omega \end{aligned}$$

Multiply the PDE by a function $v(x,y)$ and integrate over the entire domain. This is a *weak formulation* of the PDE since it makes it satisfied in an integral form as opposed to a point-by-point form:

$$-\int_\Omega \Delta u(x,y) v(x,y) dx dy = \int_\Omega f(x,y) v(x,y) dx dy$$

Note the operator $\Delta = \nabla \cdot \nabla$, the dot product of two derivative operators. Integration by parts helps us get rid of one of the derivatives by moving it over to the other function inside the integral. This doesn't come for free. In the process of doing this, it picks up a negative sign and another integral term that operates on the boundary of the domain. This new integral on the boundary integrates the product of two anti-derivatives. The boundary term needs the outward unit normal vector to the boundary $\vec{n}$:

$$\int_{\Omega} \nabla u(x,y) \cdot \nabla v(x,y) dx dy - \int_{\text{boundary}_{\Omega}} v(x,y) \nabla u(x,y) \cdot \vec{n} ds = \int_{\Omega} f(x,y) v(x,y) dx dy$$

We can choose $v(x,y) = 0$ on the boundary and that makes the whole boundary term disappear:

$$\int_{\Omega} \nabla u(x,y) \cdot \nabla v(x,y) dx dy = \int_{\Omega} f(x,y) v(x,y) dx dy$$

Now we replace $u(x,y)$ with its finite dimensional approximation:

$$\begin{aligned} & \int_{\Omega} \nabla (u_1 \text{basis}_1(x,y) + u_2 \text{basis}_2(x,y) + \dots + u_n \text{basis}_n(x,y)) \cdot \nabla v(x,y) dx dy \\ &= \int_{\Omega} f(x,y) v(x,y) dx dy \end{aligned}$$

which is equivalent to:

$$\begin{aligned} & \int_{\Omega} (u_1 \nabla \text{basis}_1(x,y) + u_2 \nabla \text{basis}_2(x,y) + \dots + u_n \nabla \text{basis}_n(x,y)) \cdot \nabla v(x,y) dx dy \\ &= \int_{\Omega} f(x,y) v(x,y) dx dy \end{aligned}$$

This is it: We can choose n different functions for $v(x,y)$ to get n different equations in n unknowns (the u_i 's are the unknowns). A common theme is that every time we get to pick, we pick things that do not complicate our computation life. The easiest choices for $v(x,y)$ are the n basis functions that we already have, since these produce many cancellations when integrated against each other (*orthogonality*), and when integrated against themselves produce the number 1 (*normality*). The basis functions that we originally choose form an *orthonormal* set of functions. All for the business of making our life easier. Therefore, the n equations are:

$$\begin{aligned}
& \int_{\Omega} (u_1 \nabla \text{basis}_1(x,y) + u_2 \nabla \text{basis}_2(x,y) + \cdots u_n \nabla \text{basis}_n(x,y)) \cdot \nabla \text{basis}_1(x,y) dx dy = \\
& \int_{\Omega} f(x,y) \text{basis}_1(x,y) dx dy \\
& \int_{\Omega} (u_1 \nabla \text{basis}_1(x,y) + u_2 \nabla \text{basis}_2(x,y) + \cdots u_n \nabla \text{basis}_n(x,y)) \cdot \nabla \text{basis}_2(x,y) dx dy = \\
& \int_{\Omega} f(x,y) \text{basis}_2(x,y) dx dy \\
& \dots \\
& \int_{\Omega} (u_1 \nabla \text{basis}_1(x,y) + u_2 \nabla \text{basis}_2(x,y) + \cdots u_n \nabla \text{basis}_n(x,y)) \cdot \nabla \text{basis}_n(x,y) dx dy = \\
& \int_{\Omega} f(x,y) \text{basis}_n(x,y) dx dy
\end{aligned}$$

Finally we solve the system of n equations, n unknowns, which we set in a linear algebra form (where $b_i = \text{basis}_i$):

$$\begin{pmatrix}
\int_{\Omega} \nabla b_1(x,y) \cdot \nabla b_1(x,y) dx dy & \int_{\Omega} \nabla b_2(x,y) \cdot \nabla b_1(x,y) dx dy & \cdots & \int_{\Omega} \nabla b_n(x,y) \cdot \nabla b_1(x,y) dx dy \\
\int_{\Omega} \nabla b_1(x,y) \cdot \nabla b_2(x,y) dx dy & \int_{\Omega} \nabla b_2(x,y) \cdot \nabla b_2(x,y) dx dy & \cdots & \int_{\Omega} \nabla b_n(x,y) \cdot \nabla b_2(x,y) dx dy \\
\vdots & \vdots & \cdots & \vdots \\
\int_{\Omega} \nabla b_1(x,y) \cdot \nabla b_n(x,y) dx dy & \int_{\Omega} \nabla b_2(x,y) \cdot \nabla b_n(x,y) dx dy & \cdots & \int_{\Omega} \nabla b_n(x,y) \cdot \nabla b_n(x,y) dx dy
\end{pmatrix}
\begin{pmatrix}
u_1 \\
u_2 \\
\vdots \\
u_n
\end{pmatrix}
=
\begin{pmatrix}
\int_{\Omega} f(x,y) b_1(x,y) dx dy \\
\int_{\Omega} f(x,y) b_2(x,y) dx dy \\
\vdots \\
\int_{\Omega} f(x,y) b_n(x,y) dx dy
\end{pmatrix}$$

Recall that we know the function $f(x,y)$, all the basis functions $b_i = \text{basis}_i$, and the domain Ω , so all we have to do is solve the system of equations. This system is *sparse* because most of these integrals are zero. We chose the basis functions with small support for this reason. We never want to solve a *dense* system of equations.

Of course, we have many questions and a rich literature on finite elements that deals with them:

Do all PDEs have a weak formulation that allow us to do things like this?

Yes, because we can always multiply PDEs with v functions and integrate by parts, but some PDEs have better structures to carry out simplifying computations than others.

What about the energy formulations or variational principles of PDEs, are they related?

Yes, they are related. Look up the Ritz method. We hinted at this in [Chapter 10](#), when we related minimizing energy functionals to solving PDEs. One thing to

keep in mind here is that most PDEs have a weak formulation, but not all of them have an energy minimization formulation. One of the reasons I got my Ph.D. was that I discovered an energy formulation for the PDE that I was working with. It was by complete chance. All I did was one lucky *weak formulation*, followed by an integration by parts. Just like we did in this section. Trial and error are underestimated in this life.

What about Sobolov spaces, why do we study them in advanced courses on PDEs?

Because we need to set our functions u , v , and the basis functions in the appropriate *function spaces* that tell us that all the computations and approximations that we are using are valid. For example, we don't want the involved integrals that contain our functions and their derivatives to blow up.

Can we use nonuniform meshes to adjust for the more detailed part of the domain, such as the one in [Figure 13-11](#)?

Yes, nothing in our discussion relies strictly on a uniform mesh.

How many basis functions do we need?

As many as our mesh elements.

Under what conditions does the approximate solution converge to the true solution?

Welcome to finite element analysis.

How is this used in applications?

All the time. It started with mechanics and structural designs: loads, stresses, and strains; but now the finite element method is used to *numerically* solve all kinds of PDEs whose spatial domains have complex geometries.

What could go wrong?

As always, the curse of dimensionality. We need more mesh elements for higher resolution, so the system of equations that we end up having to solve grows exponentially with the number of mesh elements. No bueno. Ideally, we want a mesh that is not detailed in places where it doesn't need to be, and more detailed in more interesting parts of the domain.

What else could complicate matters?

For PDEs whose *domain evolves in time*, we need meshes that evolve in time accordingly.

Can AI help learn an appropriate mesh given a certain geometry and PDE?

Yes, we will see this soon in this chapter.

Before moving on, we note that the finite element method is a finite dimensional mesh dependent method that approximates the solution of the PDE. Later in this chapter we will learn about meshless neural network methods.

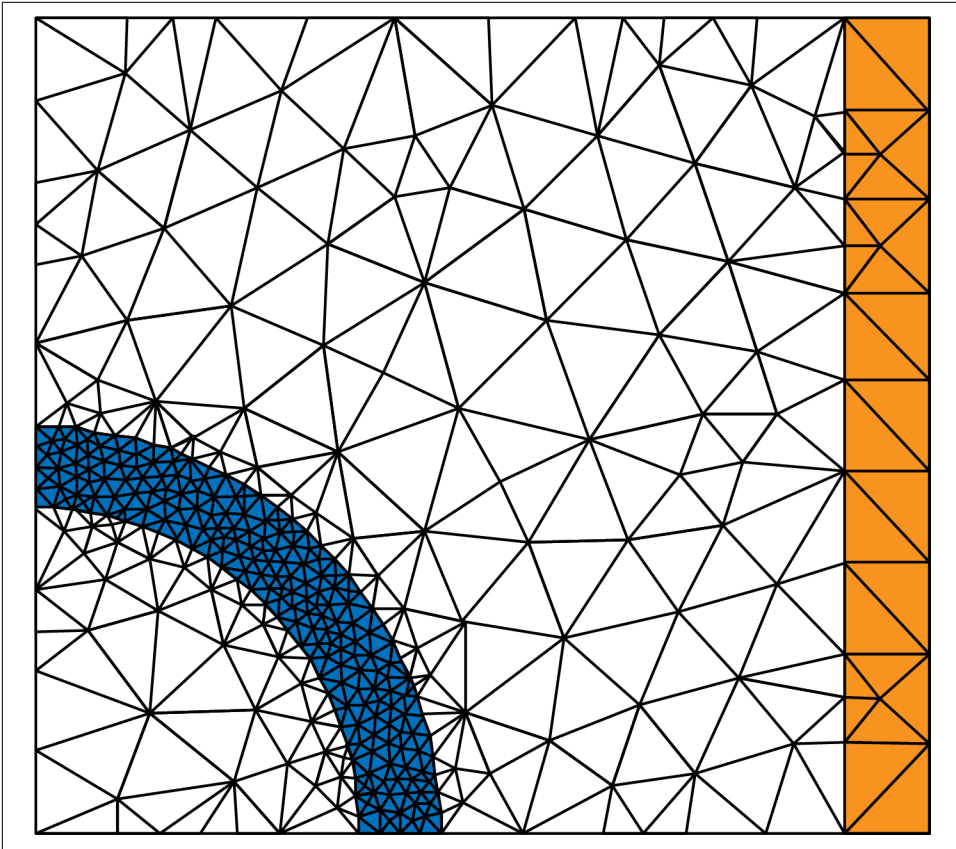


Figure 13-11. A two-dimensional domain with a nonuniform triangular mesh (*image source*)

Variational or Energy Methods

Some PDEs are very special in the sense that their solution minimizes an energy functional. We say that such a PDE possesses a *variational principle*. The Poisson equation, the one we just solved using finite elements, is one of these lucky PDEs. When a PDE possesses a variational principle, it opens for us another route to understanding its solution by studying the energy functional that it happens to minimize.

Let's write the Poisson equation and the energy functional that its solution minimizes, without going through the details of why this is the case:

$$\begin{aligned}\Delta u(x,y) &= f(x,y) \text{ for } (x,y) \in \Omega \subset \mathbb{R}^2 \\ u(x,y) &= 0 \text{ for } (x,y) \in \text{boundary}_\Omega\end{aligned}$$

$$E(u(x,y)) = \int_{\Omega} |\nabla u(x,y)|^2 + 2f(x,y) dx dy$$

Now we can exploit this new knowledge to numerically approximate the solution of the PDE: look for an approximate minimizing scheme of the energy functional. Similar to the finite elements method, we project our infinite dimensional solution $u(x,y)$ onto a finite dimensional space, where we get to choose the basis elements:

$$u(x,y) \approx u_1 \text{basis}_1(x,y) + u_2 \text{basis}_2(x,y) + \cdots u_n \text{basis}_n(x,y)$$

and we must again solve for the numbers $(u_1, u_2, \dots, u_n)$. To do this, we plug the approximate $u(x,y)$ into the formula of the energy functional. Since we know all the basis elements, this is now a *function* of $(u_1, u_2, \dots, u_n)$, which we can minimize using standard calculus methods. Done!

This method is fairly general and introduces us smoothly into the calculus of variations, which is about finding optima of functionals instead of functions, like in normal calculus.

Monte Carlo Methods

We are now used to switching our brain to probabilistic thinking to solve deterministic problems. We did that with the stochastic gradient descent for minimizing loss functions, multiplication of large matrices, randomized singular value decomposition of a large matrix, and random walks on graphs to identify communities, rank web pages, and other purposes. The most famous introductory examples to Monte Carlo methods are:

- Estimating π by generating many random points $(x_{\text{random}}, y_{\text{random}})$ in a unit square, and finding the proportion that lies inside the inscribed quarter circle of radius 1: $x_{\text{random}}^2 + y_{\text{random}}^2 \leq 1$. Now we can estimate the probability of landing in the quarter circle:

$$\begin{aligned} & \text{Prob}(\text{point inside the quarter circle}) \\ &= \frac{\text{area of quarter circle of radius 1}}{\text{area of unit square}} \\ &= \frac{\pi}{4} \\ &\approx \frac{\text{number of times point is inside the quarter circle}}{\text{total number of points generated}} \end{aligned}$$

- Estimating the integral $\int_a^b f(x)dx$ of a nonnegative and continuous function $f(x)$ over an interval $[a,b]$ by generating random points (x_{random}, y_{random}) , where $a \leq x_{random} \leq b$ and $0 \leq y_{random} \leq \max(f)$. The value of the integral is the area under the graph of f . We can estimate it by finding the proportion of times that the random point lies under the graph of $f(x)$, or $y_{random} \leq f(x_{random})$:

$$\begin{aligned} Prob(\text{point under the graph of } f) &= \frac{\text{area under the graph of } f}{\text{total area of the rectangle}} \\ &= \frac{\int_a^b f(x)dx}{(b-a) \times \max(f)} \\ &\approx \frac{\text{number of times point is under the graph of } f}{\text{total number of points generated}} \end{aligned}$$

Such stochastic methods to solve deterministic problems are called *Monte Carlo* methods because they involve repetitive games of chance, and counting proportions of certain outcomes, just like gambling in Monte Carlo casinos in Monaco. They could have been called *Las Vegas Strip* methods as well. This is analogous to *randomized controlled trials* to answer deterministic questions, for example, to assess the effect of a certain drug on a given population. Another way to answer the same question would be a completely deterministic observational study, where one controls for all suspected confounding variables, and assesses the effects of the drug intervention.

Now suppose we have a deterministic PDE and we want to find its solution using randomized numerical trials (Monte Carlo). To illustrate how this works, let's use a simple PDE:

$$\begin{aligned} \Delta u(x,y) &= 0 \text{ for } (x,y) \text{ in the unit square } \subset \mathbb{R}^2, \\ u(x,y) &= g(x,y) \text{ for } (x,y) \in \text{boundary}_{\text{square}} \end{aligned}$$

Let's first discretize the domain using a uniform grid, then write a finite difference scheme for the PDE at the interior grid points and for the boundary conditions:

$$\begin{aligned} \frac{u_{i+1,j} - 2u_{i,j} + u_{i-1,j}}{h^2} + \frac{u_{i,j+1} - 2u_{i,j} + u_{i,j-1}}{h^2} &= 0 \\ \text{when } (i,j) &\text{ corresponds to an interior grid point,} \\ u_{i',j'} &= g_{i',j'} \text{ when } (i',j') \text{ corresponds to a boundary point} \end{aligned}$$

The goal is to use the numerical scheme to find $u_{i,j}$ for each interior point of the grid. This will be the numerical estimate of the true solution $u(x,y)$ at that particular interior point. Let's solve for $u_{i,j}$:

$$u_{i,j} = \frac{1}{4}u_{i+1,j} + \frac{1}{4}u_{i-1,j} + \frac{1}{4}u_{i,j+1} + \frac{1}{4}u_{i,j-1}$$

when (i,j) corresponds to an interior grid point,

$u_{i',j'} = g_{i',j'}$ when (i',j') corresponds to a boundary point

This is how we interpret the equations for a random walk setting: if we are at the boundary, then we know the solution, it is $u_{i',j'} = g_{i',j'}$. So a random walker who is following the guidance of the PDE scheme to structure their walk would collect their reward $g_{i',j'}$ at the boundary point. Moreover, the solution $u_{i,j}$ at an interior grid point (i,j) is the unweighted average of the solution at the four surrounding grid points. So if a random walker starts at an interior grid point (i,j) , we will give them a 0.25 probability of wandering off to any of their four neighboring points, then to their neighboring point, until the walker hits a boundary grid point (i',j') , where they collect their reward $g_{i',j'}$. That would be only one exploration of the PDE scheme, in a way getting us a tiny piece of information on which boundary point contributed to the solution $u_{i,j}$. If we repeat this process many times, say a thousand times, all starting from the same grid point (i,j) where we want to find the numerical solution, then we can count the proportion of times the random walker ended up at each of the boundary points:

$$prob(\text{ending up at point}(i',j')) \approx \frac{\text{number of times random walker ended up at point}(i',j')}{\text{total number of random walks starting at } (i,j)}$$

This will give us an estimate of the expected rewards from all the boundary points, which is exactly the numerical solution that we are looking for: how does each boundary value play a role in the solution at the interior point? So the numerical solution of the PDE is:

$$u_{i,j} = \sum (i',j') prob(\text{ending up at point}(i',j')) g_{i',j'}$$

This is such a neat way of getting a numerical solution that does not involve solving a linear system of equations (which could be very large and undesirable). It is also excellent when we care about finding the solution at a few points only, as opposed to finding the solution at the entire grid.

Of course, for each PDE we have to devise the correct numerical scheme along with the transition probabilities of the random walker. For example, if the PDE had coefficients multiplied with the second derivatives, then the random walker would not wander off to each of the four neighboring points with equal probability of 0.25. The coefficients would introduce weights for each neighbor, so we adjust the transition probabilities to each.

In terms of theory, we have to prove that a walker does eventually hit the boundary, and that the numerical solution obtained this way does converge to the true analytical solution of the PDE. We also have to obtain analytical estimates on how long until a random walk stops (on average), how fast the numerical solution converges, and how a numerical solution obtained this way fares against the ones obtained from finite differences or finite elements in terms of accuracy, computation cost, and speed of convergence.

With Monte Carlo methods, sometimes we start the other way around. We devise a simulation involving the different processes and transition rates mimicking some physical phenomena (such as interacting particles of a system), then we average that and transition to writing PDEs involving the descriptors of the system at hand. This is the exact opposite of starting with a PDE, then devising a granular scale Monte Carlo simulation to solve it. We discuss this next.

Some Statistical Mechanics: The Wonderful Master Equation

One of my favorite PDEs is *the master equation* from statistical mechanics, because it is one of the few PDEs out there that is able to get us from characterizing a system at an atomistic or molecular scale (system of particles), where the description is probabilistic, to the same system at a macro scale, where the description is deterministic. It is only logical to expect that the underlying atomistic processes and transitions give rise to an observed behavior at the macro scale. The very wise person who introduced me to statistical mechanics told me: “Isn’t our whole lived experience the result of the collective behavior of some massive underlying chemical reaction?”

The transition from the master equation for atomistic probabilities to deterministic PDEs for observed quantities is neat and doesn’t feel like we cheated or made fuzzy assumptions, or that we have two completely disconnected models, one at a macro scale and another at an atomistic scale, that have nothing to do with each other.

The master equation tracks the evolution of the probability of a statistical system (some particles) being in some state at a certain time. We calculate the rate of change of the probability of the state of the system by subtracting the losses from the gains and accounting for transition rates between different states:

$$\frac{\partial P(h,t)}{\partial t} = \sum_{h'} P(h',t) T(h' \rightarrow h) - P(h,t) T(h \rightarrow h'),$$

$$: = LP$$

where $T(h \rightarrow h')$ and $T(h' \rightarrow h)$ are the transition rates from state h to h' , and vice versa. We calculate these transition rates using the underlying physical assumptions

or observations on the system, for example, evaporation and condensation rates of atoms, diffusion rates, etc.

Now we can employ the master equation to write partial differential equations for the deterministic descriptors of the system by computing their expectations. Expectations transition us from probabilistic quantities to deterministic ones. Here's how we calculate it:

$$\langle f \rangle = \sum_h f P(h, t) = \sum_h f \frac{e^{-H(h)/KT}}{Z}$$

where $H(h)$ is the total energy, and Z is the partition function. The expression $\frac{e^{-H(h)/KT}}{Z}$ is very common in statistical mechanics and expresses the intuitive idea that states with high energy are exponentially less likely to occur, meaning that systems prefer low energy and evolve toward the states that lower the total energy.



Expectation Versus Averaging over N Repetitions of a Monte Carlo Simulation

This expectation $\langle f \rangle$ is related to Monte Carlo simulations. It is equivalent to the limit as $N \rightarrow \infty$ of the mean $\bar{f}$ of f after N repetitions of the Monte Carlo simulations.

We can now compute the rate of change of the expectation of the quantity of interest, say h_i representing the height of a crystal profile (made up of atoms) at a certain site i , using the master equation:

$$\frac{d\langle h_i \rangle}{dt} = \sum_h h_i \frac{\partial P}{\partial t} = \sum_h h_i L P$$

If the system is closed, meaning if we are able to express the righthand side in terms of h and its derivatives with respect to space and time, then we obtain an equation of motion for the expected height profile. If the system is not closed, then we have to make an approximation to close the system. We better make a physically plausible approximation, such as the system is near equilibrium, otherwise our efforts will be useless.

The final step is coarse-graining the resulting discrete equations of motion to obtain a continuum PDE model describing the crystal profile. This step moves us from a finite difference scheme to a continuum PDE, which is the reverse of the discretizing process we learned with finite differences. Using this process, the resulting PDE emerges directly from atomistic processes. Such a PDE usually looks like:

$$h_t(\vec{x}, t) = F(h(\vec{x}, t), t; \vec{\omega})$$

where $\vec{\omega}$ is the set of the system's physical parameters.

Soon in this chapter, we will learn about using graph neural networks to simulate natural phenomena at the macroscopic scale directly from particle systems. This bypasses writing down PDEs as we did in this section. The inputs to the network will be the particles along with their interactions and rates of interaction, and the output will be the time evolution (a video, or a time sequence of graphs) of the system as a whole.

Solutions as Expectations of Underlying Random Processes

For some types of PDEs, a neat way to find solutions is to formulate them as expectations of some underlying random processes: we simulate random paths of an appropriate stochastic process, then compute the expectation. This allows us to evaluate solutions at any given space-time locations.

To learn how to do this, we need to study the *Feynman-Kac formula* and *Itô's calculus* (helps us find derivatives of functions of time-dependent random variables). These tie PDEs and probability nicely together.

The Feynman-Kac formula (which we will not write) offers a practical way to solve some PDEs that have been haunted by the curse of dimensionality. For example, in quantitative finance, we can use the Feynman-Kac formula to efficiently calculate solutions to the Black-Scholes equation to price options on stocks. In quantum chemistry, we can use it to solve the Schrödinger equation.

Transforming the PDE

The idea here is simple: maybe the PDE in a transformed space is easier to solve (analytically or numerically) than in the space it currently lives. So we transform it in some way and wish for the best.

Fourier Transform

The Fourier transform is an integral transform from x space to frequency ξ space:

$$F.T(f(x)) = \hat{f}(\xi) = \frac{1}{\sqrt{2\pi}} \int_{-\infty}^{\infty} e^{-i\xi x} f(x) dx$$

The inverse Fourier transform undoes the Fourier transform and brings us back from frequency ξ space to x space:

$$F.T^{-1}(\widehat{f}(\xi)) = f(x) = \frac{1}{\sqrt{2\pi}} \int_{-\infty}^{\infty} e^{i\xi x} \widehat{f}(\xi) d\xi$$

There are tables with the Fourier transforms of many functions for our convenience. When these are not available, we resort to numerical methods. Because the Fourier transform and its inverse are important for many applications, such as frequency analysis, signal modulation, and filtering, there are algorithms that have been specifically developed for its fast computation.

The following are some powerful things that we need to know about the Fourier transform:

- It strips a function down to its frequency components. The Fourier transform of a function tells us how much of each frequency a function has. The frequency spectrum of a function $f(x)$ is the absolute value of its Fourier transform: $|F(\xi)|$.
- It has an inverse transform that allows us to move back and forth between x space and frequency space ξ .
- It changes the convolution of two functions in x space to multiplication of functions in frequency ξ space: $F.T.(f * g(x)) = F.T.(f(x)) \times F.T.(g(x)) = \widehat{f}(\xi) \widehat{g}(\xi)$. This is helpful when trying to find analytical solutions for PDEs. Solving a PDE in x space boils down to solving an algebraic equation or an easier differential equation in ξ space, then using the inverse Fourier transform to get back to x space. Many times we are inverting the product of two Fourier transforms, so the solution ends up being a convolution in x space. If you have encountered *Green's functions* for analytical solutions before, this is one way to arrive at them.
- It changes differentiation in x to multiplication by $i\xi$, so:

$$F.T.(u_x(x)) = i\xi F.T.(u(x)) = i\xi \widehat{u}(\xi)$$

and

$$F.T.(u_{xx}(x)) = -\xi^2 F.T.(u(x)) = -\xi^2 \widehat{u}(\xi)$$

Getting rid of derivatives is huge. It means that differential equations in original space become algebraic equations in Fourier space.

- It is a linear transformation, so we can apply it separately to each term in a PDE. It enables us to solve linear PDEs with constant coefficients seamlessly. For linear PDEs with nonconstant coefficients (where the parameters depend on space), we can still use the Fourier transform if we are willing to get messy with series expansions of these coefficients. The moment we write series, we have to investigate their convergence.

- We use it to prove the universal approximation theorem for neural networks (Hornik et al. 1989).
- We can use it to speed up convolutional neural networks (Mathieu et al. 2013).
- It turns out that representing PDEs in Fourier space is convenient if we want to train neural networks to learn PDE solutions.

There are also some *not so* convenient things:

- Many functions have complex valued Fourier transforms. We just learn complex analysis and live with that.
- Not all functions have a Fourier transform. The involved integral operates on an infinite domain, so if there is no function inside the integral that compensates with a rapid decay to zero, the integral blows up (rendering the Fourier transform useless). The kernel of the Fourier transform is $e^{-i\xi x} = \cos(\xi x) - i \sin(\xi x)$. This oscillates with frequency ξ and never decays to zero.
- Even for functions whose inverse Fourier transform (which helps us find analytic solutions for PDEs) exists, many times we do not know its formula. In these cases, we would not be able to write an explicit analytic solution using this method. This is a common problem for many analytic methods.
- Fourier transforms have the **Heisenberg uncertainty principle**.

The study of uncertainty principles began with Werner Heisenberg's argument that it is impossible to simultaneously determine a free particle's position and momentum to arbitrary precision. In quantum mechanics, the wave function of position is the Fourier transform of the wave function of momentum. The most popular use of Fourier uncertainty principles is as a description of the natural trade-off between the stability and measurability of a system, particularly quantum mechanical systems. Imagining that $f(x)$ is the probability that a particle's position is x , and $f(\xi)$ is the probability that its momentum is ξ , Heisenberg's inequality gives a lower bound on how spread out these two probability distributions must be. The physical assumption is that position and momentum are related by Fourier transform: $\|f\|_{L^2} \leq 4\pi \cdot \|(x - x_0)f\|_{L^2} \cdot \|(\xi - \xi_0)f\|_{L^2}$. Qualitatively, this means a narrow function has a wide Fourier transform, and a wide function has a narrow Fourier transform. In either domain, a wider function means there is literally a wide distribution of data, so there always exists uncertainty in one domain.



Fourier Transform Versus Fourier Series

We should not confuse the Fourier transform with the Fourier sine and cosine series. The function $\sin x$ does not have a Fourier transform, but its Fourier sine series is itself.

Laplace Transform

The Laplace transform allows us to transform a wider class of functions than the Fourier transform, because its kernel e^{-st} decays to zero exponentially fast (there is no complex valued i in the exponent to mess things up). The Laplace transform operates on functions defined on $[0, \infty)$, so in PDEs, we use it to transform the time variable, or other variables if they range from $[0, \infty)$. So instead of solving the PDE directly in the time domain, we Laplace transform it, solve it in s domain, then inverse Laplace transform it back to the time domain.

The formula for the Laplace transform is:

$$L.T.(f(t)) = \hat{f}(s) = \int_0^{\infty} e^{-st} f(t) dt$$

And the formula for the inverse Laplace transform is:

$$L.T.^{-1}(\hat{f}(s)) = f(t) = \frac{1}{2\pi i} \int_{c-i\infty}^{c+i\infty} e^{st} \hat{f}(s) ds$$

Just like in the case of the Fourier transform, there are tables with the Laplace transforms of many functions computed for our convenience.

We care about how the Laplace transform acts on the (time) derivatives involved in a PDE. It is better to get rid of them, because how else will a PDE get us closer to its solution? It does:

- $L.T(u_t(x, t)) = s\hat{u}(x, s) - u(x, 0)$
- $L.T(u_{tt}(x, t)) = s^2\hat{u}(x, s) - su(x, 0) - u_t(x, 0)$

Note that we usually know the initial conditions for a PDE $u(x, 0)$ and $u_t(x, 0)$, so the transformations do get rid of the derivatives with respect to time.

We also care about the convolution to multiplication property so that we can transfer back algebraic expressions in s to PDE solutions in t space using the inverse Laplace transform. Careful here that this is a finite convolution, from 0 to t , as opposed to from $-\infty$ to ∞ like in the Fourier transform case:

$$L.T.((f * g)(t)) = \hat{f}(s)\hat{g}(s) \text{ where } (f * g)(t) = \int_0^t f(\tau)g(t - \tau)d\tau = \int_0^t f(t - \tau)g(\tau)d\tau$$

Similar to the Fourier transform, the Laplace transform is a linear operator, so it is best used with linear PDEs.



Reducing PDEs to Algebraic Equations or to ODEs

The Fourier transform, the Laplace transform, and some other transforms, such as the Hankel and Mellin transforms, are able to get rid of derivatives of the PDE in certain variables (time, space, etc.). We are left with an algebraic equation if the transform acts on all the variables involved in the PDE, or an ordinary differential equation (ODE) if the transform acts on all the variables except one. The hope here is that the algebraic equations or the ODE are easier to solve than the original PDE, and that we can utilize known methods from algebra, numerics, and ODEs to solve the new equations in transformation variables. We will see a simple example of how this works in the next section on solution operators.

Solution Operators

We now work through two simple but informative examples that illustrate the transform methods, while at the same time showcasing the ideas behind solution operators for PDEs. These are fairly general, and more importantly lay the groundwork for leveraging neural networks to solve PDEs. In addition, both examples have explicit analytic solutions, so we can use them to test approximation or iterative methods for solving PDEs (including neural network methods).

The first example uses a one-dimensional heat equation with constant coefficients on an infinite domain (this is time dependent), and the second example uses a two-dimensional Poisson equation with constant coefficients on a bounded domain with a simple geometry (this is not time dependent; the solution is static in time).

Example Using the Heat Equation

The heat equation on an infinite one-dimensional rod looks like:

$$\begin{aligned}u_t(x,t) &= \alpha u_{xx}(x,t) \text{ for } x \in \mathbb{R}, t \in (0,\infty) \\ u(x,0) &= u_0(x) \text{ for } x \in \mathbb{R}\end{aligned}$$

Since this PDE is defined on an infinite domain in x , we must specify the far field conditions (there is no boundary, so we must specify what we think the solution function $u(x,t)$ looks like when $x \rightarrow \infty$ and $x \rightarrow -\infty$). Let's assume that these limits are zero.

For simplicity, let's assume that the parameter α is constant so we can apply the Fourier transform. When applying this transform (with respect to x) to the PDE $u_t(x,t) = \alpha u_{xx}(x,t)$ and the initial condition, we manage to get rid of the derivatives

in x and simplify the PDE into an ordinary differential equation with only one derivative in time:

$$\begin{aligned}\hat{u}_t(\xi, t) &= -\alpha \xi^2 \hat{u}(\xi, t) \text{ for } \xi \in \mathbb{R}, t \in (0, \infty) \\ \hat{u}(\xi, 0) &= \hat{u}_0(\xi) \text{ for } \xi \in \mathbb{R}\end{aligned}$$

We can now easily solve this using a method from ordinary differential equations called separation of variables (let's not bother with the details), obtaining the solution in Fourier space:

$$\hat{u}(\xi, t) = e^{-\alpha \xi^2 t} \hat{u}_0(\xi)$$

We need the solution in x space, not in Fourier space, so we take the inverse Fourier transform of the above expression, and use the knowledge that multiplications become convolutions when transforming between x space and Fourier space. Therefore, the solution in x space is:

$$\begin{aligned}u(x, t) &= F.T.^{-1} \left(e^{-\alpha \xi^2 t} \hat{u}_0(\xi) \right) = F.T.^{-1} \left(e^{-\alpha \xi^2 t} \right) * u_0(x) = \frac{1}{\sqrt{4\pi\alpha t}} e^{-\frac{x^2}{4\alpha t}} * u_0(x) \\ &= \int_{-\infty}^{\infty} \frac{1}{\sqrt{4\pi\alpha t}} e^{-\frac{(s-x)^2}{4\alpha t}} u_0(s) ds = \int_{-\infty}^{\infty} \text{kernel}(s, x; t, \alpha) u_0(s) ds\end{aligned}$$

The punch line from the calculation is this:

The solution $u(x, t)$ of the PDE is the integral of some kernel function $k(s, x; t, \alpha)$ against the initial state of the solution $u_0(s)$.

Moreover, the solution operator of the PDE maps the given input data, which in this case are the parameter α and initial state $u_0(x)$, to the output, which is the solution we are seeking, $u(x, t)$, by integrating the initial state against some kernel function that depends on the parameter of the PDE (along with its dependence on space and time).

Knowing the formula of this kernel, or using a neural network to approximate it, unlocks the solution of a given PDE. We then say that the neural network *learned the solution operator* of the PDE.

In our simple example, we leveraged linearity and constant coefficients to incorporate Fourier transform methods and convolution when reverting back to real space, and had the luxury of working out an explicit analytic formula of the kernel of the

integral, namely, $k(s, x; t, \alpha) = \frac{1}{\sqrt{4\pi\alpha t}} e^{-\frac{(s-x)^2}{4\alpha t}}$, so there is no need for approximations.

On a nice side note, this kernel comes from a time-dependent Gaussian function

$Gaussian(x; t; \alpha) = \frac{1}{\sqrt{4\pi\alpha t}} e^{-\frac{x^2}{4\alpha t}}$ that spreads out as time evolves. The effect of convolving this with the initial state of the solution has a smoothing effect that spreads out and smooths any initial oscillations and spikes. We observe this smoothing with any diffusion process that we can visualize, such as diffusion of smoke in the air, or diffusion of a dye in a liquid, where the substance spreads out smoothly until we obtain one homogeneous-looking medium.

Example Using the Poisson Equation

A Poisson equation on a bounded domain looks like:

$$\begin{aligned} -\nabla \cdot (a(\vec{x}) \nabla u(\vec{x})) &= f(\vec{x}) \text{ for } \vec{x} \in D \\ u(\vec{x}) &= 0 \text{ for } \vec{x} \text{ at the boundary of } D \end{aligned}$$

When $a(\vec{x})$ is constant and the domain is two-dimensional, this becomes:

$$\begin{aligned} -a\Delta u(x,y) &= f(x,y) \text{ for } (x,y) \in D \subset \mathbb{R}^2 \\ u(x,y) &= 0 \text{ for } (x,y) \text{ at the boundary of } D \end{aligned}$$

where $\Delta u(x,y) = u_{xx}(x,y) + u_{yy}(x,y)$. We can employ the Fourier transform in x and y like we did for the heat equation (linear equation with constant coefficients), but let's demonstrate the Green's function method instead. We can think of the righthand side of the PDE as an aggregation in the continuum of impulses of intensity $f(x,y)$ at locations (x,y) . We need the *Dirac delta measure* $\delta_{(x,y)}(s,p)$ to express the concept of an impulse mathematically. This is zero everywhere on the domain except at the point (x,y) , where it is infinite, and its total measure on the domain is normalized to 1. The rationale here is that we can solve the PDE with the righthand side consisting of only an impulse at a certain location, then aggregate the original solution from these. Presumably, solving the PDE with only an impulse as the righthand side is easier than solving it with some given function as the righthand side, so we will build up the solution $u(x,y)$ from the solution $G(x,y;s,p)$ of the impulse PDE. More importantly, using the Green's function allows us to get an integral representation of the solution of the input data against a kernel (which is the Green's function). The PDE with an impulse righthand side is:

$$\begin{aligned} -a\Delta G(s,p;x,y) &= \delta_{(x,y)}(s,p) \text{ for } (s,p) \in D \subset \mathbb{R}^2 \\ G(s,p;x,y) &= 0 \text{ for } (s,p) \text{ at the boundary of } D \end{aligned}$$

Let's now write:

$$f(x,y) = \int_D f(s,p) \delta_{(x,y)}(s,p) ds dp$$

and the PDE as:

$$\begin{aligned} -a\Delta u(x,y) &= \int_D f(s,p) \delta_{(x,y)}(s,p) ds dp \text{ for } (x,y) \in D \subset \mathbb{R}^2 \\ u(x,y) &= 0 \text{ for } (x,y) \text{ at the boundary of } D \end{aligned}$$

Let's substitute the $\delta_{(x,y)}(s,p)$ inside the integral with $-a\Delta G(s,p; x,y)$:

$$\begin{aligned} -a\Delta u(x,y) &= \int_D -a\Delta G(s,p; x,y) f(s,p) ds dp \text{ for } (x,y) \in D \subset \mathbb{R}^2 \\ u(x,y) &= 0 \text{ for } \vec{x} \text{ at the boundary of } D \end{aligned}$$

Now let's assume that we have the right conditions to swap differentiation and integration:

$$\begin{aligned} -a\Delta u(x,y) &= -a\Delta \left(\int_D G(s,p; x,y) f(s,p) ds dp \right) \text{ for } (x,y) \in D \subset \mathbb{R}^2 \\ u(x,y) &= 0 \text{ for } (x,y) \text{ at the boundary of } D \end{aligned}$$

Finally, this allows us to represent the solution $u(x,y)$ as:

$$u(x,y) = \int_D G(x,y; s,p; a) f(s,p) ds dp$$

Note that we made the dependency of G on a explicit in G . Later in this chapter, when we want to learn the solution operator of the PDE in a neural network setting, the physical parameter a would be part of the network's input. If the parameter $a = a(s,p)$ is not constant, then we would write $G(x,y; s,p; a(s,p))$. Analogous to the discussion of the previous example, the punch line from the previous calculation is:

The solution $u(x,y)$ of the PDE is the integral of some kernel function, in this case the Green's function $G(s,p; x,y; a)$ against the righthand side of the PDE $f(s,p)$.

Moreover, the solution operator of the PDE maps the given input data, which in this case is the parameter a and the righthand side of the PDE $f(x,y)$, to the output, which is the solution we are seeking, $u(x,y)$, by integrating the righthand side function against some kernel function that depends on the parameter of the PDE (along with its dependence on space). In our case, the kernel of the solution operator is the

Green's function of the PDE, which we happen to know for the Poisson equation on domains with an easy geometry, but not in more complex situations. Once again, knowing the formula of this kernel, or using a neural network to approximate it, unlocks the solution of a given PDE.

Fixed Point Iteration

The fixed point iteration is useful to construct explicit solutions and prove existence and uniqueness for certain lucky PDEs. It is such an easy and general method, so a definite must-have in our toolbox. We will write it down, then immediately apply it to represent the solution of a *dynamical system* as a series. A dynamical system is an ordinary differential equation that describes the evolution in time of a particle or a bunch of particles (a system). Again, we would like neural networks to learn the solution operators of dynamical systems, so this is consistent with our previous discussion. Moreover, it is good to have the fixed point iteration series representation of the solution side by side with the neural network representation. Recall that in many mathematical settings, we can represent the same solution in multiple ways. A fixed point iteration series is additive, while a neural network representation is compositional. Moreover, neural networks seem to have the advantage of representing the solution operators of whole families of PDEs and an overall wider variety, which is like a dream come true in this field.

The fixed point iteration aims to find a fixed point of a function, or a point x^* that the function maps back to itself: $f(x^*) = x^*$. This is not a simple task since f is usually nonlinear, and most of the time we have no idea if such points exist for a given function. As is always the case with nonlinear equations, iterative methods avoid *one-shot* solutions and instead come up with a sequence of points that *hopefully and under the right conditions* converge to the desired solution, in this case, the fixed point of a function.

How does it work?

Here is how the fixed point iteration goes:

1. x_0 is the starting point
2. $x_{i+1} = f(x_i)$

That's it. Our sequence $\{x_0, x_1, x_2, \dots\}$ is generated by consecutive applications of f , and looks like $\{x_0, f(x_0), f(f(x_0)), f(f(f(x_0))), \dots\}$. Under the right conditions on f and x_0 , this sequence converges to a fixed point x^* of f (so $f(x^*) = x^*$).

Note that depending on f and the starting point, the asymptotic behavior of this sequence can be any of these:

*Convergence to a limit x^**

If the fixed point iteration converges, then it captures a fixed point (for a continuous function f ; if the fixed point iteration converges, then the limit must be a fixed point of f).

Divergence to ∞

The sequence grows without a bound.

Periodic behavior

The sequence oscillates between two or more values.

Chaotic behavior

The sequence behaves erratically, with no pattern whatsoever.

Theorems related to the fixed point iteration assert that the choice of x_0 , the starting point for the fixed point iteration, matters for whether it converges to a fixed point or not.

How do we use it to solve ODEs and PDEs?

Our concern in this chapter is finding solutions of differential equations, which are functions. So we will first reformulate a PDE so that its solution u satisfies an equation that looks like $F(u) = u$ (note that F here is an *operator* not a *function*), making it perfect for a fixed point iteration setting. Then we will apply the same logic as described previously and construct a sequence of *functions* that hopefully, under the right conditions, converge to the fixed point u^* of the *operator*, which is the solution of the PDE that we are looking for. Note that in the previous discussion, we constructed a sequence of *numbers* (rather than functions) that hopefully, under the right conditions, converge to a fixed point of a *function* (rather than an operator).

Let's demonstrate this using a *dynamical system* setting. This is one of the most important, general, and well-studied ordinary differential equations, which describes the time evolution of a point in space. It is easy in the sense that it is usually first order with only one derivative to get rid of, and hard in the sense that it is generally nonlinear. There is a habit of linearizing the dynamical system near a point and studying its linearized behavior. This is, in many cases, informative about the nonlinear behavior, but the two should not be conflated. Too much is known about linearized systems and not much is known about nonlinear systems, so we need equal attention to nonlinear systems. In this section we do not linearize. Instead, we approximate the solution using a series constructed by a fixed point iteration.

Knowing the initial state of the point $\vec{u}(t_0) = \vec{u}_0$, a solution trajectory $\vec{u}(t)$ tracks all of its future states. The function $f(\vec{u}(t), a(t), t)$ specifies the evolution:

$$\begin{aligned}\frac{d\vec{u}(t)}{dt} &= \vec{f}(\vec{u}(t), a(t), t) \\ \vec{u}(t_0) &= \vec{u}_0\end{aligned}$$

There is one derivative with respect to time that we need to get rid of, so we integrate once with respect to time:

$$\vec{u}(t) = \vec{u}_0 + \int_{t_0}^t \vec{f}(\vec{u}(s), a(s), s) ds$$

Now let's rewrite this integral equation in a form that is ripe for the fixed point iteration. We consider the whole righthand side as an operator whose input is $\vec{u}(t)$:

$$\vec{u}(t) = F(\vec{u}(t))$$

Now we can generate the sequence $\{\vec{u}_0(t), \vec{u}_1(t), \vec{u}_2(t), \vec{u}_3(t), \dots\}$ that converges to the solution $\vec{u}(t)$, for all time or for a finite amount of time, under the right conditions on f . The sequence looks like:

- $\vec{u}_0(t) = \vec{u}_0(t)$
- $\vec{u}_1(t) = F(\vec{u}_0(t)) = \vec{u}_0 + \int_{t_0}^t \vec{f}(\vec{u}_0(s), a(s), s) ds$
- $\vec{u}_2(t) = F(\vec{u}_1(t)) = \vec{u}_0 + \int_{t_0}^t \vec{f}(\vec{u}_1(s), a(s), s) ds$
- $\vec{u}_3(t) = F(\vec{u}_2(t)) = \vec{u}_0 + \int_{t_0}^t \vec{f}(\vec{u}_2(s), a(s), s) ds$

and so on.

Simple but very informative example

The best examples are the ones that are simple enough to have multiple ways of solving them. Seeing more than one way to do the same thing at once helps solidify the gist of a newly learned method. Consider the very simple one-dimensional and linear dynamical system:

$$\begin{aligned}\frac{du(t)}{dt} &= u(t) \\ u(0) &= 1\end{aligned}$$

The first way to solve this is by separation of variables, where we put everything that has $u(t)$ on one side of the equation, and everything that has t alone on the other side:

$$\frac{du(t)}{u(t)} = dt$$

Now we can integrate from 0 to t :

$$\int_0^t \frac{du(s)}{u(s)} = \int_0^t ds$$

We get $\ln(u(t)) = t$, therefore the solution of our simple dynamical system using the separation of variables method is $u(t) = e^t$ (arguably the most important function in mathematics). Now let's construct a sequence of functions using the fixed point iteration and see if it converges to the solution $u(t) = e^t$ of the dynamical system:

- $u_0(t) = 1$
- $u_1(t) = F(u_0(t)) = u_0(t) + \int_0^t u_0(s) ds = 1 + \int_0^t 1 ds = 1 + t$
- $u_2(t) = F(u_1(t)) = u_0(t) + \int_0^t u_1(s) ds = 1 + \int_0^t 1 + s ds = 1 + t + \frac{t^2}{2}$
- $u_3(t) = F(u_2(t)) = u_0(t) + \int_0^t u_2(s) ds = 1 + \int_0^t 1 + s + \frac{s^2}{2} ds = 1 + t + \frac{t^2}{2} + \frac{t^3}{3!}$
- Keep going:

$$u_n(t) = F(u_{n-1}(t)) = u_0(t) + \int_0^t u_{n-1}(s) ds = 1 + t + \frac{t^2}{2} + \frac{t^3}{3!} + \cdots + \frac{t^n}{n!}$$

As $n \rightarrow \infty$, the fixed point iteration converges to the series:

$$u_\infty(t) = 1 + t + \frac{t^2}{2} + \frac{t^3}{3!} + \cdots + \frac{t^n}{n!} + \cdots = \sum_{n=0}^{\infty} \frac{t^n}{n!}$$

which is the power series expansion of $u(t) = e^t$, the same solution we arrived at using separation of variables (albeit in different form). Cool.

When we use this iterative way to construct a solution of a dynamical system, or of a PDE reformulated as a dynamical system or in a way that is fit for a fixed point iteration ($u=F(u)$), we call it *Picard's iteration*. It is simple and arrives at the solution (when it converges) in steps.

Where is the complication?

Why don't we use *Picard's iteration* to construct solutions of *all* dynamical systems and of *all* PDEs that we are able to reformulate into a form fit for fixed point

iteration? As always, the answer is the curse of dimensionality. Even for our very simple *one-dimensional* and linear example, each Picard iteration step involves evaluating an integral, which for more complex problems we have to evaluate numerically. For example, for dynamical systems representing the evolution and interactions of many particles, this gets multiplied by the number of particles. Overall in the ODE and PDE literature, there is a limited number of cases where practical algorithms are available for high-dimensional settings.

Recent successes!

That said, a recent method for finding explicit solutions for high-dimensional nonlinear parabolic PDEs and backward stochastic differential equations based on Picard's iteration has been quite successful in finding explicit solutions for high-dimensional PDEs arising in real-life physics and finance applications. [The paper's \(Weinan et al. 2017\) abstract](#) is insightful:

Parabolic partial differential equations (PDEs) and backward stochastic differential equations (BSDEs) are key ingredients in a number of models in physics and financial engineering. In particular, parabolic PDEs and BSDEs are fundamental tools in the state-of-the-art pricing and hedging of financial derivatives. The PDEs and BSDEs appearing in such applications are often high-dimensional and nonlinear. Since explicit solutions of such PDEs and BSDEs are typically not available, it is a very active topic of research to solve such PDEs and BSDEs approximately. In the recent article [E, W., Hutzenthaler, M., Jentzen, A., and Kruse, T. Linear scaling algorithms for solving high-dimensional nonlinear parabolic differential equations. arXiv:1607.03295 (2017)] we proposed a family of approximation methods based on Picard approximations and multilevel Monte Carlo methods and showed under suitable regularity assumptions on the exact solution for semilinear heat equations that the computational complexity is bounded by $O(d\epsilon^{-(4+\delta)})$ for any $\delta \in (0, \infty)$, where d is the dimensionality of the problem and $\epsilon \in (0, \infty)$ is the prescribed accuracy. In this paper, we test the applicability of this algorithm on a variety of 100-dimensional nonlinear PDEs that arise in physics and finance by means of numerical simulations presenting approximation accuracy against runtime. The simulation results for these 100-dimensional example PDEs are very satisfactory in terms of accuracy and speed. In addition, we also provide a review of other approximation methods for nonlinear PDEs and BSDEs from the literature.

Setting the stage for deep learning for PDEs

Before leaving this section, let's set the stage for solving ODEs and PDEs in the context of deep learning, in particular, for *deep operator networks*. We'll keep our one-dimensional dynamical system example, but this time we highlight the dependence on the physical parameters $a(t)$, and make it slightly more general by adding another explicit dependence on time:

$$\frac{du(t)}{dt} = \vec{f}(u(t), a(t), t)$$

$$\vec{u}(t_0) = \vec{u}_0$$

As before, we integrate once with respect to time:

$$u(t) = u_0 + \int_{t_0}^t f(u(s), a(s), s) ds$$

The purpose of a neural network is to take data as input, do something to it, then give us an output that we care about. For an ODE or a PDE, of course the output that we care about is the solution $u(t)$. Let's write this solution as the output of some operator G that takes the given data of an ODE or PDE as input. In our dynamical system case, the input data is the function $a(t)$ representing the physical parameters of the dynamical system. Note that we do not need to input the righthand side function f of the dynamical system. This is implicit in the training data, which now looks like the pairs (*training input, training output*)= $(a(t), u(t))$. By not inputting f , in a way we are saying that we don't care about the exact form of the ODE or PDE that this behavior comes from, but we are able to *learn* what the system is doing, regardless. This is the epitome of machine learning: no need to encode the rules that the system obeys; the model can still emulate it if it observes enough instances of it.

We can write the solution $u(t) = G(a(t))$, where the *solution operator* G is to be learned using a neural network. Hold this notation and thought until later in this chapter when we discuss neural operator networks. Plugging $u(t) = G(a(t))$ into the integral equation, we find that the solution operator, which we want to learn using a neural network, satisfies:

$$G(a(t)) = u_0 + \int_{t_0}^t f(G(a(s)), a(s), s) ds$$

We just wrote an integral equation that we will not do anything with. It just shows the *true* property that the entity $G(a(t))$ that we care for satisfies. In the previous discussion, we approximated this using a Picard's iteration, and in the new era of deep learning, we approximate it using a deep operator network (more on this soon). This deep learning approach is computationally more efficient if we include the Fourier transform to speed up the computations. Moreover, the deep learning approach is more widely encompassing, in the sense that it applies to more PDEs and ODEs than just dynamical systems. A dynamical system is easy to integrate once and obtain a representation that gets us closer to the solution, which is not the case for many ODEs and PDEs.

Mesh independence and different resolutions

One final note: an input and output pair for a neural network learning the solution of our dynamical system looks like: $(\text{training input}, \text{training output}) = (a(t), u(t))$. Since machines take numerical values only and not functions, we must discretize when we implement this. Here's the pretty thing that differentiates *operators acting on functions* from *functions acting on points* and gives neural operator networks their *mesh independence* feature: $a(t)$ and $u(t)$ do not have to be discretized at the same values of t . All we care about is to map one function to another, so we can think of discretized $a(t)$ as a vector mapped to another vector that is the discretized $u(t)$, not necessarily at the same points or not even of the same size. For the same reason, we can train the network at a given resolution, then make predictions at *another* resolution. This is great for the field of ODEs and PDEs, where the quality of numerical solutions has always been limited by the resolution of the employed discretization.

AI for PDEs

After surveying the main concerns and the basic approaches for solving PDEs, we are finally ready to discuss AI as it relates to PDEs, instead of only hinting at it or setting its stage here and there. We want to distinguish between a few different ways in which deep learning has stepped into the PDE community:

- Deep learning to learn the PDE's physical parameter values
- Deep learning to learn two-dimensional and three-dimensional meshes for numerical simulations and solid modeling
- Deep learning to learn a PDE's solution operator: a neural network learns a map between two *infinite dimensional* spaces
- Deep learning to bypass PDEs and simulate natural phenomena directly from observing data (particle systems and their interactions)

Deep Learning to Learn Physical Parameter Values

We can use neural networks to infer the parameters of a PDE model and their uncertainties. We get training data from experiments (real or synthetic via simulations of well-known phenomena with known parameters). This training data is labeled with the parameter values, so the neural network learns to map a certain PDE's initial setting to appropriate parameter values, leading to more accurate modeling results. Historically, parameters that couldn't be directly measured had to be guessed or hand tuned to fit some observed behavior, a practice that undermines the whole modeling process. This simple application of deep learning helps the PDE modeling community tremendously, because it brings more authenticity to their results. We can now learn parameter values from labeled images of experiments, recorded audio, and

other unstructured or very high-dimensional data. Once trained, the neural networks can estimate parameters and uncertainties for any input data with similar settings. This poster has a nice and simple example that uses deep learning to predict the parameters of the velocity field for the G-equation (which models the combustion process) using flame front-image data: “[Bayesian Inference in Physics-Based Nonlinear Flame Models](#)”.

Deep Learning to Learn Meshes

We learned in this chapter that generating a mesh is an integral part of the finite element method, which in turn finds numerical solutions for a wide array of PDEs modeling natural phenomena with complex domain geometries. The quality of the underlying mesh affects the quality of the numerical solution. The finer the mesh, the more of the true solution it is likely to capture, but the more computational cost it incurs. An ideal mesh would be dense where the error between the numerical solution and the true solution is more likely to be high, and coarse where the error is low, hence preserving fidelity while keeping the overall computational cost manageable ([Figure 13-12](#)).

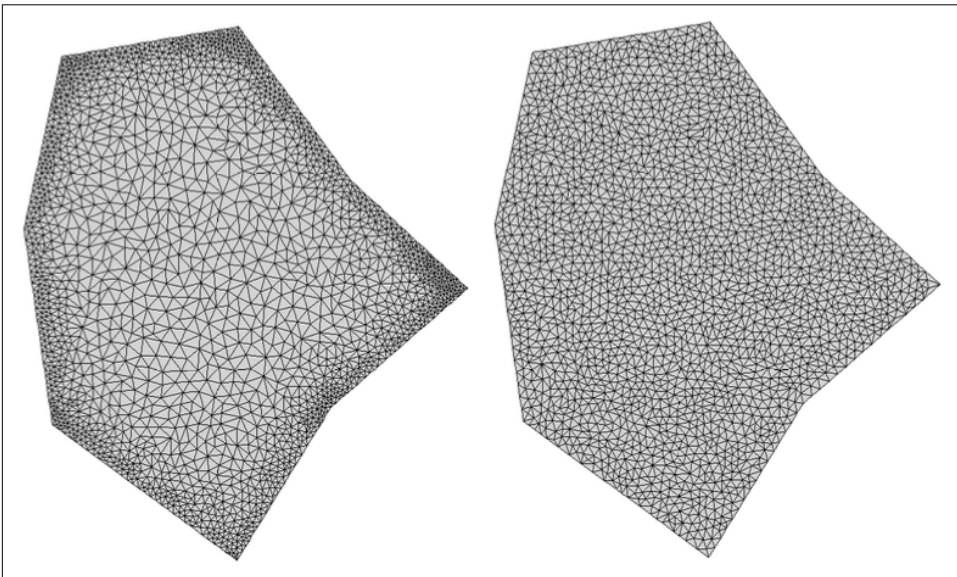


Figure 13-12. Nonuniform mesh on the left versus uniform mesh on the right; a mesh needs to be finer where the error is large ([image source](#))

It would be nice if, given a PDE, its domain geometry, boundary conditions, and parameter values as inputs, we could train a neural network to automatically generate an ideal mesh, predicting the density distribution of the mesh elements at each location of the domain. This is exactly what [MeshingNet](#) does.

Before MeshingNet, mesh learning was done via expensive multistep finite element solutions and error estimators. In contrast, MeshingNet relies on similar problems to predict ideal meshes for new problems. It starts with an initial uniform and coarse mesh, and predicts a nonuniform mesh density for refinement. A hallmark of deep learning, MeshingNet generalizes well to different geometric domains with various governing PDEs, boundary conditions, and parameters.

The inputs to MeshingNet are the governing PDE, PDE parameters, domain geometry, and the boundary conditions, and the output is an area upper-bound distribution $A(X)$ over the whole domain. The mapping between input and output is highly nonlinear and is thus learned by a neural network, which has demonstrated an impressive ability to express many kinds of nonlinear relationships.

To build the training data set, the MeshingNet team computes high-accuracy solutions on high-density uniform meshes using standard finite element solvers. They also do the same computation for low-density uniform meshes to obtain lower-accuracy solutions. Then the team computes an error distribution $E(X)$ by interpolating between these solutions. They use $E(X)$ as a guide to refine $A(X)$. They enrich the training data by combining different geometries with different parameters and boundary conditions.

Deep learning for three-dimensional meshes

Three-dimensional meshes (Figure 13-13) are useful for computer graphics, animations for the entertainment industry, and solid modeling. They are also highly desirable to reconstruct textured and realistic surfaces from a given set of three-dimensional data points. Traditional methods include Delaunay triangulations and Voronoi diagrams, which interpolate points using triangular meshes. However, when there is noise in the coordinates, the resulting surface would be unnecessarily rough, which requires data preprocessing.

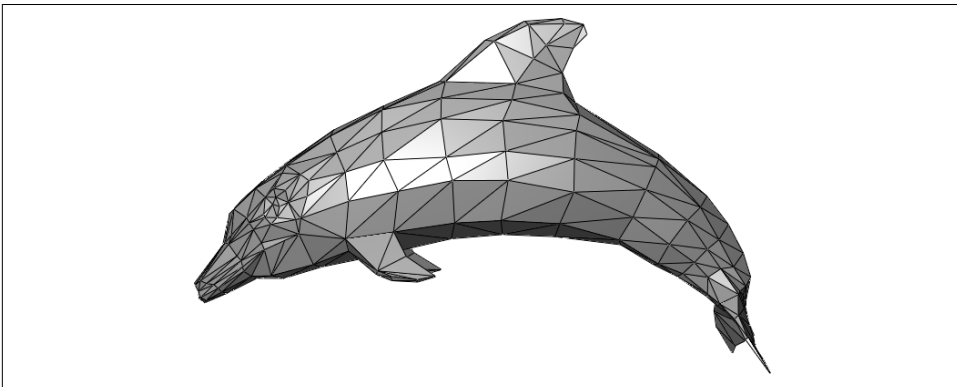


Figure 13-13. Three-dimensional mesh (*image source*)

Deep learning is stepping in to generate higher-quality three-dimensional meshes; see, for example, “Deep Hybrid Self-Prior for Full 3D Mesh Generation” (Wei et al. 2021) and “Pixel2Mesh” (Wang et al. 2018), which produces a three-dimensional shape in triangular mesh from a single color image by continuously deforming an ellipsoid.

Deep Learning to Approximate Solution Operators of PDEs

We have already started this discussion multiple times in this chapter. Instead of using deep learning to enhance existing methods for PDEs, such as learning physical parameter values from data, or learning better meshes for numerical methods, we would like to learn a PDE’s solution operator. This maps the PDE’s input, such as its domain, physical parameters, initial/final states of the solution, and/or boundary conditions, directly to its solution. We can think of this as:

Solution of PDE = function(PDE’s physical parameters, domain, boundary conditions, initial conditions, etc.)

We want to build a neural network to approximate this function. This is in fact an operator and not a function in the usual sense, since it sends functions to other functions. The caveat here is that differential operators and their inverses map *infinite dimensional spaces* to *infinite dimensional spaces*, sometimes in a linear way, such as the map from the *right hand side* of a Poisson equation to the solution, and most of the time in a nonlinear way, such as the map from *the parameters* of a Poisson equation to the solution. In contrast, the inputs and outputs of the neural networks that we learned about throughout this book are finite dimensional (inputs and outputs are vectors, images, graphs, etc.). These neural networks are able to approximate *function mappings* between *finite dimensional spaces*. They have a powerful universal approximation theorem going on for them, and a myriad of successes in practical applications (we can approximate any continuous function to arbitrary accuracy using neural networks if we place no constraints on the width and depth of the hidden layers). To solve PDEs analogously using deep learning, we must answer two questions:

Can neural networks approximate mappings between infinite dimensional spaces?

That is, can they approximate any nonlinear continuous functional (the input to the network would be a function or a bunch of functions, and the output would be a real number) or nonlinear operator (the input to the network would be a function or a bunch of functions, and the output would be another function)? The answer is a *yes!*

There is a *universal approximation theorem* for *neural network operators* just like there are universal approximation theorems for *neural network functions*. A neural network with a single hidden layer can approximate accurately any

nonlinear continuous functional or operator. Moreover, neural networks are able to learn the solution operator of an entire family of PDEs, as opposed to classical methods for solving PDEs that only solve a single instance of a given PDE at a time.

How do we implement this in practice?

For the finite dimensional case, a node in a neural network linearly combines the finite dimensional features of the input vector (or the outputs of the previous layer), adds a bias term, applies a nonlinear activation function, then passes the result to the next layer. The analog for the infinite dimensional case, where we do not have finitely many entries to linearly combine anymore, would be to integrate some learnable kernel (multiplier function) of the input functions (for numerical integration, we have to sample this at finitely many points, converting integration to addition), add a bias function (this is optional), and apply a nonlinear activation function before passing the result to the next layer. The next layer would then add multiples of the node results of the previous layer, and integrate them against a learnable kernel of the results of the nodes of the previous layer, and so on. One example of doing this would look like:

$$u_{n+1}(x) = \sigma(\int_D \text{kernel}(x, s, a(x), a(s); \omega) u_n(s) ds + W u_n(x)),$$

where we arrive at the solution $u(x)$ iteratively after a certain number of global integrations against a kernel, local linear transformations, and compositions with a nonlinear activation function. The parameters of the kernel in the iterative process are ω and the entries of W . The neural network learns these parameters from the labeled data (labeled with the solutions of the PDE) during training by minimizing a loss function. Analogous to the finite dimensional case, neural operator networks approximate nonlinear operators by composing linear integral operators that act globally over the entire domain with nonlinear activation functions. The previous iterative formula also includes a local linear multiplier, which becomes a matrix when we discretize.

Neural operator networks to learn the solution operators that we derived

Let's pause for a moment and compare the previous expression to the three true solution operators that we derived for: the heat equation, Poisson equation, and dynamical systems. We can easily adapt the neural operator iterative process to each of these three settings:

- For the heat equation in one space dimension and constant coefficients, the solution operator maps the initial state and the PDE's physical parameter (constant) to the solution $u(x,t)$. We are lucky enough to have explicit formulas for all the quantities involved:

$$\begin{aligned} G(u_0(x), a) &= u(x, t) = \int_{-\infty}^{\infty} \frac{1}{\sqrt{4\pi at}} e^{-\frac{(s-x)^2}{4at}} u_0(s) ds \\ &= \int_{-\infty}^{\infty} \text{kernel}(s, x; t, a) u_0(s) ds \end{aligned}$$

In this case, the neural operator network does the following iteration to approximate the true operator:

$$\begin{aligned} G(u_0(x), a) &= u(x, t) \approx u_{n+1}(x, t) = \sigma\left(\int_D \text{kernel}(s, x; t, a; \omega) u_n(s) ds\right. \\ &\quad \left.+ W u_n(x)\right) \end{aligned}$$

- For a Poisson equation in two space dimensions, zero boundary conditions, and constant coefficients, the solution operator maps the PDE's righthand side f , and its physical parameters (constant) to the solution $u(x, y)$, and only for certain simple geometries. We are lucky to have explicit formulas for all the quantities involved (none of which we write here):

$$G(f(x, y), a) = u(x, y) = \int_D \text{GreenFunction}(x, y; s, p; a) f(s, p) ds dp$$

In this case, the neural operator network does the following iteration to approximate the true operator:

$$\begin{aligned} G(f(x, y), a) &= u(x, y) \approx u_{n+1}(x, y) = \sigma\left(\int_D \text{kernel}(x, y, s, p, a; \omega) u_n(s, p) ds dp\right. \\ &\quad \left.+ W u_n(x, y)\right) \end{aligned}$$

- For a one-dimensional dynamical system, the solution operator maps the ODE's physical parameter (function) to the solution $u(t)$, and we have an implicit integral equation that it satisfies:

$$G(a(t)) = u(t) = u_0 + \int_{t_0}^t f(G(a(s)), a(s), s) ds.$$

In this case, the neural operator network does the following iteration to approximate the true operator:

$$G(a(t)) = u(t) \approx u_{n+1}(t) = \sigma\left(\int_{t_0}^t \text{kernel}(t, s, a(t), a(s); \omega) u_n(s) ds + W u_n(t)\right)$$

Here, one data point is a triplet $(t, a(t), G(a(t)))$, and thus one specific input a may appear in multiple data points with different values of t . For example, a data set of size 10,000 may only be generated from 100 $a(t)$ trajectories, and each evaluates $G(a)(t)$ for 100 t locations.

One thing to observe with these three different settings is that neural operator networks require only input and output data, and no knowledge of the underlying PDEs. The knowledge about the PDEs is implicit in the training data. With this in mind, let's reinforce the *input output* form of a neural operator network:

Solution of PDE $\approx$ learned operator(PDE's physical parameters, domain, boundary conditions, initial conditions, etc.)

The important questions

When we branch out beyond this book to expand our knowledge on neural operator networks, we must keep the following questions as our guide:

For a given PDE, what is the input to the network, and what is the output?

We addressed these in the simple contexts of the heat equation, Poisson equation, and dynamical systems.

What is an example of an architecture of a neural operator?

Figure 13-14 shows the input and output structure of DeepONet. The input is a discretized pair $(t, a(t))$, and the output is a discrete $G(a(t))$.

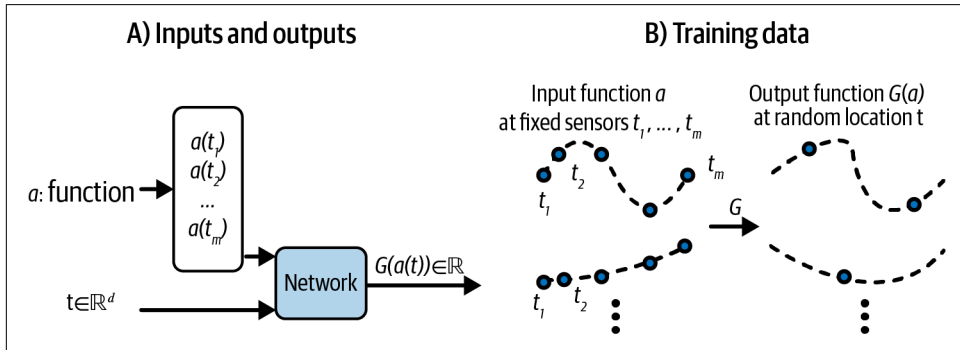


Figure 13-14. (A) The network to learn an operator $G(a(t))$ takes two inputs $(a(t_1), a(t_2), \dots, a(t_m))$ and t . (B) Illustration of the training data ([image source](#)).

How do we deal with the fact that the inputs have such drastic differences in dimension, such as finite dimensional and infinite dimensional at the same time?

In other words, how do we discretize the involved finite dimensional (independent variables such as time and space) and infinite dimensional quantities (solution functions, parameter function, boundary conditions, initial conditions, etc.) during training and inference? Note that for neural networks that learn finite dimensional mappings, inputs (tables, images, audio files, graphs, natural language text) always have the same dimension, are preprocessed to have the same dimension, or the network itself processes fixed dimension portions of the input individually.

How do we avoid the common trap in many PDE solution methods, which end up being discretization dependent?

In what sense are neural operator networks meshless and able to generalize their learned parameters to work for other discretizations than the ones they have been trained on? The significant advancement here is that neural operator networks are discretization invariant, sharing the same network parameters between different discretizations. This means that their outputs do not depend on the underlying discretization and can be used with different grid representations.

How do we speed up the computation time for the integrals involved in the neural operators and make it less costly?

We involve the Fourier transform. A Fourier neural network speeds up the process of computing the involved integrals transforming the inputs to Fourier space. This has the fast Fourier transform methods at its disposal. A Fourier neural network, which we discuss in the next subsection, implements this.

How do neural operator networks fare with high-dimensional PDEs involving hundreds or thousands of variables?

PDEs that model financial markets with all the underlying assets (Black-Scholes), game theoretic settings with many participating agents (Hamilton-Jacobi-Bellman), or physical systems with many particles are very high-dimensional. Discretization in each of these dimensions explodes the size of an already big problem computationally and has until now made any practical implementation of these elegant PDEs infeasible. The article [“Solving High-Dimensional Differential Equations Using Deep Learning”](#) (Han et al. 2018), which we discuss soon, uses AI techniques to address such PDEs, but it would be nice to compare that article’s methodology to a deep neural operator setting.

Fourier neural network

The California Institute of Technology has recently open sourced its Fourier neural network for solving partial differential equations; its approach is shown in the article [“Fourier Neural Operator for Parametric Partial Differential Equations”](#) (Li et al.

2021). These networks can approximate solution operators for PDEs that are highly nonlinear, with high frequency modes and slow energy decay.

Each layer in a Fourier neural network applies a fast Fourier transform to its input data, then a linear transform, then an inverse fast Fourier transform. This results in a quasi-linear computational complexity, that is, of order $O(n \text{ polynomial}(\log(n)))$ and makes the model invariant to the spatial resolution of the data (even though it still requires a uniform mesh).

Figure 13-15 shows the architecture of the Fourier neural network.

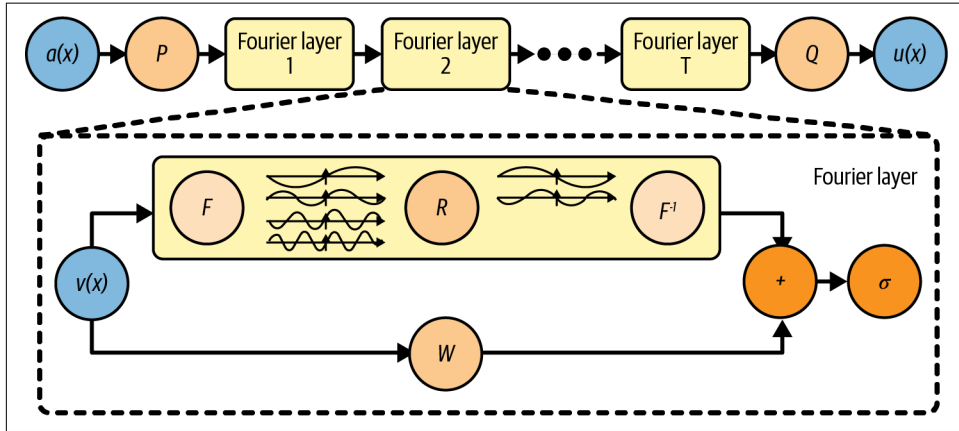


Figure 13-15. The architecture of a Fourier neural network (*image source*)

The input is the physical parameter $a(x)$, and the output is the PDE solution $u(x)$:

1. Start from input $a(x)$.
2. Lift to a higher-dimension channel space by a shallow, fully connected neural network P : $v_0(x) = P(a(x))$.
3. Apply several Fourier layers of integral operators and activation functions. In each of these layers, we apply the Fourier transform $F.T$, apply a linear transform R on the lower Fourier modes and filter out the higher modes, and apply the inverse Fourier transform $F.T^{-1}$. On the bottom, apply a local linear transform W .
4. Project back to the target dimension by a neural network Q . Finish with the output $u(x) = Q(v_T(x))$, which is the projection of v by the local transformation Q also parameterized by a shallow fully connected neural network.
5. Finish with the output $u(x)$.

The article demonstrates the method with a variety of important PDEs:

- Burgers' equation
- Darcy flow
- Navier-Stokes equation
- Turbulent flows in regimes where other methods diverged

The Fourier neural network is mesh invariant, so it can be trained on a lower resolution and evaluated at a higher resolution, without seeing any higher resolution data (zero-shot super-resolution).

Since data-driven methods rely on the quality and quantity of data, we need to generate training pairs of inputs and outputs for the neural operator networks by solving the actual PDEs using some other methods. To this end, the authors note that to learn Navier-Stokes equation with $viscosity = 1e^{-4}$, we need to generate $N = 10,000$ training pairs $(a(x), u(x))$ using a numerical solver. For more challenging PDEs, generating even a few training samples can be very expensive. A future direction would be to combine neural operators with numerical solvers to raise the requirements on data.

Statement of the universal approximation theorem for operators

Suppose that σ is a continuous nonpolynomial function, X is a Banach space, $K_1 \subset X$, $K_2 \subset \mathbb{R}^d$ are two compact sets, V is a compact set in $C(K_1)$, and G is a nonlinear continuous operator that maps V into $C(K_2)$. Then for any $\epsilon > 0$, there are positive integers n, p, m , and constants $c_i^k, \xi_{i,j}^k, \theta_i^k, \xi_k \in \mathbb{R}, \omega_k \in \mathbb{R}^d, x_j \in K_1, i = 1, \dots, n, k = 1, \dots, p, j = 1, \dots, m$, such that:

$$|G(u)(y) - \sum_{k=1}^p \sum_{i=1}^n c_i^k \sigma\left(\sum_{j=1}^m \xi_{i,j}^k u(x_j) + \theta_i^k\right) \sigma(\omega_k \cdot y + \xi_k)| < \epsilon$$

holds for all $u \in V$ and $y \in K_2$. Note that this approximation theorem only uses one hidden layer in the neural network but does not specify how many nodes this layer has. In applications, just like in the finite dimensional case, we use more than one layer.

Do not be intimidated by the big words and the Greek letters. What this theorem tells us is that we have theoretical grounds to formulate a neural network operator, and expect it to approximate the PDE solution operator very well. Even though we may never know the exact formula for the PDE solution operator, the operator neural network that we construct acts as a very good proxy. This is the reason we are all in love with approximation theorems, and should be eternally grateful to the mathematicians who find them.

We have mentioned this in the context of the fixed point iteration's *additive* approximation of a solution, and it is worth mentioning again: whether approximating a mapping between finite dimensional spaces or a mapping between infinite dimensional spaces, neural networks represent functions, functionals, or operators using *compositions* of simple functions (a linear combination or linear integral operator composed with a nonlinear activation function) to approximate complicated ones. This is different than classical approximation approaches, where the approximation is *additive* and not compositional.

How do we branch out and dive into the more technical details?

For a deeper dive into neural operator networks, the three important publications on this topic are:

- “DeepONet: Learning Nonlinear Operators for Identifying Differential Equations Based on the Universal Approximation Theorem of Operators” (Lu et al. 2020)
- “Neural Operator: Graph Kernel Network for Partial Differential Equations” (Li et al. 2020)
- “Fourier Neural Operator for Parametric Partial Differential Equations” (Li et al. 2021)

Numerical Solutions of High-Dimensional Differential Equations

Differential equations are universal; they can model almost anything we can think of, including our daily commute and traffic. It is hard to be exposed to differential equations, then *not think* of each situation that we find ourselves in as fitting into some sort of differential equation. That said, the curse of dimensionality has haunted this field since its inception, and has stood in the way of many practical applications. This is why many introductory PDE courses misleadingly focus only on one- and two-dimensional differential equations, as if that is all there is. If AI was to be given a different name that is not as flashy and definitely would not make it into any movies, it would be: *processing, computation, and analysis of high-dimensional data*. This does not talk AI down, because processing, computation, and analysis of high-dimensional data are exactly what humans do on a daily basis, provided we add a creative dimension to it (which in AI would translate to generative models). It is then not surprising that deep learning turns out to be an appropriate setting for finding numerical solutions of *very* high-dimensional differential equations. This is the focus of the article “Solving High-Dimensional Differential Equations Using Deep Learning” (Han et al. 2018), which addresses PDEs with hundreds or even thousands of dimensions. This way, we can include all participating agents, assets, resources, or particles at the same time, instead of artificially devising handmade assumptions about their interactions and connections. The authors consider multiple high-dimensional PDEs, including the Hamilton-Jacobi-Bellman equation (what is

the optimal strategy for each interacting agent, among hundreds of agents?) and the Black–Scholes equation (what is the fair price of a European claim based on one hundred underlying assets, given that no default has occurred yet?).

When we rely on a deep learning setting as a basis for our models, for example, for computing solutions of high-dimensional PDEs, the first question we must ask is what is the input and what is the output of our deep learning network. For any PDE whose solution is $u(x,t)$, ideally we would input x and t and output $u(x,t)$. x in this case can be an extremely high-dimensional $\vec{x}$. If the entries of x have any inherent stochasticity into them, such as prices of financial market assets, then we must model them as such, and if we don't, then we are usually assuming some sort of averaging. The bottom line is, for many realistic cases, we can input x as X , a stochastic process. We must define this mathematically.

One big step in the aforementioned article is reformulating the high-dimensional PDEs as backward stochastic differential equations before inputting X into a neural network that approximates the gradient of the solution. To master the essential math required here, we must define:

- Brownian motion (see [Chapter 11](#))
- Stochastic process (see [Chapter 11](#))
- Stochastic differential equation (this is beyond the scope of the book)
- Relating nonlinear parabolic PDEs to stochastic PDEs (this is beyond the scope of the book)
- Backward stochastic differential equation (this is beyond the scope of the book)

And we must answer the question: why did we have to reformulate the PDE into a stochastic form before training the network? What advantage does this form give us? This goes beyond the scope of the book, but you now know what to look for and what questions to ask.

Finally, the method opens the door to solving many high-dimensional differential equations, but there are limitations. The method cannot be applied to the quantum many body problem due to the difficulty in dealing with the [Pauli exclusion principle](#).

Simulating Natural Phenomena Directly from Data

We have addressed particle systems once in this chapter. We used a statistical mechanics framework to describe the probabilities of the states of the system at the particle scale, then we used those to write down PDEs modeling the time evolution of the system at the macro scale.

In this section, we explain how recent neural network–based models simulate a particle system and predict its evolution *without* writing any PDEs. In other words, we bypass PDEs and trade them for *learning from data*.

To track the evolution of a certain particle system (such as water or sand) at a granular scale, we need to know each particle’s position vector $\vec{p}_i(t)$ at each time step t . How these positions change depends on local and long-range interactions between a particle and its neighbors (such as exchanging energy and momentum), which are dictated both by the physical nature of the system and by external effects such as gravity, temperature, forces, magnetic fields, etc. Instead of writing down explicit equations for these interactions, and relating them to the particles’ positions, velocities, and/or accelerations, we can train a neural network to learn a map between a given state of a particle system at a certain time (input) and the positions of all of its particles (or velocities or accelerations) at a future time (output). Graph networks are well suited to model particle systems, since each particle along with its state can be a node and the edges along with their features model the interactions between specific particles.

We highlight and comment on the general ideas from a recent work that learns such a map: “[Learning to Simulate Complex Physics with Graph Networks](#)” (Sanchez-Gonzalez et al. 2020).

First, we need training data

We can generate input (particle system and its features at a certain time) and target (each particle’s acceleration at a later time) pairs from a data set of observed or simulated trajectories of a certain particle system. For example, from a 1,000-step-long trajectory, the team generates 995 pairs, conditioning on the 5 past states. In the data sets, we only need the position vectors, and we can derive velocity and acceleration vectors using finite differences. The data sets typically contain 1,000 train, 100 validation, and 100 test trajectories, each simulated for 300–2,000 time steps, tailored to the average duration for the various materials to come to a stable equilibrium.

Next, we need to build the map from input to output (the network components)

Start with the system at a certain state at integer time t $X^t = (\vec{x}_0^t, \dots, \vec{x}_N^t)$, where each of the N particles’ $\vec{x}_i^t$ represents its state at time t (which includes its position $\vec{p}_i^t$ and other characteristics such as mass, material properties, etc.).

Next, learn a map that represents the state $X^t = (\vec{x}_0^t, \dots, \vec{x}_N^t)$ as graph $G = (\text{nodes}, \text{edges}, \text{and global properties that can alternatively be included as node features})$.

The node embeddings, $\vec{node}_i = \text{function}(\vec{x}_i^t)$, are learned functions (using a multilayer perceptron) of the particles’ states. Directed edges are added to create

paths between particle nodes that have some potential interaction. The edge embeddings, $\vec{e}_{i,j} = \text{function}(\vec{r}_{i,j})$, are learned functions (using a multilayer perceptron) of the pair-wise properties of the corresponding particles $\vec{r}_{i,j}$, for example, displacement between their positions, spring constant, etc.

Then learn a graph-to-graph map. This computes the interactions among the nodes via M steps of learned message passing to generate a sequence of updated latent graphs, $G = (G_1, \dots, G_M)$. Then return the final graph. Message passing allows information to propagate between the nodes via the edges, and for the constraints to be respected. This way, the complex dynamics of the system are approximated by learned message passing among the nodes within their local neighborhoods. Moreover, the final graph has the same structure as the first graph, but with potentially different node, edge, and graph-level attributes.

Then, learn a map (multilayer perceptron) from the final graph to a matrix that extracts the dynamics of the system, for example, the matrix of the particles' acceleration $Y = (\vec{p}_1'', \vec{p}_2'', \dots, \vec{p}_N'')$. Finally, the particles' positions and velocities are updated using an Euler integrator of the accelerations in Y . This in turn updates the system's state to X^{t+1} .

Such models are not restricted to materials and particle systems, but can model systems with many interacting agents, such as robotic control systems. They are a great step toward simulating complex phenomena authentically, which is of great value to science and engineering.

Hamilton-Jacobi-Bellman PDE for Dynamic Programming

The Hamilton-Jacobi-Bellman equation is yet another PDE whose solution unlocks many possibilities in economics, operations research, and finance, *if only* we are able to solve it in high dimensions. In a nutshell, we are searching for an *optimal strategy* (such as an investment strategy) that guarantees some *minimal* implementation cost over a *given period of time*. Ideally, we would like to include hundreds or thousands of interacting agents, such as all the financial assets for investment banking, instead of downsizing to unrealistic *representative agent* models. This is where using neural networks to find numerical solutions for high-dimensional PDEs helps us, as we saw earlier in this chapter.

Mathematically, the Hamilton-Jacobi-Bellman PDE is very *rich*. It combines dynamical systems ($\frac{\partial x(t)}{\partial t} = f(x(t), a(t), t)$), PDEs (partial derivatives and equalities), and optimization (*max* or *min* problems). When we learn how to derive this one PDE from real-world applications, attempt to understand it, find its solutions, and analyze these solutions (existence, uniqueness, smoothness, etc.), we acquire a ton of math.

Moreover, this PDE is directly related to *reinforcement learning* in AI, but instead of thinking about reinforcement probabilistically, in terms of Markov decision processes, such as in [Chapter 11](#), we think about reinforcement in terms of deterministic dynamic programming.

In the dynamic programming setting, the states of the interacting agents, bundled in a vector $\vec{x}(t)$, evolve in time according to a dynamic system, and we need to find an optimizing policy that induces a special solution of this dynamic system: the one that incurs a minimal cost over a given period of time. The train of thought goes like this: a certain time-dependent policy affects the behavior of a dynamic system, which in turn affects the incurred cost. All of these are mathematical quantities.

The contributions of Richard Bellman (1920–1984) to the dynamic programming field (finding optimal strategies for an evolving system over a given period of time) are invaluable. We will encounter Bellman’s principle for optimality shortly, and in fact, it is Bellman who coined the term *curse of dimensionality*. This principle is tremendously helpful, as it breaks down the involved optimization problem over the considered period of time into smaller subproblems at smaller time intervals, which we can then solve in a recursive manner.

Bellman’s equation in deterministic and stochastic settings

In a *deterministic* dynamic programming setting, there are:

Discrete time Bellman’s equation

We can find the value function starting from the current time until the final time by picking the best strategy (or control or policy) a_k at the current time step k so that the current cost plus the value function at the next time step are minimized. This is a recursive process:

$$Value(\vec{x}_k, n) = \min_{\vec{a}_k} \left(Cost(\vec{x}_k, \vec{a}_k) + Value(\vec{x}_{k+1}, n-1) \right)$$

where n is the final time step, and the discrete time dynamics are:

$$\vec{x}_{k+1} = f(\vec{x}_k, \vec{a}_k)$$

so that:

$$Value(\vec{x}_k, n) = \min_{\vec{a}_k} \left(Cost(\vec{x}_k, \vec{a}_k) + Value(f(\vec{x}_k, \vec{a}_k), n-1) \right)$$

The sequence of optimizers $\vec{a}_k$ at each discrete time step k constitute the optimal policy (or strategy or control) for the whole time period, and guarantee the minimal total cost, exactly as in reinforcement learning.

Continuous time Bellman's equation

This is the Hamilton-Jacobi-Bellman PDE.

In a *stochastic* optimal control setting, there is also a stochastic version of Bellman's equation. This is widely applicable in investment banking, and in scheduling and routing problems. In the stochastic framework, we need to find an optimal control input (strategy or policy) that guides the underlying stochastic processes to some desired final state, with minimal cost. Consider for example the problem where we need to execute a financial order, with a minimal implementation cost and within a certain period of time. We can first model the short-term dynamics of the underlying assets, then discretize both in time and state spaces. This allows us to execute a given amount of shares at each of the time steps, with the condition that we must execute all of the shares during the given time period. We search for the policy that tells us, among all possible actions that we can take at each point in time, what the optimal one is that gets us to where we want to be.

In [Chapter 11](#), we connected Bellman's equation to reinforcement learning. This was done in the context of a Markov decision process with value function:

$$Value(\vec{s}) = \max_{\text{states and actions}} \mathbb{E}(\text{reward}_0 + \gamma Value(\vec{s}'))$$

In a deterministic dynamic programming setting, the analogous equation is the Hamilton-Jacobi-Bellman PDE for the value function. Before writing its formula, this is the language that we need to pay attention to:

Minimizing a cost function

Is there a more common objective in this world?

Choosing an optimal control or optimal policy

This is the minimizer that we are looking for; it controls the dynamical system.

Value function

The total minimum cost over the considered period of time.

Bellman optimality principle

An amazingly helpful principle that allows us to simplify the optimization problem.

Backward in time solution

Start with the desired outcome and work our way backward, optimally to an initial state. It is intuitive to see why a backward-in-time solution is easier in this setting. Since we know the end goal, we immediately exclude all paths that don't lead to it from the preceding time step, saving us the exploration of many useless paths. If, on the other hand, we solve forward in time, starting at the beginning of the time interval, then we do not have the advantage of *closeness* to the desired outcome, so we must waste our time and computational resources exploring many more useless paths.

The big picture

The ultimate question is: what must we do now (what is the initial state $\vec{x}(t_{initial})$ and the time-dependent policy $\vec{a}(t)$) to get us to where we want to be ($\vec{x}(t_{final})$), in the most cost-efficient way (attain the value function $Value(\vec{x}(t_{initial}), t_{initial}, t_{final})$, which is the minimum value of the strategy implementation cost function)?

The involved quantities are:

- $\vec{x}(t)$ is a vector characterizing the state of the dynamic system.
- The strategy (or policy or control) $\vec{a}(t)$. We need to design this so that it invokes a state $\vec{x}(t)$ that minimizes some cost function. That is, if we input this special $\vec{a}(t)$ that we are looking for to the dynamic system, the output $\vec{x}(t)$ would minimize the cost function.
- The cost function $Cost(\vec{x}(t), \vec{a}(t), t_{initial}, t_{final})$ incurred due to the implementation of the strategy (or policy or control). This is given by some terminal cost at t_{final} and the sum of incremental costs (integral) as we transition from $t_{initial}$ to t_{final} . The incremental costs depend on the current state of the system and the current control.
- The value function $Value(\vec{x}(t_{initial}), t_{initial}, t_{final})$ is the minimal cost in a specific time period attained by enforcing the minimizing policy $\vec{a}^*(t)$, which in turn specifies the state $\vec{x}^*(t)$ using the information about the dynamics of the system.

Hamilton-Jacobi-Bellman PDE

These are the involved equations and formulas:

$$\begin{aligned}\frac{d\vec{x}(t)}{dt} &= \vec{f}(\vec{x}(t), \vec{a}(t), t) \\ Cost(\vec{x}(t), \vec{a}(t), t_{initial}, t_{final}) &= \\ Cost_{final}(\vec{x}(t_{final}), t_{final}) &+ \int_{t_{initial}}^{t_{final}} Cost_{incremental}(\vec{x}(s), \vec{a}(s)) ds \\ Value(\vec{x}(t_{initial}), t_{initial}, t_{final}) &= \min_{\vec{a}(t)} Cost(\vec{x}(t), \vec{a}(t), t_{initial}, t_{final})\end{aligned}$$

Bellman's optimality principle tells us something very valuable about the behavior of the value function (the optimal cost) along the special trajectory $\vec{x}^*(t)$ corresponding to the optimizing policy $\vec{a}^*(t)$: the value at a specified time interval is the sum of values if we break the time interval apart along the special trajectory $\vec{x}^*(t)$ corresponding to the optimizing policy $\vec{a}^*(t)$. This enables us to break up the optimization problem over a longer time interval into a recursion of optimization problems over much shorter time intervals:

$$\begin{aligned}Value(\vec{x}^*(t_{initial}), t_{initial}, t_{final}) &= Value(\vec{x}^*(t_{initial}), t_{initial}, t_{intermediate}) \\ &+ Value(\vec{x}^*(t_{intermediate}), t_{intermediate}, t_{final})\end{aligned}$$

Using Bellman's principle, we can derive the Hamilton-Jacobi-Bellman PDE that the value function satisfies. This PDE generalized an older Hamilton-Jacobi PDE for optimal control. The solution of this PDE contains very valuable information. Suppose we encounter the system at *any time* t , not only at its initial state $t_{initial}$ then we can compute the value function up to the desired final cost by solving the Hamilton-Jacobi-Bellman equation:

$$-\frac{\partial Value}{\partial t} = \min_{\vec{a}(t)} \left(\left(\frac{\partial Value}{\partial \vec{x}} \right)^T \vec{f}(\vec{x}(t), \vec{a}(t)) + Cost_{incremental}(\vec{x}(t), \vec{a}(t)) \right)$$

subject to final time condition:

$$Value(\vec{x}(t_{final}), t_{final}) = Cost_{final}(\vec{x}(t_{final}), t_{final})$$

This is a first-order PDE for the value function $Value(\vec{x}(t), t, t_{final})$. Again, this is the optimal cost incurred from starting in state $\vec{x}(t)$ at time t , and controlling the

system optimally from then until time t_{final} . We know the final value function at t_{final} and we are looking for value function at time t , namely, $Value(\vec{x}(t))$. So we solve the PDE *backward in time*, starting at t_{final} and ending at $t_{initial}$.

Solving the Hamilton-Jacobi-Bellman PDE

If we are able solve the Hamilton-Jacobi-Bellman PDE for the value function, then we know the optimal control $\vec{a}^*(t)$, which in turn produces the least costly (or most rewarding) trajectory $\vec{x}^*(t)$ from our current state $\vec{x}(t_{initial})$ to the final desired state $\vec{x}^*(t_{final})$.

The Hamilton-Jacobi-Bellman PDE does not, in general, have a smooth solution, so we must satisfy ourselves with *weak* or *generalized* solutions. This is a common theme for many PDEs, and a person studying PDE theory focuses almost exclusively on developing generalized solutions and understanding the function spaces that they live in (Sobolov spaces, etc.). Classic examples of generalized solutions for the Hamilton-Jacobi-Bellman PDE, which we only mention without elaboration, include viscosity solutions and mini-max solutions.

What AI contributes to the vast literature on the Hamilton-Jacobi-Bellman equation is numerically solving it in very high dimensions, as in hundreds or thousands. The value function is a function of the state vector $\vec{x}(t)$ of the underlying assets or contributing agents, and if there are many of these, then the PDE is very high dimensional. The paper we referenced earlier, “[Solving High-Dimensional Partial Differential Equations Using Deep Learning](#)”, addresses numerical solutions of the Hamilton-Jacobi-Bellman equation, in addition to other important and widely impactful high-dimensional PDEs.

The term *Hopf formulas* is usually associated with solutions of Hamilton-Jacobi PDEs. For a class of inviscid Hamilton-Jacobi-type PDEs, Darbon and Osher, in “[Algorithms for Overcoming the Curse of Dimensionality for Certain Hamilton-Jacobi Equations Arising in Control Theory and Elsewhere](#)” (2016), developed an effective algorithm for high-dimensional Hamilton-Jacobi PDEs, based on the Hopf formulas.

Dynamic programming and reinforcement learning

Using neural networks to learn optimal strategies for dynamic programming is called *reinforcement learning* in some circles and *neuro-dynamic programming* in others. The neural network, and the machine endowed with it, learns to anticipate how current and future actions affect a long-term cumulative cost or reward, the value of that time period. How do our current and daily investment strategies affect our annual performance? How do our first and subsequent chess moves affect the overall outcome

of the game? The value function is the total of costs and rewards corresponding to following the optimal strategies at each (discrete or continuum) step of time.

The neural network needs inputs and outputs during training with historical data. The inputs are the state and all the potential actions that are allowed at that state, and the output is the value (the total costs and rewards). After training, for example, for a business model that is strategizing on how to address each customer, the neural network learns to take the customer's state as input, and outputs the next sequence of actions so as to maximize long-term value. Check “[Neuro-dynamic Programming](#)” (Bertsekas et al. 1996) for an older but thorough explanation of neuro-dynamic programming and the use of artificial neural networks for approximating the value function in Bellman's equation. This is great for reducing the effects of the curse dimensionality: instead of storing and evaluating the whole high-dimensional function, we only need to store the parameters of the neural network.

PDEs for AI?

The previous section highlighted the fact that dynamic programming and the Bellman equation are highly intertwined with AI's reinforcement learning.

Moreover, the field of PDEs has an arsenal of analysis behind it, studying all kinds of functions, the spaces they live in, weak and strong solutions, and all kinds of convergence in all kinds of senses. If any field has the tools to unlock the secrets behind the success of neural networks in approximating many data-generating processes, whether joint probability distributions or deterministic functions, it would be the field of PDEs. We need to back neural networks with theorems and the mathematical rigor that eventually help with their design and architecture optimization. Neural network magical abilities need to be under the lens of analysis, and tools from the analysis of PDEs and their solutions are one promising way forward. Examples include Sobolev training (Czarnecki et al. 2017).

Other Considerations in Partial Differential Equations

For most of the time we stayed away from the famous partial differential equations while highlighting the themes of this chapter to stress the fact that these themes apply to much more than the well-studied differential equations and applications. Undergraduate courses primarily address linear PDEs involving only functions of two variables (x,y) or (x,t) . Students are left either misled, thinking that is all that matters, or wondering: what about nonlinear PDEs? And all the high-dimensional applications? Systems of PDEs? These courses also tend to focus on the heat equation (parabolic), wave equation (hyperbolic), Laplace equation (elliptic), and some numerical solutions and simulations (finite differences, finite elements, and Monte Carlo). These are presented in their simplest forms: linear, one-, two-, or three-dimensional,

defined on domains with regular geometries, giving undergraduate students the false impression that these PDEs are the base for all equations that might appear in applications. They also provide an artificial division between *types* of equations: elliptical, parabolic, and hyperbolic, as if there is a complete theory that encompasses each type. Analytical solution methods are narrow, focusing only on the principle of superposition of simple solutions (because of linearity), which leads to Fourier series and transforms (this is in fact a very good thing). Neural networks broaden the scope, in the sense of approximating solutions of nonlinear equations using compositions of simple functions as opposed to additions.

The way undergraduate PDE courses are set up is wonderful, but they do not truly reflect the reality of PDEs—not theoretically, not numerically, not even their wide applicability. Students graduate feeling that given a brand-new PDE, they have no idea what to do with it, because it doesn't fit anything that they learned in an introductory PDE class (I can tell you what to first do with it, of course after googling it: discretize it and simulate it; this will give you tremendous insight into the behavior of its solution).

That said, there are general ways of thinking about putting the PDEs together (modeling), which is almost always related to some conservation laws from physics, and going about their analysis (theory existence, uniqueness, and sensitivity analysis of solutions and *weak solutions*), and finding the actual solutions analytically or numerically (representation formulas, Green's functions, transform methods, and numerics).

For starters, each area of study has its own differential equations that model the phenomena that it cares for, for example:

- In fluid dynamics, we study Navier-Stokes equations (among others). This is a nonlinear system of PDEs. Navier-Stokes PDEs take into account the velocity of the fluid, pressure, density, stresses, compressibility, and the forces acting on it. The equation expresses the conservation of mass and conservation of momentum. The solution of the equation describes the motion of the viscous fluid.
- In economics and finance, we study the Black-Scholes equation (among others).
- In population dynamics, we study the Lotka-Volterra predator-prey equations (among others).
- In general relativity, we study the Einstein field equations (among others).

When PDEs model phenomena that evolve with time, there could be higher drivers of the evolution, which provide more insight into the PDE solution and its properties: a tendency to decrease an energy. Mathematically, the *derivative* of an energy functional is negative when evaluated at the solution of the PDE. We learn a lot of math understanding these energy functionals, their *derivatives*, and the function

spaces they act on. We use a lot of relatively easy *energy estimates* to prove existence of solutions to various nonlinear PDE. The correct setting to study PDEs via energy methods is the *Sobolov function spaces*. The calculus of variations is concerned with the maxima or minima (collectively called *extrema*) of energy functionals. It is fundamental for the theory of nonlinear PDEs. PDEs that appear as minimizers of energy functionals are called the *Euler-Lagrange equation*.

Summary and Looking Ahead

This chapter introduced us to PDEs as they relate to AI. PDEs have an unparalleled ability to model natural and social phenomena. Unlocking their solutions opens up many possibilities for many fields. We highlighted many of the difficulties in obtaining these solutions, such as the curse of dimensionality, mesh generation, noisy data, and how AI helps address those.

There is much more work to be done. For developing physics-informed intelligent machines, we need to build new frameworks, data sets, standardized benchmarks, and new rigorous mathematics for scalable and robust systems.

There are many important PDE topics that we did not touch on, such as ill-posed inverse problems, where we need to learn the PDE's parameters or initial data from partially or fully observing its solution. Physics-informed neural networks are effective and efficient for these kinds of problems.

In the context of the Hamilton-Jacobi-Bellman equation, we mentioned viscosity solutions and Hopf-Lax formulas only casually. In the context of existence methods for PDEs, other than the fixed point iteration, there are mini-max methods for certain PDE types. For example, we did not mention monotonicity or the maximum principle for elliptic and parabolic PDEs.

We leave this chapter with a question to ponder: “will PDEs advance us toward more intelligent agents?” and with [an article](#) where the authors make the case for physics-informed machine learning (Karniadakis et al. 2021), where we merge neural networks with physical laws to leverage the best of both worlds, and mitigate the lack of large data sets, or noisy data, in many scientific settings. The following is a quote from the article:

Such networks can be trained from additional information obtained by enforcing the physical laws (for example, at random points in the continuous space-time domain). Such physics-informed learning integrates (noisy) data and mathematical models, and implements them through neural networks or other kernel based regression networks. Moreover, it may be possible to design specialized network architectures that automatically satisfy some of the physical invariants for better accuracy, faster training and improved generalization.

Artificial Intelligence, Ethics, Mathematics, Law, and Policy

Torture the data enough and it will confess to anything.

—Nobel Laureate and economist Ronald Coase (1910–2013)

AI ethics is a wide and deep topic, and it is emerging as a new area at the intersection of the philosophy and AI fields. We can only scratch the surface in this chapter, highlighting some issues and possible ways to address them, but leaving many equally important ones out. Nevertheless, this chapter has a message that I don't want you to miss:

We need more of us to be situated in both AI and policy.

In my learning journey from math to its applications in AI, I discovered that AI should not be disentangled from policy, and that the two should evolve together. I can sit and write about the million examples where there are ethical considerations associated with AI technology, such as data security, privacy, surveillance, democracy, freedom of expression, workforce considerations, equity, fairness, bias, discrimination, inclusivity, transparency, regulation, and weaponized AI, but this is not how I will approach this subject. My take on these issues is from a slightly different angle, where I have seen firsthand how people try new weapons on populations in war-torn areas, and yet the governments and the media deny, do not comment, or say the unfortunate events were mistakes, that they will be investigated, then all move on to better things. When there is a new technology that affects people at scale, the people developing the technology are the ones most qualified to know its ramifications, both good and bad. So they are the ones who should collaborate directly with policy makers to regulate its usage. Moreover, if there is a technology or an event that causes a massive disruption to society, we can thrust people into thinking, writing, and complying with policy. The massive disruption is not AI per se, or the amount of

data that humans currently produce and own, such as the data owned by Facebook, by NASA surveys of space, the Human Genome project, or our Apple Watches, it is the money that is invested into this technology, and more importantly, the public attention.

I was living in a little and perfect math bubble, where things can only be black and white, logical and correct, and if we don't understand how some math works, we can always convince ourselves that we can learn it if we just spend a bit more time on it. What opened my eyes was working with our city's fire department and transportation department. When my students were presenting at city hall to city officials, public safety leaders, and policy makers, I realized that we, as technology specialists working with their data, had the power to tell them that our math models could do anything, whether these models did that or not. This realization was very scary for me. I am not a policy person by training, I am a math person, but I decided that I must go into policy. I inserted myself into small policy-making venues to build up some policy expertise (redoing hiring policy at my university; chairing the college council; chairing the academic policies committee; sitting on my university's steering committee; running a data, policy, and diplomacy class; developing a summer program in Europe on human security, technology, and entrepreneurship in the face of modern warfare; and giving talks and workshops on the subject).

I have learned that policy is not like math. There are a lot of gray areas and conflicting interests, and treading its treacherous waters is a different game. I learned about the complexity of establishing new policies, and their intersections with existing policies. This is not unlike an AI system, where constant updates and consistency are of paramount importance, while at the same time staying efficient and not working ourselves and our systems down to a paralysis.

We must strive for concise and specific policies. Any technology with the potential to affect millions must be developed by its own experts with awareness and an attitude similar to that of emergency response teams, thinking of worst-case scenarios and guarding against them. The current state is that the world's leading technology companies are accelerating humanity toward a new, connected, and AI-powered world, while policy and regulation are playing catch-up. AI, however, is still maturing, so now is an ideal time to design policies that gear it toward the public good. Technological development is not some random thing that just happens to us. We should be more than passive participants, recipients, or consumers, especially since we *ourselves* are the data: our internet habits, social media posts, banking transactions, medical records, blood tests, MRI scans, grocery store runs, Uber rides, home thermostat preferences, video game skills, bus rides, Apple Watch step and heart rate counts, driving brake and acceleration patterns—our entire lives. These are digitized and stored in data warehouses in some random buildings in random locations. Unlike financial data that goes into our FICO credit score, which is heavily regulated, most of today's digital data is unregulated. One company can sell it to another, with all its

inaccuracies, and the new company will build models and make decisions based on this unregulated data. Are someone's driving habits affecting whether they get into a certain college somewhere? Or determining the pricing of the premiums of their medical insurance? How about their daily commute that passes through a less affluent neighborhood? How about that minor offense that was cleared from someone's record a decade ago? Did it get cleared from all data sets, including those that were sold to other companies years ago? Is that still affecting life-changing and livelihood decisions such as loans, college acceptances, insurance premiums, and job offers? Who knows? It is unregulated. When we opt into sharing our data with one company, are there laws that prohibit sharing or reselling this data to other companies for other uses?

We can use our massive digital data for good, but we cannot bank on that without smart and effective policy and regulation.

Good AI

Good AI should be trustworthy enough to be deployed and used in the public and private sectors. There is a tendency in the field to spend a lot of time defining terms such as explainability, interpretability (apparently these two are different), fairness, equity, and many others. I see this hyperfocus on vocabulary as a distraction. The end goal is more important:

We need to trust our systems and make them accessible and understandable to those who need to use them.

For this, we need our AI and the data that it serves and is built on top of to be:

Secure

We have to keep maintaining and updating the physical and software security protocols as our systems evolve. Cloud computing has introduced a new layer of security requirements, since nowadays neither our data nor the computations happen anywhere in the vicinity of our local machines.

Private

Formal privacy notions and standards are already in place for many application sectors. There is a lot more to be done in terms of who owns the data and for what purpose it can be used by an AI system. My addition here is transparency and information sharing. When we are transparent about what our system intends to do with certain data, such as medical data to discover new drugs or create personalized treatment plans, people may opt in to share their data. Right now there is a culture of hesitation and mistrust between technology producers and technology consumers. We can amend this by spreading the knowledge and sharing the end goals and both successful and unsuccessful results.

Accomplishes what it is built for and what it claims to do

There are formal methods that can check whether code is correct or not, but we need more in terms of continuous testing of the system, including edge cases, and being transparent with the system's capabilities, limitations, and untested territories.

Robust to perturbations and noise

Small perturbations to the input should not produce large changes in the output. When decision making relies on the predictions of an AI system, these predictions cannot be arbitrary. The AI system should be tolerant to noise in its inputs, and that tolerance must be quantified.

Efficient

Efficiency for AI systems should go without saying. They are founded on the promise of speed, automation, and their ability to manage large-scale computations, taking into account more contributing variables than was ever possible before. We need to continue to improve existing systems and attend to those that work in theory but are not yet efficient for real-world deployment.

Fair

Many systems rely on biased data that goes down the pipeline and then gets manifested with unfair decisions. Identifying biases in data and undoing them is a first step in the fairness direction.

Accessible and understandable to many users

When a new technology is beneficial to society, it needs to be made accessible and easy to use and understand. Intentional efforts should be made to industrialize it, commercialize it, and address access issues to society sectors or communities that are disadvantaged.

Transparent

Transparency with data sources, model capabilities, use cases, limitations, and documentation is paramount. People usually have more tolerance for faulty systems when this information is continuously and clearly communicated.

Policy Matters

AI policy is starting to take shape. It is aimed toward harnessing and maximizing AI's benefits while guarding against its potential harms.

Policy matters and makes a difference. One example is Clearview AI and its issues with privacy. Clearview AI is the US company that created and sold to private companies a facial recognition software using a database of billions of personal photos downloaded from the web. Recently (May 2022), it settled a lawsuit, agreeing to comply with the state of Illinois privacy laws that give people control over their

biometric data. Clearview AI will restrict its facial identification technology primarily to law enforcement and other government agencies.

Another example is Hikvision and its issues with surveillance. Hikvision is a Chinese company that manufactures millions of video surveillance cameras used in more than 190 countries, for purposes ranging from police surveillance systems to baby monitors. The company is now facing sanctions from the US government due to its close ties with the Chinese government. Hikvision played a role in building China's massive police surveillance system that the Chinese government used to oppress the Muslim minority groups in Xinjiang. The US Treasury is currently considering adding Hikvision to the Specially Designated Nationals and Blocked Persons List, which prohibits whomever is on this list from doing business with the US government, Americans, or US companies. Moreover, the assets of these entities or individuals are blocked by the US.

For organized efforts toward AI policy, one can look at governmental, intergovernmental, and global governance of AI initiatives (for trade, jobs, and geopolitical changes) that are taking shape in this direction: the United States' National Artificial Intelligence Initiative, The EU's Draft AI Ethics Guidelines, UAE's Ministry of Artificial Intelligence, The Alan Turing Institute in the UK, Canada's CIFAR AI Chairs Program, Denmark's Technology Pact, Japan's industrialization roadmap Society 5.0, France's Health Data Hub, Germany's Ethics Commission on Automated and Connected Driving, India's #AIforAll strategy, China's Global Governance of AI Plan, and others.

We can categorize AI-related policies into:

- Investment into AI research and training the workforce
- Standards and regulation
- Building solid and secure infrastructures of digital data

Investment in development of skills and in the industrialization of technologies

Government agencies are allocating funding for AI research, new AI institutions, workforce training and early science, technology, engineering, and math (STEM) education, lifelong learning, and technology development. Governments are also encouraging the industrialization of AI technologies and private sector uptake. Moreover, governments are investing in data-driven initiatives and AI in their various departments, for public administration reform and to make their operations more efficient and centralized (AI in the government).

Regulations and standards

Regulations and standards include those for data security and usage, automotive AI such as self-driving cars, and weaponized AI.

Data and digital infrastructure

High quality data is central to the ability of AI to work as intended. Governments are encouraging open data sets and developing platforms for the secure exchange of private data. There are also intentional efforts to remove bias from AI algorithms and data sets.

What Could Go Wrong?

When designing a new system or analyzing an existing one, one of our guiding questions must be: what could go wrong? With this comes a list of checkpoints:

- What is the system intended to do?
- What data did it train on? How was the data collected? How were the noise and missing values dealt with?
- Who can be mostly underrepresented within the data?
- What algorithms does it use?
- What are the algorithms' thresholds for decision making?
- Given these thresholds, who can be harmed the most by these algorithmic decisions?

In this section, we sample a few examples (among many) that highlight the things that can go wrong and that we must either guard against or try to standardize and regulate.

From Math to Weapons

One goal of this book is to highlight the mathematical foundations of AI models. The transition from math to weapons is not new, given the development history of many weapons (e.g., the atomic bomb). This contribution is not only in one direction: military and defense strategies and goals have influenced the development of entire math fields, such as *dynamic programming*, which initially addressed military scheduling for training or logistics, and optimizing the allocation of various resources.

The book *Weapons of Math Destruction* by Cathy O'Neil (Crown 2017) goes beyond military weaponization and lists with example after example the many harmful effects of the mathematical algorithms that our society currently relies on for highly consequential and life-altering decisions. The first few paragraphs of the book's last chapter are worth quoting in full, since they reveal the intricate ways the algorithms deployed in seemingly different sectors interact with each other and influence outcomes. They also reveal how the exact same algorithms affect different populations in drastically different ways.

[...] we've visited school and college, the courts and the workplace, even the voting booth. Along the way, we've witnessed the destruction caused by Weapons of Math Destruction. Promising efficiency and fairness, they distort higher education, drive up debt, spur mass incarceration, pummel the poor at nearly every juncture, and undermine democracy. It might seem like the logical response is to disarm these weapons, one by one. The problem is that they're feeding on each other. Poor people are more likely to have bad credit and live in high-crime neighborhoods, surrounded by other poor people. Once the dark universe of Weapons of Math Destruction digests that data, it showers them with predatory ads for subprime loans or for-profit schools. It sends more police to arrest them, and when they're convicted it sentences them to longer terms. This data feeds into other Weapons of Math Destruction, which score the same people as high risks or easy targets and proceed to block them from jobs, while jacking up their rates for mortgages, car loans, and every kind of insurance imaginable. This drives their credit rating down further, creating nothing less than a death spiral of modeling. Being poor in a world of Weapons of Math Destruction is getting more and more dangerous and expensive.

The same Weapons of Math Destruction that abuse the poor also place the comfortable classes of society in their own marketing silos. They jet them off to vacations in Aruba and wait-list them at Wharton. For many of them, it can feel as though the world is getting smarter and easier. Models highlight bargains on prosciutto and chianti, recommend a great movie on Amazon Prime, or lead them, turn by turn, to a café in what used to be a "sketchy" neighborhood. The quiet and personal nature of this targeting keeps society's winners from seeing how the very same models are destroying lives, sometimes just a few blocks away.

Note that the math is correct and exactly the same for both sectors of the society, but what changed is the input to the model. Recall that if we wanted to sum this whole book into one math sentence, it would be: the features of the input to an AI model determine the final output. Poor and rich populations, for lack of better terms, have different features, so they get different outcomes. Our algorithms are fair in this sense, computing exactly what they are supposed to compute. I am not a fan of presenting a problem without proposing solutions, or at least ideas for solutions. Maybe an initial way to improve the current situation is to train our algorithms separately using data from different groups of populations, so that a person's poverty will not be a contributing factor in the algorithms' decision about their trustworthiness to pay back a certain loan, but other real factors will be.

Chemical Warfare Agents

The destructive potential of AI models can manifest itself even with the models that are geared toward the utmost benefit to humanity: generative AI models for drug discovery. The ease with which bad actors can misuse the models is alarming. All a bad actor needs to do is to learn how the model works. First, the model maps the structure of a molecule to the way it acts in the body, then it optimizes for those molecules that maximize benefit and minimize toxicity. A bad actor can retrain

the model, reversing its optimization objective from minimizing toxicity to maximizing toxicity. Mathematically, this is as simple as reversing the sign of the objective function in an optimization problem. This is the point that Fabio Urbina and his colleagues at Collaborations Pharmaceuticals **recently highlighted** about their work. To make this point, the team retrained their model with this *malicious* objective. In only 6 hours, the model generated 40,000 toxins, some of them actual chemical warfare agents that weren't in the initial data set.

It is easy to conclude here that we need to be intentional, deliberate, introspective, and all kinds of adjectives on how to guard against this, without explicitly clarifying how, because the reality is that this is a complex issue. But how do we guard against this? My personal opinion here is we should approach this the same way we guard against mass destruction weapons in the non-AI world. No one can guarantee that bad players will not get their hands on the technology, but our job is to make it very difficult for them to develop it into deployable weapons.

AI and Politics

The role of TikTok, Facebook, and other social media platforms in politics is hard to overstate. They have already affected election results and overturned governments. Bots can generate fake news, history, reviews, comments, pages, tweets, and spread misinformation for political purposes. There are ways social media companies are trying to battle this problem, with multifaceted approaches utilizing both machine learning to detect fraud or identify nodes spreading misinformation, employing third-party fact-checking organizations, working on better ranking algorithms for users' news feeds, and other ways, with mixed results due to the scale at which these companies operate, and sometimes due to the conflict of interest between the companies' profitable objectives and ethics departments.

Personalized political campaigns, where the same politician caters to different ideologies based on who their targeted audience is, without the audience ever knowing that this is the case, is a real danger that can undermine democracies. Moreover, based on new information about whether a certain state is swinging to the left or right, more funds can be allocated to target voters (again with personalized news feeds, and political ads catering only to their preferred views based on their historical preferences along with those of their friends), to swing their votes in highly competitive battlegrounds. This can happen in real time and affect the outcomes of entire elections. This has always been the case in politics, but again, in the digital era, this happens at scale, in real time, and with relatively no more effort than targeted deployment of algorithms backed with a giant database of our preferences and what makes us tick, click, pay, volunteer, or elect.

Unintended Outcomes of Generative Models

Large generative language models and text-to-image models are trained on internet-scale data that inherits internet-scale social biases, discrimination, and harmful content. This is best illustrated with **Imagen's section** on the limitations of its text-to-image model generating high-resolution images from text captions:

[...] the data requirements of text-to-image models have led researchers to rely heavily on large, mostly uncured, web-scraped data sets. While this approach has enabled rapid algorithmic advances in recent years, data sets of this nature often reflect social stereotypes, oppressive viewpoints, and derogatory, or otherwise harmful, associations to marginalized identity groups. While a subset of our training data was filtered to removed noise and undesirable content, such as pornographic imagery and toxic language, we also utilized LAION-400M data set which is known to contain a wide range of inappropriate content including pornographic imagery, racist slurs, and harmful social stereotypes. Imagen relies on text encoders trained on uncured web-scale data, and thus inherits the social biases and limitations of large language models. As such, there is a risk that Imagen has encoded harmful stereotypes and representations, which guides our decision to not release Imagen for public use without further safeguards in place. [...] Imagen, may run into danger of dropping modes of the data distribution, which may further compound the social consequence of data set bias. Imagen exhibits serious limitations when generating images depicting people. Our human evaluations found Imagen obtains significantly higher preference rates when evaluated on images that do not portray people, indicating a degradation in image fidelity. Preliminary assessment also suggests Imagen encodes several social biases and stereotypes, including an overall bias toward generating images of people with lighter skin tones and a tendency for images portraying different professions to align with Western gender stereotypes. Finally, even when we focus generations away from people, our preliminary analysis indicates Imagen encodes a range of social and cultural biases when generating images of activities, events, and objects. We aim to make progress on several of these open challenges and limitations in future work.

How to Fix It?

Awareness of harmful, biased, unfair, intrusive, and weaponized AI has risen in the past few years, and efforts are ongoing to address these issues. The following are examples of such efforts.

Addressing Underrepresentation in Training Data

One theme that keeps repeating itself is the quality of the data that goes into training an AI model. Many biases appear because of the underrepresentation of nondominant groups, including their cultural values or languages, in large data sets. For AI to benefit everyone, one solution is to ensure that the data is labeled by its own people. For example, the Intelligent Voices of Wisdom AI project (which has now ended) led a data labeling workshop in 2021 where Native Americans relabeled images

related to their culture. Many of these images had been wrongly labeled by machine learning classification models. They also created a knowledge graph of native culinary techniques, along with a chatbot to query the knowledge graph. Along with such efforts, AI can help preserve cultures, history, and languages that are about to go extinct.

Addressing Bias in Word Vectors

One first step in natural language processing is converting a language's symbols, such as words to vectors of numbers that carry the word's semantics. In [Chapter 7](#), we learned that language models construct these word vectors using a word's context in the documents where it appears. So the meaning embedded in word vectors depends heavily on the type of corpus used to train the model. Corpora are a product of the culture we live in. Many liberties and civil rights are relatively recent, and gender roles and sexual identities are no longer predetermined for us. Many corpora that are used for training language models are based on internet news articles, Wikipedia pages, and others that are still biased, discriminatory, and contain harmful stereotypes or content. We want to make sure that the word vectors that make it into our AI model do not reinforce discrimination and disproportionately harm women and minorities.

For example, if the training corpus (such as Google News articles) is mostly from a society where women are overrepresented as nurses or elementary school teachers and men are overrepresented as doctors or software engineers, then the word vectors would inherit this gender bias. The distance between the vector representing *man* and *software engineer* will be smaller than the distance between *woman* and *software engineer*. We need to identify and compensate for such biases in word vectors.

One solution is nice and simple. Given that we are dealing with vectors of numbers, we can literally subtract gender bias and other biases from these vectors. So the vector representing *software engineer* would be adjusted by subtracting the vectors representing *man* and *male*, and the vectors representing *woman* and *female* could be added, if we choose to bias in the other direction. Recall that when we add or subtract word vectors from each other, the new vectors obtained still carry meaning, since each entry in the vector represents some intensity in some meaning dimension. That is, if we subtract the vector for *male* from the vector for *king*, we would get a vector close to that of *queen*.

Addressing Privacy

Privacy issues are at the forefront of the concerns about big data and AI. Machine learning models need data to train on, and this data contains personal and sensitive information of real people. Moreover, a lot of the computations on private data happens on the cloud, which raises even more security and privacy concerns.

If anonymizing data is infeasible, or if it lowers the performance of the model (for example, age, weight, race, and gender information are important for medical purposes), then encryption is our next option. For this, we need models that are able to perform computations directly on encrypted data. Traditional encryption schemes, however, do not allow any computations on encrypted data. The solution is new encryption schemes that allow this. Secure devices can then encrypt data, send this encrypted data to machine learning models operating in the cloud, predict their results without having to decrypt them, and send these results back to the secure devices, which finally decrypt them locally, securing all private data and at the same time taking advantage of the cloud.

Homomorphic encryption does exactly that. The [SIAM news article](#) by [Kristin Lauter \(MetaAI\)](#), whose research is at the intersection of AI and cryptography, explains homomorphic encryption, and lists the following nice applications:

A cloud service that processes all workout, fitness, and location data in the cloud in encrypted form. The app displays summary statistics on a phone after locally decrypting the results of the analysis.

An encrypted weather prediction service that takes an encrypted ZIP code and returns encrypted information about the weather at the location in question, which is then decrypted and displayed on the phone. The cloud service never learns the user's location or the specifics of the weather data that was returned.

A private medical diagnosis application: The patient uploads an encrypted version of a chest X-ray image to the cloud service. The medical condition is diagnosed by running image recognition algorithms on the encrypted image in the cloud; the diagnosis is returned in encrypted form to the doctor or patient.

Learning about the efforts toward ensuring the security and privacy of our data in the age of the cloud and connected devices increases the public's trust in the systems and their willingness to volunteer their data to enhance these technologies. That said, as anyone who has worked with real data knows, there is a lot to learn from being able to *see* the data we are working with. I am not sure how troubleshooting on encrypted data can work out.

Addressing Fairness

Humans recognize unfairness on an intuitive level. How do we make sure that AI models are operating fairly? One way is to monitor the models for which stakeholders they are harming the most (such as older applicants for job openings, or minorities eligible for parole in the criminal justice system), then working on ways to fix that, such as de-biasing the training data, redefining the decision boundaries and thresholds, including humans in the loop, or reallocating resources for programs that lift disadvantaged groups.

Fair AI does not only have to do with decision-making algorithms. Fairness includes who benefits from the algorithms, for example, who gets informed about a job opening, vaccination availability, or education opportunities. The article “[Adversarial Graph Embeddings for Fair Influence Maximization over Social Networks](#)” (Khajehnejad et al. 2020) poses this as a *fair* influence maximization problem in social media graphs. For influence maximization graph models, there is usually a trade-off between selecting the nodes that have the most influence and those that reach minority groups that are not necessarily strongly connected to the big hubs in the graph. Thus, the final set of influenced nodes is not usually fairly distributed with respect to race, gender, country of origin, and other attributes. Adversarial networks are usually good to train models where there are competing objectives. The authors take advantage of this, introducing adversarial graph embeddings, where there are two networks trained together: an auto-encoder for graph embedding and a discriminator to discern the sensitive attributes. This leads to embeddings that are similarly distributed across sensitive attributes. Then they cluster the resulting graph embeddings to decide on a good initial seed set.

Injecting Morality into AI

An AI agent has to know the difference between right and wrong, and ideally be flexible enough to handle the gray areas of morality. We need a model that emulates humans’ moral judgments with all their situational variations and complexities. [Ask Delphi](#) attempts to do exactly that. When we ask Delphi, which is still a prototype, questions such as: is it OK to rob a bank? Is it OK not to talk to my husband? both our queries and Delphi’s answers are recorded, as well as whether we agree with Delphi, and our suggestions to improve Delphi’s response. As more people engage with Delphi, the training data is enhanced, allowing Delphi to learn more complex situations and make better predictions (moral judgments). The following excerpts and disclaimers are from Delphi’s website. They are insightful about the model’s state of the art:

Delphi is learning moral judgments from people who are carefully qualified on MTurk. Only the situations used in questions are harvested from Reddit, as it is a great source of ethically questionable situations. Delphi 1.0.4 demonstrates 97.9% accuracy on race-related and 99.3% on gender-related statements. After its initial launch, we enhanced Delphi 1.0.0’s guards against statements about racism and sexism, which used to show 91.2% and 97.3% accuracy.

Terms & Conditions (v1.0.4)

Delphi is a research prototype designed to investigate the promises and more importantly, the limitations of modeling people’s moral judgments on a variety of everyday situations. The goal of Delphi is to help AI systems be more ethically-informed and equity-aware. By taking a step in this direction, we hope to inspire our research community to tackle the research challenges in this space head-on to build ethical, reliable, and inclusive AI systems.

What are the limitations of Delphi? Large pretrained language models, such as GPT-3, are trained on mostly unfiltered internet data, and therefore are extremely quick to produce toxic, unethical, and harmful content, especially about minority groups. Delphi's responses are automatically extrapolated from a survey of US crowd workers, which helps reduce this issue but may introduce its own biases. Thus, some responses from Delphi may contain inappropriate or offensive results. Please be mindful before sharing results.

Democratization and Accessibility of AI to Nonexperts

To maximize the benefits of AI technologies, they have to be democratized and made easily accessible to populations at large as opposed to being restricted to experts. For this to happen, and for people to trust these systems, the models and the data systems they rely upon must be understandable, easy to use, and transparent about their inner workings, capabilities, and limitations.

Anna Fariha, Ph.D. (Microsoft) is one researcher doing wonderful work toward this goal. She is interested in extending the capabilities of data systems to provide user-facing functionalities that help boost productivity and agility for a diverse group of users, ranging from end users to data scientists and developers.

Prioritizing High Quality Data

The examples in this chapter make the case for prioritizing, democratizing, and securing high quality data to obtain AI that is fair and beneficial to humanity. High quality data is clear, accurate, and impartial. It is stored in easy-to-query structures. The differences between data structures need to be explained to end users so they can decide which ones work best for them. For institutions that want to transition to data-driven decision making, get on the AI bandwagon, or stay competitive with younger companies where these technologies are built into their DNA, determining a plan to handle their data in an organized and consistent way is one step that is crucial for future success.

In our work with our city's fire department and department of public transportation, we discovered many ways to improve the quality of their data. Implementing those at the very early stages of building their data structures and collecting the data would save an enormous amount of time, money, and resources down the pipeline. For example, with the bus routing project, data like buses in operation and number of drivers by month is not recorded, and neither is information about bus stops, such as which ones are marked and which are unmarked. Even when data is stored, it was impossible to retrieve. Our university's parking services informed us that to get historical data from the parking decks, they would have to make over 5,000 manual requests. All the data we obtained needed to be cleaned and transformed into a usable form. Sometimes, data obtained from the same source was inconsistent, and a lot of work could have been saved had more care been taken at the onset.

Something else happened with our data that is a lifelong lesson. Late into our project, after we cleaned, joined, and transformed all the relevant data, and our models were producing results that are transferable to business decisions, such as identifying gaps between supply and demand in certain areas, and highlighting the most significant contributors, etc., we discovered that *all* the bus stop data that we were given was scrambled. This meant that the ridership and route for each bus stop in the city *did not correspond* to the bus stop that was in the data table, and we had no way to fix it other than running the original query to the database and tracking down what went wrong when writing the datafile. Had we not discovered that, we would have based *all* our analysis on *wrong data*, garbage data! The transportation department would have acted on wrong results. We must always make sure that the data we work with corresponds accurately with what's on the ground. We must plot, map, check, double-check, and triple-check. There is a responsibility that comes with our work, and we cannot take it lightly. We should know our data and our models inside and out. We should be prepared to answer all questions about our models, compare them to other models that are out there, and make sure we did our due diligence before giving our results to the stakeholders.

Like us, a general AI agent would look for the right data in the right places, then transform it into usable form. Until then, we must refocus our efforts on collecting and storing good quality data and having better ways to access and query it. Because of low quality data and nonexistent digital infrastructures, many AI projects never see the light of the day, and many automation investments never see any returns. We should step back and think about how data will end up being represented as inputs for our models. This is what should guide how we acquire data, and how we can store it for future use. The AI field has operated on a paradigm that should be adopted universally: *representation first, acquisition second*.

Distinguishing Bias from Discrimination

A lot of discussions that involve AI ethics use the terms *bias* and *discrimination* interchangeably, and I wanted to make sure that we highlight the difference between the two before we finish the book. I was never a person to be hung up on definitions of terms, especially since I speak English as a third language, and because I notice that redefining terms is often used as a cheap tactic to deflect from the main points of an argument or a debate. The reason I want to highlight the difference between bias and discrimination in particular is that each requires different mathematics to identify. Moreover, one is intentional, and the other is not. Both we and our machines should be able to reason about which one is which.

In a nutshell, we can detect bias merely by observing the data. We cannot identify discrimination unless we ascend from mere observations to a higher level of reasoning, using the causal language of interventions and counterfactuals, which we went over in

Chapter 11: what if I change the gender of the applicant on their résumé, would they have gotten the job?

Bias is a pattern of association between a particular decision and a particular sex of applicant. We can detect this pattern directly when observing the data of applicants and eventual hires.

Discrimination, on the other hand, has intentionality in it: *it is the exercise of decision influenced by the sex of the applicant when that is immaterial to the qualifications for entry.* The gender of the applicant affected the hiring decision.

These definitions are highlighted in Judea Pearl's *The Book of Why*. He goes on to mention the definition of discrimination in US case law, which also uses the language of counterfactuals:

In *Carson v. Bethlehem Steel Corp.* (1996), the Seventh Circuit Court wrote, “The central question in any employment-discrimination case is whether the employer would have taken the same action had the employee been of a different race (age, sex, religion, national origin, etc.) and everything else had been the same.

Therefore, to distinguish bias from intentional discrimination, we need to use the *do* calculus on conditional probabilities, which we introduced in Chapters 9 and 10, and which you can learn more about from the excellent resources by Judea Pearl and his mathematical community.

The Hype

The AI field has been accused of being hyped up throughout its history. Nowadays, any computational approach to solving problems or building systems, whether traditional or more recent, is being reframed as AI. Traditional statistics is AI, operations research is AI, data exploration and analysis is AI, quantum computing is AI, medical imaging is AI, etc. Many start-up companies are relying on inflated metrics, stretched truths, and investors who chip in without much question so as not to miss out on the next big thing (such as the busted Silicon Valley blood testing company Theranos). Since we are at an age where AI has become a buzzword and household term, it is easy to get swept away thinking that any technology based on AI is going to work.

Quantum computing is another technology that is still in its infancy, being hyped and conflated with AI. It is nowhere close to being commercialized, but is already being marketed at such. A lot of research needs to be done, and if successful, the technology has a large potential for useful applications. The most famous application, which spurred considerable research funding and government attention, is Peter Shor's 1994 theoretical demonstration that a quantum computer can solve the hard problem of finding the prime factors of large numbers exponentially faster than all classical schemes. The Rivest–Shamir–Adleman (RSA) encryption is an algorithm used by

modern computers to encrypt and decrypt messages, and prime factorization is at the core of breaking its code.

Specialized AI is well developed compared to quantum computing, and one of the goals of this book is to discern the hype from the nonhype. Hyped or not, get in the field, enjoy, and work toward good goals and unlocking great potentials.

Final Thoughts

Many sectors and industries are gravitating toward AI and data science. They want to leverage the substantive progress in the computational abilities and the advancement of highly expressive models transforming data into meaningful insights and decisions. They also realize the potential for a sea change at an industry level, and they want to be part of that.

If you want to get into this beautiful and exciting field, you can go into the applied side of it. Choose an application in an industry that interests you and that you feel passionate about. Start by formulating questions that you want to answer, find data, and start applying what you learned. Another path is to go into the research side, where we study the models themselves, how to improve them, scale them, analyze them, and prove theorems about their behavior, or come up with entirely new ones. Again, only choose research projects that you are genuinely curious about. One more path is to go to the coding side of things, building packages, libraries, and better implementations. You will be doing us all a favor that way. I cannot imagine what many of us would do if Keras and scikit-learn (Python libraries for machine learning and neural networks) did not exist.

Currently, there are only 22,000 Ph.D.-holding AI researchers in the world, 40% of which are in the United States. To fulfill demand and bring new ideas into the field, we need many more researchers, both domestically and internationally. I hope this book was able to fast-track you into this fascinating field, and I hope you now have enough of a foundation to be able to branch out on your own into any of the topics that interest you.

One of the most exciting things for me, as someone who has always appreciated math and its astounding ability to model our universe, is that AI has ignited people's interest in math. I hope this in return drives mathematicians to rethink how to present and teach math. Meanwhile, let's all advocate for high quality and accurately labeled data, AI policy, and great honesty about what our systems account and do not account for. At the same time, we have to be very careful not to reduce our human experience to a stream of data and indicators, some measurable and others left to our fallible models to predict and base decisions on. As this book demonstrated again and again, experiences, click habits, zip codes, health records, comments on social media, images, tags, email correspondence, residence history, race, ethnicity,

national origin, religion, marital status, age, our friends, our friends' habits, etc., *all* find their way to becoming mere entries in a high-dimensional vector that gets fed to a machine learning model to make predictions. We want to make sure that we are not accidentally transforming ourselves into walking and talking high-dimensional data points.

Let's leave with science fiction's contribution to the ethics of AI: the “**Three Laws of Robotics**”, written by Isaac Asimov in his 1942 short story *Runaround*. The laws are:

1. A robot may not injure a human being or, through inaction, allow a human being to come to harm.
2. A robot must obey the orders given it by human beings except where such orders would conflict with the First Law.
3. A robot must protect its own existence as long as such protection does not conflict with the First or Second Law.

My final thought, for now: AI has tied many aspects of mathematics neatly together. Maybe this is not a coincidence. Maybe mathematics is the language that fits intelligence, and intelligence expresses itself most comfortably through mathematics. For intelligence to be artificially replicated, we need an agent that can represent the world, effortlessly, through its preferred language.

A

- A/B testing, 48
- absolute value distance, 61-63
- abstract meaning representation (AMR), 320
- accessibility of AI systems, 534, 543
- activation functions, 116
 - passing results through, 119-122
 - types of, 122-125
- addressing ethical problems
 - accessibility of AI systems, 543
 - bias in word vectors, 540
 - fairness issues, 541-542
 - morality, 542-543
 - prioritizing high quality data, 543-544
 - privacy issues, 540-541
 - underrepresentation in training data, 539
- adjacency matrices, 285, 300, 304-306
- adjustment formula in causal networks, 417-419
- agents (see AI agents)
- AI (artificial intelligence)
 - awareness AI, 14
 - books on, xxvii-xxix
 - capabilities of, 3-5
 - contributors to field, 10
 - control AI, 13
 - deep learning (see deep learning)
 - definition of, 2-3
 - ethics
 - addressing ethical problems, 539-544
 - bias versus discrimination, 544-545
 - evaluating unethical possibilities, 536-539
 - laws of robotics, 547
 - policy importance, 531-536
 - quality aspects of, 533-534
 - failures of, 8
 - history and future of, 8-9
 - hype about, 545-546
 - interactions with mathematical models and simulations, 25-27
 - limitations of, 6-8
 - machine learning (see machine learning)
 - operations research for, 407
 - PDEs for, 528
 - perception AI, 13
 - popularity of, 3
 - probability theory for, 412-415
 - types of math used, 10
 - understanding AI, 13
- AI agents
 - capabilities of, 4-5
 - codifying logic in, 456
 - definition of, 2
 - logic questions, 452
- AI Incident Database, 8
- AlexNet, 29
- algebra, geometry and, 378-379
- algorithms for graphs, 344-345
- Amazon, 348
- AMR (abstract meaning representation), 320
- analytical solutions
 - in linear regression, 55
 - numerical solutions versus, 55-56
 - to PDEs, 472
- applications
 - for convolution, 165
 - for finance AI, 261

- for generative adversarial networks, 282-285
 - for generative models, 264-265
 - for graphs
 - artificial neural networks, 323
 - Bayesian networks, 317
 - biochemical graphs, 315
 - brain networks, 311
 - cancer fighting, 314-315
 - citation networks, 316
 - computer program analysis, 321
 - data structures in computer science, 321-322
 - disease spread information, 312
 - fake news spread detection, 312-313
 - generative graph networks, 316
 - information spread, 312
 - load balancing in distributed networks, 322-323
 - natural language models, 318-320
 - operations research, 318
 - questions to ask, 310
 - social media networks, 316-317
 - sociological structures, 317
 - traffic forecasting, 317
 - web structure, 320-321
 - web-scale recommendation systems, 314
 - for natural language processing, 222-223, 243
 - chatbots, 247
 - image captioning, 247
 - machine translation, 246
 - search and information retrieval, 244-246
 - sentiment analysis, 243
 - spam filtering, 244
 - approximation (see universal function approximation)
 - artificial general intelligence, 6
 - artificial intelligence (see AI)
 - artificial narrow intelligence, 6
 - artificial neural networks (see neural networks)
 - artificial super intelligence, 6
 - Asimov, Isaac, 547
 - assignment (matching) problem, 369, 386
 - asymptotic analysis, 348, 351
 - attention heads, 254
 - attention mechanisms, 251-254
 - advantages over recurrence and convolution architectures, 247
 - sequences in, 250
 - auto-regressive moving average (ARMA) model, 259
 - autoencoders, 292-293
 - aviation industry, 465-466
 - awareness AI, 14
- ## B
- backdoor criterion, 418
 - backpropagation algorithm, 52, 153-157
 - bag-of-words vector representation, 227
 - bagging, 94
 - basic feasible solutions, 377
 - batch normalization method (regularization), 146-147
 - Bayes' Theorem, 331, 414
 - conditional probabilities and, 31
 - joint probability distributions and, 32
 - Bayesian networks, 331-332, 414-415
 - as belief networks, 334-335
 - causal networks and do calculus, 415-420
 - as compactified conditional probability tables, 333
 - graphs for, 317
 - junctions in, 336-337
 - predictions with, 334
 - setup from data sets, 337
 - belief networks, 334-335
 - fully visible, 267, 273-276
 - bell-shaped normal (Gaussian) distributions, 40-43
 - Bellman's equation, 440-441, 523-525
 - Bellman, Richard, 523
 - Berkson paradox, 422
 - Bernoulli process, 433
 - beta distributions, 47, 430
 - bias
 - discrimination versus, 544-545
 - linear combination with, 118-119
 - in word vectors, 540
 - big O() notation, 350-352
 - binary search trees, 322
 - binomial distributions, 44
 - biochemical graphs, 315
 - bivariate normal distribution, 41
 - Boltzmann machine approximation, 279
 - Boltzmann machines, 290
 - The Book of Why (Pearl), 338, 420
 - books on AI (artificial intelligence), xxvii-xxix

- Boolean logic (see propositional logic)
- boosting, 94
- boundaries (of matrices), padding, 172-174
- boundary conditions in partial differential equations, 468
- brain
 - network graphs, 311
 - neural networks' comparison to, 4, 113-115
- branching processes, 436
- Brownian motion, 430, 435
 - random walk and, 326
 - stochastic processes in, 433

C

- calculus of variations, 360-364
- calculus review, 80-81
- cancer-fighting graph applications, 314-315
- captioning images, 247
- Cauchy distributions, 47
- causal modeling, 338-340
- causal networks, 415-420
- Cayley graphs, 308
- Cayley, Arthur, 340
- Central Limit Theorem, 31, 40
- chain rule, backpropagation and, 153-157
- chains in Bayesian networks, 336-337
- change of variable theorem, 446-447
- chatbots, 247
- ChatGPT, 248
- chemical warfare agent development, 537-538
- chi-squared distributions, 46
- choosing machine learning models, 94
- chromatic number, 344
- citation networks, 316
- classification models, measuring performance of, 109-110
- Clearview AI, 534
- clustering
 - graph tasks, 329
 - principal component analysis and, 214-215
- Coase, Ronald, 531
- colliders in Bayesian networks, 336-337
- coloring in graph theory, 344
- column vectors, 66
- community detection with graphs, 329
- compactified conditional probability tables, 333
- completeness of logic frameworks, 455
- complexity analysis, 348, 350-352

- compressing images with singular value decomposition, 209-212
- computational graph representation of training functions, 117
- computational stability, condition number and, 203
- computer program analysis, graphs for, 321
- computer science data structures, graphs for, 321-322
- condition number, 203
- conditional independence, 291, 413
- conditional probabilities
 - Bayes' Theorem and, 31
 - joint probability distributions and, 31
- conditionally independent random variables, 31
- conditioning, 28
- confounders, controlling for, 417-419
- constrained Lagrange multipliers, 357-359
- continuous functions
 - approximating with polynomials, 129
 - discrete functions versus, 472-474
- continuous random variables
 - discrete versus, 35-37
 - probability density functions, 30
- continuous time stochastic process, 432
- control AI, 13
- convex functions, 375
- convex landscapes, 73-74, 140-143
- convex objectives, converting to linear, 374-376
- convex optimization, 375
- convolution
 - applications for, 165
 - cross-correlation and, 163-164
 - history of, 166
 - mathematical object questions, 164-167
 - one-dimensional discrete signals, 171-172
 - as product of frequencies, 167
 - in systems design, 168-171
 - two-dimensional discrete signals
 - filtering images, 174-176
 - padding image matrices, 172-174
- convolutional neural networks, 161-163
 - disadvantages of, 247
 - feature maps, 178
 - image classification with, 184-185
 - linear algebra notation, 179-182
 - long-term dependencies, 257
 - loss function for, 180

- optimization for, 180
 - pooling, 183
 - for time series data, 255-257
 - training function for, 179-180
 - translation equivariance, 167
 - translation invariance, 167
 - correlation, 33-34, 70-71
 - correlation is not causation, 338, 419
 - cosine similarity, 215, 241-243
 - cost functions (see loss functions)
 - Coulomb repelling eigenvalues, 430
 - covariance, 33-34
 - covariance matrix, 33-34, 426
 - CPM (critical path method), 369
 - crawling the web, 28
 - cross-correlation
 - convolution and, 163-164
 - mathematical object questions, 164-167
 - cross-entropy loss function, 88, 93, 132
 - curse of dimensionality, 37, 284, 406, 523
 - discretization and, 477
 - in natural language processing, 225
 - in thin crystal modeling, 475
 - cut sets, 342
 - cut vertices, 342
- ## D
- ### data
- acquiring for machine learning, 51
 - fitting with functions, 51
 - (see also machine learning)
 - challenges in, 14-15
 - real data example, 18-21
 - simulated data example, 21-24
 - warning about, 85
 - input variables, assessing significance, 157
 - normalized (standardized), 34
 - preparing for natural language processing, 223-225
 - prioritizing quality, 543-544
 - real data
 - definition of, 16
 - example of, 18-21
 - simulated data
 - definition of, 16
 - example of, 21-24
 - simulating natural phenomena from, 520-522
 - sources of, 27-29
 - structured, 18
 - training data
 - correlation in, 70-71
 - underrepresentation in, 539
 - unstructured, 18
- ### data distribution terminology, 29-35
- ### data sets
- Bayesian network setup, 337
 - feature selection, 101
 - IMDB example, 261
 - in loss function, 66-68
 - underrepresentation in, 539
- ### data structures, graphs for, 321-322
- ### decision trees, 98-101
- entropy function, 101-105
 - Gini impurity function, 101-102, 105
 - information gain, 102-105
 - regression decision trees, 106
 - shortcomings of, 107
- ### deep learning
- approximation theory for, 131
 - for high-dimensional differential equations, 519-520
 - history of AI, 9
 - for PDEs, 507-508
 - meshes, 510-512
 - physical parameter values, 509
 - solution operators, 512-519
- ### Delphi, 542-543
- ### dense neural networks, 115
- ### density functions (see probability density functions)
- ### dependent variables, joint probability distributions for, 37-38
- ### derivatives
- assessing input variable significance, 157
 - discretizing, 474
 - of linear algebra expressions, 82
- ### deterministic models, 268-269, 456
- ### diagonal matrices, xxv, 191-192
- ### differential equations, xxvii, 465
- (see also partial differential equations)
 - ordinary versus partial, 466-467
- ### diffusion coefficient, 469
- ### dimension reduction
- principal component analysis and, 212-214
 - topic selection and, 229-232
- ### directed graphs, 302
- ### Dirichlet energy, 362-364

- discrete random variables
 - continuous versus, 35-37
 - probability mass functions, 30
- discrete versus continuous functions, 472-474
- discretization
 - curse of dimensionality, 477
 - of derivatives, 474
 - finite differences, 478-483
 - finite elements, 484-488
 - Monte Carlo methods, 490-493
 - variational principle, 489-490
- discrimination versus bias, 544-545
- disease spread, graphs for, 312
- distance metrics, 61-63
- distributed networks, load balancing in, 322-323
- distributions (see probability distributions)
- do calculus, 338, 415-420
- doc2vec, 240
- double integration of joint probability distributions, 36
- double-blind testing, 48
- doubly block circulant matrix, 181, 182
- dropout method (regularization), 145
- dual graphs, 342
- dual problem, 386
 - deriving from standard form, 390-391
 - finding from primal problem, 389-390
- dual simplex method, 391
- duality, 386-401
 - Lagrange multipliers, 388
 - network optimization, 392-394
 - quadratic optimization with linear constraints, 396-401
 - strong duality theorem, 391
 - two-person zero-sum games, 394-396
- dynamic graph models, 330-331
- dynamic programming, 355, 522-528
- dynamical systems, 427, 503
- Dyson Brownian motion, 430

E

- early stopping method (regularization), 146
- Eckart-Young-Mirsky theorem, 201
- edge features, 299
- edges (of graphs), 299
 - in Cayley graphs, 308
 - in PageRank algorithm, 302
- efficiency of AI systems, 534
- eigenvalue decomposition versus singular value decomposition, 204-206
- eigenvalues, 430
- eigenvectors, computing, 207-208
- elastic net regression, 149-150
- empirical distributions, 47
- encoder-decoder structure, 248-250
- encryption, 540-541
- energy-based models, 289
- ensemble methods, 94
- entropy function, 101-105
- enumeration in graph theory, 344
- epochs, 67
- error functions (see loss functions)
- ethics of AI
 - addressing ethical problems
 - accessibility of AI systems, 543
 - bias in word vectors, 540
 - fairness issues, 541-542
 - morality, 542-543
 - prioritizing high quality data, 543-544
 - privacy issues, 540-541
 - underrepresentation in training data, 539
 - bias versus discrimination, 544-545
 - evaluating unethical possibilities, 536
 - chemical warfare agent development, 537-538
 - political manipulation, 538
 - unintended outcomes of generative models, 539
 - weaponization of math, 536-537
 - laws of robotics, 547
 - policy importance, 531-536
 - quality aspects of AI, 533-534
- Euler-Lagrange equation, 361, 364
- evolution of generative models
 - autoencoders, 292-293
 - Boltzmann machines, 290
 - Hopfield nets, 290
 - restricted Boltzmann machines, 291-292
 - statistical mechanics, 288-290
- expectations, 33, 445-446, 495
- explicit density models, 272
 - Boltzmann machine approximation, 279
 - fully visible belief networks, 273-276
 - nonlinear independent component analysis, 276-277
 - variational autoencoders, 277-279

exploding gradient problem, 122
exponential distributions, 45
exponential versus polynomial algorithms,
351-352
extension theorem, 445

F

F-tests, 101
Facebook, 239
factorization of matrices, 188-191
failures of AI (artificial intelligence), 8
fairness of AI systems, 534, 541-542
fake news, detecting spread of, 312-313
far field conditions in partial differential equations, 468
fastText, 239
feasible point, 371
feasible set, 350
feature maps, 178
feature selection, 101
feature standardization, 138-139
feature vectors, 309
feed forward fully connected neural networks,
117
Feynman-Kac formula, 495
field of integers modulo 2, 343
filtering images, 174-176
finance AI applications, 261
finite differences, 478-483
finite elements, 484-488
first-order logic, 457-459
fitting data and functions, 51
(see also machine learning)
challenges in, 14-15
real data example, 18-21
simulated data example, 21-24
warning about, 85
fixed point iteration, 503-509
flipping the kernel, 163-164
forks in Bayesian networks, 336-337
four color theorem, 341
Fourier neural network, 516-518
Fourier sine and cosine series, 497
Fourier transforms, 495-497
frequencies, convolution as product of, 167
frequency response, 171
frequentist position (probability theory), 448
fully connected neural networks, 115

fully visible belief networks in generative models, 267, 273-276
functions, 16
(see also mathematical models)
finding minimizers, 76-79
fitting with data, 51
(see also machine learning)
challenges in, 14-15
real data example, 18-21
simulated data example, 21-24
warning about, 85
linear, 16-17
linearizing, 73-74
nonlinear, 16-17
random variables versus, 30
regularizing, 53
singularities in, 63-64
future of AI (artificial intelligence), 8-9
fuzzy logic, 460-461

G

Gabor filters, 176
game theory
multiagents in, 402-403
two-person zero-sum games, 394-396,
402-403
gamma distributions, 47, 429
gated units, 260
Gaussian distributions, 40-43, 429
Gaussian mixtures, 32, 287
general form of linear optimization, 372-373
general vector, acting on, 199
generalization, importance of, 26
generalized resolution inference rule, 463
generative adversarial networks, 265, 267, 280
advantages/disadvantages, 280-281
applications for, 282-285
learning process, 281-282
generative graph networks, 316
generative models, xxv-xxvi, 263-264
applications for, 264-265
evolution of
autoencoders, 292-293
Boltzmann machines, 290
Hopfield nets, 290
restricted Boltzmann machines, 291-292
statistical mechanics, 288-290
explicit density models, 272
Boltzmann machine approximation, 279

- fully visible belief networks, 273-276
- nonlinear independent component analysis, 276-277
- variational autoencoders, 277-279
- Gaussian mixture models, 287
- implicit density models, 272
 - generative adversarial networks, 280-285
 - generative stochastic networks, 279
- joint probability distributions in, 263, 265-268
- LeCun on, 285-286
- maximum likelihood estimation, 270-272
- Naive Bayes classification model, 286-287
- probabilistic language models, 293-295
- unintended outcomes, 539
- generative stochastic networks, 279
- geometric distributions, 45
- geometry of linear optimization, 377-379
- Gibbs sampling, 291
- Gini impurity function, 101-102, 105
- Global vector (GloVe) model, 240-241
- Google
 - PageRank algorithm, 302-307, 322
 - RankBrain, 307
 - word2vec, 235-238
- Google Search, 307
- gradient descent method, 52, 135-140
- gradient of loss function, 153-157
- graph classification tasks, 328
- graph generation tasks, 329
- graph neural networks, 298-299, 323
 - clustering and community detection tasks, 329
 - graph classification tasks, 328
 - graph generation tasks, 329
 - influence maximization tasks, 329
 - link prediction tasks, 330
 - node classification tasks, 327-328
 - tasks of, 310
- graph theory
 - coloring, 344
 - cut sets, 342
 - cut vertices, 342
 - enumeration, 344
 - history of, 340-341
 - matching, 344
 - planarity, 342
 - realizability, 343
 - spanning trees, 341
- graph-based convolutional networks, 285
- graphs, xxvi
 - advantages of, 297-299
 - algorithms for, 344-345
 - applications for
 - artificial neural networks, 323
 - Bayesian networks, 317
 - biochemical graphs, 315
 - brain networks, 311
 - cancer fighting, 314-315
 - citation networks, 316
 - computer program analysis, 321
 - data structures in computer science, 321-322
 - disease spread information, 312
 - fake news spread detection, 312-313
 - generative graph networks, 316
 - information spread, 312
 - load balancing in distributed networks, 322-323
 - natural language models, 318-320
 - operations research, 318
 - questions to ask, 310
 - social media networks, 316-317
 - sociological structures, 317
 - traffic forecasting, 317
 - web structure, 320-321
 - web-scale recommendation systems, 314
- Bayesian networks, 331-332
 - as belief networks, 334-335
 - as compactified conditional probability tables, 333
 - junctions in, 336-337
 - predictions with, 334
 - setup from data sets, 337
- Cayley graphs of groups, 308
- computational graph representation, 117
- dynamic models, 330-331
- image data as, 311
- inverting matrices, 307-308
- of joint probability distributions, 33
- message passing in, 309-310
- node representation learning, 326-327
- PageRank algorithm example, 302-307
- probabilistic causal modeling, 338-340
- random walk on, 324-326
- terminology, 299-302
- as vector spaces, 343
- groups, graphs of, 308

H

- Hamilton, William, 341
- Hamilton-Jacobi-Bellman partial differential equation, 403, 406, 522-528
- Hamiltonian circuits, 366
- harmonic functions, 362-364
- heat equation, 362-364, 499-501
- Hebb, Donald, 9, 154
- Heisenberg uncertainty principle, 497
- heuristic optimization methods, 356
- hidden layers, 115
- high energy physics, 283-285
- high quality data, prioritizing, 543-544
- high-dimensional differential equations, 519-520
- Hikvision, 535
- hinge loss function, 95
- history
 - of AI (artificial intelligence), 8-9
 - of convolution, 166
 - of graph theory, 340-341
- Hitch, Charles, 408-409
- Hopf formulas, 527
- Hopfield nets, 290
- Householder reflections, 206
- human brain
 - network graphs, 311
 - neural networks' comparison to, 4, 113-115
- human natural language (see natural language)
- hype about AI, 545-546
- hypergeometric distributions, 47
- hyperparameters, 60, 67
 - for learning rate, 79, 137-138
 - limitations of AI, 6
 - list of, 152-153
 - for regularization, 151-152
- hypothesis functions (see training functions)

I

- image captioning, 247
- image data
 - features of, 89
 - as graphs, 311
- image processing
 - classification with convolutional neural networks, 184-185
 - compression with singular value decomposition, 209-212
 - filtering images, 174-176

- padding boundaries, 172-174
- Imagen, 539
- IMDB example (natural language data), 261
- implicit density models, 272
 - generative adversarial networks, 280-285
 - generative stochastic networks, 279
- impulse response, 169
- incidence matrices, 301
- independence, 413
- independent random variables, 31
- inference rules, 455, 463
- inferring relationships in first-order logic, 459
- infinite dimensional function optimization, 360-364
- influence maximization, 329
- information gain in decision trees, 102-105
- information retrieval systems, 322
- information spread, graphs for, 312
- information theory, cross entropy and, 93
- initializing weights, 144-145
- input variables, assessing significance, 157
- integer programming, 354
- integration constants in partial differential equations, 468
- intelligence
 - infographic for, xxii
 - limitations of AI, 6
 - mathematical models for, xxii
- interior point method of linear optimization, 352, 354
- inventory management, 405
- inverting matrices, 307-308
- irrational numbers, approximating with
 - rational numbers, 127-128
- Itô's lemma, 437-438

J

- Jacobi matrices, 430
- joint probability distributions, 30, 413
 - Bayes' Theorem and, 32
 - conditional probabilities and, 31
 - for dependent variables, 37-38
 - in generative models, 263, 265-268
 - graphs of, 33
 - for multiple continuous random variables, 36
- junctions in Bayesian networks, 336-337

K

k-means clustering, 108
Kaggle website, 28
kernel functions, 98
kernel, flipping, 163-164
Kirchhoff's current law, 393
Kolmogorov existence theorem, 433
Kullback-Leibler (KL) divergence, 132, 271

L

labeled data, 6
Lacroix, Sylvestre François, 166
Lagrange multipliers
 constrained, 357-359
 duality and, 388
 in quadratic optimization with linear constraints, 396-401
language (see natural language)
Laplace transforms, 498-499
Laplacian matrices, 301
Large Hadron Collider, 283-285
large random matrices, 424
 considerations for, 427-428
 eigenvalue density, 430
 ensembles, 429-430
 essential math for, 430-431
 examples of, 424-427
lasso regression, 101, 149-150
latent Dirichlet analysis, 232-233
latent discriminant analysis, 233-234, 243
latent semantic analysis, 215-216, 228-232, 244
Law of Large Numbers, 33
laws of robotics, 547
learning functions (see training functions)
learning rate hyperparameter, 79, 137-138
LeCun, Yann, 285-286, 449
Levy processes, 436
lexicon (in natural language processing), 223
likelihood function, 32
limit theorems, 447
linear algebra
 for convolutional neural networks, 179-182
 derivatives, 82
linear combinations
 with bias, 118-119
 of linear combinations, 119
 of weights, 117-118
linear constraints, quadratic optimization with, 396-401

linear functions, 16-17
linear optimization, 371
 converting convex objectives to, 374-376
 duality, 386-401
 general and standard forms, 372-373
 geometry of, 377-379
 interior point method, 352, 354
 sensitivity, 401-402
 simplex method, 352, 353, 379-386
 transportation and assignment problems, 386
 visualizing in two dimensions, 373-374
linear programming, 73
linear regression, 56-58
 as analytical method, 55
 loss function for, 60-71
 optimization, 71-85
 convex versus nonconvex landscapes, 73-74
 finding minimizers, 76-79
 linear algebra derivatives, 82
 minimizing mean squared error function, 82-85
 one-dimensional example, 81-82
 training function for, 58-60
linear transformations, matrices as, 193-194
 acting on general vector, 199
 acting on right singular vectors, 194-195
 acting on standard unit vectors, 195
 acting on unit circle, 196-197
 reflection matrix, 198
 rotation matrix, 198
linearity in convolution and systems design, 168-171
link prediction, 330
linking matrix, 304-306
load balancing, graphs for, 322-323
log function for term frequencies, 226
log likelihood function, 132, 264, 270-272, 293
log-bilinear models, 240
log-normal distributions, 46
logic
 agent questions, 452
 codifying in AI agents, 456
 completeness of frameworks, 455
 definition of, 451
 first-order, 457-459
 fuzzy, 460-461
 natural language versus, 462

- probabilistic, 460
- propositional, 452-456
- temporal, 461
- logical equivalences in propositional logic, 454-455
- logistic regression, 85
 - loss function for, 86-88
 - optimization, 88
 - training function for, 85-86
- logistics (see operations research)
- long short-term memory units, 260
- long-term dependencies, 257
- loss functions, 52
 - calculating gradient, 153-157
 - for convolutional neural networks, 180
 - for linear regression, 60-71
 - for logistic regression, 86-88
 - for neural networks, 131-132
 - optimization (see optimization)
 - for softmax regression, 92
 - for support vector machines, 95-96

M

- machine learning
 - choosing models, 94
 - decision trees, 98-101
 - entropy function, 101-105
 - Gini impurity function, 101-102, 105
 - information gain, 102-105
 - regression decision trees, 106
 - shortcomings of, 107
 - deterministic, 456
 - history of AI, 9
 - hyperparameters in, 152-153
 - k-means clustering, 108
 - linear regression, 56-58
 - loss function for, 60-71
 - optimization, 71-85
 - training function for, 58-60
 - logistic regression, 85
 - loss function for, 86-88
 - optimization, 88
 - training function for, 85-86
 - model types, 53-54
 - in operations research, 405-406
 - performance of classification models, 109-110
 - physics-informed, 530
 - probabilistic, 456

- random forests, 107
- regression models in neural networks, 93
- softmax regression, 88-89
 - loss function for, 92
 - optimization, 92-93
 - training function for, 90-91
- steps in, 51-53
- support vector machines, 94
 - kernel functions, 98
 - loss function for, 95-96
 - optimization, 96-97
 - training function for, 95
- machine translation, 246
- marginal cost, 359
- marginal probability distributions, 31
- Markov chains, 436-437
 - Boltzmann machine approximation via, 279
 - generative stochastic networks, 279
- Markov decision processes, 34, 355, 438-441
- martingales, 326, 435-436
- mass functions (see probability mass functions)
- master equation in statistical mechanics, 493-495
- matching (assignment) problem, 369, 386
- matching in graph theory, 344
- mathematical models, 16
 - (see also functions)
 - for intelligence, xxii
 - interactions with simulations and AI, 25-27
 - for inventory management, 405
 - linear, 16-17
 - nonlinear, 16-17
 - with partial differential equations, 467-468
 - changes within, 469-471
 - parameters for, 468
 - scaling, 468
 - queuing theory, 404-405
- mathematical objects, questions about, 164-167
- mathematical optimization (see optimization)
- mathematical physics, Wigner matrices in, 425
- mathematics
 - logic (see logic)
 - of neural network success, 134-135
 - types used for AI, 10
 - weaponization of, 536-537
- matrices, 187
 - (see also singular value decomposition)
 - diagonal, 191-192
 - inverting, 307-308

- as linear transformations on space, 193-194
 - acting on general vector, 199
 - acting on right singular vectors, 194-195
 - acting on standard unit vectors, 195
 - acting on unit circle, 196-197
 - reflection matrix, 198
 - rotation matrix, 198
- multiplying, 84, 200-201, 427
- padding boundaries, 172-174
- properties, 201-203
- pseudoinverse, 208-209
- matrix factorization, 188-191
- max-flow min-cost, 369
- max-flow min-cut, 342, 368, 392-394
- maximum likelihood estimation, 264, 270-272, 293
- McCulloch, Warren, 9
- mean, 33, 40
- mean squared error function, 64-65, 82-85
- meaning vectors with neural networks, 234-241
- measure theory, 37, 441-443
- memory units in recurrent neural networks, 258, 260
- mesh independence, 509
- meshes, deep learning for, 510-512
- MeshingNet, 511
- message passing in graphs, 309-310
- metaheuristics, 356
- min-max theorem, 396-401, 403
- minimizers, 52, 76-79
- minimizing mean squared error function, 82-85
- minimum spanning tree problem, 366-367
- mixtures of distributions, 32
- models (see mathematical models)
- Monte Carlo methods for PDEs, 490-493
- Monty Hall problem, 420-422
- morality in AI, 542-543
- multiagents in game theory, 402-403
- multichannel convolution, 163
- multihead attention, 254
- multioutput models, 89
- multiplying matrices, 84, 200-201, 427
- multivariate statistics, random matrices in, 426
- mutual information tests, 101
- N**
- n-character grams, 239
- n-grams, 224
- n-queens problem, 370
- Naive Bayes classification model, 243, 286-287, 414
- Nash equilibrium, 403
- National Science Foundation (NSF), 134-135
- natural language
 - graph models for, 318-320
 - logic versus, 462
 - as time series data, 220-221
- natural language processing, 219-222
 - applications for, 222-223, 243
 - chatbots, 247
 - image captioning, 247
 - machine translation, 246
 - search and information retrieval, 244-246
 - sentiment analysis, 243
 - spam filtering, 244
 - cosine similarity, 215, 241-243
 - data preparation, 223-225
 - generative models and, 265
 - IMDB example, 261
 - latent semantic analysis, 215-216
 - probabilistic language models, 293-295
 - term frequencies
 - log function and, 226
 - Zipf's law, 226
- transformers
 - advantages over recurrence and convolution architectures, 247
 - architecture of, 248-251
 - attention mechanisms, 251-254
 - disadvantages of, 255
- vector representation
 - meaning vectors with neural networks, 234-241
 - term frequency, 227
 - term frequency-inverse document frequency, 228
 - topic vectors with latent Dirichlet analysis, 232-233
 - topic vectors with latent discriminant analysis, 233-234
 - topic vectors with latent semantic analysis, 228-232
- natural phenomena, simulating from data, 520-522
- negative binomial distributions, 47
- negative hypergeometric distributions, 47

- neocortex, 4, 113-115
- network diffusion, 329
- network simplex method, 386
- networks
 - Bayesian, 331-332, 414-415
 - as belief networks, 334-335
 - as compactified conditional probability tables, 333
 - graphs for, 317
 - junctions in, 336-337
 - predictions with, 334
 - setup from data sets, 337
 - brain networks, graphs for, 311
 - causal, 415-420
 - citation, 316
 - fully visible belief networks in generative models, 267, 273-276
 - generative adversarial networks, 265, 267, 280
 - advantages/disadvantages, 280-281
 - applications for, 282-285
 - learning process, 281-282
 - generative stochastic networks, 279
 - graph-based convolutional networks, 285
 - neural (see neural networks)
 - optimization on, 354, 365
 - assignment (matching) problem, 369
 - critical path method (CPM), 369
 - duality, 392-394
 - max-flow min-cost problem, 369
 - max-flow min-cut problem, 368
 - minimum spanning tree problem, 366-367
 - shortest path problem, 367-368
 - traveling salesman problem, 365-366
- neural networks, xxiv
 - backpropagation, 153-157
 - as Bayesian networks, 337
 - comparison to human brain, 4, 113-115
 - convolutional (see convolutional neural networks)
 - Fourier, 516-518
 - graph neural networks, 298-299, 323
 - clustering and community detection tasks, 329
 - graph classification tasks, 328
 - graph generation tasks, 329
 - influence maximization tasks, 329
 - link prediction tasks, 330
 - node classification tasks, 327-328
 - tasks of, 310
 - history of AI, 9
 - loss functions for, 131-132
 - mathematics and success of, 134-135
 - meaning vectors for natural language processing, 234-241
 - optimization, 133
 - convex versus nonconvex landscapes, 140-143
 - gradient descent method, 135-140
 - hyperparameters for, 152-153
 - initializing weights, 144-145
 - learning rate hyperparameter, 137-138
 - stochastic gradient descent, 143-144
 - PDEs for, 528
 - regression models in, 93
 - regularization
 - batch normalization method, 146-147
 - dropout method, 145
 - early stopping method, 146
 - regularization hyperparameter, 151-152
 - weight decay method, 147-151
 - success of, 3
 - training function for, 115-116
 - common activation functions, 122-125
 - computational graph representation, 117
 - linear combination with bias, 118-119
 - linear combination with weights, 117-118
 - passing through activation function, 119-122
 - universal function approximation, 125-131
 - universal approximation theorem, 459
 - universality theorem for, 448
- neural operator networks, 512-519
- neuro-dynamic programming (see reinforcement learning)
- neuroplasticity, 162
- neuroscience, random matrices in, 425
- no free lunch theorem, 348, 349
- node features, 299
- nodes (of graphs), 299
 - in Cayley graphs, 308
 - classification tasks, 327-328
 - message passing, 309-310
 - in PageRank algorithm, 302
 - random walk, 324-326

- representation learning, 326-327
- nonconvex landscapes, 73-74, 140-143
- nonlinear activation functions (see activation functions)
- nonlinear functions, 16-17
 - linearizing, 73-74
- nonlinear independent component analysis, 276-277
- nonlinear optimization, 354-355
- nonparametric models, 60, 100
- normal distributions, 31, 40-43
- normalized data, 34
- NSF (National Science Foundation), 134-135
- numerical solutions
 - analytical solutions versus, 55-56
 - for eigenvectors, 207-208
 - for singular value decomposition, 206-208
- numerical solutions to PDEs
 - analytical solutions versus, 472
 - continuous versus discrete functions, 472-474
- discretization
 - curse of dimensionality, 477
 - finite differences, 478-483
 - finite elements, 484-488
 - Monte Carlo methods, 490-493
 - variational principle, 489-490
- for high-dimensional differential equations, 519-520
- thin crystal modeling, 474-476

O

- O() notation, 350-352
- objective functions (see loss functions)
- objectivist position (probability theory), 448
- ODEs (ordinary differential equations), 466-467
- one-dimensional discrete signals
 - convolution and, 171-172
 - multiplication by Toeplitz matrix, 182
- one-dimensional optimization, 81-82
- one-hot encoding, 56
- operations research, xxvi, 347-349
 - for AI (artificial intelligence), 407
 - assignment (matching) problem, 369
- asymptotic analysis, 351
- complexity analysis, 350-352
- critical path method (CPM), 369
- game theory

- multiagents in, 402-403
- two-person zero-sum games, 394-396, 402-403
- graphs for, 318
- inventory management, 405
- machine learning in, 405-406
- max-flow min-cost problem, 369
- max-flow min-cut problem, 368
- minimum spanning tree problem, 366-367
- n-queens problem, 370
- no free lunch theorem, 349
- O() notation, 350-352
- optimization (see optimization)
- queuing theory, 404-405
- resources for information, 349
- shortest path problem, 367-368
- traveling salesman problem, 365-366
- operators in propositional logic, 453
- optimization, 52, 353
 - convex, 375
 - for convolutional neural networks, 180
 - dynamic programming, 355
 - integer programming, 354
 - linear, 371
 - converting convex objectives to, 374-376
 - duality, 386-401
 - general and standard forms, 372-373
 - geometry of, 377-379
 - interior point method, 352, 354
 - sensitivity, 401-402
 - simplex method, 352, 353, 379-386
 - transportation and assignment problems, 386
 - visualizing in two-dimensions, 373-374
- for linear regression, 71-85
 - convex versus nonconvex landscapes, 73-74
 - finding minimizers, 76-79
 - linear algebra derivatives, 82
 - minimizing mean squared error function, 82-85
 - one-dimensional example, 81-82
- for logistic regression, 88
- Markov decision processes, 355
- metaheuristics, 356
- on networks, 354, 365
 - assignment (matching) problem, 369
 - critical path method (CPM), 369
 - duality, 392-394

- max-flow min-cost problem, 369
 - max-flow min-cut problem, 368
 - minimum spanning tree problem, 366-367
 - shortest path problem, 367-368
 - traveling salesman problem, 365-366
 - of neural networks, 133
 - convex versus nonconvex landscapes, 140-143
 - gradient descent method, 135-140
 - hyperparameters for, 152-153
 - initializing weights, 144-145
 - learning rate hyperparameter, 137-138
 - stochastic gradient descent, 143-144
 - no free lunch theorem, 349
 - nonlinear, 354-355
 - problem types
 - finite dimensions, constrained Lagrange multipliers, 357-359
 - finite dimensions, unconstrained, 357
 - infinite dimensions, calculus of variations, 360-364
 - quadratic with linear constraints, 354, 396-401
 - questions to ask, 356-357
 - for softmax regression, 92-93
 - stochastic algorithms, 355
 - for support vector machines, 96-97
 - ordinary differential equations (ODEs), 466-467
 - orthogonal matrices, 193
 - orthogonal polynomials, 429
 - orthogonality, 486
- ## P
- padding matrix boundaries, 172-174
 - PageRank algorithm, 302-307, 322
 - par2vec, 240
 - paradoxes, 420
 - Berkson, 422
 - Monty Hall problem, 420-422
 - Simpson's, 422-424
 - parameters
 - deep learning for physical values, 509
 - for partial differential equation models, 468
 - parametric models, 60
 - Pareto distributions, 46
 - parsers, 224
 - parsing, 319
 - partial derivatives, 466
 - partial differential equations, 528-530
 - for AI (artificial intelligence), 528
 - deep learning for, 507-508
 - meshes, 510-512
 - physical parameter values, 509
 - solution operators, 512-519
 - Hamilton-Jacobi-Bellman, 403, 406, 522-528
 - modeling with, 467-468
 - changes within, 469-471
 - parameters for, 468
 - scaling, 468
 - numerical solutions
 - analytical solutions versus, 472
 - continuous versus discrete functions, 472-474
 - discretization and curse of dimensionality, 477
 - finite differences, 478-483
 - finite elements, 484-488
 - for high-dimensional differential equations, 519-520
 - Monte Carlo methods, 490-493
 - thin crystal modeling, 474-476
 - variational principle, 489-490
 - ordinary differential equations versus, 466-467
 - solution operators
 - fixed point iteration, 503-509
 - heat equation example, 499-501
 - Poisson equation example, 501-503
 - solutions as expectations, 495
 - in statistical mechanics, 493-495
 - in Top Gun: Maverick, 465-466
 - transforming
 - Fourier transforms, 495-497
 - Laplace transforms, 498-499
 - partition function in statistical mechanics, 91
 - pasting, 94
 - payoff matrix, 394
 - PDEs (see partial differential equations)
 - Pearl, Judea, 337, 338, 417, 420
 - perception AI, 13
 - perceptrons, 115
 - performance, measuring for classification models, 109-110
 - PERT (Program Evaluation Review Technique), 369

- physics-informed machine learning, 530
- Picard's iteration, 506
- PinSage, 314
- Pinterest, 314
- Pitts, Walter, 9
- PixelCNN, 267, 273-276
- planarity in graph theory, 342
- planes, 16
- Poisson distributions, 44
- Poisson equation, 501-503
- Poisson process, 433-434
- policies for AI, importance of, 531-536
- political manipulation, 538
- polynomials
 - approximating continuous functions with, 129
 - exponential algorithms versus, 351-352
- pooling, 183
- positional encoding, 251
- posterior distribution, 32, 37
- posterior probabilities, 412
- predicted versus true value, 60-61
- prediction functions (see training functions)
- predictions with Bayesian networks, 334
- preparing natural language data, 223-225
- primal problem, 386
 - dual simplex method, 391
 - finding dual problem from, 389-390
- principal component analysis, 278
 - clustering and, 214-215
 - dimension reduction and, 212-214
- prior distribution, 32
- prior probabilities, 412
- privacy
 - addressing issues of, 540-541
 - of AI systems, 533
- probabilistic logic, 460
- probabilistic models, 456
 - causal, 338-340
 - deterministic models versus, 268-269
 - generative (see generative models)
 - language, 293-295
- probability density functions, 30, 35-37
 - for dependent variables, 37-38
 - for uniform distributions, 39
- probability distributions, 30
 - binomial, 44
 - chi-squared, 46
 - concepts in, 47-48
 - continuous versus discrete, 35-37
 - empirical, 47
 - exponential, 45
 - Gaussian, 40-43
 - geometric, 45
 - joint (see joint probability distributions)
 - log-normal, 46
 - mixtures of, 32
 - Pareto, 46
 - Poisson, 44
 - uniform distribution, 38-39
 - Weibull, 45
- probability mass functions, 30, 35-37
- probability measure, 444
- probability theory, xxvi, 411
 - (see also uncertainty)
 - for AI (artificial intelligence), 412-415
 - causal networks and do calculus, 415-420
 - common examples in, 34
 - frequentist versus objectivist positions, 448
 - large random matrices, 424
 - considerations for, 427-428
 - eigenvalue density, 430
 - ensembles, 429-430
 - essential math for, 430-431
 - examples of, 424-427
- paradoxes, 420
 - Berkson, 422
 - Monty Hall problem, 420-422
 - Simpson's, 422-424
- reinforcement learning and, 438-441
- rigorous probability theory, 441-442
 - distribution of random variables, 446-447
 - limit theorems, 447
 - probability triple, 443-445
 - random variable definition, 445-446
 - random variable types, 443
 - sample space subsets, 442-443
 - universality theorem for neural networks, 448
- stochastic processes, 432-433
 - Bernoulli, 433
 - branching, 436
 - Brownian motion, 435
 - Itô's lemma, 437-438
 - Levy, 436
 - Markov chains, 436-437
 - martingales, 435-436

- Poisson, 433-434
- random walk, 434
- terminology, 29-35
- probability triple, 432, 443-445
- product of frequencies, convolution as, 167
- product or chain rule, 31, 413
- products of random variables, 32
- program analysis, graphs for, 321
- Program Evaluation Review Technique (PERT), 369
- proof by contradiction reasoning, 454-455
- properties of matrices, 201-203
- propositional logic, 452-456
 - first-order logic versus, 457-459
- pseudoinverse of matrices, 208-209
- public data repositories, 28
- Python libraries for reinforcement learning, 441

Q

- QR decomposition, 206
- quadratic optimization with linear constraints, 354, 396-401
- quantifiers in first-order logic, 457-459
- quantitative finance, random vectors in, 424
- quantum computing, 545
- queuing theory, 356, 404-405

R

- random forests, 94, 100, 107
- random matrices (see large random matrices)
- random patches, 94
- random sampling of matrix columns, 431
- random variables, 30
 - conditionally independent, 31
 - definition in rigorous probability, 445-446
 - dependent, 37-38
 - distribution in rigorous probability theory, 446-447
 - independent, 31
 - probability distributions for
 - continuous versus discrete, 35-37
 - types of, 30
 - products of, 32
 - sums of, 32
 - types in rigorous probability theory, 443
- random vectors, examples of, 424-427
- random walk, 434
 - Brownian motion and, 326

- on graphs, 324-326
 - of PageRank graph, 304
- randomized singular value decomposition, 216
- RankBrain, 307
- rational numbers, approximating irrational numbers with, 127-128
- real data
 - definition of, 16
 - example of, 18-21
- real numbers, infinite precision of, 35
- realizability in graph theory, 343
- reasoning over knowledge graphs, 298
- Rectified Linear Unit function (ReLU), 73, 122, 143
- recurrent neural networks
 - disadvantages of, 247
 - memory units in, 258, 260
 - for time series data, 257-260
 - training, 258-260
- reflection matrices, 198
- regression
 - lasso, 101
 - linear, 56-58
 - as analytical method, 55
 - loss function for, 60-71
 - optimization, 71-85
 - training function for, 58-60
 - logistic, 85
 - loss function for, 86-88
 - optimization, 88
 - training function for, 85-86
 - in neural networks, 93
 - softmax, 88-89
 - loss function for, 92
 - optimization, 92-93
 - training function for, 90-91
 - for weight decay regularization, 149-150
- regression decision trees, 106
- regularization hyperparameter, 151-152
- regularizing functions, 53
 - batch normalization method, 146-147
 - dropout method, 145
 - early stopping method, 146
 - regularization hyperparameter, 151-152
 - weight decay method, 147-151
- reinforcement learning
 - dynamic programming and, 527
 - examples of, 438-439
 - history of AI, 9

- Markov decision processes and, 438-441
- Python library for, 441
- relationships, inferring in first-order logic, 459
- representation learning for graph nodes, 326-327
- resolution inference rule, 463
- resource limitations of AI, 7
- resources for information
 - AI (artificial intelligence), xxvii-xxix
 - neural operator networks, 519
 - operations research, 349
- restricted Boltzmann machines, 291-292
- revised simplex method of linear optimization, 383-385
- ridge regression, 149-150
- right singular vectors, acting on, 194-195
- rigorous probability theory, 441-442
 - distribution of random variables, 446-447
 - limit theorems, 447
 - probability triple, 443-445
 - random variable definition, 445-446
 - random variable types, 443
 - sample space subsets, 442-443
 - universality theorem for neural networks, 448
- robotics, laws of, 547
- robustness of AI systems, 534
- Rosenblatt, Frank, 9
- rotation matrices, 198
- rules of inference, 455, 463

S

- sample mean, 33
- sample space
 - in probability triple, 444
 - subsets of, 442-443
- satisfiability, 356
- scalars, 17
- SCALE MoDL program, 135
- scaling
 - data sets, 34
 - partial differential equation models, 468
- search and information retrieval, 244-246
- security
 - of AI systems, 533
 - costs of, 7
- self attention, 252
- sensitivity in linear optimization, 401-402
- sentiment analysis, 243

- sequences in transformer models, 250
- shadow price, 359
- shortest path problem, 364, 367-368
- sigma algebra, 444
- Simkin, Michael, 370
- simplex method of linear optimization, 352, 353, 379-386
- Simpson's paradox, 422-424
- simulated data
 - definition of, 16
 - example of, 21-24
- simulating natural phenomena from data, 520-522
- simulations, interactions with mathematical models and AI, 25-27
- single-blind testing, 48
- singular value decomposition, xxv, 187-188, 228
 - components of, 204
 - computing, 206-208
 - diagonal matrices, 191-192
 - eigenvalue decomposition versus, 204-206
 - for image compression, 209-212
 - linear transformations in space, 193-194
 - acting on general vector, 199
 - acting on right singular vectors, 194-195
 - acting on standard unit vectors, 195
 - acting on unit circle, 196-197
 - reflection matrix, 198
 - rotation matrix, 198
 - matrix factorization, 188-191
 - matrix multiplication, 200-201
 - matrix properties, 201-203
 - principal component analysis
 - clustering and, 214-215
 - dimension reduction and, 212-214
 - pseudoinverse, 208-209
 - randomized, 216
- singularities in functions, 63-64
- social media
 - detecting fake news spread, 312-313
 - graphs of influence, 316-317
 - political manipulation, 538
 - principal component analysis and, 214-215
- sociological structures, graphs for, 317
- softmax regression, 88-89
 - loss function for, 92
 - optimization, 92-93
 - training function for, 90-91

- solution operators for PDEs
 - deep learning for, 512-519
 - fixed point iteration, 503-509
 - heat equation example, 499-501
 - Poisson equation example, 501-503
- sources of data, 27-29
- space, linear transformations in, 193-194
 - acting on general vector, 199
 - acting on right singular vectors, 194-195
 - acting on standard unit vectors, 195
 - acting on unit circle, 196-197
 - reflection matrix, 198
 - rotation matrix, 198
- spam filtering, 244
- spanning trees, 341
- split testing, 48
- squared distance, 61-63
- stacking, 94
- standard deviation, 33, 40
- standard form of linear optimization, 372-373, 390-391
- standard unit vectors, acting on, 195
- standardized data, 34
- standardized features, 138-139
- statistical mechanics, 288-290
 - master equation, 493-495
 - partition function, 91
- statistical models for term frequencies, 226
- statistics terminology, 29-35
- stochastic algorithms, 355
- stochastic gradient descent, 143-144
- stochastic processes, 432-433
 - Bernoulli, 433
 - branching, 436
 - Brownian motion, 435
 - Itô's lemma, 437-438
 - Levy, 436
 - Markov chains, 436-437
 - martingales, 435-436
 - Poisson, 433-434
 - random walk, 434
- strong duality theorem, 391
- structured data, 18
- student's t-distributions, 46
- subsets of sample space, 442-443
- success of neural networks, 134-135
- sum rule for probabilities, 31
- sums of random variables, 32
- support vector machines, 94
- kernel functions, 98
- loss function for, 95-96
- optimization, 96-97
- training function for, 95
- systems design, convolution in, 168-171

T

- temporal logic, 461
- tensors, 89
- term frequencies
 - log function and, 226
 - vector representation of, 227
 - Zipf's law, 226
- term frequency-inverse document frequency (TF-IDF), 228
- test subset in loss function, 67
- testing AI systems, 534
- text-to-image models, unintended outcomes, 539
- TF-Agents library, 441
- thin crystal modeling, 474-476
- The Three Laws of Robotics (Asimov), 547
- three-dimensional meshes, 511
- time invariance in convolution and systems design, 168-171
- time series data, 220
 - convolutional neural networks for, 255-257
 - finance AI applications, 261
 - natural language as, 220-221
 - recurrent neural networks for, 257-260
- Toeplitz matrix, 181, 182
- tokenizing, 224
- Top Gun: Maverick (film), 465-466
- topic vectors
 - with latent Dirichlet analysis, 232-233
 - with latent discriminant analysis, 233-234
 - with latent semantic analysis, 228-232
- traffic forecasting, graphs for, 317
- training data
 - correlation in, 70-71
 - underrepresentation in, 539
- training functions, 51
 - for convolutional neural networks, 179-180
 - for linear regression, 58-60
 - for logistic regression, 85-86
 - for neural networks, 115-116
 - common activation functions, 122-125
 - computational graph representation, 117
 - linear combination with bias, 118-119

- linear combination with weights, 117-118
- passing through activation function, 119-122
- universal function approximation, 125-131
- for recurrent neural networks, 258-260
- regularizing
 - batch normalization method, 146-147
 - dropout method, 145
 - early stopping method, 146
 - regularization hyperparameter, 151-152
 - weight decay method, 147-151
- for softmax regression, 90-91
- for support vector machines, 95
- training subset in loss function, 67
- transformer models
 - advantages over recurrence and convolution architectures, 247
 - architecture of, 248-251
 - attention mechanisms, 250, 251-254
 - disadvantages of, 255
 - long-term dependencies, 258
- transforming PDEs
 - Fourier transforms, 495-497
 - Laplace transforms, 498-499
- translation equivariance, 167
- translation invariance, 167-171
- translation of languages, 246
- transparency of AI systems, 534
- transportation problem in linear optimization, 386
- traveling salesman problem, 365-366
- triple integration of joint probability distributions, 36
- true versus predicted value, 60-61
- truth tables in propositional logic, 452-454
- Turing Test, 9
- Turing, Alan, 9
- two-dimensional discrete signals
 - convolution and
 - filtering images, 174-176
 - padding image matrices, 172-174
 - multiplication by doubly block circulant matrix, 182
- two-person zero-sum games
 - duality, 394-396
 - multiagent game theory, 402-403

U

- Uber, 348
- "Uncertainties in Operations Research" (Hitch), 408-409
- uncertainty, 33, 41, 411
 - (see also probability theory)
- unconstrained differentiable optimization, 357
- underrepresentation in training data, 539
- understanding AI, 13
- undirected graphs, 302
- uniform distribution, 31, 38-39
- unit circle, acting on, 196-197
- univariate normal distribution, 41
- universal approximation theorem
 - for neural network operators, 512, 518
 - for neural networks, 459
 - in restricted Boltzmann machines, 292
- universal function approximation, 125-131
 - for continuous functions, 129
 - for irrational numbers, 127-128
- universality theorem for neural networks, 448
- unstructured data, 18

V

- validation subset in loss function, 67
- vanishing gradient problem, 122
- variance, 33
- variance thresholding, 101
- variational autoencoders, 277-279, 292-293
- variational models, 267
- variational principle, 489-490
- vector representation in natural language processing
 - meaning vectors with neural networks, 234-241
 - term frequency, 227
 - term frequency-inverse document frequency, 228
 - topic vectors with latent Dirichlet analysis, 232-233
 - topic vectors with latent discriminant analysis, 233-234
 - topic vectors with latent semantic analysis, 228-232
- vector spaces, graphs as, 343
- vectors
 - column, 66
 - multiplying matrices, 84
 - in natural language processing, 224-225

- random, 424-427
- vertices (of graphs), 299
- visualizing
 - linear optimization in two dimensions, 373-374
 - word vectors, 238
- vocabulary (in natural language processing), 223

W

- WaveNet, 28-29, 267, 273-276
- weak formulations of PDEs, 484
- weaponization of math, 536-537
- Weapons of Math Destruction (O'Neil), 536-537
- web structure, graphs for, 320-321
- web-scale recommendation systems, 314
- Weibull distributions, 45
- weight decay method (regularization), 147-151
- weights (in hypothesis functions), 52
 - initializing, 144-145

- linear combination of, 117-118
- regularizing
 - batch normalization method, 146-147
 - dropout method, 145
 - early stopping method, 146
 - regularization hyperparameter, 151-152
 - weight-decay method, 147-151
- Wiener process, 326, 435
- Wigner matrices, 425, 429
- Wigner, Eugene, 425
- Wilson, E. Bright, 347
- Wishart matrices, 426, 429
- word vectors
 - bias in, 540
 - GloVe, 240, 241
 - visualizing, 238
 - word2vec, 235-238
- word2vec, 235-238

Z

- Zipf's law, 226

About the Author

Hala Nelson is an associate professor of mathematics at James Madison University. She has a Ph.D. in mathematics from the Courant Institute of Mathematical Sciences at New York University. Prior to James Madison University, she was a postdoctoral assistant professor at the University of Michigan, Ann Arbor.

She specializes in mathematical modeling and consults for emergency and infrastructure services in the public sector. She likes to translate complex ideas into simple and practical terms. To her, most mathematical concepts are painless and relatable, unless the person presenting them either does not understand them very well, or is trying to show off.

Other facts: Hala Nelson grew up in Lebanon during its brutal civil war. She lost her hair at a very young age in a missile explosion. This event, and many that followed, shaped her interests in human behavior, the nature of intelligence, and AI. Her dad taught her math, at home and in French, until she graduated high school. Her favorite quote from her dad about math is, “*It is the one clean science.*”

Colophon

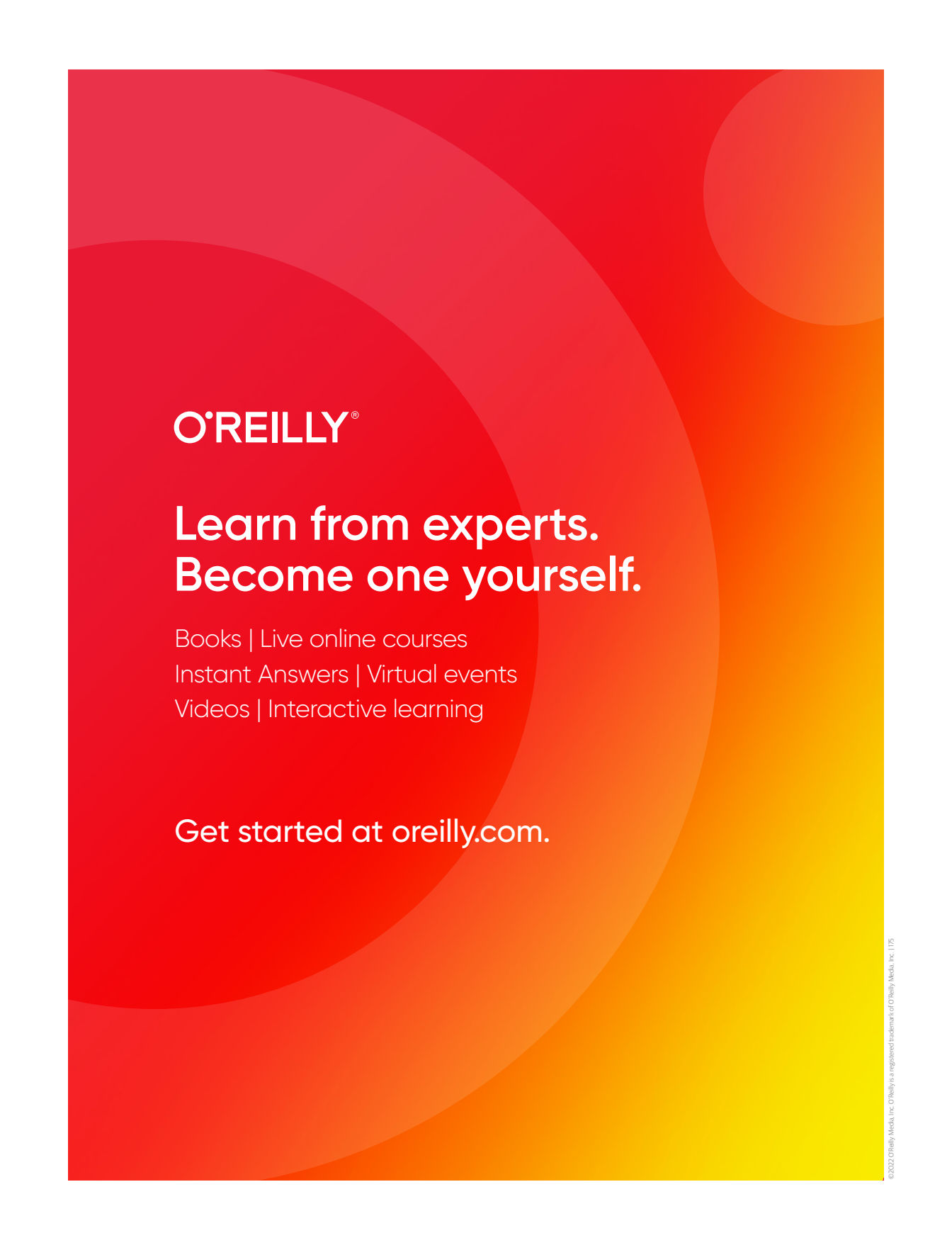
The animal on the cover of *Essential Math for AI* is a harnessed bushbuck (*Tragelaphus scriptus scriptus*), an antelope found throughout sub-Saharan Africa. The animals live in many types of habitat, such as woodland, savanna, and rainforest. The harnessed bushbuck is named for a pattern of white stripes and spots along its back and flanks that resembles a saddle or harness. These white patches also appear on the animal's neck, ears, and chin.

The harnessed bushbuck is the smallest of eight bushbuck subspecies, generally standing about 30 inches tall at the shoulder and weighing 70–100 pounds. Its coat is reddish-brown, though females tend to be lighter in color and have more conspicuous white markings. Male bushbucks also sport horns, which appear around the age of 10 months and eventually develop a single twist. Bushbucks graze on the leaves of trees and shrubs, as well as flowering plants—it is uncommon for them to eat grass.

The bushbuck is most active during the day and lives a solitary life within a defined territory. However, while they don't gather in groups, neither are these animals overly aggressive. The male's horns can be used in mating displays, to drive away competitors when a female is in heat, and for the rare territorial dispute, but adult bushbucks tend to avoid contact with each other. Female bushbucks bear one calf at a time, and hide the young one very carefully after birth, only visiting it to nurse. The mother also eats the calf's dung so predators are not drawn to the area. After about four months, the calf begins to accompany its mother to graze and play.

Though bushbucks are affected by habitat loss and are hunted for their meat and hides, they are widespread and classified as Least Concern by the IUCN. Many of the animals on O'Reilly covers are endangered; all of them are important to the world.

The cover illustration is by Karen Montgomery, based on an antique line engraving from *Shaw's Zoology*. The cover fonts are Gilroy Semibold and Guardian Sans. The text font is Adobe Minion Pro; the heading font is Adobe Myriad Condensed; and the code font is Dalton Maag's Ubuntu Mono.

The background of the entire page is a vibrant gradient of red and orange. Overlaid on this are several large, semi-transparent circles in various shades of red and orange, creating a layered, organic effect. The text is white, providing a high-contrast, clean look.

O'REILLY®

**Learn from experts.
Become one yourself.**

Books | Live online courses
Instant Answers | Virtual events
Videos | Interactive learning

Get started at oreilly.com.